DEVELOPER DOCUMENTATION

SAP RAP
Complete Developer Guide

ABAP RESTful Application Programming Model, end to end — from database table to Fiori button.

Platform SAP S/4HANA (on-premise / private cloud) · ABAP Platform 2021+ · SAP HANA 2021 · Eclipse ADT

Covers CDS interface & projection views line by line · metadata extensions · service definition & binding · read-only Fiori reports · behavior definitions · behavior pool classes · managed CRUD · draft · validations & determinations · actions & buttons · feature control & authorization · unmanaged · managed with unmanaged save · the save sequence · EML · numbering, locking and ETag

Chapters 28

Contents

00 How to Read This Guide, Naming and Build Order

01 RAP Fundamentals

02 Data Model and Tables

03 The Basic / Interface View, Line by Line

04 The Projection (Consumption) View, Line by Line

05 Metadata Extension and UI Annotations

06 Service Definition and Service Binding

07 Building a Read-Only Fiori Report, Step by Step

08 The Behavior Definition (BDEF), Keyword by Keyword

09 Behavior Pool Classes (ABP), Line by Line

10 Full CRUD Managed Application, Step by Step

11 Draft Handling

12 Validations and Determinations

13 Actions, Functions and Buttons

14 Feature Control and Authorization

15 The Unmanaged Scenario

16 Managed with Unmanaged Save, and Additional Save

17 The RAP Transaction and the Save Sequence

18 EML (Entity Manipulation Language) Reference

19 Numbering, Locking and ETag

20 Messages, Troubleshooting and Cheat Sheet

21 Side Effects

22 Value Helps and Text Associations

23 Testing and Debugging

24 Performance and Production Readiness

25 Transport and Deployment (DEV → QA → PROD)

26 Anti-Patterns: What Not To Do

27 Interview Preparation

SAP RAP — Complete Developer Documentation

> ABAP RESTful Application Programming Model, end to end. Target platform: **SAP S/4HANA (on-premise / private cloud), ABAP Platform 2021+, HANA 2021, Eclipse ADT**. Everything here also applies to ABAP Cloud / BTP ABAP Environment unless explicitly marked *on-premise only*.

Status of the code in this documentation

Read this before copying anything into a system.

The **declarative syntax** — behavior definition language, CDS annotations, BDEF keywords, the save sequence, the phase rules — was verified against SAP's published material (see Sources).

The **ABAP code examples have not been activated or executed in an SAP system.** They are written to be correct and idiomatic, and they show the right structure, but individual statements must be validated in your target release before use. RAP syntax, generated method signatures and framework class names differ across ABAP Platform releases.

Confidence	Applies to
High — verified against SAP sources	BDEF syntax, save-sequence order, phase rules, `with additional save` / `with unmanaged save` placement, CDS annotation names
Medium — standard, widely documented patterns	Managed behavior pool code (Chapters 09–14, 16), EML statements
Treat as conceptual reference	Chapter 15 — Unmanaged Scenario. See the notice at the top of that chapter and its recheck list

Two habits that make the release question mostly go away:

1. **Generate, do not type.** Use `Ctrl + 1` for every handler and saver method signature, and for draft tables. The generated artifact is authoritative for your release; an example in a document is not.
2. **Activate as you go.** Build in the order given in Chapter 10, §10.2, activating each object before the next.

The fastest source of known-good, activated code on your own system is the **ABAP Flight Reference Scenario** (`/DMO/` packages) — it ships a complete managed *and* unmanaged implementation of the same Travel/Booking model used throughout this guide.

How to read this documentation

Read Chapters 01–20 in order the first time, then use the whole set as a reference. Every chapter uses **one running example** (a Travel / Booking business object) so the same object names carry from the database table to the Fiori button.

Part I — Building blocks (read in order)

#	Chapter	What you get
01	RAP Fundamentals	Architecture, every layer, managed vs unmanaged decision table, request lifecycle
02	Data Model and Tables	Tables, UUID vs semantic keys, admin fields, draft tables
03	Basic / Interface View	`ZR_TravelTP` **line by line** — composition vs association, `$projection` , `@Semantics`
04	Projection View	`ZC_TravelTP` line by line — `provider contract` , `redirected to` , virtual elements
05	Metadata Extension & UI Annotations	Facets, line items, field groups, data points, criticality
06	Service Definition & Binding	`expose` , V2/V4, UI vs Web API, publishing
07	Read-Only Fiori Report	Complete walkthrough — **no BDEF, no ABAP**

Part II — Behavior

#	Chapter	What you get
08	Behavior Definition (BDEF)	**Every keyword**, base + projection
09	Behavior Pool Classes	Global / handler / saver anatomy, every `FOR ...` signature, `failed` / `reported` / `mapped`
10	CRUD Managed Walkthrough	Full managed app, step by step
11	Draft Handling	Lifecycle, `%tky` , the two lock phases, `Prepare`
12	Validations & Determinations	Triggers, worked implementations
13	Actions, Functions & Buttons	Every action type, and how each becomes a **button**
14	Feature Control & Authorization	DCL vs global vs instance vs feature control, **end-to-end worked scenario**

Part III — Advanced implementation types

#	Chapter	What you get
15	Unmanaged Scenario	Own buffer, all handlers, own lock, own save *(conceptual reference code)*
16	Managed with Unmanaged Save / Additional Save	Both hybrids, `save_modified`, the late-numbering pattern
17	The Save Sequence	Phases, point of no return, the allowed/forbidden matrix
18	EML Reference	Every statement, derived types, `%cid` / `%tky` / `%pid` / `%control`
19	Numbering, Locking, ETag	Five numbering scenarios, lock vs ETag, `total etag`

Part IV — Making it work in practice

#	Chapter	What you get
20	Messages, Troubleshooting & Cheat Sheet	Message classes, state messages, symptom→cause index, one-page cheat sheet
21	Side Effects	Why the UI shows stale values, and the one-line fix
22	Value Helps & Text Associations	Complete F4 example: searchable, dependent, with write-back
23	Testing & Debugging	EML console classes, ABAP Unit, `$metadata`, breakpoints, `ST05` / `SAT` / `SQLM`
24	Performance & Production Readiness	Set-based access, paging, mass EML, go-live checklist
25	Transport & Deployment	DEV → QA → PROD, what does **not** transport, post-import checklist
26	Anti-Patterns	18 wrong/right pairs, and a review checklist
27	Interview Preparation	Core and senior questions with spoken answers, plus a revision sheet

A single concatenated file is also generated — **SAP-RAP-Complete-Guide.md** — and a PDF, **SAP-RAP-Complete-Developer-Guide.pdf**.

How each chapter is structured

From Chapter 01 onwards, every chapter follows the same shape so you can navigate by position:

Section	Answers
What is it?	Definition, in the first paragraph
Why do we need it?	The problem it solves
Where is it used?	Which layer / which object
Syntax	The declarations, with every option
Complete example	Working code in the running Travel scenario
Runtime behavior	What actually happens, and when
Common errors	Symptom → cause → fix
Interview answer	A concise spoken answer
Production considerations	What matters once real users and real volume arrive

Chapters 21–27 follow it strictly. Chapters 01–20 were written before the format was fixed, so the first sections appear under topic-specific headings, but each one ends with **Interview answer** and **Production considerations**, and most also carry a **Checklist**.

The running example

```
ZR_TravelTP   (root)         <- a trip booked by an agency for a customer
   +-- _Booking              <- composition (child)
       ZR_BookingTP          <- the individual flight bookings
```

Plus read-only associations to master data: Agency, Customer, Currency, Carrier.

Object naming used throughout (and why)

RAP does not force a naming convention, but SAP's own reference applications use the pattern below. Using it consistently is the single biggest readability win in a RAP project, because the prefix tells you **which layer** an object belongs to before you open it.

Prefix	Layer	Example	Notes
`Z...` (table)	Database	`ZTRAVEL`	DDIC table or CDS table entity
`ZI_`	Basic **I**nterface view	`ZI_Agency`	Reusable, no UI annotations, no behavior
`ZR_` + `TP`	**R**oot / interface view for **T**ransactional **P**rocessing	`ZR_TravelTP`	The BO model layer
`ZC_` + `TP`	**C**onsumption / projection view	`ZC_TravelTP`	One per service. UI-facing
`ZP_`	Alternative for projection	`ZP_TravelTP`	Some teams use `ZP_`; pick one and stay with it
`ZBP_`	**B**ehavior **P**ool (ABAP class)	`ZBP_R_TravelTP`	Mirrors the BDEF name
`ZSD_`	**S**ervice **D**efinition	`ZSD_TRAVEL`	What is exposed
`ZUI_` ... `_O4`	Service **B**inding, OData **V4**, **UI**	`ZUI_TRAVEL_O4`	`_O2` for V2, `ZAPI_` for Web API
`Z..._D`	Draft table	`ZTRAVEL_D`	One per draft-enabled entity
`ZE_`	CDS abstract entity	`ZE_TravelParams`	Action parameters, event payloads
`ZI_..._VH`	Value-help provider view	`ZI_AgencyVH`	Chapter 22

> **Project note:** keep the existing system's Z/Y prefix habits. If your package already uses e.g. `ZGUL_`, prepend it: `ZGUL_R_TravelTP`. Consistency inside the package beats matching SAP demos.

Development objects you will create

For a **full transactional app** with draft, 13 objects:

#	Object type	ADT: *New ->*	Name in this guide
1–2	Database tables	Database Table	`ZTRAVEL`, `ZBOOKING`
3–4	Draft tables	Database Table *(via `Ctrl + 1` quick fix)*	`ZTRAVEL_D`, `ZBOOKING_D`
5–6	CDS view entities	Data Definition	`ZR_TravelTP`, `ZR_BookingTP`
7	Behavior Definition	Behavior Definition	`ZR_TravelTP`
8	Behavior Implementation class	*(quick fix from BDEF)*	`ZBP_R_TravelTP`
9–10	CDS projection views	Data Definition	`ZC_TravelTP`, `ZC_BookingTP`
11	Projection Behavior Definition	Behavior Definition	`ZC_TravelTP`
12	Metadata Extension	Metadata Extension	`ZC_TravelTP`
13	Service Definition + Binding	Service Definition / Binding	`ZSD_TRAVEL` / `ZUI_TRAVEL_O4`

Plus a message class (SE91) and an exception class for translatable messages. For a **read-only report**, 5 objects and no ABAP — see Chapter 07.

Prerequisites and tooling

- **Eclipse** with ABAP Development Tools (ADT). RAP artifacts (BDEF, service binding, metadata extension) **cannot** be created in SAP GUI. SE80 is not an option.
- ABAP Platform **2020+** for managed RAP with draft; **2021+** for `strict ( 2 )`, entity-level `with unmanaged save`, and RAP business events.
- `S_DEVELOP` plus a transport request, and a real development package — not `$TMP`.
- For the Fiori Elements preview: the service binding's **Publish** then **Preview**.

The three keyboard shortcuts that matter

Shortcut	What it does
`Ctrl + 1`	Quick Fix — generates behavior pools, handler and saver methods, draft tables
`Ctrl + Shift + A`	Open ABAP development object
`F3`	Navigate to definition — follow `redirected to`, associations, includes

`Ctrl + 1` is not optional knowledge. **Almost every method skeleton in RAP is generated by it**, and the generated artifact is authoritative for your release in a way that a document cannot be.

Sources

The declarative syntax in this documentation was verified against:

- SAP Help — ABAP RESTful Application Programming Model
- SAP-samples/abap-cheat-sheets — RAP Behavior Definition Language
- SAP-samples/abap-cheat-sheets — EML / ABAP for RAP
- SAP-samples/abap-platform-rap-workshops — RAP610 unmanaged save exercise
- SAP Help — Defining Unmanaged Save in the Behavior Definition
- SAP Help — Save Options
- SAP Help — Implementing the Interaction Phase and the Save Sequence
- ABAP Keyword Documentation — check_before_save, RAP Saver Method
- The **ABAP Flight Reference Scenario** (`/DMO/` packages) shipped with the system

Chapter 01 — RAP Fundamentals

1.1 What RAP actually is

RAP (ABAP RESTful Application Programming Model) is **a contract plus a runtime**.

- **The contract** — you *declare* what your business object can do (create, update, delete, actions, validations, locks, authorizations) in a dedicated language called **BDL** (Behavior Definition Language), in a file called a **Behavior Definition (BDEF)**.
- **The runtime** — the RAP framework reads that declaration and orchestrates everything around it: it parses the OData request, calls your code in a fixed order, manages a transactional buffer, handles locking, handles ETags, handles drafts, and finally commits.

The mental shift from classic ABAP is this:

Classic ABAP (module pool / ALV / BAPI)	RAP
You write the flow: PBO, PAI, `CALL FUNCTION`, `COMMIT WORK`	The framework writes the flow. You fill in *handlers*
You decide when to read and when to save	The framework decides. You only implement phases
`COMMIT WORK` wherever you like	`COMMIT WORK` is **forbidden** in most phases
Locking via `ENQUEUE_E...` calls you write	`lock master` — one word
UI is SAP GUI / ALV, tightly coupled	UI is Fiori Elements, generated from annotations

RAP is **declarative first, code second**. If you find yourself writing a lot of code for something RAP already offers declaratively (locking, ETag, field read-only-ness, draft), you are fighting the framework.

1.2 The full architecture stack

Read this diagram bottom-up. This is the order in which you build, and the order this documentation follows.

```
  +-------------------------------------------------------------+
  |  SAP Fiori Elements App  (List Report / Object Page / ALP)  |   <- generated from annotations
  +-------------------------------------------------------------+
                              |  OData V4 (or V2)
  +-------------------------------------------------------------+
7 |  Service Binding      ZUI_TRAVEL_O4                         |   protocol + version + publish
  +-------------------------------------------------------------+
6 |  Service Definition   ZSD_TRAVEL                            |   which entities are exposed
  +-------------------------------------------------------------+
5 |  Metadata Extension   ZC_TravelTP  (@UI annotations)        |   layout only, no data
  +-------------------------------------------------------------+
4 |  Projection Layer                                           |
  |    CDS Projection View      ZC_TravelTP                     |   service-specific subset
  |    Projection BDEF          ZC_TravelTP                     |   'use create; use update; ...'
  +-------------------------------------------------------------+
3 |  Behavior Layer                                             |
  |    Behavior Definition      ZR_TravelTP  (BDEF)             |   the contract
  |    Behavior Pool (class)    ZBP_R_TravelTP                  |   the implementation
  +-------------------------------------------------------------+
2 |  Business Object Model (interface / root views)             |
  |    ZR_TravelTP  --composition--> ZR_BookingTP               |   data model + associations
  +-------------------------------------------------------------+
1 |  Data Model: DDIC tables  ZTRAVEL / ZBOOKING (+ draft tables)|
  +-------------------------------------------------------------+
```

What each layer is *for* (and why it exists separately)

Layer	Purpose	Why not merge it into the one below?
1 Tables	Persistence	—
2 Interface / root view	The *single* canonical model of the business object. Field naming, associations, semantics.	One BO, many services. If you put UI annotations here, every consumer inherits them.
3 BDEF + ABP	The behavior contract and its implementation. Owned by the BO, not by a UI.	Validations must apply to *every* consumer — OData, EML from another app, background job.
4 Projection	A *service-specific view* of the BO. Only the fields and operations this app needs.	A "manager" app and an "employee" app can expose the same BO with different fields and different enabled actions.
5 Metadata extension	Pure layout. Which column where, which facet, which button.	Keeps 300 lines of `@UI` out of the projection view; can be transported/overridden separately.
6 Service definition	The published surface	You can publish two services (UI + API) from one projection.
7 Service binding	Protocol binding + activation	OData V2 for older Fiori, V4 for new; UI vs Web API.

> **The rule of thumb:** *business semantics go down, presentation goes up.* Never put `@UI` in `ZR_`. Never put business logic in `ZC_`.

1.3 The two (really four) implementation types

The **first decision** in any RAP project, taken before a single line of code:

```
managed implementation in class ZBP_R_TravelTP unique;                    " 1
unmanaged implementation in class ZBP_R_TravelTP unique;                  " 2
managed with additional save implementation in class ZBP_R_TravelTP unique;   " 3
managed with unmanaged save implementation in class ZBP_R_TravelTP unique;    " 4
```

Decision table

Question	managed	managed + additional save	managed + unmanaged save	unmanaged
Who provides the transactional buffer?	Framework	Framework	Framework	**You**
Who writes to the database?	Framework	Framework + you	**You**	**You**
Who implements CRUD handlers?	Nobody (automatic)	Nobody	Nobody	**You** (`FOR MODIFY` , `FOR READ` , `FOR LOCK`)
Draft support	Yes, declarative	Yes	Yes	Only with extra work
`persistent table` in BDEF	Required	Required	**Forbidden**	Not applicable
Typical use	Greenfield app on Z-tables	Greenfield + change docs / app log / events	Greenfield but save must call an API / late numbering	Wrapping existing BAPI / function-module logic
Effort	Lowest	Low	Medium	Highest

How to choose — in plain terms

- **Greenfield, own Z tables, no legacy logic → managed**. This covers ~80% of custom developments.
- **Managed, but on save I also need to write a change document, an application log, fire a business event, or call a BAdI → managed with additional save**. The framework still writes your table; your `save_modified` runs *in addition*.
- **Managed, but the actual persistence has to go through a released API / BAPI, or I need `CONVERT KEY` for late numbering → managed with unmanaged save**. You get all the managed goodies (buffer, lock, ETag, draft, field control) but **you** write the DB.
- **There is 15 years of existing logic in function modules with its own locking and update task, and I must not reimplement it → unmanaged**. You wrap it.

> **Common mistake:** choosing `unmanaged` because "we have existing tables". Existing *tables* are fine for managed. Choose unmanaged only when there is existing *transactional logic* you must reuse.

1.4 Vocabulary you must be fluent in

Term	Meaning
BO (Business Object)	The whole composition tree: root + all children + behavior. `ZR_TravelTP` *is* the BO.
Root entity	The top of the composition tree. Exactly one per BO. Owns the lock and the authorization by default.
Child entity	Part of the BO via `composition` . Cannot exist without its parent. Deleted when the parent is deleted.
Composition	A **parent-owns-child** relationship. Defines the BO boundary. `composition [0..*] of ZR_BookingTP as _Booking`
Association	A plain reference to *another* BO or master data. Does **not** extend the BO. `association [0..1] to ZI_Agency as _Agency`
Transactional buffer	In-memory area where all changes live until save. Nothing touches the DB before the late save phase.
Interaction phase	The phase where the consumer creates/reads/modifies. Buffer may be temporarily inconsistent.
Save sequence	The phase triggered by `COMMIT ENTITIES` that makes buffer → database.
ABP (ABAP Behavior Pool)	The global class `ZBP_...` holding local handler and saver classes.
Handler class	`lhc_...` , inherits `CL_ABAP_BEHAVIOR_HANDLER` . Implements the interaction phase.
Saver class	`lsc_...` , inherits `CL_ABAP_BEHAVIOR_SAVER` . Implements the save sequence.
EML	Entity Manipulation Language. ABAP statements (`READ ENTITIES` , `MODIFY ENTITIES` , ...) to call a RAP BO from ABAP.
Derived type	`TYPE TABLE FOR CREATE ZR_TravelTP` etc. — ABAP types the framework generates from the BDEF.
`%cid`	Content ID. A consumer-chosen temporary identifier used before a key exists.
`%key`	The key fields of an instance.
`%tky`	*Transactional key* = `%key` + `%is_draft` . **Always use `%tky` , not `%key` ,** in draft scenarios.

Term	Meaning
`%pid`	Preliminary ID in late-numbering scenarios; resolved to a real key via `CONVERT KEY`.
`%control`	Per-field flags saying which fields the consumer actually sent (`if_abap_behv=>mk-on/off`).
`mapped` / `failed` / `reported`	The three RAP response parameters. Mapping info, failed keys, messages.
Determination	Code that *changes* data automatically when a trigger condition occurs.
Validation	Code that *checks* data and rejects it with a message. Never changes data.
Action	An operation beyond CRUD, exposed as a button or an API call.
Draft	An auto-saved, not-yet-active copy of an instance, stored in a separate draft table.
Strict mode	`strict ( 1 )` / `strict ( 2 )` — turns on stricter syntax checks. Always use the highest available.

1.5 The RAP request lifecycle (what happens when a user presses Save)

Understanding this sequence explains 90% of "why doesn't my code run" questions.

```
User in Fiori presses "Create"
   |
   v
[1] OData V4 POST arrives at the service binding
   |
   v
[2] RAP runtime resolves projection -> base BDEF, checks 'use create;'
   |
   v
[3] INTERACTION PHASE
      - global authorization check      (FOR GLOBAL AUTHORIZATION)
      - instance authorization check    (FOR INSTANCE AUTHORIZATION)
      - lock                            (managed: automatic)
      - create in transactional buffer  (managed: automatic)
      - determinations 'on modify'      <- your code
      - validations 'on modify'         <- your code
      - feature control                 (FOR FEATURES / FOR INSTANCE FEATURES)
      ...user keeps editing; repeat...
   |
   v  (user presses Save -> RAP runtime issues COMMIT ENTITIES implicitly)
[4] EARLY SAVE PHASE
      - determinations 'on save'        <- your code   (run when finalize is reached)
      - finalize()                      <- your code
      - validations 'on save'           <- your code   (run when check_before_save is reached)
      - check_before_save()             <- your code
      - if anything failed -> cleanup_finalize() and return to [3]
   |
   v   ======= POINT OF NO RETURN =======
[5] LATE SAVE PHASE
      - adjust_numbers()                <- your code (late numbering only)
      - save() / save_modified()        <- framework and/or your code
      - implicit COMMIT WORK
      - cleanup()                       <- clears the buffer
   |
   v
[6] OData response returned to Fiori
```

Chapter 17 covers each phase in full, including what ABAP statements are legal in each.

1.6 Managed vs unmanaged — what code you actually write

Same business object, two implementation types. This is the clearest way to see the difference.

Managed — the behavior pool for a plain CRUD BO can be literally empty:

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.
    " nothing - create, update, delete, read, lock all work already
ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.
ENDCLASS.
```

Unmanaged — for the *same* CRUD you must implement, at minimum:

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.
    METHODS create        FOR MODIFY IMPORTING entities FOR CREATE Travel.
    METHODS update        FOR MODIFY IMPORTING entities FOR UPDATE Travel.
    METHODS delete        FOR MODIFY IMPORTING keys     FOR DELETE Travel.
    METHODS read          FOR READ   IMPORTING keys     FOR READ   Travel RESULT result.
    METHODS lock          FOR LOCK   IMPORTING keys     FOR LOCK   Travel.
```

```
    METHODS rba_Booking   FOR READ   IMPORTING keys_rba FOR READ   Travel\_Booking FULL result_requested RESULT result LINK association_links.
    METHODS cba_Booking   FOR MODIFY IMPORTING entities_cba FOR CREATE Travel\_Booking.
ENDCLASS.
```

...plus a saver class with `finalize`, `check_before_save`, `save`, `cleanup`, plus your own buffer (a global internal table in the ABP), plus your own lock objects.

That is the entire argument for managed.

1.7 Where RAP fits vs. the alternatives

You need	Use
A read-only analytical/operational report on Fiori	RAP read-only (CDS + service, no BDEF) — **Chapter 07**
A transactional Fiori app on custom tables	RAP managed — **Chapter 10**
A REST/OData API for another system to call	RAP + Web API service binding — **Chapter 06**
To call an existing SAP BO (e.g. Purchase Requisition) from your code	EML against the released `I_...TP` interface — **Chapter 18**
A classic report with SAP GUI output	Still a classic ABAP report. RAP is not for SAP GUI.
A batch job that mass-updates a RAP BO	ABAP class + EML + `COMMIT ENTITIES` — **Chapter 18**

1.8 Interview answer

> **"What is RAP, and what is the difference between managed and unmanaged?"**
>
> RAP is SAP's programming model for building OData services and Fiori apps on the ABAP platform. The data model is CDS, the transactional behavior is declared in a behavior definition, and the business logic is implemented in an ABAP behavior pool. The framework owns the transaction, the buffer, locking, ETags, draft and the OData protocol.
>
> Managed and unmanaged are **implementation types**, and the difference is who provides the transactional buffer and who performs the save. Managed: the framework does both, and you only write business logic. Unmanaged: you provide your own buffer and implement `FOR READ`, `FOR MODIFY`, `FOR LOCK`, read- and create-by-association, plus a saver class.
>
> The decision criterion is existing **transactional logic**, not existing tables — `persistent table` accepts any table. There are also two hybrids: `managed with additional save` for extra work at save time, and `managed with unmanaged save` when only the persistence is custom.

Full set of interview questions and answers: Chapter 27.

1.9 Production considerations

- **Record the implementation-type decision and why.** It is the one architectural choice that is expensive to reverse, and the next developer will otherwise assume it was arbitrary.
- **Do not mix implementation types across a composition tree** unless you have a specific reason; the save behaviour applies to the whole business object.
- **Check release availability before you design.** `strict ( 2 )`, entity-level `with unmanaged save` and RAP business events need ABAP Platform 2021+. DEV, QA and PROD must be on the same release, or code that activates in DEV will not activate in PROD.
- **Plan for more than one consumer from day one.** Even a "single Fiori app" usually acquires a background job and an API later; that is what keeps business logic in the base BDEF rather than the projection.

Chapter 02 — Data Model and Tables

2.1 The scenario

A travel agency books trips. Each **Travel** has a customer, an agency, a date range, a price and a status. Each Travel contains one or more **Bookings** (individual flights).

```
ZTRAVEL  (root)
   travel_uuid   (key, UUID)
   travel_id     (semantic / human-readable number)
   agency_id, customer_id, begin_date, end_date, booking_fee, total_price, currency_code
   description, overall_status
   <admin fields>

ZBOOKING (child)
   booking_uuid  (key, UUID)
   travel_uuid   (foreign key to parent -> this is what makes the composition work)
   booking_id
   booking_date, customer_id, carrier_id, connection_id, flight_date, flight_price, currency_code
   booking_status
   <admin fields>
```

2.2 Key design: UUID vs semantic key

This is a decision you cannot easily undo later. Decide before creating the tables.

	UUID key (`RAW(16)` / `SYSUUID_X16`)	Semantic key (e.g. `TRAVEL_ID` NUMC(8))
Generated by	RAP framework, automatically (`numbering : managed`)	You, via number range / `early numbering` / consumer
Parent-child linking	Trivial — child stores parent UUID	Needs the full parent key in every child
Draft support	Works out of the box	Works, but late numbering complicates it
Readability in DB	Poor (hex blob)	Good
URL / OData key	Ugly but stable	Clean
Recommended for	**Almost always**	Only when an external system or business requires a specific number format

The standard pattern used by SAP and used here: *both*.

- `travel_uuid` — the technical key, UUID, managed numbering. This is the RAP key.
- `travel_id` — a human-readable number filled by a **determination** from a number range object.

This gives you framework-managed keys *and* a readable business number. Chapter 12 shows the determination that fills `travel_id`.

2.3 The root table `ZTRAVEL`

```
@EndUserText.label : 'Travel - persistent table'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table ztravel {
  key client           : abap.clnt not null;
  key travel_uuid      : sysuuid_x16 not null;
  travel_id            : /dmo/travel_id;
  agency_id            : /dmo/agency_id;
  customer_id          : /dmo/customer_id;
  begin_date           : /dmo/begin_date;
  end_date             : /dmo/end_date;
  @Semantics.amount.currencyCode : 'ztravel.currency_code'
  booking_fee          : /dmo/booking_fee;
  @Semantics.amount.currencyCode : 'ztravel.currency_code'
  total_price          : /dmo/total_price;
  currency_code        : /dmo/currency_code;
  description          : /dmo/description;
  overall_status       : /dmo/overall_status;
  created_by           : abp_creation_user;
  created_at           : abp_creation_tstmpl;
  last_changed_by      : abp_lastchange_user;
  last_changed_at      : abp_lastchange_tstmpl;
  local_last_changed_at : abp_locinst_lastchange_tstmpl;
}
```

Line-by-line

Line	Meaning
`@EndUserText.label`	Short description shown in ADT and DDIC.
`@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE`	No customer append structures allowed. Use `#EXTENSIBLE_CHARACTER_NUMERIC` if the table must be appendable. Mandatory annotation — activation fails without it.
`@AbapCatalog.tableCategory : #TRANSPARENT`	Normal transparent table (1:1 with a HANA table).
`@AbapCatalog.deliveryClass : #A`	`A` = application/transaction data. Use `C` for customizing, `L` for temporary. Controls client-copy and transport behaviour.
`@AbapCatalog.dataMaintenance : #RESTRICTED`	SE16/SM30 maintenance is restricted. For RAP tables this is correct — the BO owns the data, not SE16.
`key client : abap.clnt not null`	**Mandatory first key field.** RAP requires a client field. Never omit it, even in single-client development.
`key travel_uuid : sysuuid_x16`	The technical key. `sysuuid_x16` = `RAW(16)`. This is what `numbering : managed` fills.
`travel_id : /dmo/travel_id`	Semantic number. `/dmo/...` are SAP demo data elements; in a real project use your own `ZDE_TRAVEL_ID`.
`@Semantics.amount.currencyCode : 'ztravel.currency_code'`	**Required** on every amount field. Tells the runtime and the UI which field holds the currency. Without it, CDS activation on the view above will warn and Fiori will render the amount without a currency. Note the value is `tablename.fieldname`.
`created_by : abp_creation_user`	RAP **admin field**. These five `abp_*` data elements are the standard set; using them lets the managed runtime fill them automatically (see 2.5).
`local_last_changed_at : abp_locinst_lastchange_tstmpl`	Used as the **ETag** field. Changed on every local change, including draft-related ones.

Why five admin fields and not two

Field	Data element	Filled when	Used for
`created_by`	`abp_creation_user`	On create	Audit
`created_at`	`abp_creation_tstmpl`	On create	Audit
`last_changed_by`	`abp_lastchange_user`	On any *content* change	Audit
`last_changed_at`	`abp_lastchange_tstmpl`	On any *content* change	**Total ETag** (draft resumability)
`local_last_changed_at`	`abp_locinst_lastchange_tstmpl`	On *any* change of this instance, including ones that don't change business content	**ETag** (optimistic locking)

`last_changed_at` and `local_last_changed_at` look redundant but are not:

- `local_last_changed_at` changes whenever *this* row changes → perfect for optimistic locking of *this* row.
- `last_changed_at` propagates up the composition tree → a Booking change updates the Travel's `last_changed_at`, so the whole document has one "version". That is what `total etag` needs.

SAP's recommendation, which this guide follows: `etag master LocalLastChangedAt` and `total etag LastChangedAt`, and **use different fields for the two**.

2.4 The child table `ZBOOKING`

```
@EndUserText.label : 'Booking - persistent table'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table zbooking {
  key client            : abap.clnt not null;
  key booking_uuid      : sysuuid_x16 not null;
  travel_uuid           : sysuuid_x16 not null;   " <- parent key. NOT part of the key.
  booking_id            : /dmo/booking_id;
  booking_date          : /dmo/booking_date;
  customer_id           : /dmo/customer_id;
  carrier_id            : /dmo/carrier_id;
  connection_id         : /dmo/connection_id;
  flight_date           : /dmo/flight_date;
  @Semantics.amount.currencyCode : 'zbooking.currency_code'
  flight_price          : /dmo/flight_price;
  currency_code         : /dmo/currency_code;
  booking_status        : /dmo/booking_status;
  last_changed_at       : abp_locinst_lastchange_tstmpl;
}
```

The one thing people get wrong here

`travel_uuid` is a **normal field, not a key field**.

Because `booking_uuid` is itself a UUID and therefore globally unique, the child does not need a compound key. The parent reference is just a data field that the composition's `ON` condition matches. This is the *UUID-based* modelling style and it is what managed RAP with `numbering : managed` expects.

If you instead use semantic keys, the child key **is** compound:

```
key travel_id  : /dmo/travel_id;   " part of the key
key booking_id : /dmo/booking_id;  " part of the key
```

Both work. Do not mix them inside one BO.

2.5 Draft tables

A draft-enabled entity needs a **second table** that mirrors the active one.

```
@EndUserText.label : 'Travel - draft table'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table ztravel_d {
  key client            : abap.clnt not null;
  key traveluuid        : sysuuid_x16 not null;
  travelid              : /dmo/travel_id;
  agencyid              : /dmo/agency_id;
  customerid            : /dmo/customer_id;
  begindate             : /dmo/begin_date;
  enddate               : /dmo/end_date;
  @Semantics.amount.currencyCode : 'ztravel_d.currencycode'
  bookingfee            : /dmo/booking_fee;
  @Semantics.amount.currencyCode : 'ztravel_d.currencycode'
  totalprice            : /dmo/total_price;
  currencycode          : /dmo/currency_code;
  description           : /dmo/description;
  overallstatus         : /dmo/overall_status;
  createdby             : abp_creation_user;
  createdat             : abp_creation_tstmpl;
  lastchangedby         : abp_lastchange_user;
  lastchangedat         : abp_lastchange_tstmpl;
  locallastchangedat    : abp_locinst_lastchange_tstmpl;
  "%admin" : include sych_bdl_draft_admin_inc;
}
```

Rule 0 — generate the draft table, do not design it by hand

Do not manually design draft-table technical fields unless you have a specific reason to. Generate the draft table through ADT and verify the generated structure against your release.

In ADT: open the CDS root view, place the cursor on the entity name, press `Ctrl + 1` → "*Create draft table ...*". ADT derives the field names, types and the admin include from the current entity definition and the current release's draft framework.

This matters because the draft table is an **interface to the RAP draft runtime**, not a table you own. Its exact required structure is release- and tooling-dependent, and it has changed across ABAP Platform releases (late-numbering key fields and admin-include content in particular). A hand-typed structure that activates today can break on the next upgrade; a generated one tracks the framework.

The listing above is shown so you can **read and review** a generated draft table, not as a template to type from.

What the generated structure has to satisfy

These are the properties to check on the generated result — they are the constraints, expressed as review criteria rather than as a typing recipe:

1. **One draft table per draft-enabled entity.** Travel and Booking each need their own.
2. **Field names correspond to the CDS *element* names, not the DDIC field names of the active table.** In the generated example above the active table has `travel_uuid` while the draft table has `traveluuid`, because the CDS view exposes the element as `TravelUUID`. Treat the exact transformation as *whatever ADT generated for your release* — verify it, do not assume it.
3. **Types are compatible** with the corresponding CDS elements.
4. **A client field is present.**
5. **The draft admin include is present:** `abap "%admin" : include sych_bdl_draft_admin_inc;` It supplies the technical fields the RAP transactional engine needs (draft creation and change timestamps, the draft administrative data UUID, the operation code, the "has active entity" indicator, and others — the exact set comes from the include and varies by release, which is another reason to include it rather than to list the fields yourself).
6. *(Late numbering)* an additional key field for the draft UUID is required. Let ADT add it; the name and type are release-specific.

If a regenerated draft table differs from your existing one after an upgrade, take the generated version as correct and adjust — not the other way round.

> **Never read or write a draft table with ABAP SQL.** Its contents are framework-owned. Use EML. This applies to reporting, clean-up jobs and debugging alike.

2.6 Data elements and domains

Use your own data elements. Reusing `/DMO/...` is fine for learning but wrong in a real project — they belong to SAP's demo package and can change.

A minimal set for this example:

Data element	Domain / type	Note
`ZDE_TRAVEL_ID`	`NUMC(8)`	Semantic travel number
`ZDE_BOOKING_ID`	`NUMC(4)`	Booking item number
`ZDE_OVERALL_STATUS`	`CHAR(1)` with fixed values `O` / `A` / `X`	Open / Accepted / Cancelled
`ZDE_TOTAL_PRICE`	`CURR(16,2)`	Needs `@Semantics.amount.currencyCode`
`ZDE_CURRENCY_CODE`	`CUKY(5)`	Check table `TCURC`

Fixed values and the UI

If you put **fixed values on the domain** of `ZDE_OVERALL_STATUS`, Fiori Elements automatically renders a dropdown with those texts — no value help object, no annotation. This is the cheapest possible value help and it is often overlooked.

For anything with more than a handful of values, or values that are master data, use a **CDS value help association** instead (Chapter 04, section 4.6).

2.7 Alternative: CDS table entities (ABAP Cloud)

In ABAP Cloud / newer on-premise releases you can replace the DDIC table with a **CDS table entity**:

```
@EndUserText.label : 'Travel'
define table entity ZTRAVEL
{
  key TravelUuid : sysuuid_x16;
  TravelId       : zde_travel_id;
  ...
}
```

Advantages: CDS-native syntax, element names identical to the view (so **draft table naming stops being a trap**), no DDIC editor. `persistent table ZTRAVEL` in the BDEF accepts a CDS table entity exactly like a DDIC table.

Availability: ABAP Platform 2022 / ABAP Cloud. On S/4HANA 2021 on-premise, stay with DDIC tables.

2.8 Checklist before moving to Chapter 03

- [ ] Both tables activate without warnings.
- [ ] Every amount field has `@Semantics.amount.currencyCode`; every quantity field has `@Semantics.quantity.unitOfMeasure`.
- [ ] Client field is the first key field in every table.
- [ ] Admin fields use the `abp_*` data elements.
- [ ] Draft tables (if draft is planned) were **generated by the ADT quick fix**, not hand-typed, and the generated structure was reviewed against the criteria in section 2.5.
- [ ] The child table has the parent UUID as a **non-key** field (UUID modelling).

2.9 Interview answer

> **"How do you design the key and the admin fields for a RAP business object?"**
>
> I use both a technical and a semantic key. The technical key is a UUID with `field ( numbering : managed, readonly )`, so the framework generates it — that makes parent-child linking trivial and draft works out of the box. The human-readable document number is a separate field filled by a determination declared `on save`, from a number range, so that a discarded draft does not consume a number and leave gaps.
>
> For audit I use the five standard `abp_*` data elements. Annotating them with `@Semantics.user.*` and `@Semantics.systemDateTime.*` in the CDS view and marking them `readonly` in the BDEF means the managed runtime fills them automatically — no determination at all.
>
> `LastChangedAt` and `LocalLastChangedAt` look redundant but are not: the local one changes when *this* row changes and is the ETag; the other propagates up the composition tree and is the **total** ETag that draft resumability depends on. They should be different fields.
>
> Draft tables I generate with the ADT quick fix rather than typing — the required structure is release-dependent and it is an interface to the draft runtime, not a table I own.

2.10 Production considerations

- **Use your own data elements**, not `/DMO/...`. Those belong to SAP's demo package.
- **Amount and quantity fields need `@Semantics` from day one.** Adding them later means revisiting every view and every consumer.
- **Set `@AbapCatalog.dataMaintenance : #RESTRICTED`.** The business object owns the data; SE16 edits bypass every validation you wrote.
- **Plan archiving before go-live** for transactional tables. RAP gives you no housekeeping.
- **Table changes need conversion in the target system.** Check `SE11` after an import that alters a table with data.

- **Draft tables grow.** Schedule the clean-up job (Chapter 11, §11.8) as part of go-live, not later.
- `#NOT_EXTENSIBLE` **is a decision.** If a customer or a later project must append fields, choose an extensible category now.

Chapter 03 — The Basic / Interface View, Line by Line

3.1 What this layer is and is not

The interface view (`ZR_TravelTP`) is **the business object's data model**. It is the one place that defines:

- which database fields exist in the BO and what they are called
- the composition tree (what belongs to the BO)
- associations to other BOs / master data
- semantic annotations (amount, quantity, user, date-time)

It **must not** contain:

- `@UI.*` annotations (those go in the metadata extension, Chapter 05)
- `@Search.*` for a specific app
- anything service-specific

> If two different Fiori apps need the same business object, they share `ZR_TravelTP` and have their own `ZC_...` projections. That only works if `ZR_` stays neutral.

3.2 `DEFINE VIEW` vs `DEFINE VIEW ENTITY` vs `DEFINE ROOT VIEW ENTITY`

Syntax	Generates a DDIC SQL view?	Use it?
`define view ZX as select from ...`	Yes (`@AbapCatalog.sqlViewName` required)	**Legacy.** Only for old releases or when you must expose an SQL view.
`define view entity ZX as select from ...`	No	**Yes** — the modern form. Faster activation, better syntax checks, no SQL view artifact.
`define root view entity ZX as select from ...`	No	**Yes** — for the *root* of a composition tree.

Everything in this documentation uses **view entities**. If you see `@AbapCatalog.sqlViewName` in a tutorial, that tutorial predates ABAP Platform 1909.

The word `root` does exactly one thing: it marks this entity as the **top of a composition tree**, which is a precondition for attaching a behavior definition that has children.

3.3 The root interface view `ZR_TravelTP` — full source

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Travel - interface view (RAP root)'
@Metadata.ignorePropagatedAnnotations: true
@ObjectModel.usageType: { serviceQuality: #X, sizeCategory: #S, dataClass: #MIXED }
define root view entity ZR_TravelTP
  as select from ztravel as Travel

  composition [0..*] of ZR_BookingTP as _Booking

  association [0..1] to /DMO/I_Agency   as _Agency   on $projection.AgencyId   = _Agency.AgencyID
  association [0..1] to /DMO/I_Customer as _Customer on $projection.CustomerId = _Customer.CustomerID
  association [0..1] to I_Currency      as _Currency on $projection.CurrencyCode = _Currency.Currency
{
  key travel_uuid             as TravelUUID,

      travel_id               as TravelId,
      agency_id               as AgencyId,
      customer_id             as CustomerId,
      begin_date              as BeginDate,
      end_date                as EndDate,

      @Semantics.amount.currencyCode: 'CurrencyCode'
      booking_fee             as BookingFee,
      @Semantics.amount.currencyCode: 'CurrencyCode'
      total_price             as TotalPrice,
      currency_code           as CurrencyCode,

      description             as Description,
      overall_status          as OverallStatus,

      @Semantics.user.createdBy: true
      created_by              as CreatedBy,
      @Semantics.systemDateTime.createdAt: true
      created_at              as CreatedAt,
      @Semantics.user.lastChangedBy: true
      last_changed_by         as LastChangedBy,
      @Semantics.systemDateTime.lastChangedAt: true
      last_changed_at         as LastChangedAt,
      @Semantics.systemDateTime.localInstanceLastChangedAt: true
      local_last_changed_at   as LocalLastChangedAt,
```

```
      /* Associations */
      _Booking,
      _Agency,
      _Customer,
      _Currency
}
```

3.4 Line by line

The header annotations

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
```

Declares whether a DCL (access control / CDS role) is expected for this view.

Value	Meaning	When to use
`#CHECK`	A DCL exists and **is** applied. Activation warns if no DCL is found.	Production views where row-level authorization is needed.
`#NOT_REQUIRED`	No DCL. No warning.	Interface views where authorization is enforced *one layer up* (in the projection) or in the BDEF.
`#NOT_ALLOWED`	DCL must not exist.	Rare — technical/helper views.
`#PRIVILEGED_ONLY`	Only privileged callers.	Internal framework views.

Where to put the DCL in RAP — a design decision, not a fixed rule.

DCL can be applied at whichever CDS layer the authorization rule actually belongs to:

Put the DCL on the…	When
Projection (`ZC_TravelTP`)	Different services over the same BO need **different** row-level restrictions — e.g. a manager app and a self-service app. The interface view then stays `#NOT_REQUIRED`.
Interface view (`ZR_TravelTP`)	The restriction is an **invariant of the data model itself** — every consumer, present and future, must be subject to it. Common for sensitive data (HR, payroll) where an unrestricted read must not be possible at any layer.
Both	An invariant baseline on the interface view, plus a narrower service-specific filter on the projection. DCL conditions at different layers are combined, so the effective result is the intersection.

The example above uses `#NOT_REQUIRED` because this BO's restriction is service-specific. If your security architecture says the rule belongs to the reusable model, put it on `ZR_` and set `#CHECK` there instead — that is equally valid.

What drives the choice is **reuse and security architecture**: how many services will consume this model, and whether an unfiltered read of the interface view is acceptable. Decide it deliberately and write the decision down; it is hard to change once several services depend on the model. See Chapter 14 for the implementation and for how DCL relates to BDEF authorization.

```
@EndUserText.label: 'Travel - interface view (RAP root)'
```

Description. Max 60 characters. Shown in ADT, in the where-used list, and — if you forget a label on the projection — in the UI.

```
@Metadata.ignorePropagatedAnnotations: true
```

Important and frequently misunderstood. By default, a CDS view *inherits* annotations from the entities it selects from. With `true`, that inheritance is switched off: this view sees only the annotations written in this file.

Use `true` on interface views. Reason: you do not want a stray `@UI.lineItem` from a lower view silently leaking into your BO and then into every service. It makes the view self-contained and predictable. (`@Semantics` you write *here* still applies — you are only blocking *propagated* ones.)

```
@ObjectModel.usageType: { serviceQuality: #X, sizeCategory: #S, dataClass: #MIXED }
```

Classification metadata for the VDM. It has no runtime effect on RAP, but SAP's CDS checks (and ATC) expect it on interface views.

Sub-annotation	Values	Meaning
`serviceQuality`	`#A` (transactional/high) … `#X` (extreme, no guarantee)	Expected response-time class. `#X` = "not classified".
`sizeCategory`	`#S` , `#M` , `#L` , `#XL` , `#XXL`	Expected result-set size.
`dataClass`	`#TRANSACTIONAL` , `#MASTER` , `#ORGANIZATIONAL` , `#CUSTOMIZING` , `#META` , `#MIXED`	What kind of data.

For a transactional BO root, `{ serviceQuality: #D, sizeCategory: #S, dataClass: #TRANSACTIONAL }` is also a common and more accurate choice.

The entity declaration

```
define root view entity ZR_TravelTP
```

```
  as select from ztravel as Travel
```

Token	Meaning
`define`	Start of a CDS entity definition.
`root`	This entity is the root of a composition tree. **Required** if the BDEF has child entities. Omitting it produces "*Entity must be a root entity*" when you activate the BDEF.
`view entity`	Modern CDS view — no generated DDIC SQL view.
`ZR_TravelTP`	The CDS entity name. Must match the file/object name in ADT. Case-insensitive at activation, but the spelling you use here is the one that appears in EML and the BDEF, so be consistent.
`as select from ztravel`	The data source. Can be a DDIC table, a CDS table entity, or another CDS view.
`as Travel`	Alias for the data source, used on the left side of field assignments and in join conditions.

The composition

```
composition [0..*] of ZR_BookingTP as _Booking
```

This single line is what makes `ZR_BookingTP` **part of this business object**.

Part	Meaning
`composition`	A parent-owns-child relationship. Establishes the BO boundary.
`[0..*]`	Cardinality: a Travel has zero or more Bookings.
`of ZR_BookingTP`	The child entity. It **must** declare the matching `association to parent`.
`as _Booking`	The name you use in the BDEF (`_Booking`), in EML (`\_Booking`), and in UI facets. Convention: leading underscore.

There is no `on` condition. That is not an omission — a composition derives its join condition from the child's `association to parent` clause. The child owns the linking logic.

Composition vs association — the decision rule

Ask	If yes →	If no →
Does the child have any meaning without the parent?	association	composition
Should deleting the parent delete it?	composition	association
Should it be locked together with the parent?	composition	association
Should it be saved in the same LUW as part of the same document?	composition	association
Is it master data (Customer, Agency, Currency, Plant)?	**association**, always	—

Concretely: *Booking* is a composition (a booking outside a travel is meaningless). *Customer* is an association (a customer exists independently and is owned by another BO).

Cardinalities

Cardinality	Meaning
`[0..1]`	Optional single child
`[1..1]`	Mandatory single child
`[0..*]`	Zero or many — the normal case for item tables
`[1..*]`	At least one — the framework does **not** enforce this; you enforce it with a validation on save

> `[1..*]` on a composition does not make RAP reject a Travel with no Bookings. Cardinality is metadata for the optimizer and the UI, not a constraint. Write a validation (Chapter 12).

Associations

```
association [0..1] to /DMO/I_Agency as _Agency on $projection.AgencyId = _Agency.AgencyID
```

Part	Meaning
`association [0..1] to /DMO/I_Agency`	A reference to another entity. Not part of this BO.
`as _Agency`	The alias.
`on $projection.AgencyId = _Agency.AgencyID`	The join condition.

`$projection` vs the source alias — this matters.

You write	It means
`$projection.AgencyId`	Use the field **as it appears in the SELECT list of this view** (i.e. after the `as AgencyId` rename).
`Travel.agency_id`	Use the field **from the data source**, with its DDIC name.

Both work here. Prefer `$projection` because:

- it survives a change of data source,
- it works when the field is calculated (e.g. `cast(...)` or a `case`), which a direct source reference cannot do.

When must you use the source alias instead? When the field is *not* in the SELECT list. If you want to join on `ztravel~agency_id` but you do not expose `AgencyId`, you must write `Travel.agency_id`.

Associations are *not* joins

A CDS association is a **lazy** join. It is only resolved when the association is actually used (a path expression, a `_Agency.Name` in a projection, an OData `$expand`, or a UI value help). If nobody uses it, no join is executed. That is why exposing 10 associations costs nothing, while 10 `LEFT OUTER JOIN`s cost a lot.

This is one of the strongest reasons to model with associations rather than joins in CDS.

Exposing an association's field directly

```
_Agency.Name as AgencyName,     " path expression – creates an implicit LEFT OUTER JOIN when used
```

Legal, and useful for a read-only report. Avoid it in a transactional interface view — it makes the field non-updatable and drags the joined table into every read.

The field list

```
key travel_uuid as TravelUUID,
```

Part	Meaning
`key`	Marks this as a key element of the CDS entity. **This — not the DDIC key — is what RAP uses as the BO key.**
`travel_uuid`	Source field name.
`as TravelUUID`	The CDS element name. This is the name used in the BDEF, in EML, in the draft table, and in OData.

Naming convention for elements: `UpperCamelCase`, no underscores. `travel_uuid` → `TravelUUID`. This is not cosmetic: OData property names come from here, and Fiori Elements conventions assume it.

> Do not expose the `client` field. CDS handles client dependency automatically. Exposing `mandt` breaks the OData model.

```
      travel_id   as TravelId,
      agency_id   as AgencyId,
```

Plain fields, renamed. No annotation needed.

`@Semantics` annotations — what each one actually buys you

```
@Semantics.amount.currencyCode: 'CurrencyCode'
booking_fee as BookingFee,
```

Declares that `BookingFee` is an amount whose currency is in the element `CurrencyCode` **of this same view** (note: element name, not `table.field` as in the DDIC table).

What it gives you:

- correct decimal-shift rendering in Fiori (JPY has 0 decimals, KWD has 3),
- the currency shown next to the number,
- aggregation/conversion support in analytical scenarios,
- a CDS activation **warning** if missing.

```
@Semantics.quantity.unitOfMeasure: 'UnitOfMeasure'
```

Same for quantities. Both annotations must point at an element that is actually in the SELECT list.

```
@Semantics.user.createdBy: true
created_by as CreatedBy,

@Semantics.systemDateTime.createdAt: true
created_at as CreatedAt,

@Semantics.user.lastChangedBy: true
last_changed_by as LastChangedBy,
```

```
@Semantics.systemDateTime.lastChangedAt: true
last_changed_at as LastChangedAt,

@Semantics.systemDateTime.localInstanceLastChangedAt: true
local_last_changed_at as LocalLastChangedAt,
```

These five are **functional, not documentation**. In a *managed* RAP BO they trigger automatic behaviour:

Annotation	Effect in managed RAP
`@Semantics.user.createdBy`	Framework fills the field with `sy-uname` on create. You do **not** write a determination.
`@Semantics.systemDateTime.createdAt`	Framework fills the timestamp on create.
`@Semantics.user.lastChangedBy`	Framework fills on every change.
`@Semantics.systemDateTime.lastChangedAt`	Framework fills on every change. Enables `total etag` support.
`@Semantics.systemDateTime.localInstanceLastChangedAt`	Framework fills on every local change. Enables `etag master` support.

For the ETag annotations, the field type must be `utclong`, `timestamp` or `timestampl` (`abp_locinst_lastchange_tstmpl` is `timestampl` — correct).

> Combined with the BDEF's `readonly` on these fields (Chapter 08), this is the complete admin-field solution. Zero lines of ABAP.

Exposing associations at the end

```
      /* Associations */
      _Booking,
      _Agency,
      _Customer,
      _Currency
```

An association declared in the header is **not** available to consumers unless you also list it in the SELECT list. Forgetting this is the classic cause of:

> "*_Booking is not a valid element of ZR_TravelTP*" when you write the BDEF or the projection.

Exposing `_Booking` (the composition) is **mandatory** — the BDEF needs it.

3.5 The child interface view `ZR_BookingTP`

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Booking - interface view'
@Metadata.ignorePropagatedAnnotations: true
define view entity ZR_BookingTP
  as select from zbooking as Booking

  association to parent ZR_TravelTP as _Travel
    on $projection.TravelUUID = _Travel.TravelUUID

  association [0..1] to /DMO/I_Customer   as _Customer   on $projection.CustomerId   = _Customer.CustomerID
  association [0..1] to /DMO/I_Carrier    as _Carrier    on $projection.CarrierId    = _Carrier.AirlineID
  association [0..1] to /DMO/I_Connection as _Connection on  $projection.CarrierId    = _Connection.AirlineID
                                                         and $projection.ConnectionId = _Connection.ConnectionID
{
  key booking_uuid    as BookingUUID,
      travel_uuid     as TravelUUID,
      booking_id      as BookingId,
      booking_date    as BookingDate,
      customer_id     as CustomerId,
      carrier_id      as CarrierId,
      connection_id   as ConnectionId,
      flight_date     as FlightDate,
      @Semantics.amount.currencyCode: 'CurrencyCode'
      flight_price    as FlightPrice,
      currency_code   as CurrencyCode,
      booking_status  as BookingStatus,
      @Semantics.systemDateTime.localInstanceLastChangedAt: true
      last_changed_at as LastChangedAt,

      /* Associations */
      _Travel,
      _Customer,
      _Carrier,
      _Connection
}
```

The three differences from the root view

1. **No `root` keyword.** A child is not a root.

2. `association to parent` — this is a special association type, not a normal one:
   `abap association to parent ZR_TravelTP as _Travel on $projection.TravelUUID = _Travel.TravelUUID` | Rule | Detail | |---|---| | Exactly one per child entity | A child has exactly one parent. | | No cardinality allowed on the left | It is implicitly `[1..1]`. Writing `association [1..1] to parent` is a syntax error. | | Must be exposed in the SELECT list | Same rule as any association. | | Supplies the composition's join condition | This is why the parent's `composition` line has no `on`. |
3. `TravelUUID` **is exposed but is not a key.** It must be in the SELECT list so the `association to parent` can reference it via `$projection`.

3.6 Multi-level compositions

Compositions nest arbitrarily:

```
ZR_TravelTP  (root)
   +-- _Booking      ZR_BookingTP
         +-- _BookingSupplement   ZR_BookSupplTP
```

- `ZR_BookingTP` gets a `composition [0..*] of ZR_BookSupplTP as _BookingSupplement`.
- `ZR_BookSupplTP` gets `association to parent ZR_BookingTP as _Booking`.
- Only `ZR_TravelTP` is `root`. Intermediate nodes are plain `view entity`.
- **A grandchild may also declare an association back to the root** for convenience:
  `abap association [1..1] to ZR_TravelTP as _Travel on $projection.TravelUUID = _Travel.TravelUUID` This is normal and useful — you often need the root key in a child's validation.

Practical limit: SAP supports deep trees, but every level adds a lock scope, a draft table and a handler class. Three levels is a lot. If you are at four, question the model.

3.7 Common building blocks in interface views

`cast`

```
cast( overall_status as zde_overall_status preserving type ) as OverallStatus,
```

Retypes a field. `preserving type` keeps the underlying built-in type and only swaps the data element — that is what you want when the source is a generic `char1` and you need your own data element (with its fixed values) so the UI shows a dropdown.

`case`

```
case overall_status
  when 'O' then 2      " yellow / critical
  when 'A' then 3      " green  / positive
  when 'X' then 1      " red    / negative
  else 0
end                              as OverallStatusCriticality,
```

Calculated criticality. Feeds `@UI.criticality` in the metadata extension (Chapter 05) to colour a status column. Note it is calculated *here* in the interface view so that every consumer gets it — this is business semantics, not layout.

Arithmetic and `@EndUserText.label` on an element

```
@EndUserText.label: 'Trip duration (days)'
dats_days_between( begin_date, end_date ) as TripDuration,
```

Currency and unit conversion

```
currency_conversion(
  amount               => total_price,
  source_currency      => currency_code,
  target_currency      => cast( 'EUR' as abap.cuky ),
  exchange_rate_date   => begin_date,
  error_handling       => #SET_TO_NULL )  as TotalPriceEUR,
```

`@ObjectModel.text.element` — text for a code

```
@ObjectModel.text.element: ['OverallStatusText']
overall_status as OverallStatus,
_StatusText.Text as OverallStatusText,
```

Tells the UI: when displaying `OverallStatus`, show `OverallStatusText`. The rendering (`#TEXT_ONLY`, `#TEXT_FIRST`, ...) is then controlled with `@UI.textArrangement` in the metadata extension.

3.8 What *not* to do in the interface view

Anti-pattern	Why it hurts	Do instead
`@UI.lineItem` etc.	Every service inherits your layout	Metadata extension on `ZC_`
`LEFT OUTER JOIN` to master data	Always executed, even when unused	`association`
`WHERE` clause filtering business data (`where deleted = ''`)	Hidden filter surprises every consumer; also breaks managed CRUD (you can create a row the view then hides)	Filter in the projection, or model a proper status
Client field in the SELECT list	Breaks OData	Omit it
`union` / `union all`	A BO root with a union cannot be managed (no single persistent table)	One view per source, or an unmanaged BO
Aggregation (`sum(...)` , `group by`)	Not updatable, cannot be a transactional root	Separate analytical view
Field names with underscores	Non-standard OData properties; draft table naming becomes ambiguous	`UpperCamelCase`

> **The hard rule for managed RAP:** the root interface view must be a **1:1 projection of one persistent table** — no joins that multiply rows, no aggregation, no union. The managed runtime maps `INSERT` / `UPDATE` / `DELETE` straight onto `persistent table` . Anything that breaks the 1:1 mapping breaks the managed save.

3.9 Checklist

- [ ] Root view is `define root view entity` .
- [ ] Composition declared on the parent, `association to parent` on the child.
- [ ] Every association **and** the composition appear in the SELECT list.
- [ ] Element names are `UpperCamelCase` , no client field.
- [ ] All five admin fields carry their `@Semantics` annotations.
- [ ] Every amount has `@Semantics.amount.currencyCode` .
- [ ] No `@UI` annotations anywhere.
- [ ] Activates with zero warnings.

3.10 Interview answer

> **"What is the difference between a composition and an association?"**
>
> A composition defines the **boundary of the business object**: the child is owned by the parent, cannot exist without it, is locked with it, saved in the same LUW, and deleted when the parent is deleted. An association is just a reference to something outside the BO — master data, or another business object — and is completely independent.
>
> The composition is declared on the parent with `composition [0..*] of ZR_BookingTP as _Booking` , and the child declares the matching `association to parent` . There is no `on` condition on the composition, because the join condition comes from the child's parent association.
>
> The practical test: would deleting the parent delete it, and should it be locked with the parent? A booking inside a travel, yes — composition. A customer, no — association, always.
>
> One thing worth adding: a CDS association is **lazy**. It is only resolved when something actually uses it, so exposing ten associations costs nothing, whereas ten `LEFT OUTER JOIN` s cost every read. That is the main reason to model with associations rather than joins.

3.11 Production considerations

- **Keep `ZR_` free of `@UI` annotations.** Every future service inherits whatever you put there. Use `@Metadata.ignorePropagatedAnnotations: true` so nothing leaks up from below either.
- **Verify association cardinalities against real data.** CDS does not enforce `[0..1]` ; a wrong cardinality silently multiplies rows when the association is used in a path expression.
- **For managed RAP the root must be a 1:1 projection of one persistent table.** No joins that multiply rows, no aggregation, no union — the managed save maps directly onto `persistent table` .
- **Avoid a `where` clause on the interface view.** A hidden filter surprises every consumer and can make a created instance invisible.
- **Calculate criticality and other business-semantic derivations here**, not in the metadata extension, so every consumer gets them consistently.
- **Element names are the OData property names.** Renaming one later is a breaking change for any external consumer.

Chapter 04 — The Projection (Consumption) View, Line by Line

4.1 Why a projection layer exists at all

A projection view is a **service-specific window onto the business object**.

The BO (`ZR_TravelTP`) has 40 fields, 6 actions, and 3 child levels. A given app might need 12 fields, 2 actions, and 1 child level. The projection is where you say so.

Three things the projection gives you that nothing else can:

1. **Field subsetting per service.** The manager app exposes `BookingFee` ; the employee app does not.
2. **Independent lifecycle.** You can add a field to `ZR_` without changing any running service.
3. **UI attachment point.** Value helps, search, text arrangement and the metadata extension all hang off the projection, not the BO.

> **"Projection view" and "consumption view" mean the same thing in RAP.** SAP's terminology for the VDM calls the top layer "consumption"; RAP calls the `as projection on` construct a "projection view". Same object. This documentation uses **projection**.

4.2 `as projection on` vs `as select from`

This is the core syntactic difference from Chapter 03.

```
define view entity ZC_TravelTP as projection on ZR_TravelTP { ... }
```

	`as select from`	`as projection on`
Can join, filter, aggregate	Yes	**No** (only `where` , and only in limited form)
Inherits the behavior of the source	No	**Yes** — this is how the BDEF projection attaches
Can rename fields	Yes	Yes
Can add calculated fields	Yes	Only via `virtual` elements or by exposing an existing one
Fields must exist in the source	—	Yes, all of them
Used for	Modelling	Service exposure

`as projection on` is deliberately restrictive. It guarantees that the projection is a *true subset* of the BO, which is what makes it safe for the behavior projection to reuse the BO's behavior 1:1.

> If you find yourself needing a join in a projection, you need a new interface view, not a cleverer projection.

4.3 The root projection `ZC_TravelTP` — full source

```
@EndUserText.label: 'Travel - projection view for Fiori UI'
@AccessControl.authorizationCheck: #CHECK
@Metadata.allowExtensions: true
@Search.searchable: true
define root view entity ZC_TravelTP
  provider contract transactional_query
  as projection on ZR_TravelTP
{
  key TravelUUID,

      @Search.defaultSearchElement: true
      TravelId,

      @Consumption.valueHelpDefinition: [{ entity: { name: '/DMO/I_Agency', element: 'AgencyID' } }]
      @ObjectModel.text.element: ['AgencyName']
      AgencyId,
      _Agency.Name as AgencyName,

      @Consumption.valueHelpDefinition: [{ entity: { name: '/DMO/I_Customer', element: 'CustomerID' } }]
      @ObjectModel.text.element: ['CustomerName']
      CustomerId,
      _Customer.LastName as CustomerName,

      BeginDate,
      EndDate,

      @Semantics.amount.currencyCode: 'CurrencyCode'
      BookingFee,
      @Semantics.amount.currencyCode: 'CurrencyCode'
      TotalPrice,
```

```
      @Consumption.valueHelpDefinition: [{ entity: { name: 'I_Currency', element: 'Currency' } }]
      CurrencyCode,

      @Search.defaultSearchElement: true
      Description,

      OverallStatus,
      OverallStatusCriticality,

      CreatedBy,
      CreatedAt,
      LastChangedBy,
      LastChangedAt,
      LocalLastChangedAt,

      /* Associations */
      _Booking : redirected to composition child ZC_BookingTP,
      _Agency,
      _Customer,
      _Currency
}
```

4.4 Line by line

`@EndUserText.label`

```
@EndUserText.label: 'Travel - projection view for Fiori UI'
```

Appears in the OData `$metadata` as the entity set label and, if no `@UI.headerInfo` is defined, in the app title.

`@AccessControl.authorizationCheck: #CHECK`

```
@AccessControl.authorizationCheck: #CHECK
```

This is the layer where DCL (CDS access control) belongs. `#CHECK` means: *a DCL role exists for this entity and it will be applied to every read.*

The matching DCL object (`ZC_TravelTP` — same name, object type *Access Control*):

```
@EndUserText.label: 'Access control for Travel'
@MappingRole: true
define role ZC_TRAVELTP {
  grant select on ZC_TravelTP
    where ( AgencyId ) = aspect pfcg_auth( ZTRAVEL_AUTH, AGENCY_ID, ACTVT = '03' );
}
```

Part	Meaning
`@MappingRole: true`	The role is automatically assigned to users who have the PFCG authorization. Without it the role has to be assigned explicitly.
`grant select on ZC_TravelTP`	Read access. DCL controls **read only** — write access is controlled by the BDEF's `authorization master`.
`aspect pfcg_auth( ... )`	Bind to a classic PFCG authorization object. The user's authorized `AGENCY_ID` values become a `WHERE` filter.

> **Key point:** DCL filters *reads*. It does **not** stop an update. Modification authorization is the BDEF's job (`FOR GLOBAL AUTHORIZATION` / `FOR INSTANCE AUTHORIZATION`, Chapter 14). Many teams assume DCL covers everything. It does not.

`@Metadata.allowExtensions: true`

```
@Metadata.allowExtensions: true
```

Mandatory if you want a metadata extension. It declares that this view may be annotated from a separate Metadata Extension object.

If you forget it, activating the metadata extension fails with "*Extension of view ZC_TravelTP is not allowed*".

Note it is mutually exclusive with `@Metadata.ignorePropagatedAnnotations: true` in practice — on a projection you want extensions, so use `allowExtensions`.

`@Search.searchable: true`

```
@Search.searchable: true
```

Turns on the **free-text search box** in the Fiori List Report. Requires at least one element annotated with `@Search.defaultSearchElement: true`.

```
@Search.defaultSearchElement: true
TravelId,
```

This field participates in free-text search. Optional refinements:

```
@Search.defaultSearchElement: true
@Search.fuzzinessThreshold: 0.8       " HANA fuzzy search, 0.0 – 1.0
@Search.ranking: #HIGH                " weight this field higher in the result ranking
Description,
```

`@Search.fuzzinessThreshold` is a real HANA feature: `0.8` finds "Barcelona" when the user types "Barcelna". Values below ~0.7 produce noise.

`provider contract transactional_query`

```
define root view entity ZC_TravelTP
  provider contract transactional_query
  as projection on ZR_TravelTP
```

This one clause decides what kind of service this projection is.

Contract	Meaning	Needs a BDEF?
`transactional_query`	A transactional BO projection. Read **and** write. The behavior comes from the projection BDEF.	**Yes**
`transactional_interface`	Used on an interface-layer projection for reuse between services (an intermediate layer).	Yes
(omitted)	Plain read-only projection. No behavior at all.	**No**
`analytical_query`	An analytical query (`@Analytics.query: true` style).	No

For a read-only Fiori report (Chapter 07), omit `provider contract` entirely. Adding `transactional_query` without a BDEF causes activation error "*No behavior definition found for ZC_TravelTP*".

`key TravelUUID,`

The key of the projection must be the key of the BO. You cannot re-key a projection.

Field exposure and renaming

```
      TravelId,
      BookingFee,
```

Just list the element name. The name from `ZR_` carries through.

Renaming is allowed:

```
      TravelId as TravelNumber,
```

It changes the OData property name. Do it only when you must match an external contract — it breaks the mental link between the layers, and the BDEF projection still refers to the **base** name.

Fields you omit simply do not exist in the service. They are not read, not sent, not writable. This is a real performance lever on wide tables.

Path expressions for text fields

```
      _Agency.Name as AgencyName,
```

Legal in a projection — path expressions along an **exposed association** are one of the few "calculated" things a projection may do.

Caveats:

- The resulting field is **read-only**. You cannot update `AgencyName`.
- It triggers a `LEFT OUTER JOIN` whenever read. Fine for a handful of fields.
- The association (`_Agency`) must also be exposed in the SELECT list.

`@ObjectModel.text.element`

```
      @ObjectModel.text.element: ['AgencyName']
      AgencyId,
      _Agency.Name as AgencyName,
```

Declares that `AgencyName` is the descriptive text of `AgencyId`. The UI can then display "Aurora Travel (070001)" instead of "070001". How it is arranged is decided in the metadata extension:

```
@UI.textArrangement: #TEXT_FIRST      " Aurora Travel (070001)
@UI.textArrangement: #TEXT_LAST       " 070001 (Aurora Travel)
```

```
@UI.textArrangement: #TEXT_ONLY        " Aurora Travel
@UI.textArrangement: #TEXT_SEPARATE    " two separate columns
```

`@Consumption.valueHelpDefinition` — the F4 help

```
      @Consumption.valueHelpDefinition: [{ entity: { name: '/DMO/I_Agency', element: 'AgencyID' } }]
      AgencyId,
```

Part	Meaning
`[{ ... }]`	An **array** — a field can have several value helps (the UI shows a dropdown to pick which).
`entity.name`	The CDS entity providing the values. Must be exposed in the same service **or** be a released/searchable entity.
`entity.element`	Which element of that entity supplies the value for `AgencyId`.

Value help with additional binding (multi-field / filtered)

For a dependent value help — e.g. Connection depends on Carrier:

```
      @Consumption.valueHelpDefinition: [{
        entity          : { name: '/DMO/I_Connection', element: 'ConnectionID' },
        additionalBinding: [
          { localElement: 'CarrierId', element: 'AirlineID', usage: #FILTER_AND_RESULT },
          { localElement: 'FlightPrice', element: 'Price',   usage: #RESULT }
        ]
      }]
      ConnectionId,
```

`usage`	Meaning
`#FILTER`	The local value restricts the value-help list.
`#RESULT`	The selected row writes back into the local field.
`#FILTER_AND_RESULT`	Both.

This is how you build "pick a flight and the price fills itself in" without a single line of ABAP.

Value help sourced from domain fixed values

If the underlying data element has domain fixed values, **you need no annotation at all** — the UI renders a dropdown automatically. Only reach for `@Consumption.valueHelpDefinition` when the values come from a table.

`@Consumption.filter`

```
      @Consumption.filter: { selectionType: #INTERVAL, multipleSelections: true, mandatory: false }
      BeginDate,
```

Controls how the field behaves in the filter bar.

Sub-annotation	Values
`selectionType`	`#SINGLE`, `#INTERVAL`, `#RANGE`, `#HIERARCHY_NODE`
`multipleSelections`	`true` / `false`
`mandatory`	`true` forces the user to fill it before the list loads — useful on large tables
`defaultValue` / `defaultValueHigh`	Prefill

`mandatory: true` on a selective field is the single most effective way to stop a List Report from reading 10 million rows on open.

`redirected to` — the most important line in the file

```
      _Booking : redirected to composition child ZC_BookingTP,
```

What problem it solves: `ZR_TravelTP._Booking` points at `ZR_BookingTP`. But the service exposes `ZC_BookingTP`. Without redirection, navigating from the Travel to its Bookings in OData would land on the *interface* entity, which is not part of the service. Activation fails with "*_Booking must be redirected*".

`redirected to` re-targets the association **inside this service**.

The three forms

Form	Use for
`redirected to composition child ZC_BookingTP`	A **composition** to a child projection
`redirected to parent ZC_TravelTP`	The child's `association to parent`

Form	Use for
`redirected to ZC_SomeOther`	A plain **association** to another projection in the same service

```
" In ZC_TravelTP (parent):
_Booking : redirected to composition child ZC_BookingTP,

" In ZC_BookingTP (child):
_Travel  : redirected to parent ZC_TravelTP,
```

When do you *not* redirect?

Associations to entities that stay outside the service — `_Agency`, `_Customer`, `_Currency`. They point at released interface views used only for value help and text. Leave them as-is.

Rule: **redirect everything inside your BO tree; leave everything outside alone.**

4.5 The child projection `ZC_BookingTP`

```
@EndUserText.label: 'Booking - projection view'
@AccessControl.authorizationCheck: #CHECK
@Metadata.allowExtensions: true
define view entity ZC_BookingTP
  as projection on ZR_BookingTP
{
  key BookingUUID,
      TravelUUID,
      BookingId,
      BookingDate,

      @Consumption.valueHelpDefinition: [{ entity: { name: '/DMO/I_Customer', element: 'CustomerID' } }]
      CustomerId,

      @Consumption.valueHelpDefinition: [{ entity: { name: '/DMO/I_Carrier', element: 'AirlineID' } }]
      CarrierId,

      @Consumption.valueHelpDefinition: [{
        entity: { name: '/DMO/I_Connection', element: 'ConnectionID' },
        additionalBinding: [ { localElement: 'CarrierId', element: 'AirlineID', usage: #FILTER_AND_RESULT } ] }]
      ConnectionId,

      FlightDate,
      @Semantics.amount.currencyCode: 'CurrencyCode'
      FlightPrice,
      CurrencyCode,
      BookingStatus,
      LastChangedAt,

      /* Associations */
      _Travel : redirected to parent ZC_TravelTP,
      _Customer,
      _Carrier,
      _Connection
}
```

Note:

- **No** `root` and **no** `provider contract` on the child. The contract is declared once, on the root.
- `_Travel : redirected to parent ZC_TravelTP` — mandatory.

4.6 Virtual elements — calculated fields in the projection

When you need a field that cannot be derived in CDS (a call to a class, a conversion, a lookup that SQL cannot express):

```
      @ObjectModel.virtualElement: true
      @ObjectModel.virtualElementCalculatedBy: 'ABAP:ZCL_TRAVEL_VE_CALC'
      @EndUserText.label: 'Days until departure'
      virtual DaysUntilDeparture : abap.int4,
```

The class implements `IF_SADL_EXIT_CALC_ELEMENT_READ`:

```
CLASS zcl_travel_ve_calc DEFINITION PUBLIC FINAL CREATE PUBLIC.
  PUBLIC SECTION.
    INTERFACES if_sadl_exit_calc_element_read.
ENDCLASS.

CLASS zcl_travel_ve_calc IMPLEMENTATION.
  METHOD if_sadl_exit_calc_element_read~get_calculation_info.
    " tell the framework which real fields you need in order to calculate
    APPEND 'BEGINDATE' TO et_requested_orig_elements.
```

```
  ENDMETHOD.

  METHOD if_sadl_exit_calc_element_read~calculate.
    DATA lt_data TYPE STANDARD TABLE OF zc_traveltp.
    lt_data = CORRESPONDING #( it_original_data ).
    LOOP AT lt_data ASSIGNING FIELD-SYMBOL(<fs>).
      <fs>-DaysUntilDeparture = <fs>-BeginDate - cl_abap_context_info=>get_system_date( ).
    ENDLOOP.
    ct_calculated_data = CORRESPONDING #( lt_data ).
  ENDMETHOD.
ENDCLASS.
```

Costs to know before you use one:

- Virtual elements are **not filterable and not sortable** on the database. The framework reads the rows first, then calculates. Sorting/filtering on a virtual element pulls the full result set.
- They run **per page** of the result, so the class is called repeatedly.
- They are **read-only**.

Use them for display-only derivations on small result sets. For anything filterable, calculate in the interface view instead.

4.7 Adding a `where` clause

A projection may carry a `where`:

```
define view entity ZC_TravelActiveTP
  provider contract transactional_query
  as projection on ZR_TravelTP
{
  ...
}
where OverallStatus <> 'X'
```

This creates a **service-specific filter** — e.g. an app that only shows non-cancelled travels.

Be careful: in a transactional projection, a `where` clause means a user can create an instance that then disappears from the list (because it does not match the filter). RAP does not warn you. If you filter, make sure the filtered attribute is read-only or defaulted so this cannot happen.

4.8 Common activation errors in this layer and their real cause

Error	Real cause
"Association _Booking must be redirected"	Missing `redirected to composition child`
"No behavior definition found for ZC_TravelTP"	`provider contract transactional_query` present but no projection BDEF created
"Extension of view ... is not allowed"	Missing `@Metadata.allowExtensions: true`
"Element ... does not exist in ZR_TravelTP"	Field not exposed in the interface view, or a typo in the CamelCase spelling
"Use of ... not permitted in a projection view"	You wrote a join / aggregation; move it to an interface view
"The provider contract of the root ... differs"	`provider contract` accidentally repeated on the child
Value help does not appear in the app	The value-help entity is not exposed in the service **and** is not a released entity; or `@Metadata.allowExtensions` blocked propagation

---

4.9 Checklist

- [ ] `as projection on`, not `as select from`.
- [ ] `provider contract transactional_query` **only** on the root, **only** if a BDEF exists.
- [ ] `@Metadata.allowExtensions: true` on every projection you will annotate.
- [ ] `@AccessControl.authorizationCheck: #CHECK` plus a DCL, or a conscious decision not to.
- [ ] Every composition redirected with `composition child`; every parent association with `parent`.
- [ ] Value helps on all code fields that are not domain fixed values.
- [ ] Key field of the projection = key field of the BO.

4.10 Interview answer

> **"Why do we need a projection view?"**
>
> Because the business object model and the service are different concerns. The interface view is the canonical model — all fields, the composition tree, semantics. The projection is a **service-specific window** onto it: the subset of fields this app needs, the value helps, the DCL, and the attachment point for the metadata extension.
>
> That gives three things. One business object can serve several services with different fields and different enabled operations. You can extend the model without touching a running service. And business semantics stay separate from presentation.
>
> It is written with `as projection on`, which is deliberately restrictive — no joins, no aggregation — so the projection is guaranteed to be a true subset, which is what makes the projection behavior definition safe.
>
> Two details interviewers probe: `provider contract transactional_query` is what makes it a transactional service and requires a projection BDEF — leave it off and you get a read-only service, which is exactly right for a report. And every composition must be `redirected to composition child`, every parent association `redirected to parent`, otherwise navigation would land on the interface entity, which is not part of the service.

4.11 Production considerations

- **Expose only the fields the service uses.** Every extra field is read on every Object Page load and every `$expand`.
- **Decide the DCL layer deliberately** (§4.4) and write the decision down — it is hard to change once several services depend on the model.
- **Value-help entities must also be exposed in the service definition**, or the F4 is empty in the target system. This is a classic QA-only failure.
- **Renaming a projection field is a breaking change** for any external consumer of the Web API. Version the service binding rather than rename.
- **Virtual elements are display-only.** They are computed in ABAP per page and cannot be filtered or sorted on the database.
- **A `where` clause on a transactional projection can hide instances the user just created.** If you filter, make the filtered field read-only or defaulted.

Chapter 05 — Metadata Extension and UI Annotations

5.1 What a metadata extension is

A **Metadata Extension (MDE)** is a separate development object that adds annotations to a CDS view without touching the view's source.

```
ZC_TravelTP  (Data Definition)        <- data: which fields exist
ZC_TravelTP  (Metadata Extension)     <- layout: where each field appears
```

Same name, different object type. ADT: *New → Other ABAP Repository Object → Metadata Extension*.

Prerequisite: the view must carry `@Metadata.allowExtensions: true`.

Why bother — three concrete reasons

1. **Volume.** A real List Report + Object Page needs 150–400 lines of `@UI`. Mixed into the projection view, the data model becomes unreadable.
2. **Layering.** MDEs have a `@Metadata.layer` — you can ship a `#CORE` layer and let a customer override it with an `#INDUSTRY` or `#CUSTOMER` layer, without a modification.
3. **Separate transport.** Layout changes travel independently of model changes.

You *can* write `@UI` directly in the projection view. It works. For anything beyond a 5-field demo, do not.

5.2 Anatomy of an MDE

```
@Metadata.layer: #CORE

@UI: {
  headerInfo: { typeName: 'Travel',
                typeNamePlural: 'Travels',
                title:       { type: #STANDARD, value: 'TravelId' },
                description: { type: #STANDARD, value: 'Description' } },
  presentationVariant: [ { sortOrder: [ { by: 'TravelId', direction: #DESC } ],
                           visualizations: [ { type: #AS_LINEITEM } ] } ]
}
annotate entity ZC_TravelTP with
{
  @UI.facet: [ ... ]

  @UI.lineItem:       [ { position: 10, importance: #HIGH } ]
  @UI.identification: [ { position: 10 } ]
  @UI.selectionField: [ { position: 10 } ]
  TravelId;

  ...
}
```

Element	Meaning
`@Metadata.layer: #CORE`	The layer of this extension. Order of precedence: `#CORE` < `#LOCALIZATION` < `#INDUSTRY` < `#PARTNER` < `#CUSTOMER`. Higher layers win. Use `#CORE` for your own app.
Annotations **before** `annotate entity`	Entity-level annotations (`headerInfo` , `presentationVariant` , `selectionPresentationVariant` , `chart`).
`annotate entity ZC_TravelTP with { ... }`	Element-level annotations. Every element you annotate **must exist** in the view.
`TravelId;`	The element the preceding annotations apply to. Note the **semicolon**, not a comma.

> **Syntax trap:** in an MDE, elements are terminated with `;`. In a CDS view, with `,`. Getting this wrong produces a confusing parser error.

You do **not** have to list every element. Unlisted elements are still in the OData service (they are just not placed on any UI position) — they remain available for `$select`, filters via the adapt-filters dialog, and personalisation.

5.3 The four annotation groups you cannot avoid

Annotation	Controls	Appears in
`@UI.headerInfo`	App title, object title, object subtitle, icon	List Report header, Object Page header
`@UI.lineItem`	**Columns of the table**	List Report table, and tables inside Object Page sections
`@UI.selectionField`	**Fields in the filter bar**	List Report filter bar
`@UI.facet`	**Sections of the Object Page**	Object Page

Annotation	Controls	Appears in
`@UI.identification`	**Fields inside a facet** of type `#IDENTIFICATION_REFERENCE`	Object Page general-information section
`@UI.fieldGroup`	Fields grouped under a qualifier, placed by a facet	Object Page sub-sections

Everything else is refinement.

5.4 `@UI.headerInfo`

```
@UI: {
  headerInfo: {
    typeName:       'Travel',
    typeNamePlural: 'Travels',
    title:          { type: #STANDARD, value: 'TravelId' },
    description:    { type: #STANDARD, value: 'Description' },
    imageUrl:       'ImageUrl',
    typeImageUrl:   'sap-icon://flight'
  }
}
```

Field	Renders as
`typeName`	"Travel" — used in "Create Travel", "Delete Travel?" dialogs
`typeNamePlural`	"Travels" — the List Report page title
`title`	The big line on the Object Page header
`description`	The subtitle under it
`typeImageUrl`	An icon (any `sap-icon://` name)
`imageUrl`	An element containing a URL to a real image per instance

`type` can be `#STANDARD` (a plain field) or `#AS_CONNECTED_FIELDS` / `#WITH_INTENT_BASED_NAVIGATION`.

5.5 `@UI.lineItem` — the table columns

```
@UI.lineItem: [ { position: 20, label: 'Travel Number', importance: #HIGH } ]
TravelId;
```

Property	Meaning
`position`	Column order. Use gaps of 10 so you can insert later without renumbering.
`label`	Overrides the data element label for this column only.
`importance`	`#HIGH` / `#MEDIUM` / `#LOW`. Drives responsive hiding: on a phone only `#HIGH` columns survive. This is the one property people skip and then wonder why the phone layout is unusable.
`criticality`	Name of an element holding 0–3. Colours the cell.
`type`	`#STANDARD` (default), `#FOR_ACTION`, `#FOR_INTENT_BASED_NAVIGATION`, `#AS_CHART`, `#AS_DATAPOINT`, `#WITH_URL`
`hidden`	`true`, or the name of a boolean element for conditional hiding
`qualifier`	Lets you define **several different line-item sets** on the same entity (see 5.11)

Criticality

```
@UI.lineItem: [ { position: 90, criticality: 'OverallStatusCriticality',
                  criticalityRepresentation: #WITH_ICON } ]
OverallStatus;
```

Value in the criticality element	Colour
`0`	Neutral (grey)
`1`	Negative (red)
`2`	Critical (yellow/orange)
`3`	Positive (green)

`criticalityRepresentation`: `#WITH_ICON` (default) or `#WITHOUT_ICON` (colour only).

The criticality element is calculated in the **interface view** (Chapter 03, section 3.7) — it is business semantics, not layout.

Putting an action button in the table toolbar

```
@UI.lineItem: [ { position: 10, importance: #HIGH },
                { type: #FOR_ACTION, dataAction: 'acceptTravel', label: 'Accept Travel' } ]
TravelId;
```

The second array entry does not describe a column — it declares a **button**. It has to hang off *some* element; by convention the first key-ish field. See Chapter 13 for the full action story.

5.6 `@UI.selectionField` — the filter bar

```
@UI.selectionField: [ { position: 10 } ]
TravelId;

@UI.selectionField: [ { position: 20 } ]
AgencyId;

@UI.selectionField: [ { position: 30 } ]
OverallStatus;
```

Only fields with `@UI.selectionField` appear in the filter bar by default. Every other field is still filterable through *Adapt Filters*.

Combine with `@Consumption.filter` (in the projection view) to control mandatory / interval / default value.

5.7 `@UI.facet` — the Object Page structure

This is the part that confuses people most, so here is the model:

```
Object Page
 +-- Header facets            (facet with purpose: #HEADER)
 +-- Section 1                (facet type #COLLECTION or #IDENTIFICATION_REFERENCE)
 |     +-- Subsection 1.1     (facet type #FIELDGROUP_REFERENCE)
 |     +-- Subsection 1.2
 +-- Section 2                (facet type #LINEITEM_REFERENCE -> a child table)
```

```
@UI.facet: [
  // ---------- Header ----------
  { id:             'TravelHeaderStatus',
    purpose:        #HEADER,
    type:           #DATAPOINT_REFERENCE,
    position:       10,
    targetQualifier: 'OverallStatusDP' },

  // ---------- Section: General Information ----------
  { id:       'TravelGeneral',
    purpose:  #STANDARD,
    type:     #COLLECTION,
    label:    'General Information',
    position: 10 },

  {   id:             'TravelData',
      parentId:       'TravelGeneral',
      purpose:        #STANDARD,
      type:           #IDENTIFICATION_REFERENCE,
      label:          'Travel Data',
      position:       10 },

  {   id:               'TravelPrices',
      parentId:         'TravelGeneral',
      purpose:          #STANDARD,
      type:             #FIELDGROUP_REFERENCE,
      label:            'Prices',
      position:         20,
      targetQualifier: 'PriceGroup' },

  // ---------- Section: Bookings (child table) ----------
  { id:            'BookingSection',
    purpose:       #STANDARD,
    type:          #LINEITEM_REFERENCE,
    label:         'Bookings',
    position:      20,
    targetElement: '_Booking' }
]
```

Facet properties

Property	Meaning
`id`	Unique name. Needed so children can point at it with `parentId`.

Property	Meaning
`parentId`	Makes this facet a **subsection** of the named facet.
`purpose`	`#STANDARD` (a section) or `#HEADER` (in the page header)
`position`	Order
`label`	Section title
`type`	What the facet renders — see table below
`targetQualifier`	For `#FIELDGROUP_REFERENCE` / `#DATAPOINT_REFERENCE`: which qualifier to render
`targetElement`	For `#LINEITEM_REFERENCE`: which **association** to render as a table

Facet types

`type`	Renders
`#IDENTIFICATION_REFERENCE`	All fields annotated with `@UI.identification`
`#FIELDGROUP_REFERENCE`	All fields annotated with `@UI.fieldGroup` carrying the matching `qualifier`
`#LINEITEM_REFERENCE`	A **table** of the associated entity (`targetElement: '_Booking'`), using that entity's own `@UI.lineItem`
`#COLLECTION`	A container that groups subsections. Has no content of its own.
`#DATAPOINT_REFERENCE`	A single KPI / status datapoint (usually in the header)
`#CHART_REFERENCE`	A chart defined with `@UI.chart`
`#CONTACT_REFERENCE`	A contact card from `@Communication.contact`
`#ADDRESS_REFERENCE`	An address block

5.8 `@UI.identification` and `@UI.fieldGroup`

```
@UI.identification: [ { position: 10 } ]
TravelId;

@UI.identification: [ { position: 20 } ]
AgencyId;

@UI.fieldGroup: [ { qualifier: 'PriceGroup', position: 10 } ]
BookingFee;

@UI.fieldGroup: [ { qualifier: 'PriceGroup', position: 20 } ]
TotalPrice;

@UI.fieldGroup: [ { qualifier: 'PriceGroup', position: 30 } ]
CurrencyCode;
```

- `@UI.identification` → picked up by the facet of type `#IDENTIFICATION_REFERENCE`.
- `@UI.fieldGroup` with `qualifier: 'PriceGroup'` → picked up by the facet whose `targetQualifier` is `'PriceGroup'`.

The qualifier string is free text; it is just the link between the field and the facet. Getting a letter wrong means the section renders empty — a very common and very silent failure.

5.9 `@UI.dataPoint` — KPIs and header status

```
@UI.dataPoint: { qualifier: 'OverallStatusDP',
                 title: 'Status',
                 criticality: 'OverallStatusCriticality' }
OverallStatus;

@UI.dataPoint: { qualifier: 'TotalPriceDP',
                 title: 'Total Price',
                 targetValue: 5000,
                 criticalityCalculation: {
                   improvementDirection: #MINIMIZE,
                   deviationRangeHighValue: 4000,
                   toleranceRangeHighValue: 3000 },
                 visualization: #PROGRESS }
TotalPrice;
```

Property	Meaning
`qualifier`	Referenced from a `#DATAPOINT_REFERENCE` facet
`title`	Label

Property	Meaning
`criticality`	Element with 0–3
`criticalityCalculation`	Let the UI derive criticality from thresholds instead
`improvementDirection`	`#MINIMIZE` / `#MAXIMIZE` / `#TARGET`
`visualization`	`#NUMBER`, `#PROGRESS`, `#RATING`
`targetValue`	For progress/rating

5.10 The complete MDE for `ZC_TravelTP`

```
@Metadata.layer: #CORE

@UI: {
  headerInfo: { typeName: 'Travel',
                typeNamePlural: 'Travels',
                title:       { type: #STANDARD, value: 'TravelId' },
                description: { type: #STANDARD, value: 'Description' },
                typeImageUrl: 'sap-icon://flight' },
  presentationVariant: [ { sortOrder: [ { by: 'TravelId', direction: #DESC } ],
                           visualizations: [ { type: #AS_LINEITEM } ] } ]
}
annotate entity ZC_TravelTP with
{
  @UI.facet: [
    { id: 'StatusHeader',  purpose: #HEADER, type: #DATAPOINT_REFERENCE,
      position: 10, targetQualifier: 'OverallStatusDP' },
    { id: 'PriceHeader',   purpose: #HEADER, type: #DATAPOINT_REFERENCE,
      position: 20, targetQualifier: 'TotalPriceDP' },

    { id: 'GeneralColl',  purpose: #STANDARD, type: #COLLECTION,
      label: 'General Information', position: 10 },
    { id: 'TravelData',   parentId: 'GeneralColl', type: #IDENTIFICATION_REFERENCE,
      label: 'Travel Data', position: 10 },
    { id: 'PriceData',    parentId: 'GeneralColl', type: #FIELDGROUP_REFERENCE,
      label: 'Prices', position: 20, targetQualifier: 'PriceGroup' },
    { id: 'AdminData',    parentId: 'GeneralColl', type: #FIELDGROUP_REFERENCE,
      label: 'Administrative Data', position: 30, targetQualifier: 'AdminGroup' },

    { id: 'BookingList',  purpose: #STANDARD, type: #LINEITEM_REFERENCE,
      label: 'Bookings', position: 20, targetElement: '_Booking' }
  ]

  @UI.hidden: true
  TravelUUID;

  @UI.lineItem:       [ { position: 10, importance: #HIGH },
                        { type: #FOR_ACTION, dataAction: 'acceptTravel',  label: 'Accept Travel' },
                        { type: #FOR_ACTION, dataAction: 'rejectTravel',  label: 'Reject Travel' } ]
  @UI.identification: [ { position: 10 },
                        { type: #FOR_ACTION, dataAction: 'acceptTravel',  label: 'Accept Travel' },
                        { type: #FOR_ACTION, dataAction: 'rejectTravel',  label: 'Reject Travel' } ]
  @UI.selectionField: [ { position: 10 } ]
  TravelId;

  @UI.lineItem:       [ { position: 20, importance: #HIGH } ]
  @UI.identification: [ { position: 20 } ]
  @UI.selectionField: [ { position: 20 } ]
  @UI.textArrangement: #TEXT_FIRST
  AgencyId;

  @UI.hidden: true
  AgencyName;

  @UI.lineItem:       [ { position: 30, importance: #MEDIUM } ]
  @UI.identification: [ { position: 30 } ]
  @UI.selectionField: [ { position: 30 } ]
  @UI.textArrangement: #TEXT_FIRST
  CustomerId;

  @UI.hidden: true
  CustomerName;

  @UI.lineItem:       [ { position: 40, importance: #MEDIUM } ]
  @UI.identification: [ { position: 40 } ]
  @UI.selectionField: [ { position: 40 } ]
  BeginDate;

  @UI.lineItem:       [ { position: 50, importance: #MEDIUM } ]
  @UI.identification: [ { position: 50 } ]
  EndDate;

  @UI.fieldGroup:     [ { qualifier: 'PriceGroup', position: 10 } ]
  BookingFee;

  @UI.lineItem:       [ { position: 60, importance: #HIGH } ]
  @UI.fieldGroup:     [ { qualifier: 'PriceGroup', position: 20 } ]
```

```
  @UI.dataPoint:      { qualifier: 'TotalPriceDP', title: 'Total Price' }
  TotalPrice;

  @UI.fieldGroup:     [ { qualifier: 'PriceGroup', position: 30 } ]
  CurrencyCode;

  @UI.lineItem:       [ { position: 70, importance: #LOW } ]
  @UI.identification: [ { position: 60 } ]
  @UI.multiLineText:  true
  Description;

  @UI.lineItem:       [ { position: 80, importance: #HIGH,
                          criticality: 'OverallStatusCriticality' } ]
  @UI.identification: [ { position: 70, criticality: 'OverallStatusCriticality' } ]
  @UI.selectionField: [ { position: 50 } ]
  @UI.dataPoint:      { qualifier: 'OverallStatusDP', title: 'Status',
                        criticality: 'OverallStatusCriticality' }
  @UI.textArrangement: #TEXT_ONLY
  OverallStatus;

  @UI.hidden: true
  OverallStatusCriticality;

  @UI.fieldGroup: [ { qualifier: 'AdminGroup', position: 10 } ]
  CreatedBy;
  @UI.fieldGroup: [ { qualifier: 'AdminGroup', position: 20 } ]
  CreatedAt;
  @UI.fieldGroup: [ { qualifier: 'AdminGroup', position: 30 } ]
  LastChangedBy;
  @UI.fieldGroup: [ { qualifier: 'AdminGroup', position: 40 } ]
  LastChangedAt;

  @UI.hidden: true
  LocalLastChangedAt;
}
```

And the child MDE, `ZC_BookingTP`

```
@Metadata.layer: #CORE

@UI: { headerInfo: { typeName: 'Booking', typeNamePlural: 'Bookings',
                     title: { type: #STANDARD, value: 'BookingId' } } }
annotate entity ZC_BookingTP with
{
  @UI.facet: [ { id: 'BookingData', purpose: #STANDARD,
                 type: #IDENTIFICATION_REFERENCE, label: 'Booking', position: 10 } ]

  @UI.hidden: true
  BookingUUID;
  @UI.hidden: true
  TravelUUID;

  @UI.lineItem:       [ { position: 10, importance: #HIGH } ]
  @UI.identification: [ { position: 10 } ]
  BookingId;

  @UI.lineItem:       [ { position: 20, importance: #HIGH } ]
  @UI.identification: [ { position: 20 } ]
  BookingDate;

  @UI.lineItem:       [ { position: 30, importance: #HIGH } ]
  @UI.identification: [ { position: 30 } ]
  CustomerId;

  @UI.lineItem:       [ { position: 40, importance: #HIGH } ]
  @UI.identification: [ { position: 40 } ]
  CarrierId;

  @UI.lineItem:       [ { position: 50, importance: #HIGH } ]
  @UI.identification: [ { position: 50 } ]
  ConnectionId;

  @UI.lineItem:       [ { position: 60, importance: #MEDIUM } ]
  @UI.identification: [ { position: 60 } ]
  FlightDate;

  @UI.lineItem:       [ { position: 70, importance: #HIGH } ]
  @UI.identification: [ { position: 70 } ]
  FlightPrice;

  @UI.identification: [ { position: 80 } ]
  CurrencyCode;

  @UI.lineItem:       [ { position: 80, importance: #MEDIUM } ]
  @UI.identification: [ { position: 90 } ]
  BookingStatus;

  @UI.hidden: true
  LastChangedAt;
}
```

Why the child needs its own `@UI.lineItem`: the `#LINEITEM_REFERENCE` facet on the parent renders the child table using the **child's** line items. If you forget them, the Bookings section appears with no columns.

5.11 Qualifiers — several layouts on one entity

```
@UI.lineItem: [ { position: 10, importance: #HIGH },                      // default set
                { position: 10, qualifier: 'Compact', importance: #HIGH } ] // alternative set
TravelId;
```

Then reference the qualified set from a facet or a presentation variant:

```
{ id: 'CompactBookings', type: #LINEITEM_REFERENCE, targetElement: '_Booking',
  targetQualifier: 'Compact', label: 'Bookings (compact)', position: 30 }
```

Useful when the same child entity is rendered in two places with different columns.

5.12 Useful annotations that are easy to miss

Annotation	Effect
`@UI.hidden: true`	Removes from the UI entirely. **Always hide UUID keys** — the user must never see them.
`@UI.hidden: 'HideFlagElement'`	Conditional hiding driven by a boolean element.
`@UI.multiLineText: true`	Renders a text area instead of a single-line input.
`@UI.textArrangement`	`#TEXT_FIRST` / `#TEXT_LAST` / `#TEXT_ONLY` / `#TEXT_SEPARATE`
`@UI.masked: true`	Renders as `****`. For sensitive fields.
`@UI.isImageURL: true`	Element contains a URL, rendered as an image.
`@Semantics.imageUrl: true`	Same intent, semantic layer.
`@UI.presentationVariant`	Default sort and grouping of the list
`@UI.selectionPresentationVariant`	A named combination of filter + presentation = a **saved view / tab**
`@Consumption.semanticObject: 'TravelObject'`	Enables intent-based navigation to other apps
`@UI.connectedFields`	Render several fields as one composed line
`@Communication.contact`	Renders a contact quick-view card

Multiple tabs (view switching) in a List Report

```
@UI: { selectionPresentationVariant: [
  { qualifier: 'AllTravels',      text: 'All',
    selectionVariant:    { parameters: [] },
    presentationVariant: { visualizations: [ { type: #AS_LINEITEM } ] } },
  { qualifier: 'OpenTravels',     text: 'Open',
    selectionVariant:    { parameters: [ { name: 'OverallStatus', value: 'O' } ] },
    presentationVariant: { visualizations: [ { type: #AS_LINEITEM } ] } },
  { qualifier: 'AcceptedTravels', text: 'Accepted',
    selectionVariant:    { parameters: [ { name: 'OverallStatus', value: 'A' } ] },
    presentationVariant: { visualizations: [ { type: #AS_LINEITEM } ] } }
] }
```

Gives the user "All | Open | Accepted" tabs above the table, with counts, for free.

5.13 Why the UI does not reflect your annotation change

In order of likelihood:

1. **Browser cache.** The Fiori preview aggressively caches `$metadata`. Press `F12` → right-click reload → *Empty cache and hard reload*, or use the preview's own *clear cache and refresh*. **This is the cause more than half the time.**
2. **The MDE is not activated** — activating the view does not activate the extension.
3. `@Metadata.allowExtensions: true` **missing** on the projection view.
4. **The service binding needs to be re-published/re-activated** after a metadata change (V2 more often than V4).
5. **Qualifier typo** — `'PriceGroup'` vs `'Pricegroup'`. Silent; the section just renders empty.
6. **You annotated the interface view instead of the projection.** The service exposes the projection; `@Metadata.ignorePropagatedAnnotations: true` on `ZR_` deliberately blocks the leak.
7. **The element is not exposed in the projection.** An MDE can only annotate what the view has.

5.14 Checklist

- [ ] `@Metadata.layer: #CORE` at the top.
- [ ] Elements terminated with `;`.
- [ ] Every UUID key is `@UI.hidden: true`.
- [ ] Every `@UI.lineItem` has an `importance`.
- [ ] Every `targetQualifier` in a facet has a matching `qualifier` on a field.
- [ ] The child entity has its own `@UI.lineItem` set.
- [ ] `positions` use gaps of 10.

5.15 Interview answer

> **"How does the Fiori Elements layout get generated?"**
>
> Entirely from CDS annotations, normally in a separate **metadata extension** object rather than in the view itself. Four groups carry most of the work: `@UI.headerInfo` for the titles, `@UI.lineItem` for the table columns, `@UI.selectionField` for the filter bar, and `@UI.facet` for the structure of the Object Page — where a facet of type `#IDENTIFICATION_REFERENCE` renders the `@UI.identification` fields, `#FIELDGROUP_REFERENCE` renders a `@UI.fieldGroup` matched by qualifier, and `#LINEITEM_REFERENCE` renders an associated entity as a table using **that entity's own** line items.
>
> I keep them in a metadata extension for three reasons: volume — a real app is 150 to 400 lines of `@UI`; layering — the extension has a `@Metadata.layer` so a higher layer can override without a modification; and separate transport of layout from model.
>
> The prerequisite is `@Metadata.allowExtensions: true` on the projection view.
>
> The practical gotcha: when an annotation change appears to have no effect, it is usually the browser cache, an unactivated extension, or a qualifier typo — a mismatched `targetQualifier` renders an empty section silently.

5.16 Production considerations

- **Always set** `importance` on line items. Without it the phone layout is unusable — only `#HIGH` columns survive at narrow widths.
- **Hide UUID keys** with `@UI.hidden: true`. Users must never see them.
- **Use position gaps of 10** so later insertions do not require renumbering everything.
- **Layout is a transportable object of its own.** Activating the view does not activate the extension; both must be in the transport.
- **Qualifier typos fail silently.** Add "every section renders content" to the post-import test list (Chapter 25).
- **Clear the browser cache before concluding anything** about an annotation change, and tell testers to do the same — otherwise you will chase phantom defects.
- **Translate labels.** A `label:` hard-coded in the MDE overrides the data element text and is a common source of untranslated UI.

Chapter 06 — Service Definition and Service Binding

6.1 Two objects, two jobs

Object	Answers the question	Transportable artifact
Service Definition (`ZSD_TRAVEL`)	*What* is exposed? Which entities, under which names?	Yes
Service Binding (`ZUI_TRAVEL_O4`)	*How* is it exposed? Which protocol, which version, is it published?	Yes

One service definition can feed **several** bindings — e.g. one OData V2 binding for an older Fiori app and one OData V4 binding for a new one, both serving exactly the same model.

```
ZC_TravelTP + ZC_BookingTP
        |
        v
   ZSD_TRAVEL            (Service Definition)
     |        |
     v        v
ZUI_TRAVEL_O4   ZAPI_TRAVEL_O4   (Service Bindings: UI vs Web API)
```

6.2 The Service Definition

ADT: *New → Other ABAP Repository Object → Business Services → Service Definition*.

```
@EndUserText.label: 'Service definition for Travel'
define service ZSD_TRAVEL {
  expose ZC_TravelTP    as Travel;
  expose ZC_BookingTP   as Booking;

  // master data / value help entities
  expose /DMO/I_Agency     as Agency;
  expose /DMO/I_Customer   as Customer;
  expose /DMO/I_Carrier    as Carrier;
  expose /DMO/I_Connection as Connection;
  expose I_Currency        as Currency;
}
```

Line by line

Line	Meaning
`define service ZSD_TRAVEL`	The service definition name.
`expose ZC_TravelTP as Travel;`	Publishes the CDS entity under the **OData entity set name** `Travel` .
`as Travel`	Optional alias. Without it the OData entity set is called `ZC_TravelTP` , which appears in every URL and in the UI's model. Always alias.

Which entities to expose

Entity	Expose?	Why
Root projection	Yes	The main entity set
Child projections	**Yes**	Even though they are only reached via navigation, the OData metadata needs the entity type
Value-help entities	Yes, **unless released**	A `@Consumption.valueHelpDefinition` pointing at an entity that is neither exposed nor released produces an empty F4
Text entities used by `@ObjectModel.text.element`	Yes	Same reason
Interface views `ZR_...`	**No**	Never expose the BO model layer. It bypasses the projection's field subset and DCL

> **Rule:** if it appears in a `@Consumption.valueHelpDefinition` , a `redirected to` , or an `@ObjectModel.text.element` , it has to be exposed — or be a released C1 entity such as `I_Currency` .

Exposing another service definition

```
define service ZSD_TRAVEL {
  expose ZC_TravelTP as Travel;
  expose ZSD_VALUEHELPS;          " include everything from another service definition
}
```

Useful to factor a shared set of value-help entities out of five services.

6.3 The Service Binding

ADT: *New → Other ABAP Repository Object → Business Services → Service Binding*. You choose a **binding type** at creation and it cannot be changed afterwards.

The binding types

Binding type	Protocol	Use for
OData V4 — UI	OData V4	New Fiori Elements apps. **The default choice.**
OData V2 — UI	OData V2	Older Fiori Elements floorplans, SAPUI5 apps that require V2, Analytical List Page on older releases
OData V4 — Web API	OData V4	Machine-to-machine integration. No UI annotations in the metadata
OData V2 — Web API	OData V2	Legacy integration
InA — UI	InA	SAP Analytics Cloud / analytical consumption
SQL	SQL service	Direct SQL consumption from outside (e.g. SAC, Data Warehouse Cloud)

UI vs Web API — the real difference

	UI binding	Web API binding
`@UI.*` annotations in `$metadata`	Yes	**No** (stripped)
Draft actions exposed	Yes	Yes
Intended consumer	Fiori Elements / UI5	Another system, iFlow, external partner
`$metadata` size	Large	Small
Preview button in ADT	Yes	No

Creating both from one service definition is normal and costs nothing.

V2 vs V4 — how to choose

Choose OData **V4** when	Choose OData **V2** when
New development (default)	You must support an existing V2-only UI5 app
You want `strict ( 2 )`, deep create, better `$batch`	You need the Analytical List Page on an older stack
You want draft with the modern UI	An external consumer only speaks V2

V4 restriction to know: in OData V4 UI bindings, **the entity set names are taken from the `expose ... as` alias**, and navigation properties come from the association names. V2 is more forgiving about naming but less capable.

6.4 Creating and activating the binding

1. *New → Service Binding*, name `ZUI_TRAVEL_O4`, binding type *OData V4 - UI*, service definition `ZSD_TRAVEL`.
2. Activate the binding object.
3. Press **Publish** (the button in the binding editor, left of the service-URL box). - Publishing generates the runtime service artifacts and the local service URL. - The status must go to **Published (Local)**.
4. The **Service URL** appears, e.g. `/sap/opu/odata4/sap/zui_travel_o4/srvd/sap/zsd_travel/0001/`
5. Select an entity set on the right and press **Preview** → the Fiori Elements preview opens in a browser.

> On S/4HANA on-premise the binding must additionally be exposed through the ICF and, for a real Launchpad tile, through `/IWFND/MAINT_SERVICE` or the SAP Gateway "Activate and Maintain Services" app. For **development preview**, Publish + Preview is enough.

Service versions

The `0001` in the URL is the **service version**. Adding a new version lets you change the model incompatibly while old consumers keep using `0001`.

- In the binding editor: *Add Version*.
- Each version points at its own service definition.
- Only relevant for Web API bindings consumed by systems you do not control. For an internal UI, stay at `0001`.

6.5 Testing the service without a UI

From the browser

URL suffix	Returns
`.../0001/$metadata`	The full EDM model — the single most useful debugging artifact
`.../0001/Travel`	All travels (JSON)
`.../0001/Travel?$top=5`	First 5
`.../0001/Travel?$filter=OverallStatus eq 'O'`	Filtered
`.../0001/Travel?$expand=_Booking`	With children
`.../0001/Travel(TravelUUID=guid'...',IsActiveEntity=true)`	One instance (note the draft key part!)
`.../0001/Travel?$select=TravelId,TotalPrice&$orderby=TotalPrice desc`	Projection + sort
`.../0001/Travel/$count`	Row count

`$metadata` **is your friend.** If a field, an action or a value help does not behave, look at `$metadata` first. If it is not there, the problem is in CDS/BDEF, not in the UI.

From ADT

Right-click the service binding → *Open with* → *Service Consumption* / use the **Preview**. There is also the **OData V4 test tool** at `/sap/bc/adt/businessservices/odatav4/feap` in some releases.

From ABAP

Do **not** call your own service over HTTP from ABAP. Use EML (Chapter 18) — same BO, no network, no serialization, full transactional integration.

6.6 Making it a real Fiori Launchpad app (on-premise)

The Preview button is a development tool. For end users you need one of:

Option A — Fiori Elements app in BAS / VS Code (recommended)

1. SAP Business Application Studio → *Create Project from Template* → *SAP Fiori application*.
2. Data source: *Connect to a System* → your S/4 system → select the OData service.
3. Floorplan: *List Report Object Page*.
4. Main entity: `Travel`, navigation: `_Booking`.
5. Deploy to ABAP (`npm run deploy`) → creates a BSP application.
6. Create the Launchpad tile in the *Launchpad App Manager* app or via `/UI2/FLPD_CUST`.

Option B — no UI5 project at all Use the *Manage Launchpad Spaces / Pages* apps with the **"SAP Fiori elements v4 app descriptor"** route where supported — the service binding alone is sufficient, and the app is generated at runtime. Availability depends on release.

Option C — Custom UI5 app Full freedom, full cost. Only when Fiori Elements genuinely cannot do it.

6.7 Common problems

Symptom	Cause
"Service is not published"	Press **Publish** in the binding editor
Preview opens with an empty table and no error	No data in the table, **or** DCL filtering everything out, **or** a mandatory filter is unfilled
Preview shows "No matching entity type found"	Entity exposed in the service definition but the binding was not re-published after the change
Value help is empty	The value-help entity is not exposed in the service definition
Action button missing	Not `use action` in the projection BDEF, or not annotated with `type: #FOR_ACTION`, or browser cache
`$metadata` HTTP 500	Usually an inconsistent CDS/BDEF pair; activate BDEF, projection BDEF, views and binding in that order
Changes to the MDE do not show	Re-publish the binding and clear the browser cache
"Entity ... does not have a behavior definition"	`provider contract transactional_query` on a projection with no projection BDEF

The activation order that avoids 90% of these

```
1. DDIC tables
2. Interface views (child first, then root)
3. Behavior definition (base)
4. Behavior pool class
```

```
5. Projection views (child first, then root)
6. Projection behavior definition
7. Metadata extensions
8. Service definition
9. Service binding  -> then Publish
```

When something is inconsistent, re-activating in this order usually fixes it without any code change.

6.8 Interview answer

> **"How do you expose a RAP business object as an OData V4 service?"**
>
> Two objects. The **service definition** says *what* is exposed — `expose ZC_TravelTP as Travel;` — and must include the child entities plus every value-help and text entity, or the F4 comes back empty. The **service binding** says *how*: protocol and version, OData V4 UI for a Fiori app or V4 Web API for machine-to-machine, where the UI annotations are stripped out. You activate the binding and then **Publish** it, which generates the runtime artifacts and the service URL.
>
> One service definition can feed several bindings, so the same model can serve a Fiori app and an API without duplication.
>
> The prerequisite on the CDS side is `provider contract transactional_query` on the root projection plus a projection behavior definition. Omit the contract and you get a read-only service — which is what you want for a report.
>
> For diagnosis, `$metadata` is the source of truth: if an entity, an action or an annotation is not there, the problem is in CDS or the BDEF and there is no point looking at the UI.

6.9 Production considerations

- **Publishing is a per-system step.** The binding object transports; treat publication in QA and PROD as an explicit post-import task and agree who has the authorization to do it (Chapter 25).
- **On-premise you also need the ICF node active** and, in hub or V2 scenarios, the service registered in the gateway.
- **Version Web API bindings** consumed by external systems. Changing `$metadata` without a new version breaks the consumer.
- **Do not expose `ZR_` interface views.** That bypasses the projection's field subset and its DCL.
- **A UI binding's `$metadata` is large.** For integration consumers always use a Web API binding.
- **Re-publish after metadata changes** and clear the OData metadata cache, or annotation changes will not be visible.
- **Never call your own service over HTTP from ABAP.** Use EML — same BO, no network, full transactional integration.

Chapter 07 — Building a Read-Only Fiori Report, Step by Step

7.1 The point of this chapter

This is the **smallest useful RAP application**: a Fiori List Report with search, filters, sorting, export to Excel, personalisation and an Object Page — with **no behavior definition, no behavior pool, no ABAP class, no draft table**.

If your requirement is "the business wants to see this data in Fiori instead of SE16/ALV", this chapter is the whole answer. Do not build a transactional app for it.

What you build — 4 objects

```
1.  ZI_SalesOrderItem      Data Definition   (interface view)   <- optional but recommended
2.  ZC_SalesOrderItem      Data Definition   (projection view)
3.  ZC_SalesOrderItem      Metadata Extension
4.  ZSD_SALESORDER_RPT     Service Definition
5.  ZUI_SALESORDER_RPT_O4  Service Binding
```

(Five objects if you count the binding separately from the definition. No BDEF, no class.)

What it costs

Roughly 45 minutes for the first one, 10 minutes for the tenth.

7.2 Scenario

Report on **sales order items**: order, item, material, customer, quantity, net value, delivery status. Header fields come from `VBAK`, item fields from `VBAP`.

We will use the standard SAP CDS views where they exist and fall back to tables where they do not. For an on-premise S/4HANA system, `I_SalesOrderItem` and `I_SalesOrder` are released and preferable to reading `VBAK` / `VBAP` directly. The example below reads the tables so it works everywhere; swap in the released views if they are available.

7.3 Step 1 — The interface view `ZI_SalesOrderItem`

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Sales Order Item - interface view'
@Metadata.ignorePropagatedAnnotations: true
@ObjectModel.usageType: { serviceQuality: #C, sizeCategory: #L, dataClass: #TRANSACTIONAL }
define view entity ZI_SalesOrderItem
  as select from vbap as Item
    inner join   vbak as Header on Header.vbeln = Item.vbeln

  association [0..1] to I_Customer  as _Customer on  $projection.SoldToParty = _Customer.Customer
  association [0..1] to I_Product   as _Product  on  $projection.Material    = _Product.Product
  association [0..1] to I_Currency  as _Currency on  $projection.Currency    = _Currency.Currency
  association [0..1] to I_SalesOrganization as _SalesOrg
                                          on  $projection.SalesOrganization = _SalesOrg.SalesOrganization
{
  key Item.vbeln            as SalesOrder,
  key Item.posnr            as SalesOrderItem,

      Header.auart          as SalesOrderType,
      Header.vkorg          as SalesOrganization,
      Header.vtweg          as DistributionChannel,
      Header.spart          as Division,
      Header.kunnr          as SoldToParty,
      Header.erdat          as CreationDate,
      Header.audat          as DocumentDate,

      Item.matnr            as Material,
      Item.arktx            as MaterialDescription,
      Item.werks            as Plant,

      @Semantics.quantity.unitOfMeasure: 'OrderQuantityUnit'
      Item.kwmeng           as OrderQuantity,
      Item.vrkme            as OrderQuantityUnit,

      @Semantics.amount.currencyCode: 'Currency'
      Item.netwr            as NetAmount,
      Header.waerk          as Currency,

      Item.abgru            as RejectionReason,

      // ---- calculated: overall item status, business semantics, lives here ----
      case when Item.abgru <> ''       then 'Rejected'
           when Item.kwmeng = 0        then 'Zero Qty'
           else 'Open'
```

```
      end                  as ItemStatus,

      case when Item.abgru <> ''       then 1   // red
           when Item.kwmeng = 0        then 2   // yellow
           else 3                               // green
      end                  as ItemStatusCriticality,

      /* Associations */
      _Customer,
      _Product,
      _Currency,
      _SalesOrg
}
```

Why an interface view at all, for a report?

You could put everything in the projection. Keep the two layers anyway:

- The joins, the `case` expressions and the associations are **reusable**. The next report that needs "sales order items with status" reuses `ZI_SalesOrderItem`.
- The projection stays short and readable.
- If you later need the same data as an API and as a UI, you make two projections over one model.

For a genuinely throwaway one-off, a single projection over a table is acceptable. Be honest about which one you are building.

Notes on the code

Line	Why
`inner join vbak as Header on Header.vbeln = Item.vbeln`	A **join** is correct here (not an association) because header fields are always needed — every row of the report shows them. Associations are for *optional* navigation.
`key Item.vbeln, key Item.posnr`	The compound key. A CDS view **must** have a key for OData to address rows.
`@Semantics.quantity.unitOfMeasure`	Mandatory on quantity fields.
`case ... end as ItemStatusCriticality`	Calculated in the interface view because it is business semantics. The MDE will only *point at* it.
`@ObjectModel.usageType.sizeCategory: #L`	Honest sizing. Signals to the runtime and to reviewers that this can return a lot of rows.

7.4 Step 2 — The projection view `ZC_SalesOrderItem`

```
@EndUserText.label: 'Sales Order Item Report'
@AccessControl.authorizationCheck: #NOT_REQUIRED
@Metadata.allowExtensions: true
@Search.searchable: true
@ObjectModel.semanticKey: ['SalesOrder', 'SalesOrderItem']
define root view entity ZC_SalesOrderItem
  as projection on ZI_SalesOrderItem
{
      @Search.defaultSearchElement: true
  key SalesOrder,
  key SalesOrderItem,

      @Consumption.filter: { selectionType: #SINGLE, multipleSelections: true }
      SalesOrderType,

      @Consumption.filter: { selectionType: #SINGLE, multipleSelections: true, mandatory: true }
      @Consumption.valueHelpDefinition: [{ entity: { name: 'I_SalesOrganization',
                                                     element: 'SalesOrganization' } }]
      SalesOrganization,

      DistributionChannel,
      Division,

      @Consumption.valueHelpDefinition: [{ entity: { name: 'I_Customer', element: 'Customer' } }]
      @ObjectModel.text.element: ['SoldToPartyName']
      SoldToParty,
      _Customer.CustomerName as SoldToPartyName,

      @Consumption.filter: { selectionType: #INTERVAL, mandatory: true }
      CreationDate,
      DocumentDate,

      @Search.defaultSearchElement: true
      @Consumption.valueHelpDefinition: [{ entity: { name: 'I_Product', element: 'Product' } }]
      Material,

      @Search.defaultSearchElement: true
      @Search.fuzzinessThreshold: 0.8
      MaterialDescription,

      Plant,

      @Semantics.quantity.unitOfMeasure: 'OrderQuantityUnit'
      OrderQuantity,
      OrderQuantityUnit,
```

```
      @Semantics.amount.currencyCode: 'Currency'
      NetAmount,
      Currency,

      RejectionReason,
      ItemStatus,
      ItemStatusCriticality,

      /* Associations */
      _Customer,
      _Product,
      _Currency,
      _SalesOrg
}
```

The three things that make or break a read-only report

1. `root` **but no** `provider contract`**.** `abap define root view entity ZC_SalesOrderItem as projection on ZI_SalesOrderItem` `root` is needed so it can be the main entity of the service. `provider contract transactional_query` **must NOT be there** — that would demand a behavior definition. Its absence is what makes this a read-only service. RAP generates a read-only OData service automatically: `GET` works, `POST` / `PATCH` / `DELETE` return HTTP 405.
2. `mandatory: true` **on a selective filter.** `abap @Consumption.filter: { selectionType: #INTERVAL, mandatory: true } CreationDate,` `VBAP` has tens of millions of rows. Without a mandatory filter the List Report issues an unrestricted `SELECT` on open. This one annotation is the difference between a 0.4-second report and a timeout. Make at least one high-selectivity field mandatory.
3. `@ObjectModel.semanticKey`**.** `abap @ObjectModel.semanticKey: ['SalesOrder', 'SalesOrderItem']` Tells Fiori which fields are the *business* key. Used for the Object Page title, for the "Sales Order 4711 / Item 10" breadcrumb, and for draft-conflict messages in transactional apps. Costs one line, improves every dialog in the app.

7.5 Step 3 — The metadata extension

```
@Metadata.layer: #CORE

@UI: {
  headerInfo: { typeName: 'Sales Order Item',
                typeNamePlural: 'Sales Order Items',
                title:       { type: #STANDARD, value: 'SalesOrder' },
                description: { type: #STANDARD, value: 'MaterialDescription' },
                typeImageUrl: 'sap-icon://sales-order-item' },
  presentationVariant: [ { sortOrder: [ { by: 'SalesOrder',     direction: #DESC },
                                        { by: 'SalesOrderItem', direction: #ASC } ],
                           visualizations: [ { type: #AS_LINEITEM } ] } ]
}
annotate entity ZC_SalesOrderItem with
{
  @UI.facet: [
    { id: 'GeneralColl', purpose: #STANDARD, type: #COLLECTION,
      label: 'Item Details', position: 10 },
    { id: 'DocData',  parentId: 'GeneralColl', type: #IDENTIFICATION_REFERENCE,
      label: 'Document', position: 10 },
    { id: 'MatData',  parentId: 'GeneralColl', type: #FIELDGROUP_REFERENCE,
      label: 'Material & Quantity', position: 20, targetQualifier: 'MaterialGroup' },
    { id: 'AmtData',  parentId: 'GeneralColl', type: #FIELDGROUP_REFERENCE,
      label: 'Pricing', position: 30, targetQualifier: 'AmountGroup' }
  ]

  @UI.lineItem:       [ { position: 10, importance: #HIGH } ]
  @UI.identification: [ { position: 10 } ]
  @UI.selectionField: [ { position: 10 } ]
  SalesOrder;

  @UI.lineItem:       [ { position: 20, importance: #HIGH } ]
  @UI.identification: [ { position: 20 } ]
  SalesOrderItem;

  @UI.lineItem:       [ { position: 30, importance: #LOW } ]
  @UI.identification: [ { position: 30 } ]
  @UI.selectionField: [ { position: 20 } ]
  SalesOrderType;

  @UI.lineItem:       [ { position: 40, importance: #MEDIUM } ]
  @UI.identification: [ { position: 40 } ]
  @UI.selectionField: [ { position: 30 } ]
  SalesOrganization;

  @UI.lineItem:        [ { position: 50, importance: #HIGH } ]
  @UI.identification:  [ { position: 50 } ]
  @UI.selectionField:  [ { position: 40 } ]
  @UI.textArrangement: #TEXT_FIRST
  SoldToParty;

  @UI.hidden: true
  SoldToPartyName;

  @UI.lineItem:       [ { position: 60, importance: #MEDIUM } ]
  @UI.selectionField: [ { position: 50 } ]
  CreationDate;

  @UI.lineItem:   [ { position: 70, importance: #HIGH } ]
```

```
  @UI.fieldGroup: [ { qualifier: 'MaterialGroup', position: 10 } ]
  @UI.selectionField: [ { position: 60 } ]
  Material;

  @UI.lineItem:   [ { position: 80, importance: #HIGH } ]
  @UI.fieldGroup: [ { qualifier: 'MaterialGroup', position: 20 } ]
  MaterialDescription;

  @UI.fieldGroup: [ { qualifier: 'MaterialGroup', position: 30 } ]
  Plant;

  @UI.lineItem:   [ { position: 90, importance: #HIGH } ]
  @UI.fieldGroup: [ { qualifier: 'MaterialGroup', position: 40 } ]
  OrderQuantity;

  @UI.lineItem:   [ { position: 100, importance: #HIGH } ]
  @UI.fieldGroup: [ { qualifier: 'AmountGroup', position: 10 } ]
  NetAmount;

  @UI.fieldGroup: [ { qualifier: 'AmountGroup', position: 20 } ]
  Currency;

  @UI.lineItem:       [ { position: 110, importance: #HIGH,
                          criticality: 'ItemStatusCriticality' } ]
  @UI.identification: [ { position: 60, criticality: 'ItemStatusCriticality' } ]
  @UI.selectionField: [ { position: 70 } ]
  ItemStatus;

  @UI.hidden: true
  ItemStatusCriticality;
}
```

7.6 Step 4 — Service definition and binding

```
@EndUserText.label: 'Sales Order Item Report'
define service ZSD_SALESORDER_RPT {
  expose ZC_SalesOrderItem    as SalesOrderItem;
  expose I_Customer           as Customer;
  expose I_Product            as Product;
  expose I_SalesOrganization  as SalesOrganization;
  expose I_Currency           as Currency;
}
```

Then:

1. *New → Service Binding →* `ZUI_SALESORDER_RPT_O4`, type **OData V4 - UI**, definition `ZSD_SALESORDER_RPT`.
2. Activate.
3. **Publish**.
4. Select `SalesOrderItem` → **Preview**.

You now have a List Report with: free-text search, a filter bar, mandatory filters, sortable columns, column personalisation, Export to Spreadsheet, variant management, and an Object Page — none of which you wrote.

7.7 What you get for free (and what you do not)

Free

Feature	Comes from
Filter bar	`@UI.selectionField`
Free-text search	`@Search.searchable` + `@Search.defaultSearchElement`
Sort, group, column layout, personalisation	Fiori Elements standard
Export to Spreadsheet	Fiori Elements standard — no ALV needed
Saved variants / tiles	Fiori Elements standard
Object Page	`@UI.facet` + `@UI.identification`
Value helps	`@Consumption.valueHelpDefinition`
Paging (`$top` / `$skip`)	OData, automatic
Responsive / mobile layout	`importance`

Not free

Feature	You need
Any edit capability	A BDEF — this is no longer a read-only report
A button that does something	An action → needs a BDEF (see 7.9)
Charts / KPI cards	`@UI.chart` + Analytical List Page, or an Overview Page
Cross-app navigation	`@Consumption.semanticObject` + Launchpad configuration
Aggregation with drill-down	An analytical CDS view (`@Analytics.query: true`) + InA/V4 analytics
A printed PDF form	Adobe Forms / a separate output solution

7.8 Performance rules for read-only reports

These matter far more in a report than in a transactional app, because reports return many rows.

1. **Push the filter to the database.** Never read everything and filter in a virtual element.
2. **Make at least one selective field** `mandatory: true` **.**
3. **Avoid virtual elements on large result sets.** They are calculated per page in ABAP and cannot be filtered or sorted in HANA.
4. **Prefer associations to joins** for anything that is not always displayed.
5. **Do not expose fields you do not use.** The List Report issues `$select` based on visible columns, but `$expand` and Object Page reads pull everything.
6. **Avoid** `LEFT OUTER JOIN` **chains in the interface view.** Each one multiplies rows if the cardinality is wrong — and a wrong `[0..1]` on an association that is actually `[0..*]` silently produces duplicate rows.
7. **Check the generated SQL.** In ADT: run the CDS view with *Open with → Data Preview*, then use the SQL trace (`ST05`) or the *ABAP Profiler*. A report that does a full `VBAP` scan will show up immediately.
8. `@ObjectModel.usageType.sizeCategory` should be honest. It is metadata, not enforcement, but it documents intent for the next developer.

7.9 The moment a read-only report stops being read-only

The most common escalation: "*can we add a button that marks the item as reviewed?*"

That single button changes the architecture:

Before	After
No BDEF	BDEF with `action`
No behavior pool	`ZBP_...` class with the action implementation
`ZC_` without `provider contract`	`ZC_` with `provider contract transactional_query`
Interface view can be a join over `VBAK` / `VBAP`	Root view must be a 1:1 projection of **one persistent table** if the action changes data on it

Practically: if the button must write to a **Z status table**, model that Z table as the RAP root and associate the sales-order data to it — do not try to make `VBAP` the RAP root.

```
define root view entity ZR_SoItemReviewTP as select from zso_item_review as Review
  association [0..1] to ZI_SalesOrderItem as _Item
    on  $projection.SalesOrder     = _Item.SalesOrder
    and $projection.SalesOrderItem = _Item.SalesOrderItem
{ ... }
```

That keeps the transactional root clean and 1:1 with its own table, which is what managed RAP requires.

7.10 Checklist

- [ ] Projection is `define root view entity ... as projection on ...` with **no** `provider contract` .
- [ ] `@Metadata.allowExtensions: true` on the projection.
- [ ] `@ObjectModel.semanticKey` set.
- [ ] At least one filter is `mandatory: true` on a high-volume source.
- [ ] Every value-help entity is exposed in the service definition.
- [ ] Every `@UI.lineItem` has an `importance` .
- [ ] Criticality is calculated in the interface view, not in the MDE.
- [ ] Binding is **Published**, and Preview shows data.

7.11 Interview answer

> **"How would you build a read-only Fiori report?"**
>
> With RAP, but without any behavior at all — four or five objects and no ABAP.
>
> An interface view for the data model with the joins, associations and any calculated status or criticality; a projection view over it; a metadata extension for the layout; and a service definition plus an OData V4 UI service binding.
>
> The key point is the projection: `define root view entity ... as projection on ...` with **no** `provider contract transactional_query`. `root` makes it the main entity of the service; the absence of the contract is what makes it read-only. RAP then generates a read-only OData service automatically — `GET` works, `POST`, `PATCH` and `DELETE` return HTTP 405. Adding the contract without a behavior definition is an activation error.
>
> From the annotations you get the filter bar, free-text search, sorting, column personalisation, variant management, export to spreadsheet and an Object Page — all standard Fiori Elements, none of it written by you.
>
> The one thing I always add is `@Consumption.filter: { ..., mandatory: true }` on a selective field. On a large table that single annotation is the difference between a sub-second report and a timeout.

7.12 Production considerations

- **A mandatory, selective filter is not optional** on any report over a large table.
- `@ObjectModel.semanticKey` improves every dialog, breadcrumb and message in the app for one line of effort.
- **Check the generated SQL with** `ST05` before release. A full table scan shows up immediately.
- **Keep virtual elements out of filterable and sortable columns** — they force the whole result set to be read.
- **Verify association cardinalities.** A `[0..1]` that is really `[0..*]` silently duplicates rows, and in a report nobody notices until the totals are wrong.
- **Plan for the "can we add a button?" request.** That converts the report into a transactional app (§7.9); model the status on your own Z table rather than trying to make a standard table the RAP root.
- **Read-only does not mean unsecured.** DCL still applies, and it still needs a role in every system.

Chapter 08 — The Behavior Definition (BDEF), Keyword by Keyword

8.1 What the BDEF is

The Behavior Definition is written in **BDL** (Behavior Definition Language). It is a *declaration*, not code. It says what the business object can do; the behavior pool says how.

- It has the **same name as the CDS entity** it belongs to (`ZR_TravelTP`).
- ADT: right-click the root CDS view → *New Behavior Definition*, or `Ctrl + 1` on the view name.
- One BDEF covers the **whole composition tree** — root and all children in one file.

There are two BDEFs in a full RAP app:

BDEF	On	Contains
Base BDEF	`ZR_TravelTP`	The real contract: operations, locks, validations, determinations, actions
Projection BDEF	`ZC_TravelTP`	Only `use ...` statements — which of the base capabilities this service exposes

8.2 The complete base BDEF for our example

```
managed implementation in class ZBP_R_TravelTP unique;
strict ( 2 );
with draft;

define behavior for ZR_TravelTP alias Travel
persistent table ztravel
draft table ztravel_d
lock master
total etag LastChangedAt
authorization master ( instance )
etag master LocalLastChangedAt
{
  create;
  update;
  delete;

  field ( numbering : managed, readonly ) TravelUUID;
  field ( readonly ) TravelId, CreatedBy, CreatedAt, LastChangedBy, LastChangedAt,
                     LocalLastChangedAt, TotalPrice, OverallStatus;
  field ( mandatory ) AgencyId, CustomerId, BeginDate, EndDate, CurrencyCode;

  association _Booking { create; with draft; }

  determination setTravelId            on save { create; }
  determination calculateTotalPrice    on modify { field BookingFee, CurrencyCode; }
  determination setInitialOverallStatus on modify { create; }

  validation validateDates    on save { field BeginDate, EndDate; create; }
  validation validateCustomer on save { field CustomerId;          create; }
  validation validateAgency   on save { field AgencyId;            create; }

  action ( features : instance ) acceptTravel result [1] $self;
  action ( features : instance ) rejectTravel result [1] $self;
  action createTravelByTemplate [1] result [1] $self;
  factory action copyTravel [1];

  internal action recalculatePrices;

  side effects
  {
    field BookingFee affects field TotalPrice;
    action acceptTravel affects field OverallStatus;
  }

  draft action Activate optimized;
  draft action Discard;
  draft action Edit;
  draft action Resume;
  draft determine action Prepare
  {
    validation validateDates;
    validation validateCustomer;
    validation validateAgency;
  }

  mapping for ztravel
  {
    TravelUUID          = travel_uuid;
    TravelId            = travel_id;
    AgencyId            = agency_id;
    CustomerId          = customer_id;
    BeginDate           = begin_date;
    EndDate             = end_date;
```

```
    BookingFee          = booking_fee;
    TotalPrice          = total_price;
    CurrencyCode        = currency_code;
    Description         = description;
    OverallStatus       = overall_status;
    CreatedBy           = created_by;
    CreatedAt           = created_at;
    LastChangedBy       = last_changed_by;
    LastChangedAt       = last_changed_at;
    LocalLastChangedAt  = local_last_changed_at;
  }
}

define behavior for ZR_BookingTP alias Booking
persistent table zbooking
draft table zbooking_d
lock dependent by _Travel
authorization dependent by _Travel
etag master LastChangedAt
{
  update;
  delete;

  field ( numbering : managed, readonly ) BookingUUID;
  field ( readonly ) TravelUUID, BookingId, LastChangedAt;
  field ( mandatory ) CustomerId, CarrierId, ConnectionId, FlightDate;

  association _Travel { with draft; }

  determination setBookingId     on save   { create; }
  determination setBookingDate   on modify { create; }
  validation    validateFlight   on save   { field CarrierId, ConnectionId, FlightDate; create; }

  mapping for zbooking
  {
    BookingUUID   = booking_uuid;
    TravelUUID    = travel_uuid;
    BookingId     = booking_id;
    BookingDate   = booking_date;
    CustomerId    = customer_id;
    CarrierId     = carrier_id;
    ConnectionId  = connection_id;
    FlightDate    = flight_date;
    FlightPrice   = flight_price;
    CurrencyCode  = currency_code;
    BookingStatus = booking_status;
    LastChangedAt = last_changed_at;
  }
}
```

The rest of this chapter explains every one of those lines.

8.3 The header

```
managed implementation in class ZBP_R_TravelTP unique;
strict ( 2 );
with draft;
```

`managed implementation in class ... unique;`

The **implementation type**. See Chapter 01, section 1.3 for the decision.

Form	Meaning
`managed implementation in class ZBP_R_TravelTP unique;`	Framework provides buffer + save
`unmanaged implementation in class ZBP_R_TravelTP unique;`	You provide everything
`managed with additional save implementation in class ... unique;`	Framework saves, **plus** your `save_modified`
`managed with additional save with full data implementation in class ... unique;`	Same, but full instance data is passed to `save_modified`
`managed with unmanaged save implementation in class ... unique;`	**You** save, in `save_modified`
`abstract;`	No persistence at all — used for action parameter structures
`projection;`	This is a projection BDEF (see 8.10)
`interface;`	An interface BDEF for reuse between projections

`unique` means: this class is the *only* implementation of this behavior. Always specify it; it lets the framework do compile-time dispatch instead of a lookup.

> **Alternative placement.** You can instead specify the class per entity:
> `abap define behavior for ZR_TravelTP alias Travel implementation in class ZBP_R_TRAVELTP unique persistent table ztravel` This is useful when a large BO has one behavior pool per node. **But**: if you use `with unmanaged save` or `with additional save`, the class must be specified in the **header**, on the same line as `managed`. This is a real constraint that catches people converting an existing managed BO to unmanaged save.

`strict ( 2 );`

Turns on stricter syntax and semantic checks. There are three levels:

Level	Effect
(omitted)	Legacy mode. Do not use for new development.
`strict ( 1 )`	Enforces basic rules: `%tky` instead of `%key` in draft scenarios, mandatory `mapping` when names differ, etc.
`strict ( 2 )`	Everything in level 1 plus: mandatory `lock`, mandatory `authorization`, stricter feature control, stricter EML checks, `draft action Prepare` consistency

Always use the highest level your release supports. `strict ( 2 )` turns runtime surprises into activation errors. It is available from ABAP Platform 2021.

Converting an old BDEF to `strict ( 2 )` will produce a list of errors. Every one of them is a real latent defect.

`with draft;`

Enables **draft handling** for the whole BO. Consequences:

- Every entity needs a `draft table`.
- The root needs `total etag`.
- The projection BDEF needs `use draft;`.
- The OData key gains a component `IsActiveEntity`.
- `%tky` (key + `%is_draft`) must be used everywhere instead of `%key`.
- `draft action Activate / Discard / Edit / Resume` must be declared.

Omit it for a non-draft (direct-save) app. See Chapter 11 for the full draft story and for when draft is the wrong choice.

8.4 The entity behavior header

```
define behavior for ZR_TravelTP alias Travel
persistent table ztravel
draft table ztravel_d
lock master
total etag LastChangedAt
authorization master ( instance )
etag master LocalLastChangedAt
```

`define behavior for <CDS entity> alias <Alias>`

Part	Meaning
`ZR_TravelTP`	The CDS entity this behavior belongs to
`alias Travel`	A short name used **everywhere else**: in handler method signatures, in EML, in nested `association _Booking { }`, in derived types

The alias is not cosmetic. `METHODS validateDates FOR VALIDATE ON SAVE IMPORTING keys FOR Travel~validateDates.` uses it. Choose a clean noun.

Optional additions:

```
define behavior for ZR_TravelTP alias Travel
  external 'Trip'                    " the name exposed in OData metadata
  implementation in class ZBP_X unique " per-entity behavior pool
```

`persistent table ztravel`

The DDIC table (or CDS table entity) the managed runtime writes to.

- **Only for managed BOs.** An unmanaged BO has no `persistent table`.
- **Forbidden together with** `with unmanaged save` — see Chapter 16.
- The table's fields must map 1:1 to the entity's elements, either by identical names or via `mapping for`.

`draft table ztravel_d`

The draft table. Required when `with draft;` is set. See Chapter 02, section 2.5 for the naming rules.

`lock master`

Pessimistic locking. Declares that this entity owns the lock for the whole BO.

Form	Where	Meaning
`lock master`	Root entity	This entity manages the lock. Managed RAP acquires and releases it automatically for update, delete, create-by-association and actions.
`lock master unmanaged`	Root entity	You implement locking yourself in a `FOR LOCK` handler method.

Form	Where	Meaning
`lock dependent by _Travel`	Child entity	Locking is delegated to the parent via the named `association to parent`.
(none)	—	Not allowed under `strict ( 2 )` for entities that support modification.

What the framework actually does: before the first modification it writes an entry to the global lock table (the same infrastructure as `ENQUEUE`). A second user's modify request is then rejected with "*Object is locked by user X*".

A per-operation escape hatch exists: `action ( lock : none ) someAction;` — useful for an action that only reads.

`total etag LastChangedAt`

Draft-only, and mandatory for draft. Used to decide whether a draft is still *resumable*.

Scenario: you start editing a Travel (a draft is created, the active instance is exclusively locked). You close the browser. The exclusive lock expires after its configured timeout. Now another user could change the active instance. When you come back and press *Resume*, RAP compares the draft's stored total-ETag with the active instance's current one:

- equal → the active instance has not changed → resume is allowed
- different → your draft is stale → RAP tells you so

It must be declared **after** `lock master`, and it should use a **different field** than `etag master`. Requires `@Semantics.systemDateTime.lastChangedAt: true` on the field.

`authorization master ( instance )`

Declares which authorization checks the framework will call.

Form	Calls	Use when
`authorization master ( global )`	`FOR GLOBAL AUTHORIZATION`	"May this user create Travels at all?" — instance-independent
`authorization master ( instance )`	`FOR INSTANCE AUTHORIZATION`	"May this user change *this* Travel?" — depends on the data
`authorization master ( global, instance )`	Both	Most real apps
`authorization master ( none )`	Nothing	Only when authorization is genuinely not needed. Under `strict ( 2 )` you must state this explicitly rather than omit it.
`authorization dependent by _Travel`	Delegates to the parent	Child entities

Per-operation overrides live in the entity body:

```
update ( authorization : global );
action ( authorization : none ) recalculate;
```

> **Remember:** `authorization` in the BDEF controls **modification**. DCL (`@AccessControl`) controls **read**. You need both.

`etag master LocalLastChangedAt`

Optimistic concurrency control.

On read, the framework sends the field value to the client as an HTTP `ETag`. On the next write, the client sends it back in `If-Match`. If the stored value has changed in the meantime, RAP rejects the write with "*Data has been changed by another user*".

Form	Meaning
`etag master <field>`	This entity carries its own ETag
`etag dependent by _Travel`	ETag is taken from the parent

Requirements: field type `utclong` / `timestamp` / `timestampl`, plus `@Semantics.systemDateTime.localInstanceLastChangedAt: true` in the CDS view, plus `readonly` in the BDEF. The managed runtime then maintains it for you.

Lock vs ETag — they solve different problems:

	Lock (pessimistic)	ETag (optimistic)
When	Before editing starts	At the moment of saving
Prevents	A second user starting an edit	A second user's stale overwrite
Cost	Blocks others	No blocking
Needed in RAP	Yes	Yes

Use both. They are complementary, not alternatives.

8.5 The entity body — operations

```
{
  create;
  update;
  delete;
  ...
}
```

Statement	Enables
`create;`	`POST` — creating a new root instance
`update;`	`PATCH`
`delete;`	`DELETE`
(read)	**There is no `read;` statement.** Read is always available and is not declared.

Restrictions can be attached:

```
create ( authorization : global );
update ( features : instance );
delete ( precheck );
```

Addition	Meaning
`( authorization : global | instance | none )`	Which authorization check runs for this operation
`( features : instance | global )`	Enables dynamic feature control for this operation (Chapter 14)
`( precheck )`	Calls a `FOR PRECHECK` handler **before** the data enters the buffer. Lets you reject bad input early — e.g. reject a delete before the framework loads the instance

Child entity operations

Note the child in our example has `update; delete;` but **no `create;`**.

That is deliberate: a Booking is never created standalone — it is created *through the parent*, via `association _Booking { create; }`. Declaring `create;` on the child would expose a top-level "create booking" operation that has no valid parent.

8.6 `field ( ... )` — field characteristics

```
field ( numbering : managed, readonly ) TravelUUID;
field ( readonly ) TravelId, CreatedBy, CreatedAt, LastChangedBy, LastChangedAt,
                   LocalLastChangedAt, TotalPrice, OverallStatus;
field ( mandatory ) AgencyId, CustomerId, BeginDate, EndDate, CurrencyCode;
```

Characteristic	Meaning	UI effect
`readonly`	Consumer may never send a value. Framework rejects it.	Field rendered as display-only
`readonly : update`	Writable on create, locked afterwards	Editable in create, display-only in edit
`mandatory`	A value must be supplied	Red asterisk; save blocked if empty
`mandatory : create`	Required only on create	
`mandatory : update`	Required only on update	
`numbering : managed`	The framework generates a UUID for this key field	
`suppress`	The field exists in the CDS view but is completely hidden from the BO and from OData	Not in `$metadata` at all
`features : instance`	Enables **dynamic** field control (Chapter 14)	Read-only / mandatory computed per instance

Several can be combined in one parenthesis: `field ( readonly, features : instance ) OverallStatus;`

The three things this section really does

1. **`numbering : managed` on the UUID key** — this single word is what makes "the framework fills my key" true. Without it you must supply a key on create, or implement `FOR NUMBERING`.
2. **`readonly` on the five admin fields** — combined with the `@Semantics` annotations from Chapter 03, this gives you complete, code-free audit fields.
3. **`readonly` on derived fields (`TotalPrice`, `OverallStatus`)** — these are set by determinations and actions. Making them readonly stops a consumer from sending an inconsistent value, and stops the corresponding determination from looping.

> **A common mistake:** marking a field `readonly` and then trying to change it from a determination. That *works* — `readonly` restricts the **consumer**, not the provider. Your determination can set a readonly field via `MODIFY ENTITIES ... IN LOCAL MODE`.

`suppress` vs `readonly` vs `@UI.hidden`

	Visible in `$metadata`	Readable via OData	Writable
nothing	yes	yes	yes
`@UI.hidden: true`	yes	yes	yes (just not shown by default)
`field ( readonly )`	yes	yes	**no**
`field ( suppress )`	**no**	no	no

Use `suppress` for technical fields that must never leave the backend.

8.7 `association ... { }` — enabling operations across the composition

```
association _Booking { create; with draft; }
```

Part	Meaning
`association _Booking`	The composition from the CDS view
`{ create; }`	Enables **create-by-association** (`cba`): a Booking can be created *through* its Travel. This is what makes the "+" button work in the Bookings table on the Object Page.
`{ with draft; }`	The association also works on draft instances. **Required for every association inside a draft-enabled BO.**

On the child side:

```
association _Travel { with draft; }
```

The parent association needs `with draft;` too, but no `create;` — you never create a parent through a child.

If the "+" button in the child table is greyed out or missing, this is the first place to look.

8.8 Determinations

```
determination setTravelId             on save   { create; }
determination calculateTotalPrice     on modify { field BookingFee, CurrencyCode; }
determination setInitialOverallStatus on modify { create; }
```

Syntax: `determination <name> on <trigger-time> { <trigger-condition>; }`

Trigger time	Runs
`on modify`	During the **interaction phase**, immediately after the triggering change
`on save`	During the **early save phase**, when `finalize` is reached

Trigger condition	Fires when
`create`	An instance is created
`update`	An instance is updated
`delete`	An instance is deleted
`field F1, F2`	One of the listed fields is changed
`create; update;`	Either

`on modify` vs `on save` — how to choose

Use `on modify` when	Use `on save` when
The user should see the result **immediately** (a total recalculating as they type)	The value is only needed at save time
The determination depends only on the current instance	The determination needs the *final* state of the whole document
Example: `calculateTotalPrice`	Example: `setTravelId` (a number range should not be consumed for drafts that get discarded)

`setTravelId on save { create; }` is the canonical pattern: the human-readable number is drawn from the number range **only when the document is really saved**, so discarded drafts do not burn numbers.

> **Determination loop warning.** A determination that changes field X, triggered by `field X`, will call itself. RAP detects direct self-triggering and raises an error, but it cannot detect a two-step cycle (A changes B, B's determination changes A). Draw the dependency graph before writing the second determination on an entity.

Full implementations: Chapter 12.

8.9 Validations

```
validation validateDates    on save { field BeginDate, EndDate; create; }
validation validateCustomer on save { field CustomerId;          create; }
```

Same trigger syntax as determinations. Differences:

	Determination	Validation
May change data	**Yes**	**No**
May reject	No	**Yes** (via `failed` + `reported`)
`on modify` allowed	Yes	Yes
`on save` runs at	`finalize`	`check_before_save`

Validations should almost always be `on save`. An `on modify` validation fires on every keystroke-level change and makes the UI complain while the user is still typing. Use `on save` and let the user finish.

The one exception is a validation whose failure must block an *action* mid-interaction.

Full implementations: Chapter 12.

8.10 Actions

```
action ( features : instance ) acceptTravel result [1] $self;
action createTravelByTemplate [1] result [1] $self;
factory action copyTravel [1];
internal action recalculatePrices;
```

Full treatment in Chapter 13. The syntax skeleton:

```
[static|factory|internal] action [ ( <specs> ) ] <name>
    [ <input-cardinality> ]
    [ parameter <abstract-entity> ]
    [ result [ <result-cardinality> ] ( $self | <entity> ) ]
    [ deep ] ;
```

Element	Meaning
`action name;`	**Instance action** — operates on selected instances
`static action name;`	Not bound to an instance — a global button
`factory action name [1];`	Creates new instances (e.g. "copy")
`internal action name;`	Callable only from inside the BO (EML `IN LOCAL MODE`), never exposed in OData
`[1]` after the name	Input cardinality: how many instances the action processes
`result [1] $self`	Returns the (changed) instance itself — makes the UI refresh automatically
`result [0..*] ZR_BookingTP`	Returns other entities
`parameter ZE_TravelParams`	Opens a **dialog** in Fiori to collect input
`( features : instance )`	The action can be enabled/disabled per instance
`( authorization : none )`	Skip the authorization check for this action
`( lock : none )`	Do not acquire a lock (read-only actions)
`deep`	The parameter structure contains nested tables (deep action)

8.11 `side effects`

```
side effects
{
  field BookingFee affects field TotalPrice;
  action acceptTravel affects field OverallStatus;
  determine action Prepare affects messages;
}
```

Tells the **UI** what to re-read after a change. Without side effects the Fiori app does not know that changing `BookingFee` altered `TotalPrice` on the server, so the screen shows a stale total until the user refreshes.

Form	Meaning
`field A affects field B;`	Changing A requires re-reading B
`field A affects entity _Booking;`	Changing A requires re-reading the child table
`action X affects field F;`	After the action, re-read F
`action X affects entity _Booking;`	After the action, re-read the children
`action X affects messages;`	After the action, re-fetch messages
`field A affects permissions;`	Re-evaluate feature control (button enablement)
`determine action Prepare affects messages;`	Standard for draft — refresh the message list after Prepare

This is the fix for "my calculated field does not update on screen". People spend hours debugging the determination when the determination was fine and the UI simply never asked again.

Side effects can be declared in the base BDEF or the projection BDEF. Put UI-driven ones in the projection.

8.12 `mapping for`

```
mapping for ztravel
{
  TravelUUID = travel_uuid;
  TravelId   = travel_id;
  ...
}
```

Maps **CDS element names** to **DDIC field names**.

- Needed whenever the names differ (`TravelUUID` vs `travel_uuid`) — which, with the naming conventions in this guide, is always.
- Omitting it produces a syntax-check warning and, at runtime, silently unfilled fields.
- Short form when all names match apart from case and underscores: `abap mapping for ztravel corresponding;` `corresponding` does the name matching automatically. Convenient, but the explicit list is self-documenting and survives a field rename with a clear error rather than a silent gap.

The `control` variant

```
mapping for ztravel control ztravel_x corresponding;
```

`ztravel_x` is a structure with one `abap_boolean` field per data field:

```
define structure ztravel_x {
  travel_id     : abap_boolean;
  agency_id     : abap_boolean;
  customer_id   : abap_boolean;
  ...
}
```

This is needed when you implement `save_modified` yourself (unmanaged save / additional save) and want to map `%control` into an x-structure so you can issue a field-selective `UPDATE`. Chapter 16 shows this in use.

8.13 Draft actions

```
draft action Activate optimized;
draft action Discard;
draft action Edit;
draft action Resume;
draft determine action Prepare
{
  validation validateDates;
  validation validateCustomer;
  validation validateAgency;
}
```

Action	Triggered by	Does
`Edit`	The user presses **Edit** on an active instance	Creates a draft copy and locks the active instance
`Activate`	The user presses **Save**	Copies the draft onto the active instance and deletes the draft
`Discard`	The user presses **Cancel**	Deletes the draft, releases the lock

Action	Triggered by	Does
`Resume`	The user reopens an existing draft	Re-locks and continues editing
`Prepare`	The framework, before Activate, and on demand	Runs the listed validations so errors appear **in the draft**, before activation

`optimized`

`draft action Activate optimized;` enables the **optimized draft activation**: the framework re-uses the already-validated draft data instead of re-reading and re-validating everything. Significantly faster. Requirements: all validations must be listed in `Prepare`, and they must be side-effect free. Use it; there is rarely a reason not to.

Why `Prepare` matters

Without `draft determine action Prepare`, validations only run at Activate. The user fills a long form, presses Save, and gets a wall of errors. With `Prepare`, the same validations run while the draft is open and the messages appear inline as the user works.

Every validation you want the user to see before saving must be listed inside `Prepare`. Listing it is not automatic.

You can also list determinations in `Prepare`:

```
draft determine action Prepare
{
  validation validateDates;
  determination calculateTotalPrice;
}
```

8.14 The projection BDEF

Object: *Behavior Definition* on `ZC_TravelTP`. It is short by design.

```
projection;
strict ( 2 );
use draft;

define behavior for ZC_TravelTP alias Travel
use etag
{
  use create;
  use update;
  use delete;

  use association _Booking { create; with draft; }

  use action acceptTravel;
  use action rejectTravel;
  use action createTravelByTemplate;
  use action copyTravel;

  use action Activate;
  use action Discard;
  use action Edit;
  use action Resume;
  use action Prepare;
}

define behavior for ZC_BookingTP alias Booking
use etag
{
  use update;
  use delete;

  use association _Travel { with draft; }
}
```

Line by line

Line	Meaning
`projection;`	Declares this as a projection BDEF. Not `managed`.
`strict ( 2 );`	Same level as the base.
`use draft;`	Expose draft handling in this service. Required if the base has `with draft;` **and** you want drafts in this service.
`define behavior for ZC_TravelTP alias Travel`	The projection entity. Alias should match the base alias.
`use etag`	Expose the base entity's ETag. Without it, no optimistic locking on this service.
`use create; use update; use delete;`	Expose the operations. **Omitting one makes it unavailable** — this is how you build a read-only service over a transactional BO.
`use association _Booking { create; with draft; }`	Expose the composition and create-by-association.
`use action X;`	Expose an action. **An action not listed here does not appear in the service, no matter what the base BDEF or the MDE say.**

Line	Meaning
`use action Activate/Discard/Edit/Resume/Prepare;`	The draft actions must be explicitly exposed too.

What a projection BDEF can and cannot do

Can:

- Omit operations, actions, associations (subsetting)
- Add **new** determinations and validations that are specific to this service: `abap determination setDefaultForThisApp on modify { create; }`
- Add its own `side effects`
- Override field characteristics: `abap field ( readonly ) BookingFee; " read-only in this service, editable in another`
- Add a **new** action that only this service has

Cannot:

- Add `create` if the base does not have it
- Change lock, ETag, authorization, or numbering
- Change the persistent table

> **The single most common projection-BDEF bug:** you implement an action in the behavior pool, annotate the button in the MDE, and the button never appears — because you forgot `use action <name>;`. Check this before debugging anything else.

8.15 Full keyword reference

Header

Keyword	Purpose
`managed` / `unmanaged` / `abstract` / `projection` / `interface`	Implementation type
`with additional save` / `with unmanaged save`	Save enhancement (Chapter 16)
`with full data`	Pass full instance data to `save_modified`
`and cleanup`	Requires redefining `cleanup` in the saver
`implementation in class X unique`	The behavior pool
`strict ( 1 | 2 )`	Syntax-check strictness
`with draft`	Enable draft
`extensible`	Allow BDEF extensions

Entity level

Keyword	Purpose
`define behavior for E alias A`	Start an entity behavior
`external 'Name'`	OData-exposed name
`persistent table T`	Active data table (managed)
`draft table T_D`	Draft table
`lock master` / `lock master unmanaged` / `lock dependent by _A`	Pessimistic locking
`etag master F` / `etag dependent by _A`	Optimistic locking
`total etag F`	Draft resumability
`authorization master ( global | instance | none )` / `authorization dependent by _A`	Modification authorization
`early numbering` / `late numbering`	Key assignment strategy (Chapter 19)
`with additional save` / `with unmanaged save`	Per-entity save enhancement
`implementation in class X unique`	Per-entity behavior pool

Entity body

Keyword	Purpose
`create;` `update;` `delete;`	CRUD
`field ( ... ) F1, F2;`	Field characteristics
`association _A { create; with draft; }`	Composition operations

Keyword	Purpose
`determination D on modify | on save { ... }`	Auto-derivation
`validation V on modify | on save { ... }`	Checks
`action` / `static action` / `factory action` / `internal action`	Operations beyond CRUD
`function`	Read-only operation with a result
`side effects { ... }`	UI refresh hints
`mapping for T { ... }` / `mapping for T corresponding`	Element-to-field mapping
`draft action Activate | Discard | Edit | Resume`	Draft lifecycle
`draft determine action Prepare { ... }`	Pre-activation validation set
`event <name> parameter <abstract entity>;`	RAP business event
`auxiliary class ZCL_X;`	Helper class with privileged BO access
`precheck`	Call a `FOR PRECHECK` handler
`augment`	Modify a request in a projection before it reaches the base BO

8.16 Checklist

- [ ] `strict ( 2 );` present.
- [ ] Implementation type consciously chosen (Chapter 01, section 1.3).
- [ ] `lock master` on the root, `lock dependent by` on every child.
- [ ] `authorization master ( ... )` on the root, `authorization dependent by` on every child.
- [ ] `etag master` + `total etag` (if draft), on **different** fields.
- [ ] UUID key has `field ( numbering : managed, readonly )`.
- [ ] All five admin fields are `readonly`.
- [ ] Every association has `with draft;` if the BO is draft-enabled.
- [ ] `mapping for` present and complete for every entity.
- [ ] Every validation the user should see early is listed in `draft determine action Prepare`.
- [ ] `side effects` declared for every calculated field.
- [ ] Projection BDEF has a `use` for every operation, association and action the app needs.

8.17 Interview answer

> **"What is a behavior definition and what goes in it?"**
>
> It is the contract of the business object, written in BDL — a declaration, not code. It covers the implementation type and the behavior pool class; `strict ( 2 )`; whether draft is on; and then per entity the persistent and draft tables, `lock master` or `lock dependent by`, `etag master` and `total etag`, `authorization master`, which operations exist, field characteristics such as `readonly`, `mandatory` and `numbering : managed`, create-by-association, validations, determinations, actions, side effects, the draft actions, and the mapping between CDS element names and database field names.
>
> There are two behavior definitions in a full app. The **base** one on `ZR_` holds the real contract. The **projection** one on `ZC_` contains only `use` statements — `use create`, `use update`, `use action X`, `use association _Booking { create; }`. An operation or action not listed there does not exist in that service, and that is the single most common cause of a button that never appears, with no error anywhere.
>
> The value of `strict ( 2 )` is worth mentioning: it makes lock and authorization declarations mandatory, enforces `%tky` instead of `%key` in draft scenarios, and turns a class of runtime surprises into activation errors.

8.18 Production considerations

- `strict ( 2 )` **on every new BDEF.** Converting a legacy BDEF will produce a list of errors; every one of them is a real latent defect.
- **Business invariants belong in the base BDEF.** A validation declared only in the projection is bypassed by EML callers, jobs and any second service.
- `etag master` **and** `total etag` **should be different fields**, and `total etag` must come after `lock master`.
- **Use** `mapping for ... corresponding` **sparingly.** The explicit field list is self-documenting and fails loudly on a rename, where `corresponding` fails silently.
- **Audit** `side effects` **whenever you add a determination.** A new derived field without one shows stale data on screen.
- **Every validation the user should see before saving must be listed in** `draft determine action Prepare`**.** It is not automatic.
- **Generate handler skeletons with** `Ctrl + 1`**.** Hand-written `FOR ...` signatures are the number-one activation error.

Chapter 09 — Behavior Pool Classes (ABP), Line by Line

9.1 Anatomy of a behavior pool

An **ABAP Behavior Pool (ABP)** is a global class declared `FOR BEHAVIOR OF <BDEF>`. Its real content lives in its **local types** (the *Local Types* tab in ADT, the CCIMP include).

```
ZBP_R_TravelTP                          <- global class (usually almost empty)
  |
  +-- Local Types (CCIMP)
        |
        +-- lhc_Travel    : CL_ABAP_BEHAVIOR_HANDLER   <- interaction phase
        +-- lhc_Booking   : CL_ABAP_BEHAVIOR_HANDLER   <- one per entity (optional split)
        +-- lsc_ZR_TravelTP : CL_ABAP_BEHAVIOR_SAVER   <- save sequence (only if needed)
```

Creating it

Open the BDEF, put the cursor on the class name in `managed implementation in class ZBP_R_TravelTP unique;`, press `Ctrl + 1` → "*Create behavior implementation class ZBP_R_TravelTP*". ADT creates the global class with the correct `FOR BEHAVIOR OF` clause.

Then put the cursor on each `validation` / `determination` / `action` name and press `Ctrl + 1` again to generate the method. **Do this rather than typing signatures by hand.**

9.2 The global class

```
CLASS zbp_r_traveltp DEFINITION
  PUBLIC
  ABSTRACT
  FINAL
  FOR BEHAVIOR OF zr_traveltp.
ENDCLASS.

CLASS zbp_r_traveltp IMPLEMENTATION.
ENDCLASS.
```

Clause	Why
`PUBLIC`	Required
`ABSTRACT FINAL`	It is never instantiated. The RAP runtime instantiates the *local* handler classes. `ABSTRACT FINAL` is ABAP's way of saying "static-only, no instances".
`FOR BEHAVIOR OF zr_traveltp`	Binds the pool to the BDEF. This is what grants the class privileged access to the BO.

When the global class is *not* empty

Use its `PUBLIC SECTION` for data that must be shared between the **handler** (interaction phase) and the **saver** (save sequence). They are different classes with no other channel between them.

```
CLASS zbp_r_traveltp DEFINITION
  PUBLIC ABSTRACT FINAL
  FOR BEHAVIOR OF zr_traveltp.

  PUBLIC SECTION.
    " Bridge between interaction phase and save sequence.
    " Example: preliminary keys returned by a late-numbering API called in an action,
    " which must be resolved with CONVERT KEY inside save_modified.
    CLASS-DATA mapped_purchase_requisition TYPE RESPONSE FOR MAPPED i_purchaserequisitiontp.

    " Example: instances flagged during the interaction phase for extra processing at save.
    CLASS-DATA mt_travels_to_notify TYPE SORTED TABLE OF sysuuid_x16 WITH UNIQUE KEY table_line.

ENDCLASS.

CLASS zbp_r_traveltp IMPLEMENTATION.
ENDCLASS.
```

> **Caution.** `CLASS-DATA` in a behavior pool lives for the whole ABAP session, not just one RAP transaction. **Always clear it in** `cleanup` **and** `cleanup_finalize`, otherwise a second transaction in the same session inherits stale data. This is a real and hard-to-find bug class.

9.3 The handler class

```
CLASS lhc_Travel DEFINITION INHERITING FROM cl_abap_behavior_handler.
```

```
  PRIVATE SECTION.

    METHODS get_instance_authorizations FOR INSTANCE AUTHORIZATION
      IMPORTING keys REQUEST requested_authorizations FOR Travel RESULT result.

    METHODS get_global_authorizations FOR GLOBAL AUTHORIZATION
      IMPORTING REQUEST requested_authorizations FOR Travel RESULT result.

    METHODS get_instance_features FOR INSTANCE FEATURES
      IMPORTING keys REQUEST requested_features FOR Travel RESULT result.

    METHODS setTravelId             FOR DETERMINE ON SAVE
      IMPORTING keys FOR Travel~setTravelId.
    METHODS calculateTotalPrice     FOR DETERMINE ON MODIFY
      IMPORTING keys FOR Travel~calculateTotalPrice.
    METHODS setInitialOverallStatus FOR DETERMINE ON MODIFY
      IMPORTING keys FOR Travel~setInitialOverallStatus.

    METHODS validateDates    FOR VALIDATE ON SAVE IMPORTING keys FOR Travel~validateDates.
    METHODS validateCustomer FOR VALIDATE ON SAVE IMPORTING keys FOR Travel~validateCustomer.
    METHODS validateAgency   FOR VALIDATE ON SAVE IMPORTING keys FOR Travel~validateAgency.

    METHODS acceptTravel FOR MODIFY IMPORTING keys FOR ACTION Travel~acceptTravel RESULT result.
    METHODS rejectTravel FOR MODIFY IMPORTING keys FOR ACTION Travel~rejectTravel RESULT result.
    METHODS copyTravel   FOR MODIFY IMPORTING keys FOR ACTION Travel~copyTravel.

ENDCLASS.

CLASS lhc_Travel IMPLEMENTATION.
  METHOD get_instance_authorizations.
  ENDMETHOD.
  ...
ENDCLASS.
```

Rules

Rule	Detail
Inherits `CL_ABAP_BEHAVIOR_HANDLER`	Always
Implicitly `ABSTRACT FINAL`	You do not write it; the compiler applies it
Methods in `PRIVATE SECTION`	The runtime calls them through the framework, not through a normal reference
Method names are free	Except the predefined ones (`get_global_authorizations` , etc.). Match the BDEF name for readability
One handler class per entity	Not mandatory — one class can handle several entities — but strongly recommended

Naming convention

`lhc_<Alias>` — `lhc_Travel` , `lhc_Booking` . `lhc` = local handler class. `lsc_<BDEF name>` — `lsc_ZR_TravelTP` . `lsc` = local saver class. These are conventions ADT's quick fixes follow; stick to them so generated and hand-written code look alike.

9.4 Every handler method signature

Determination

```
METHODS calculateTotalPrice FOR DETERMINE ON MODIFY
  IMPORTING keys FOR Travel~calculateTotalPrice.
```

```
METHODS setTravelId FOR DETERMINE ON SAVE
  IMPORTING keys FOR Travel~setTravelId.
```

Parameter	Type	Contains
`keys`	`TABLE FOR DETERMINATION Travel~<name>`	The **keys only** of the triggered instances. Not the data — you must read it.
`reported` (implicit CHANGING)	`RESPONSE FOR REPORTED EARLY/LATE zr_traveltp`	Messages you want to return
`failed` (implicit CHANGING)	`RESPONSE FOR FAILED EARLY/LATE zr_traveltp`	Instances that failed

Validation

```
METHODS validateDates FOR VALIDATE ON SAVE
  IMPORTING keys FOR Travel~validateDates.
```

Same parameter shape. `keys` carries only keys.

Action

```
" instance action with result
METHODS acceptTravel FOR MODIFY
  IMPORTING keys FOR ACTION Travel~acceptTravel RESULT result.

" instance action without result
METHODS recalculate FOR MODIFY
  IMPORTING keys FOR ACTION Travel~recalculate.

" static action
METHODS archiveOldTravels FOR MODIFY
  IMPORTING keys FOR ACTION Travel~archiveOldTravels.

" factory action
METHODS copyTravel FOR MODIFY
  IMPORTING keys FOR ACTION Travel~copyTravel.

" action with a parameter
METHODS deductDiscount FOR MODIFY
  IMPORTING keys FOR ACTION Travel~deductDiscount RESULT result.
```

Parameter	Contains
`keys`	Keys of the selected instances, plus `%param` if the action has a `parameter`
`result`	What you return. Type `TABLE FOR ACTION RESULT Travel~<name>`
`mapped` , `failed` , `reported`	Implicit CHANGING

Feature control

```
METHODS get_instance_features FOR INSTANCE FEATURES
  IMPORTING keys REQUEST requested_features FOR Travel RESULT result.

METHODS get_global_features FOR GLOBAL FEATURES
  IMPORTING REQUEST requested_features FOR Travel RESULT result.
```

Authorization

```
METHODS get_instance_authorizations FOR INSTANCE AUTHORIZATION
  IMPORTING keys REQUEST requested_authorizations FOR Travel RESULT result.

METHODS get_global_authorizations FOR GLOBAL AUTHORIZATION
  IMPORTING REQUEST requested_authorizations FOR Travel RESULT result.
```

Precheck

```
METHODS precheck_update FOR PRECHECK
  IMPORTING entities FOR UPDATE Travel.

METHODS precheck_delete FOR PRECHECK
  IMPORTING keys FOR DELETE Travel.

METHODS precheck_cba_Booking FOR PRECHECK
  IMPORTING entities FOR CREATE Travel\_Booking.
```

Runs **before** the data enters the transactional buffer. Requires `( precheck )` in the BDEF.

Numbering (early numbering)

```
METHODS earlynumbering_create FOR NUMBERING
  IMPORTING entities FOR CREATE Travel.

METHODS earlynumbering_cba_Booking FOR NUMBERING
  IMPORTING entities FOR CREATE Travel\_Booking.
```

Unmanaged-only methods

```
METHODS create FOR MODIFY IMPORTING entities FOR CREATE Travel.
METHODS update FOR MODIFY IMPORTING entities FOR UPDATE Travel.
METHODS delete FOR MODIFY IMPORTING keys     FOR DELETE Travel.
METHODS read   FOR READ   IMPORTING keys     FOR READ   Travel RESULT result.
METHODS lock   FOR LOCK   IMPORTING keys     FOR LOCK   Travel.

METHODS rba_Booking FOR READ
  IMPORTING keys_rba FOR READ Travel\_Booking FULL result_requested
  RESULT result LINK association_links.

METHODS cba_Booking FOR MODIFY
  IMPORTING entities_cba FOR CREATE Travel\_Booking.
```

Note the `\_Booking` syntax: in ABAP source, the composition is addressed as `<Alias>\_<AssociationName>` with a **backslash**.

9.5 The three response parameters

Every handler method has these as implicit `CHANGING` parameters. You do not declare them; you just use them.

`reported` — messages

```
APPEND VALUE #( %tky = key-%tky
                %msg = NEW zcx_travel_messages(
                         severity = if_abap_behv_message=>severity-error
                         textid   = zcx_travel_messages=>begin_date_in_past
                         begindate = travel-BeginDate )
                %element-BeginDate = if_abap_behv=>mk-on )
       TO reported-travel.
```

Component	Meaning
`%tky`	Which instance the message belongs to
`%msg`	The message object (must implement `IF_ABAP_BEHV_MESSAGE`)
`%element-<Field>`	Attaches the message to a **field**, so Fiori highlights it in red
`%state_area`	A named area for **state messages** (see below)
`%path`	For messages on a child, the path from the root

`reported-travel` / `reported-booking` — one component per entity alias, lowercase. `reported-%other` — for messages not tied to any instance.

`failed` — rejection

```
APPEND VALUE #( %tky = key-%tky ) TO failed-travel.
```

Adding a key to `failed` tells the framework this instance could not be processed. In a validation, this is what actually blocks the save. **A message in `reported` without an entry in `failed` does not stop anything** — the user sees a red message and the document saves anyway. This is the single most common validation bug.

Optional detail:

```
APPEND VALUE #( %tky = key-%tky
                %fail-cause = if_abap_behv=>cause-unauthorized ) TO failed-travel.
```

`%fail-cause`	Meaning
`unspecific`	Default
`unauthorized`	Authorization failure
`locked`	Lock failure
`disabled`	Operation is disabled by feature control
`not_found`	Instance does not exist
`conflict`	ETag mismatch
`dependency`	A dependent object blocked it

`mapped` — key mapping

Only relevant for create operations.

```
APPEND VALUE #( %cid = entity-%cid
                TravelUUID = lv_new_uuid ) TO mapped-travel.
```

Tells the consumer "the instance you called `%cid = 'X'` now has key `TravelUUID = ...`". In a **managed** BO this is filled by the framework. You fill it in unmanaged BOs and in factory actions.

`reported` EARLY vs LATE

```
CHANGING reported TYPE RESPONSE FOR REPORTED EARLY zr_traveltp   " interaction + early save
CHANGING reported TYPE RESPONSE FOR REPORTED LATE  zr_traveltp   " late save phase
```

`EARLY` messages can be shown to the user and the transaction can still be corrected. `LATE` messages arrive after the point of no return — informational only.

State messages — the right way to show a persistent validation error

A normal message disappears on the next round trip. A **state message** stays attached to the instance until the validation clears it:

```
METHOD validateDates.
  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel FIELDS ( BeginDate EndDate ) WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  LOOP AT travels INTO DATA(travel).

    " 1. ALWAYS clear the state area first for this instance
    APPEND VALUE #( %tky        = travel-%tky
                    %state_area = 'VALIDATE_DATES' )
           TO reported-travel.

    " 2. Then add the message only if the check fails
    IF travel-BeginDate > travel-EndDate.
      APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.
      APPEND VALUE #( %tky        = travel-%tky
                      %state_area = 'VALIDATE_DATES'
                      %msg        = NEW zcx_travel_messages(
                                        severity = if_abap_behv_message=>severity-error
                                        textid   = zcx_travel_messages=>end_before_begin )
                      %element-BeginDate = if_abap_behv=>mk-on
                      %element-EndDate   = if_abap_behv=>mk-on )
             TO reported-travel.
    ENDIF.
  ENDLOOP.
ENDMETHOD.
```

The "clear first, then add" pattern is mandatory. Without the clearing entry, a message stays on screen even after the user fixes the data. Every state-message validation must start with it.

9.6 The message class

Messages need a class implementing `IF_ABAP_BEHV_MESSAGE`. The standard pattern is an exception class:

```
CLASS zcx_travel_messages DEFINITION
  PUBLIC
  INHERITING FROM cx_static_check
  FINAL
  CREATE PUBLIC.

  PUBLIC SECTION.
    INTERFACES if_abap_behv_message.

    CONSTANTS:
      BEGIN OF end_before_begin,
        msgid TYPE symsgid VALUE 'ZTRAVEL_MSG',
        msgno TYPE symsgno VALUE '001',
        attr1 TYPE scx_attrname VALUE '',
        attr2 TYPE scx_attrname VALUE '',
        attr3 TYPE scx_attrname VALUE '',
        attr4 TYPE scx_attrname VALUE '',
      END OF end_before_begin,

      BEGIN OF begin_date_in_past,
        msgid TYPE symsgid VALUE 'ZTRAVEL_MSG',
        msgno TYPE symsgno VALUE '002',
        attr1 TYPE scx_attrname VALUE 'BEGINDATE',
        attr2 TYPE scx_attrname VALUE '',
        attr3 TYPE scx_attrname VALUE '',
        attr4 TYPE scx_attrname VALUE '',
      END OF begin_date_in_past,

      BEGIN OF customer_unknown,
        msgid TYPE symsgid VALUE 'ZTRAVEL_MSG',
        msgno TYPE symsgno VALUE '003',
        attr1 TYPE scx_attrname VALUE 'CUSTOMERID',
        attr2 TYPE scx_attrname VALUE '',
        attr3 TYPE scx_attrname VALUE '',
        attr4 TYPE scx_attrname VALUE '',
      END OF customer_unknown.

    DATA: begindate  TYPE /dmo/begin_date,
          customerid TYPE /dmo/customer_id.

    METHODS constructor
      IMPORTING textid     LIKE if_t100_message=>t100key OPTIONAL
                severity   TYPE if_abap_behv_message=>t_severity DEFAULT if_abap_behv_message=>severity-error
                previous   LIKE previous OPTIONAL
                begindate  TYPE /dmo/begin_date OPTIONAL
                customerid TYPE /dmo/customer_id OPTIONAL.
ENDCLASS.

CLASS zcx_travel_messages IMPLEMENTATION.
  METHOD constructor.
    super->constructor( previous = previous ).
    me->begindate  = begindate.
    me->customerid = customerid.
    CLEAR me->textid.
    IF textid IS INITIAL.
```

```
      if_t100_message~t100key = if_t100_message=>default_textid.
    ELSE.
      if_t100_message~t100key = textid.
    ENDIF.
    me->if_abap_behv_message~m_severity = severity.
  ENDMETHOD.
ENDCLASS.
```

Element	Purpose
`INTERFACES if_abap_behv_message`	Makes it usable as `%msg`
`CONSTANTS BEGIN OF ... msgid/msgno`	Points at a message in message class `ZTRAVEL_MSG` (SE91)
`attr1..attr4`	Names of **class attributes** whose values fill `&1` .. `&4` in the message text
`DATA begindate`	The attribute referenced by `attr1` — **must be uppercase in** `attr1`
`m_severity`	`error` / `warning` / `information` / `success`

> Message text in SE91, message class `ZTRAVEL_MSG`, number `002`: `Begin date &1 must not be in the past`. `&1` is filled from the `BEGINDATE` attribute because `attr1 = 'BEGINDATE'`.

The quick alternative for messages without parameters

```
APPEND VALUE #( %tky = key-%tky
                %msg = new_message_with_text(
                         severity = if_abap_behv_message=>severity-error
                         text     = 'End date must be after begin date' ) )
       TO reported-travel.
```

`new_message_with_text( )` is inherited from `CL_ABAP_BEHAVIOR_HANDLER`. Convenient for prototyping. **Not translatable** — do not ship it.

There is also:

```
%msg = new_message( id = 'ZTRAVEL_MSG' number = '002'
                    severity = if_abap_behv_message=>severity-error
                    v1 = travel-BeginDate )
```

which is translatable and needs no exception class. Good middle ground.

9.7 The saver class

```
CLASS lsc_ZR_TravelTP DEFINITION INHERITING FROM cl_abap_behavior_saver.
  PROTECTED SECTION.
    METHODS finalize          REDEFINITION.
    METHODS check_before_save REDEFINITION.
    METHODS adjust_numbers    REDEFINITION.   " late numbering only
    METHODS save              REDEFINITION.   " unmanaged BOs
    METHODS save_modified     REDEFINITION.   " managed + additional/unmanaged save
    METHODS cleanup           REDEFINITION.
    METHODS cleanup_finalize  REDEFINITION.
ENDCLASS.
```

	Detail
Inherits	`CL_ABAP_BEHAVIOR_SAVER` (or `CL_ABAP_BEHAVIOR_SAVER_FAILED` — see below)
Section	`PROTECTED SECTION` — these are redefinitions of protected methods
Implicitly	`ABSTRACT FINAL`
When needed	Unmanaged BOs; managed BOs with `with additional save` or `with unmanaged save`; any BO with `late numbering`
Not needed	A plain managed BO with no save enhancement — **do not create an empty saver class**

Saver methods receive no instance data as parameters (except `save_modified`). In an unmanaged BO you read your own transactional buffer. That asymmetry is deliberate: the saver operates on the whole transaction, not on a request.

`CL_ABAP_BEHAVIOR_SAVER_FAILED`

If your save can genuinely fail in the **late** phase — e.g. it calls a BAPI that may return errors — inherit from `CL_ABAP_BEHAVIOR_SAVER_FAILED` instead. It gives `save_modified` access to `failed` and `reported`, so the consumer learns what went wrong. `sy-subrc = 8` after `COMMIT ENTITIES` signals a late-phase failure, and the consumer must issue an explicit `ROLLBACK ENTITIES` before doing anything else.

Chapter 17 covers the full sequence.

9.8 The universal handler-method pattern

Almost every method you write follows the same four steps.

```
METHOD acceptTravel.

  " 1. READ – handler methods get keys, not data
  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      FIELDS ( OverallStatus BeginDate )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels)
    FAILED DATA(read_failed).

  " 2. DECIDE – pure business logic, no DB access
  DATA update_lines TYPE TABLE FOR UPDATE zr_traveltp.
  LOOP AT travels INTO DATA(travel).
    IF travel-OverallStatus = 'X'.
      APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.
      APPEND VALUE #( %tky = travel-%tky
                      %msg = new_message( id = 'ZTRAVEL_MSG' number = '010'
                                          severity = if_abap_behv_message=>severity-error ) )
             TO reported-travel.
      CONTINUE.
    ENDIF.
    APPEND VALUE #( %tky = travel-%tky OverallStatus = 'A' ) TO update_lines.
  ENDLOOP.

  " 3. MODIFY – write back into the transactional buffer, never to the DB
  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      UPDATE FIELDS ( OverallStatus )
      WITH update_lines
    REPORTED DATA(update_reported).

  " 4. FILL RESULT – so the UI refreshes
  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel ALL FIELDS WITH CORRESPONDING #( keys )
    RESULT DATA(travels_after).

  result = VALUE #( FOR t IN travels_after ( %tky   = t-%tky
                                             %param = t ) ).
ENDMETHOD.
```

Why `IN LOCAL MODE`

```
READ ENTITIES OF zr_traveltp IN LOCAL MODE
```

Without `IN LOCAL MODE`	With `IN LOCAL MODE`
The call goes through the **full** BO stack: authorization checks, feature control, projection restrictions	The call goes **directly** to the transactional buffer
Read-only fields cannot be written	Read-only fields **can** be written
Recursion risk (determination triggers itself through the public interface)	No recursion through the public layer
Slower	Fast

Inside a behavior pool, always use `IN LOCAL MODE` unless you specifically want the full checks. Writing a `readonly` field like `OverallStatus` from an action is only possible with it.

Why `FIELDS ( ... )` instead of `ALL FIELDS`

`ALL FIELDS` reads every field of every instance. On a wide entity in a mass action, that is measurable. Read what you need.

Use `ALL FIELDS` when you are building a `result` that must return the full instance.

9.9 Splitting handler classes

One class per entity is the default. For a large root entity you can go further:

```
CLASS lhc_travel_validations DEFINITION INHERITING FROM cl_abap_behavior_handler. ... ENDCLASS.
CLASS lhc_travel_actions     DEFINITION INHERITING FROM cl_abap_behavior_handler. ... ENDCLASS.
CLASS lhc_travel_auth        DEFINITION INHERITING FROM cl_abap_behavior_handler. ... ENDCLASS.
```

The framework finds the method wherever it is, as long as each method is declared exactly once across all handler classes in the pool. Declaring the same method twice is an activation error.

9.10 Auxiliary classes

For reusable logic that several handler methods share, and that needs privileged BO access:

```
" BDEF:
define behavior for ZR_TravelTP alias Travel
...
auxiliary class ZCL_TRAVEL_HELPER;
{ ... }
```

```
CLASS zcl_travel_helper DEFINITION
  PUBLIC
  FINAL
  FOR BEHAVIOR OF zr_traveltp.

  PUBLIC SECTION.
    CLASS-METHODS calculate_total_price
      IMPORTING travel_uuid TYPE sysuuid_x16
      RETURNING VALUE(total) TYPE /dmo/total_price.
ENDCLASS.
```

`FOR BEHAVIOR OF` gives the class the right to use `IN LOCAL MODE` and `PRIVILEGED` EML against this BO. A plain class cannot.

Do not move whole handler implementations into auxiliary classes. SAP explicitly advises against it: the handler methods are the framework's entry points and hollowing them out adds a call layer without benefit. Use auxiliary classes for genuinely shared sub-logic.

9.11 What is forbidden in a behavior pool

Forbidden	Why	Instead
`COMMIT WORK` / `ROLLBACK WORK`	RAP owns the transaction. Breaks the LUW; runtime error under strict mode	Let RAP commit; `COMMIT ENTITIES` only from outside the BO
Direct `INSERT` / `UPDATE` / `DELETE` on the persistent table	Bypasses the buffer; the framework will then overwrite or conflict	EML `MODIFY ENTITIES ... IN LOCAL MODE`
`SELECT` from the persistent table for BO data	Returns the **database** state, not the buffer. In a draft app you get the active row while the user is editing a draft	`READ ENTITIES ... IN LOCAL MODE`
`MESSAGE` without `INTO`, `WRITE`, `CALL SCREEN`, `LEAVE`, `STOP`	Dynpro statements. Runtime error	`reported`
`CALL TRANSACTION`, `SUBMIT`	Same	—
`WAIT`, `CALL FUNCTION ... DESTINATION` in the **late** save phase	Would break the single database transaction	Do it in the interaction or early save phase
`CALL FUNCTION ... IN UPDATE TASK` outside the late save phase	Cannot be unregistered; breaks draft	Only in the late save phase
Reading the **draft table** with SQL	Framework-owned structure	EML

The `SELECT` rule deserves emphasis. It is the most frequent real-world defect in RAP code:

```
" WRONG – reads the database, ignores everything the user just typed
SELECT SINGLE * FROM ztravel WHERE travel_uuid = @key-TravelUUID INTO @DATA(ls).

" RIGHT – reads the transactional buffer, sees uncommitted changes
READ ENTITIES OF zr_traveltp IN LOCAL MODE
  ENTITY Travel ALL FIELDS WITH VALUE #( ( %tky = key-%tky ) )
  RESULT DATA(travels).
```

Reading **other** tables (master data, customizing) with `SELECT` is fine and normal. The rule is about the BO's own data.

9.12 Checklist

- [ ] Global class is `PUBLIC ABSTRACT FINAL FOR BEHAVIOR OF <bdef>`.
- [ ] Handler classes inherit `CL_ABAP_BEHAVIOR_HANDLER`; methods in `PRIVATE SECTION`.
- [ ] Saver class inherits `CL_ABAP_BEHAVIOR_SAVER`; methods in `PROTECTED SECTION`; only created if needed.
- [ ] Every method signature was generated by `Ctrl + 1`, not typed.
- [ ] Every validation that rejects fills **both** `failed` and `reported`.
- [ ] State messages clear their `%state_area` before adding.
- [ ] All EML inside the pool uses `IN LOCAL MODE`.
- [ ] No `SELECT` on the BO's own persistent or draft tables.
- [ ] No `COMMIT WORK`.

- [ ] `CLASS-DATA` in the global class is cleared in `cleanup` and `cleanup_finalize`.

9.13 Interview answer

> **"What is a behavior pool, and what is the difference between `failed`, `reported` and `mapped`?"**
>
> A behavior pool is a global class declared `FOR BEHAVIOR OF <bdef>`. It is implicitly `ABSTRACT FINAL` because the RAP runtime instantiates the local classes inside it, never the global class. The content lives in the local types: handler classes inheriting `CL_ABAP_BEHAVIOR_HANDLER` for the interaction phase, and — only where needed — a saver class inheriting `CL_ABAP_BEHAVIOR_SAVER` for the save sequence.
>
> The three response parameters are implicit changing parameters on every handler method. `failed` says which instances could not be processed — adding a key there is what actually blocks the operation. `reported` carries the messages, optionally with `%element-<Field>` so Fiori highlights the right field and `%state_area` for messages that persist until cleared. `mapped` carries key-mapping information for create operations — which real key was assigned to which `%cid`.
>
> The pairing is the part that catches people. For an error you fill **both** `failed` and `reported`: `failed` alone blocks the save and tells the user nothing, `reported` alone shows a red error and saves the record anyway. For a warning you fill `reported` only, deliberately.
>
> And inside the pool every EML call uses `IN LOCAL MODE` — it goes straight to the buffer, skips authorization and feature control, and lets you write `readonly` fields, which is how a determination sets a field the consumer may not touch.

9.14 Production considerations

- **Never `SELECT` the BO's own persistent or draft tables.** In a draft app that reads the active row while the user edits the draft.
- **Never `COMMIT WORK` or `COMMIT ENTITIES`** inside the pool. The BO is not the transaction owner.
- **Clear `CLASS-DATA` in both `cleanup` and `cleanup_finalize`.** Class data survives the whole session; `cleanup_finalize` is the path taken when the early save phase rejects the save.
- **Do not create an empty saver class** "just in case" — only when a saver method is actually needed.
- **Use a translatable message mechanism.** `new_message_with_text( )` is for prototyping; grep for it as a release check.
- **State-message validations must clear their `%state_area` first**, or the error stays on screen after the user fixes the data.
- **Split handler classes per entity.** It keeps the pool readable, and each method may be declared only once across all handler classes.

Chapter 10 — Full CRUD Managed Application, Step by Step

10.1 What this chapter builds

A complete, working, transactional Fiori app:

- List Report with filters and search
- Object Page with a General Information section and a Bookings table
- Create / Edit / Delete on Travel
- Create / Edit / Delete on Booking, inside the Travel
- Draft handling (auto-save, Cancel, Resume)
- Two validations, three determinations, two actions with buttons
- Optimistic and pessimistic locking
- Audit fields maintained automatically

Total hand-written ABAP: roughly 200 lines. Everything else is declaration.

10.2 Build order

Follow this order. Each step activates cleanly before you move on.

Step	Object	Chapter reference
1	`ZTRAVEL`, `ZBOOKING` tables	02
2	`ZR_BookingTP`, then `ZR_TravelTP`	03
3	`ZTRAVEL_D`, `ZBOOKING_D` draft tables (ADT quick fix from the views)	02 §2.5
4	BDEF `ZR_TravelTP`	08
5	Behavior pool `ZBP_R_TravelTP` (`Ctrl + 1`)	09
6	Message class `ZTRAVEL_MSG` (SE91) + `ZCX_TRAVEL_MESSAGES`	09 §9.6
7	Implement determinations, validations, actions	this chapter
8	`ZC_BookingTP`, then `ZC_TravelTP`	04
9	Projection BDEF `ZC_TravelTP`	08 §8.14
10	Metadata extensions	05
11	`ZSD_TRAVEL` + `ZUI_TRAVEL_O4`, Publish	06
12	Preview and test	this chapter §10.8

> **Child before parent** in steps 2 and 8. The parent's `composition of` and `redirected to composition child` need the child to exist.

10.3 The three determinations

`setInitialOverallStatus` — set a default on create

```
METHOD setInitialOverallStatus.

  " Read only the instances whose status is still empty
  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      FIELDS ( OverallStatus )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  DELETE travels WHERE OverallStatus IS NOT INITIAL.
  CHECK travels IS NOT INITIAL.

  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      UPDATE FIELDS ( OverallStatus )
      WITH VALUE #( FOR t IN travels
                  ( %tky          = t-%tky
                    OverallStatus = zbp_r_traveltp=>c_status-open ) )
    REPORTED DATA(update_reported).

  reported = CORRESPONDING #( DEEP update_reported ).
```

```
ENDMETHOD.
```

Points to note:

Line	Why
`DELETE travels WHERE OverallStatus IS NOT INITIAL.`	Idempotency. A determination can be called more than once; never blindly overwrite.
`CHECK travels IS NOT INITIAL.`	Do not issue an empty `MODIFY`.
`zbp_r_traveltp=>c_status-open`	A constant in the global ABP class, not a literal `'O'` scattered through the code.
`reported = CORRESPONDING #( DEEP update_reported ).`	Propagate any messages the inner `MODIFY` produced. Skipping this swallows errors.

Add the constants to the global class:

```
CLASS zbp_r_traveltp DEFINITION PUBLIC ABSTRACT FINAL FOR BEHAVIOR OF zr_traveltp.
  PUBLIC SECTION.
    CONSTANTS:
      BEGIN OF c_status,
        open      TYPE /dmo/overall_status VALUE 'O',
        accepted  TYPE /dmo/overall_status VALUE 'A',
        rejected  TYPE /dmo/overall_status VALUE 'X',
      END OF c_status.
ENDCLASS.
```

`calculateTotalPrice` — a rollup from the children

```
METHOD calculateTotalPrice.

  " 1. Read the Travel headers (fee + currency)
  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      FIELDS ( BookingFee CurrencyCode )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  " 2. Read the Bookings by association (this is 'read by association', rba)
  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel BY \_Booking
      FIELDS ( FlightPrice CurrencyCode )
      WITH CORRESPONDING #( travels )
    RESULT DATA(bookings).

  " 3. Sum per travel
  LOOP AT travels ASSIGNING FIELD-SYMBOL(<travel>).
    <travel>-TotalPrice = <travel>-BookingFee.
    LOOP AT bookings INTO DATA(booking)
         WHERE TravelUUID = <travel>-TravelUUID.
      IF booking-CurrencyCode = <travel>-CurrencyCode.
        <travel>-TotalPrice += booking-FlightPrice.
      ELSE.
        " Different currency -> convert. Never just add.
        <travel>-TotalPrice += zcl_travel_helper=>convert_currency(
                               amount          = booking-FlightPrice
                               source_currency = booking-CurrencyCode
                               target_currency = <travel>-CurrencyCode ).
      ENDIF.
    ENDLOOP.
  ENDLOOP.

  " 4. Write back
  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      UPDATE FIELDS ( TotalPrice )
      WITH CORRESPONDING #( travels )
    REPORTED DATA(update_reported).

  reported = CORRESPONDING #( DEEP update_reported ).

ENDMETHOD.
```

`READ ENTITIES ... BY \_Booking` is the read-by-association form. It reads the children of the given parents in one call, and the result carries the parent key (`TravelUUID`) so you can group. This is how you do a header/item rollup in RAP. Do not `SELECT` from `ZBOOKING`.

The BDEF trigger:

```
determination calculateTotalPrice on modify { field BookingFee, CurrencyCode; }
```

For the rollup to also fire when a **Booking price** changes, add a determination on the **Booking** entity that updates the parent — or, more simply, declare it on the Booking:

```
define behavior for ZR_BookingTP alias Booking
...
{
  determination calculateTotalPrice on modify { field FlightPrice, CurrencyCode; create; delete; }
}
```

with an implementation that reads `_Travel` and then triggers the parent's recalculation via an `internal action`:

```
METHOD calculateTotalPrice.       " on the Booking handler
  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Booking BY \_Travel
      FIELDS ( TravelUUID ) WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      EXECUTE recalculatePrices
      FROM CORRESPONDING #( travels )
    REPORTED DATA(exec_reported).
ENDMETHOD.
```

with `internal action recalculatePrices;` on the Travel entity. `internal` keeps it out of OData — it is a BO-internal mechanism, not a user button.

`setTravelId` — draw a number range number, on save

```
METHOD setTravelId.

  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      FIELDS ( TravelId )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  " Only instances that do not yet have a number
  DELETE travels WHERE TravelId IS NOT INITIAL.
  CHECK travels IS NOT INITIAL.

  TRY.
      cl_numberrange_runtime=>number_get(
        EXPORTING nr_range_nr = '01'
                  object      = 'ZTRAVELID'
                  quantity    = CONV #( lines( travels ) )
        IMPORTING number      = DATA(number_range_key)
                  returncode  = DATA(number_range_return_code)
                  returned_quantity = DATA(number_range_returned_quantity) ).
    CATCH cx_number_ranges INTO DATA(lx_number_ranges).
      LOOP AT travels INTO DATA(travel).
        APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.
        APPEND VALUE #( %tky = travel-%tky
                        %msg = lx_number_ranges ) TO reported-travel.
      ENDLOOP.
      RETURN.
  ENDTRY.

  " Distribute the block of numbers
  DATA(next) = number_range_key - number_range_returned_quantity.
  DATA update_lines TYPE TABLE FOR UPDATE zr_traveltp.
  LOOP AT travels INTO travel.
    next += 1.
    APPEND VALUE #( %tky     = travel-%tky
                    TravelId = next ) TO update_lines.
  ENDLOOP.

  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      UPDATE FIELDS ( TravelId )
      WITH update_lines
    REPORTED DATA(update_reported).

  reported = CORRESPONDING #( DEEP update_reported ).

ENDMETHOD.
```

Why `on save` and not `on modify`:

- A number range is a **scarce, non-transactional resource**. A number drawn during editing is consumed even if the user cancels the draft — producing permanent gaps.
- `on save { create; }` draws the number once, at activation, for instances that are really saved.

Prerequisite: create number range object `ZTRAVELID` in transaction `SNRO`, with interval `01`.

> Number ranges commit on their own database connection, which is legal in RAP (secondary connections are allowed in all phases — see Chapter 17, section 17.6).

10.4 The validations

`validateDates`

```
METHOD validateDates.

  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      FIELDS ( BeginDate EndDate )
```

```
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  LOOP AT travels INTO DATA(travel).

    " clear previous state messages for this instance
    APPEND VALUE #( %tky = travel-%tky %state_area = 'VALIDATE_DATES' ) TO reported-travel.

    IF travel-BeginDate IS INITIAL.
      APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.
      APPEND VALUE #( %tky              = travel-%tky
                      %state_area       = 'VALIDATE_DATES'
                      %msg              = NEW zcx_travel_messages(
                                            severity = if_abap_behv_message=>severity-error
                                            textid   = zcx_travel_messages=>begin_date_missing )
                      %element-BeginDate = if_abap_behv=>mk-on ) TO reported-travel.

    ELSEIF travel-EndDate IS INITIAL.
      APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.
      APPEND VALUE #( %tky            = travel-%tky
                      %state_area     = 'VALIDATE_DATES'
                      %msg            = NEW zcx_travel_messages(
                                          severity = if_abap_behv_message=>severity-error
                                          textid   = zcx_travel_messages=>end_date_missing )
                      %element-EndDate = if_abap_behv=>mk-on ) TO reported-travel.

    ELSEIF travel-EndDate < travel-BeginDate.
      APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.
      APPEND VALUE #( %tky              = travel-%tky
                      %state_area       = 'VALIDATE_DATES'
                      %msg              = NEW zcx_travel_messages(
                                            severity  = if_abap_behv_message=>severity-error
                                            textid    = zcx_travel_messages=>end_before_begin
                                            begindate = travel-BeginDate
                                            enddate   = travel-EndDate )
                      %element-BeginDate = if_abap_behv=>mk-on
                      %element-EndDate   = if_abap_behv=>mk-on ) TO reported-travel.

    ELSEIF travel-BeginDate < cl_abap_context_info=>get_system_date( ).
      " a warning, not an error: no entry in 'failed'
      APPEND VALUE #( %tky              = travel-%tky
                      %state_area       = 'VALIDATE_DATES'
                      %msg              = NEW zcx_travel_messages(
                                            severity  = if_abap_behv_message=>severity-warning
                                            textid    = zcx_travel_messages=>begin_date_in_past
                                            begindate = travel-BeginDate )
                      %element-BeginDate = if_abap_behv=>mk-on ) TO reported-travel.
    ENDIF.

  ENDLOOP.
ENDMETHOD.
```

The **warning** branch shows the mechanism clearly: a message in `reported` with no entry in `failed` is informational and does not block the save. That is exactly the difference.

`validateCustomer`

```
METHOD validateCustomer.

  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      FIELDS ( CustomerId )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  " Collect the distinct customers once, then check with one SELECT.
  " Checking inside the LOOP would be a SELECT per row.
  DATA(customers) = travels.
  SORT customers BY CustomerId.
  DELETE ADJACENT DUPLICATES FROM customers COMPARING CustomerId.
  DELETE customers WHERE CustomerId IS INITIAL.

  IF customers IS NOT INITIAL.
    SELECT FROM /dmo/customer
      FIELDS customer_id
      FOR ALL ENTRIES IN @customers
      WHERE customer_id = @customers-CustomerId
      INTO TABLE @DATA(valid_customers).
  ENDIF.

  LOOP AT travels INTO DATA(travel).

    APPEND VALUE #( %tky = travel-%tky %state_area = 'VALIDATE_CUSTOMER' ) TO reported-travel.

    IF travel-CustomerId IS INITIAL
       OR NOT line_exists( valid_customers[ customer_id = travel-CustomerId ] ).
      APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.
      APPEND VALUE #( %tky              = travel-%tky
                      %state_area       = 'VALIDATE_CUSTOMER'
                      %msg              = NEW zcx_travel_messages(
                                            severity   = if_abap_behv_message=>severity-error
                                            textid     = zcx_travel_messages=>customer_unknown
                                            customerid = travel-CustomerId )
```

```
                    %element-CustomerId = if_abap_behv=>mk-on ) TO reported-travel.
    ENDIF.
  ENDLOOP.
ENDMETHOD.
```

Read master data in one set-based `SELECT`, outside the loop. A validation runs on every save of every instance; a `SELECT SINGLE` inside a loop over a mass update is the classic RAP performance defect. Reading `/dmo/customer` with `SELECT` is correct here — it is *not* the BO's own data.

10.5 The actions

```
METHOD acceptTravel.

  " Set the status in the buffer
  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      UPDATE FIELDS ( OverallStatus )
      WITH VALUE #( FOR key IN keys
                  ( %tky          = key-%tky
                    OverallStatus = zbp_r_traveltp=>c_status-accepted ) )
    FAILED   DATA(modify_failed)
    REPORTED DATA(modify_reported).

  failed   = CORRESPONDING #( DEEP modify_failed ).
  reported = CORRESPONDING #( DEEP modify_reported ).

  " Read back and fill the action result so the UI refreshes
  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      ALL FIELDS WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  result = VALUE #( FOR travel IN travels ( %tky   = travel-%tky
                                            %param = travel ) ).
ENDMETHOD.
```

`rejectTravel` is identical with `c_status-rejected`.

The `result` with `%param` is what makes `result [1] $self` in the BDEF useful: the UI receives the changed instance and repaints without an extra round trip.

Chapter 13 covers actions in depth, including parameters, dialogs and button placement.

10.6 The complete file inventory

Object	Type	Lines (approx.)
`ZTRAVEL`, `ZBOOKING`	DDIC tables	2 × 20
`ZTRAVEL_D`, `ZBOOKING_D`	Draft tables	2 × 22 (generated)
`ZR_TravelTP`, `ZR_BookingTP`	CDS view entities	55 + 45
`ZR_TravelTP`	BDEF	90
`ZBP_R_TravelTP` + local types	ABAP classes	**~200 (the only real code)**
`ZCX_TRAVEL_MESSAGES`	Exception class	60
`ZTRAVEL_MSG`	Message class	— (SE91)
`ZC_TravelTP`, `ZC_BookingTP`	CDS projections	50 + 35
`ZC_TravelTP`	Projection BDEF	30
`ZC_TravelTP`, `ZC_BookingTP`	Metadata extensions	120 + 60
`ZSD_TRAVEL`	Service definition	10
`ZUI_TRAVEL_O4`	Service binding	— (generated)

10.7 What the framework did that you did not write

Worth listing explicitly, because it is the return on the declarative investment:

- `INSERT` / `UPDATE` / `DELETE` on both tables
- Reading with paging, filtering, sorting and `$expand`
- UUID key generation
- `CreatedBy`, `CreatedAt`, `LastChangedBy`, `LastChangedAt`, `LocalLastChangedAt`
- Lock acquisition and release, on the root, covering the children

- ETag generation and `If-Match` comparison
- Draft creation, auto-save, discard, resume, and the exclusive/optimistic lock transition
- Cascading delete of Bookings when a Travel is deleted
- Transaction boundaries, buffer management, `COMMIT`
- The whole OData V4 protocol implementation
- The whole Fiori UI

10.8 Testing

In the Fiori Elements preview

Test	Expect
Open the app	List Report, empty or with data
Create	Object Page in edit mode, a draft is created immediately
Leave `BeginDate` empty and press **Create**	Error message, save blocked, field highlighted
`EndDate` before `BeginDate`	Error on both fields
Enter a `BookingFee`	`TotalPrice` updates — **only if** `side effects` **is declared**
Add a Booking with "+"	Works — **only if** `association _Booking { create; }` is in the BDEF and `use association _Booking { create; }` in the projection BDEF
Press **Create** with valid data	Saves; `TravelId` gets a number; `CreatedBy` / `CreatedAt` are filled
Press **Accept Travel**	Status becomes Accepted, the column turns green
Open the same Travel in a second browser session and edit	*"Object is locked by user ..."*
Edit, close the browser, reopen	The draft is offered for resume
Delete a Travel that has Bookings	Both are deleted

With EML from an ABAP class

Do not rely only on the UI. A console app catches logic errors faster and can be kept as a test.

```
CLASS zcl_test_travel DEFINITION PUBLIC FINAL CREATE PUBLIC.
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
ENDCLASS.

CLASS zcl_test_travel IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.

    MODIFY ENTITIES OF zr_traveltp
      ENTITY Travel
        CREATE FIELDS ( AgencyId CustomerId BeginDate EndDate BookingFee CurrencyCode Description )
        WITH VALUE #( ( %cid         = 'CID_1'
                        AgencyId     = '070001'
                        CustomerId   = '000001'
                        BeginDate    = cl_abap_context_info=>get_system_date( ) + 30
                        EndDate      = cl_abap_context_info=>get_system_date( ) + 40
                        BookingFee   = '20.00'
                        CurrencyCode = 'EUR'
                        Description  = 'Created from a test class' ) )
      MAPPED   DATA(mapped)
      FAILED   DATA(failed)
      REPORTED DATA(reported).

    IF failed IS NOT INITIAL.
      out->write( 'CREATE failed' ).
      out->write( reported ).
      RETURN.
    ENDIF.

    COMMIT ENTITIES RESPONSE OF zr_traveltp
      FAILED   DATA(commit_failed)
      REPORTED DATA(commit_reported).

    IF commit_failed IS NOT INITIAL.
      out->write( 'COMMIT failed - a validation rejected it' ).
      out->write( commit_reported ).
      RETURN.
    ENDIF.

    out->write( |Created TravelUUID: { mapped-travel[ 1 ]-TravelUUID }| ).

  ENDMETHOD.
ENDCLASS.
```

Run with `F9` in ADT. Chapter 18 explains every statement here.

> **Note on draft:** the EML above creates an **active** instance directly, bypassing the draft. That is legal and is how background jobs and APIs should write to a draft-enabled BO. Drafts are a UI concept.

ABAP Unit with the RAP test double framework

```
CLASS ltc_travel DEFINITION FINAL FOR TESTING
  DURATION SHORT
  RISK LEVEL HARMLESS.

  PRIVATE SECTION.
    CLASS-DATA environment TYPE REF TO if_cds_test_environment.
    CLASS-METHODS class_setup.
    CLASS-METHODS class_teardown.
    METHODS setup.
    METHODS validate_dates_rejects FOR TESTING RAISING cx_static_check.
ENDCLASS.

CLASS ltc_travel IMPLEMENTATION.

  METHOD class_setup.
    environment = cl_cds_test_environment=>create( i_for_entity = 'ZR_TRAVELTP' ).
  ENDMETHOD.

  METHOD class_teardown.
    environment->destroy( ).
  ENDMETHOD.

  METHOD setup.
    environment->clear_doubles( ).
  ENDMETHOD.

  METHOD validate_dates_rejects.
    MODIFY ENTITIES OF zr_traveltp
      ENTITY Travel
        CREATE FIELDS ( AgencyId CustomerId BeginDate EndDate CurrencyCode )
        WITH VALUE #( ( %cid       = 'T1'
                        AgencyId   = '070001'
                        CustomerId = '000001'
                        BeginDate  = '20260101'
                        EndDate    = '20250101'      " end before begin
                        CurrencyCode = 'EUR' ) )
      FAILED DATA(failed).

    COMMIT ENTITIES RESPONSE OF zr_traveltp FAILED DATA(commit_failed).

    cl_abap_unit_assert=>assert_not_initial(
      act = commit_failed
      msg = 'Validation validateDates should have rejected the save' ).
  ENDMETHOD.

ENDCLASS.
```

`CL_CDS_TEST_ENVIRONMENT` replaces the underlying tables with in-memory doubles, so the test does not touch real data and runs in milliseconds. For BO-level testing there is also `CL_BUSINESS_OBJECT_ENVIRONMENT` / the RAP BO test double framework, which lets you mock a *dependent* BO your code calls via EML.

10.9 Things that will go wrong the first time

Symptom	Cause	Fix
"+" button missing in the Bookings table	`association _Booking { create; }` missing in base BDEF, or `use association _Booking { create; }` missing in the projection BDEF	Add both
Action button missing	`use action X;` missing in the projection BDEF	Add it
Action button present but greyed out	`features : instance` declared but `get_instance_features` returns `disabled`, or returns nothing	Return `%action-X = if_abap_behv=>fc-o-enabled`
`TotalPrice` does not refresh on screen	`side effects` not declared	`field BookingFee affects field TotalPrice;`
Validation message appears but the record saves	`failed` not filled	`APPEND VALUE #( %tky = ... ) TO failed-travel.`
Validation message stays after the user fixes the data	`%state_area` not cleared first	Add the clearing entry at the start of the loop
Determination reads old values	Used `SELECT` instead of `READ ENTITIES`	Use EML
Draft table activation fails	Hand-typed structure does not match what the draft runtime expects on this release	Delete it and regenerate with the ADT quick fix (`Ctrl + 1` on the CDS entity); take the generated version as correct
"Entity does not support CREATE"	`create;` missing, or the projection lacks `use create;`	Add it

Symptom	Cause	Fix
Runtime error "*COMMIT WORK not allowed*"	`COMMIT WORK` inside a handler	Remove it
Saved data does not appear in the list	A `where` clause in the projection filters it out	See Chapter 04, §4.7

10.10 Interview answer

> **"Walk me through building a transactional Fiori app with RAP."**
>
> Bottom up, in dependency order, activating each step before moving on: tables, then draft tables generated with the ADT quick fix, then the interface views — child before parent, because the parent's `composition of` needs the child to exist. Then the behavior definition and the behavior pool, generated with `Ctrl + 1`. Then the projection views, again child first, the projection behavior definition with its `use` statements, the metadata extensions, and finally the service definition and binding, which you publish.
>
> The business logic in a typical app is small — in this example three determinations, three validations and two actions, around 200 lines. Everything else is declaration: the framework provides CRUD on both tables, UUID key generation, the five admin fields, locking, ETags, draft with cancel and resume, cascading delete, the transaction boundary and the whole OData protocol.
>
> I test it with an EML console class before ever opening the UI, because that is the only place you can see `failed`, `reported` and `sy-subrc` after `COMMIT ENTITIES`.
>
> The failures to expect first time are all declarative: a missing `use action` so the button never appears, a missing `association _Booking { create; }` so the "+" is greyed out, a missing `side effects` so the calculated total looks stale, and a validation that fills `reported` but not `failed` so the record saves despite the error.

10.11 Production considerations

- **Constants, not literals.** Status values belong in the global ABP class, not scattered as `'A'` and `'O'` through the pool.
- `reported = CORRESPONDING #( DEEP ... )` **after every inner** `MODIFY`, or messages disappear.
- **Determinations must be idempotent** — they can run more than once per transaction.
- **Number ranges** `on save`, never `on modify`, or discarded drafts burn numbers.
- **Set-based master-data reads.** One `SELECT ... FOR ALL ENTRIES` outside the loop, never a `SELECT SINGLE` inside it.
- **Promote the playground class to an ABAP Unit test** rather than deleting it.
- **Run the full UI checklist (§10.8) after every transport to QA**, not just after the first — the declarative parts fail silently and only a checklist catches them.

Chapter 11 — Draft Handling

11.1 What a draft is

A **draft** is a second, parallel copy of an instance, stored in its own table, that the user edits before committing. It is the RAP answer to "the user filled in half a form and their laptop died".

```
Active instance                 Draft instance
ZTRAVEL                         ZTRAVEL_D
TravelUUID = ABC                TravelUUID = ABC
OverallStatus = 'O'             OverallStatus = 'O'
TotalPrice = 100                TotalPrice = 250     <- being edited
IsActiveEntity = true           IsActiveEntity = false
```

Both rows have the **same key**. They are distinguished by `IsActiveEntity` — which is why the RAP transactional key is `%tky` (= `%key` + `%is_draft`), not `%key`.

What draft buys you

Feature	Without draft	With draft
Unsaved work survives a browser crash	No	Yes
The user can leave and come back	No	Yes
Validation errors shown while editing	Only on save	Continuously, via `Prepare`
Two users editing the same object	Both can, last one wins	The second is told it is locked
"Cancel" discards changes cleanly	Needs UI-side state	Built in
Number of database tables	1 per entity	2 per entity
Locking complexity	Simple	Exclusive lock → optimistic lock transition

When draft is the *wrong* choice

- **APIs and background jobs.** Drafts are a UI concept. A machine consumer creates active instances directly.
- **High-frequency, single-field updates** (e.g. a "mark as read" toggle). The draft round trips cost more than the operation.
- **Objects with a very short edit lifetime**, where the user always completes the form in one go and a lost form is not expensive.
- **When you cannot afford the second table** — e.g. an entity with 200 columns and 500 million rows.

For a classic Fiori List Report / Object Page maintenance app: **use draft**. It is what the floorplan expects, and the non-draft alternative ("direct edit") has a noticeably worse UX.

11.2 Everything you must declare

1. Draft tables (Chapter 02, §2.5)

One per draft-enabled entity. **Generate them with the ADT quick fix** (`Ctrl + 1` on the CDS entity) rather than typing them — the required structure is release-dependent. Review the generated result against the criteria in Chapter 02, §2.5.

2. In the base BDEF

```
managed implementation in class ZBP_R_TravelTP unique;
strict ( 2 );
with draft;                                  " <- 1. enable draft for the BO

define behavior for ZR_TravelTP alias Travel
persistent table ztravel
draft table ztravel_d                        " <- 2. the draft table
lock master
total etag LastChangedAt                     " <- 3. mandatory for draft
authorization master ( instance )
etag master LocalLastChangedAt
{
  ...
  association _Booking { create; with draft; }   " <- 4. every association

  draft action Activate optimized;               " <- 5. the four draft actions
  draft action Discard;
  draft action Edit;
  draft action Resume;
  draft determine action Prepare                 " <- 6. validations to run early
  {
    validation validateDates;
    validation validateCustomer;
    validation validateAgency;
  }
```

```
}

define behavior for ZR_BookingTP alias Booking
persistent table zbooking
draft table zbooking_d                      " <- child needs its own draft table
lock dependent by _Travel
authorization dependent by _Travel
etag master LastChangedAt
{
  ...
  association _Travel { with draft; }       " <- and its parent association
}
```

Note: the **child does not declare draft actions**. Draft actions live on the root only — a draft is created for the whole composition tree.

3. In the projection BDEF

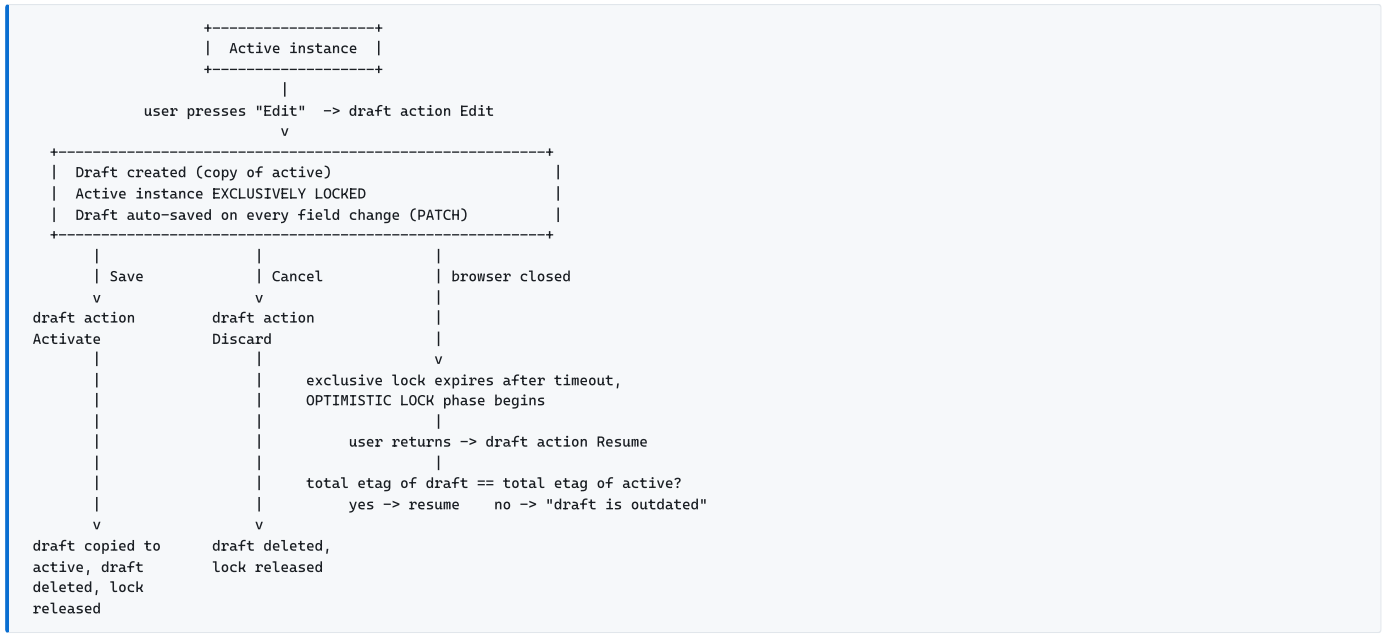
```
projection;
strict ( 2 );
use draft;                                  " <- expose draft in this service

define behavior for ZC_TravelTP alias Travel
use etag
{
  use create;
  use update;
  use delete;
  use association _Booking { create; with draft; }

  use action Activate;                      " <- all five must be exposed
  use action Discard;
  use action Edit;
  use action Resume;
  use action Prepare;
}
```

Missing any one of these five gives a subtly broken app: e.g. without `use action Prepare;` validation messages do not appear until Save.

11.3 The draft lifecycle



The four draft actions

Action	When	What it does
`Edit`	User presses **Edit** on an active instance	Copies active → draft, acquires the exclusive lock
`Activate`	User presses **Save**	Runs `Prepare`, then copies draft → active, deletes the draft, releases the lock
`Discard`	User presses **Cancel**	Deletes the draft, releases the lock. Active is untouched
`Resume`	User reopens an existing draft	Re-acquires the lock, checks total ETag, continues

`draft action Activate optimized;`

The `optimized` addition tells the framework it may skip re-running validations at activation **because `Prepare` already ran them and the data has not changed since**. Measurably faster on large documents.

Conditions:

- every validation relevant to activation is listed in `draft determine action Prepare`
- your validations do not have side effects (they must not change data — which they must not anyway)

Use it. The only reason not to is a legacy BDEF whose `Prepare` list is incomplete.

`draft determine action Prepare`

```
draft determine action Prepare
{
  validation validateDates;
  validation validateCustomer;
  validation validateAgency;
  determination calculateTotalPrice;     " determinations may be listed too
}
```

When it runs:

- automatically **before** `Activate`
- whenever the UI calls it — Fiori Elements calls `Prepare` after edits, so messages appear inline
- explicitly, from EML

Anything not listed here only runs at save time. If a user complains "the app only tells me about the error when I press Save", the fix is almost always "add the validation to `Prepare`".

Pair it with a side effect so the message list refreshes:

```
side effects
{
  determine action Prepare affects messages;
}
```

11.4 `%tky` — the single most important draft concept

```
" WRONG in a draft-enabled BO
READ ENTITIES OF zr_traveltp IN LOCAL MODE
  ENTITY Travel FIELDS ( TotalPrice )
  WITH VALUE #( ( TravelUUID = key-TravelUUID ) )   " <- which one? active or draft?
  RESULT DATA(travels).

" RIGHT
READ ENTITIES OF zr_traveltp IN LOCAL MODE
  ENTITY Travel FIELDS ( TotalPrice )
  WITH CORRESPONDING #( keys )                      " keys carries %tky
  RESULT DATA(travels).
```

`%tky` is a structure:

```
%tky = ( TravelUUID = ABC..., %is_draft = if_abap_behv=>mk-on )
```

`%is_draft` value	Means
`if_abap_behv=>mk-on` (`'01'`)	The draft instance
`if_abap_behv=>mk-off` (`'00'`)	The active instance

`strict ( 2 )` makes using `%key` in a draft-enabled BO a **syntax error** in most contexts, which is exactly what you want. Under looser modes it compiles and then reads the wrong row at runtime — a bug that looks like "my determination doesn't see the data".

Rule: in a draft-enabled BO, always use `%tky`. Never `%key`.

When you build a key table by hand:

```
DATA(keys_for_draft) = VALUE ztravel_tky_tab(
  ( TravelUUID = lv_uuid  %is_draft = if_abap_behv=>mk-on ) ).
```

11.5 Locking with draft — the two phases

This is the mechanism people find surprising, so it is worth stating precisely.

Phase 1 — exclusive lock

From the moment `Edit` (or `Create`) runs, the active instance carries an **exclusive lock**.

- Another user attempting to edit gets "*Object is locked by user X*".
- The lock survives the user's session ending. It is not a session lock.
- It expires after a configurable timeout (default in the range of hours; configured in the draft framework customizing, table `SDRAFT_...`, or via the parameter in the BDEF framework).

Phase 2 — optimistic lock

After the exclusive lock expires:

- Another user **can** now edit the active instance.
- The original draft still exists.
- When the original user presses *Resume*, RAP compares the draft's stored **total ETag** with the active instance's current one.
- Equal → nobody changed it → resume proceeds normally.
- Different → "*The object has been changed by another user. Your draft is outdated.*"

This is exactly why `total etag` is mandatory for draft, and why it must be a field that changes on **any** change in the composition tree (`LastChangedAt`, annotated with `@Semantics.systemDateTime.lastChangedAt: true`).

Checking lock state yourself

```
READ ENTITIES OF zr_traveltp IN LOCAL MODE
  ENTITY Travel
    FIELDS ( TravelId )
    WITH CORRESPONDING #( keys )
  RESULT DATA(travels).

" %is_draft tells you which copy you are on
LOOP AT travels INTO DATA(travel).
  IF travel-%is_draft = if_abap_behv=>mk-on.
    " we are working on a draft
  ENDIF.
ENDLOOP.
```

And to request a lock explicitly from outside the BO:

```
SET LOCKS OF zr_traveltp
  ENTITY Travel
  WITH VALUE #( ( TravelUUID = lv_uuid %is_draft = if_abap_behv=>mk-off ) )
  FAILED DATA(lock_failed).
```

11.6 Drafts and EML

```
" 1. Create an ACTIVE instance directly (typical for a job or an API)
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    CREATE FIELDS ( AgencyId CustomerId BeginDate EndDate CurrencyCode )
    WITH VALUE #( ( %cid = 'C1' AgencyId = '070001' ... ) )
  MAPPED DATA(mapped).
COMMIT ENTITIES.
```

No draft is involved. `%is_draft` defaults to off.

```
" 2. Work the way the UI does: Edit -> change -> Activate
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    EXECUTE Edit FROM VALUE #( ( %tky-TravelUUID = lv_uuid
                                 %tky-%is_draft  = if_abap_behv=>mk-off ) )
  MAPPED DATA(mapped_edit)
  FAILED DATA(failed_edit).

MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    UPDATE FIELDS ( Description )
    WITH VALUE #( ( %tky-TravelUUID = lv_uuid
                    %tky-%is_draft  = if_abap_behv=>mk-on
                    Description     = 'changed in the draft' ) ).

MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    EXECUTE Activate FROM VALUE #( ( %tky-TravelUUID = lv_uuid
                                     %tky-%is_draft  = if_abap_behv=>mk-on ) )
  FAILED   DATA(failed_act)
  REPORTED DATA(reported_act).

COMMIT ENTITIES.
```

Note that `Edit` is executed **against the active instance** (`%is_draft = off`) and everything after it against the **draft** (`%is_draft = on`).

```
" 3. Discard a draft
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    EXECUTE Discard FROM VALUE #( ( %tky-TravelUUID = lv_uuid
                                    %tky-%is_draft  = if_abap_behv=>mk-on ) ).
COMMIT ENTITIES.
```

```
" 4. Run Prepare explicitly to collect validation messages without saving
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    EXECUTE Prepare FROM VALUE #( ( %tky-TravelUUID = lv_uuid
                                    %tky-%is_draft  = if_abap_behv=>mk-on ) )
  REPORTED DATA(prepare_reported).
```

Useful in a test class: assert that the expected messages come back **without** committing anything.

11.7 Reading drafts

In OData the key gains `IsActiveEntity` :

```
GET /Travel(TravelUUID=guid'ABC...',IsActiveEntity=true)     <- the active instance
GET /Travel(TravelUUID=guid'ABC...',IsActiveEntity=false)    <- the draft
GET /Travel?$filter=IsActiveEntity eq false                  <- all drafts
```

Fiori Elements handles this automatically and shows a **"Draft"** marker in the list, plus a "My Drafts" filter.

The `DraftAdministrativeData` navigation gives you who owns a draft and when it was last changed:

```
GET /Travel(TravelUUID=guid'...',IsActiveEntity=false)?$expand=DraftAdministrativeData
```

11.8 Cleaning up orphaned drafts

Drafts left behind by users accumulate. Two mechanisms:

1. **Automatic expiry.** The draft framework has a configurable retention. Expired drafts are removed by the standard clean-up job. On-premise, schedule report `R_DRAFT_ENTITIES_DELETE` / the *Draft Clean-Up* application job template (the exact name varies by release — search for "draft" in the *Application Jobs* app or in `SJOBREPO`).
2. **Your own job**, if you have retention rules of your own. Use EML `Discard` , never `DELETE FROM ztravel_d` : `abap MODIFY ENTITIES OF zr_traveltp ENTITY Travel EXECUTE Discard FROM lt_old_draft_keys. COMMIT ENTITIES.` Deleting the draft table row directly leaves the lock entry and the draft admin data behind.

11.9 Non-draft transactional apps

If you decide against draft, remove `with draft;` , the draft tables, `total etag` , the four draft actions, `Prepare` , and every `with draft;` on associations. The projection BDEF drops `use draft;` and the five `use action` lines.

What changes for the user:

	Draft app	Non-draft app
Edit mode	Implicit; every change auto-saved to the draft	Explicit; changes held in the browser
Cancel	Discards the draft server-side	Discards the client-side model
Crash recovery	Full	None
Validation feedback	Continuous (via `Prepare`)	On Save only
Lock	Held from Edit to Save/Cancel	Held only during the Save request

Non-draft is the right choice for simple, single-screen edits and for services consumed by machines.

11.10 Draft troubleshooting

Symptom	Cause
Draft table activation fails	Hand-typed structure. Regenerate with the ADT quick fix rather than adjusting field by field — the exact required structure is release-dependent (see Chapter 02, section 2.5)
"*Total ETag is missing*" at BDEF activation	`with draft;` without `total etag <field>`
"*Association must be draft enabled*"	A `with draft;` is missing on an association
Edit button does nothing	`use action Edit;` missing in the projection BDEF
Validation messages only appear on Save	Validation not listed in `draft determine action Prepare`
Messages do not refresh after Prepare	`determine action Prepare affects messages;` missing from `side effects`
"*Draft is outdated*" on Resume	Correct behaviour — the active instance changed. The user must discard and start again
Two users can edit the same object	The exclusive lock expired (normal), or `lock master` is missing
`READ ENTITIES` returns nothing in a determination	Used `%key` instead of `%tky`, so it read the active instance while the user is on the draft
Drafts pile up in the database	No clean-up job scheduled
`TravelId` numbers have huge gaps	`setTravelId` is `on modify` instead of `on save` — every discarded draft burned a number

11.11 Interview answer

> **"Explain draft handling in RAP."**
>
> A draft is a second copy of the instance in its own draft table, which the user edits before committing. Both rows share the same key and are distinguished by `IsActiveEntity` — which is why the RAP transactional key is `%tky`, the key plus `%is_draft`, and why using `%key` in a draft-enabled BO reads the wrong copy.
>
> You enable it with `with draft;` in the header, a `draft table` per entity, `total etag`, and the four draft actions `Edit`, `Activate`, `Discard`, `Resume`, plus `draft determine action Prepare`, which lists the validations that should run while the draft is open so errors appear inline rather than only at save. All five must also be exposed with `use action` in the projection BDEF.
>
> Locking has two phases, and this is the part worth explaining. From `Edit`, the active instance holds an **exclusive** lock which survives the session ending and expires on a timeout. After that the **optimistic** phase begins: others may edit, and when the original user resumes, RAP compares the draft's stored **total ETag** against the active instance. If they differ the draft is declared outdated. That is what `total etag` exists for, and why it should be a different field from `etag master`.
>
> Draft is a UI concept. Background jobs and APIs create active instances directly.

11.12 Production considerations

- **Schedule the draft clean-up job before go-live.** Orphaned drafts accumulate indefinitely otherwise.
- **Never touch a draft table with ABAP SQL** — not for reporting, not for clean-up. Use `EXECUTE Discard`; deleting the row directly leaves the lock entry and the draft admin data.
- **Review the exclusive lock timeout** against how long an edit realistically takes. Too long and a user who goes to lunch blocks the document; too short and legitimate resumes fail.
- **Use** `draft action Activate optimized` and keep the `Prepare` list complete — it is measurably faster on large documents.
- **Generate draft tables with the ADT quick fix** and regenerate after an upgrade rather than patching them field by field.
- **Test the two-session conflict** as part of every QA cycle; it is the behaviour users complain about and the one least likely to be covered by a unit test.
- **Weigh the second table** for very wide or very high-volume entities before enabling draft.

Chapter 12 — Validations and Determinations

12.1 The distinction

	Determination	Validation
Purpose	**Derive** data	**Check** data
May change data	Yes	**No**
May reject	No	Yes (`failed`)
Runs at	`finalize` (on save) or during interaction (on modify)	`check_before_save` (on save) or during interaction (on modify)
Handler signature	`FOR DETERMINE ON MODIFY` / `ON SAVE`	`FOR VALIDATE ON MODIFY` / `ON SAVE`
Typical	Fill `TotalPrice` , `CreatedBy` , a status, a number	Date range, referential integrity, business rules

> A validation that changes data is a bug, even though the compiler allows it. RAP may execute validations in a different order, more than once, or skip them under `Activate optimized` . Data changed there can silently vanish.

12.2 Trigger syntax

```
determination <name> on modify { <condition>; }
determination <name> on save   { <condition>; }
validation    <name> on modify { <condition>; }
validation    <name> on save   { <condition>; }
```

Conditions

Condition	Fires when
`create`	The instance is created
`update`	The instance is updated
`delete`	The instance is deleted
`field F1, F2`	One of the named fields changes
`create; update;`	Either
`field F; create;`	F changes, **or** the instance is created

Multiple conditions are combined with `;` and mean **OR**.

```
determination calculateTotalPrice on modify { field BookingFee, CurrencyCode; create; }
```

= "run when `BookingFee` changes, or when `CurrencyCode` changes, or when a Travel is created".

`on modify` vs `on save` — the decision

	`on modify`	`on save`
Runs	Immediately after the change, during the interaction phase	In the early save phase
User sees the result	Instantly (with a `side effects` declaration)	Only after saving
Can be triggered many times per transaction	Yes	Once per transaction
Sees the final document state	No — later changes may follow	Yes
Consumes external resources	Dangerous (number ranges, API calls)	Safe

Rules of thumb:

- **Determinations that the user should see live** → `on modify` .
- **Determinations that consume a scarce resource** (number range, external ID) → `on save` .
- **Determinations that need the whole document** (a header total that depends on all items being final) → `on save` .
- **Validations** → almost always `on save` . An `on modify` validation nags the user mid-typing.

12.3 Determinations — worked examples

A) Simple default on create

```
" BDEF
determination setBookingDate on modify { create; }
```

```
METHOD setBookingDate.

  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Booking
      FIELDS ( BookingDate )
      WITH CORRESPONDING #( keys )
    RESULT DATA(bookings).

  DELETE bookings WHERE BookingDate IS NOT INITIAL.     " idempotent
  CHECK bookings IS NOT INITIAL.

  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Booking
      UPDATE FIELDS ( BookingDate )
      WITH VALUE #( FOR b IN bookings
                  ( %tky        = b-%tky
                    BookingDate = cl_abap_context_info=>get_system_date( ) ) )
    REPORTED DATA(update_reported).

  reported = CORRESPONDING #( DEEP update_reported ).
ENDMETHOD.
```

B) Derive from a foreign entity

Fill the flight price from the connection master data when carrier/connection/date change.

```
" BDEF
determination setFlightPrice on modify { field CarrierId, ConnectionId, FlightDate; }
```

```
METHOD setFlightPrice.

  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Booking
      FIELDS ( CarrierId ConnectionId FlightDate )
      WITH CORRESPONDING #( keys )
    RESULT DATA(bookings).

  DELETE bookings WHERE CarrierId IS INITIAL
                     OR ConnectionId IS INITIAL
                     OR FlightDate IS INITIAL.
  CHECK bookings IS NOT INITIAL.

  " ONE set-based read of the master data, outside the loop
  SELECT FROM /dmo/flight
    FIELDS carrier_id, connection_id, flight_date, price, currency_code
    FOR ALL ENTRIES IN @bookings
    WHERE carrier_id    = @bookings-CarrierId
      AND connection_id = @bookings-ConnectionId
      AND flight_date   = @bookings-FlightDate
    INTO TABLE @DATA(flights).

  DATA update_lines TYPE TABLE FOR UPDATE zr_traveltp\\Booking.

  LOOP AT bookings INTO DATA(booking).
    DATA(flight) = VALUE #( flights[ carrier_id    = booking-CarrierId
                                     connection_id = booking-ConnectionId
                                     flight_date   = booking-FlightDate ] OPTIONAL ).
    CHECK flight IS NOT INITIAL.
    APPEND VALUE #( %tky         = booking-%tky
                    FlightPrice  = flight-price
                    CurrencyCode = flight-currency_code ) TO update_lines.
  ENDLOOP.

  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Booking
      UPDATE FIELDS ( FlightPrice CurrencyCode )
      WITH update_lines
    REPORTED DATA(update_reported).

  reported = CORRESPONDING #( DEEP update_reported ).
ENDMETHOD.
```

Note `TYPE TABLE FOR UPDATE zr_traveltp\\Booking` — the **double backslash** addresses a child entity of the BO in a derived type. A single backslash (`zr_traveltp\_Booking`) addresses the *association*. Both appear in RAP code and they mean different things:

Syntax	Means
`zr_traveltp\\Booking`	The **entity** `Booking` inside the BO `zr_traveltp`

Syntax	Means
`zr_traveltp\_Booking`	The **association** `_Booking` from the root

C) Header rollup from children

See Chapter 10, §10.3, `calculateTotalPrice`.

D) Number range on save

See Chapter 10, §10.3, `setTravelId`.

E) Child-to-parent propagation via an internal action

When a child change must update the parent, do **not** have the child determination write to the parent directly — it creates an unclear dependency. Use an `internal action` on the parent:

```
" BDEF, Travel entity
internal action recalculatePrices;

" BDEF, Booking entity
determination notifyTravel on modify { field FlightPrice, CurrencyCode; create; delete; }
```

```
METHOD notifyTravel.                        " in lhc_Booking
  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Booking BY \_Travel
      FIELDS ( TravelUUID )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      EXECUTE recalculatePrices
      FROM CORRESPONDING #( travels )
    REPORTED DATA(exec_reported).

  reported = CORRESPONDING #( DEEP exec_reported ).
ENDMETHOD.
```

`internal action` is never exposed in OData — it is a BO-internal mechanism.

12.4 Validations — worked examples

A) Field-level check with a state message

See Chapter 10, §10.4, `validateDates`. The pattern:

```
1. READ the data
2. FOR EACH instance:
   a. APPEND a clearing entry for the %state_area
   b. IF the check fails:
        APPEND to failed
        APPEND to reported with %state_area, %msg, %element-<Field>
```

B) Referential integrity, set-based

See Chapter 10, §10.4, `validateCustomer`. The pattern:

```
1. READ the data
2. Build a distinct list of foreign keys
3. ONE SELECT with FOR ALL ENTRIES
4. LOOP and check with line_exists( )
```

C) Cross-entity validation (parent checks children)

"A Travel must have at least one Booking."

```
" BDEF, Travel entity
validation validateHasBookings on save { create; field OverallStatus; }
```

```
METHOD validateHasBookings.

  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel BY \_Booking
      FIELDS ( BookingUUID )
      WITH CORRESPONDING #( keys )
    LINK DATA(booking_links).
```

```
  LOOP AT keys INTO DATA(key).

    APPEND VALUE #( %tky = key-%tky %state_area = 'VALIDATE_BOOKINGS' ) TO reported-travel.

    IF NOT line_exists( booking_links[ source-%tky = key-%tky ] ).
      APPEND VALUE #( %tky = key-%tky ) TO failed-travel.
      APPEND VALUE #( %tky        = key-%tky
                      %state_area = 'VALIDATE_BOOKINGS'
                      %msg        = NEW zcx_travel_messages(
                                      severity = if_abap_behv_message=>severity-error
                                      textid   = zcx_travel_messages=>no_bookings ) )
             TO reported-travel.
    ENDIF.
  ENDLOOP.
ENDMETHOD.
```

`LINK DATA(booking_links)` returns the **association links** — pairs of (source key, target key) — without reading the child data. Cheaper than reading the children when all you need is "does one exist".

This is where a `composition [1..*]` cardinality would *not* have helped: cardinality is metadata, not a constraint. This validation is the enforcement.

D) Validation on delete

```
" BDEF
validation validateDeletable on save { delete; }
```

```
METHOD validateDeletable.

  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      FIELDS ( OverallStatus TravelId )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  LOOP AT travels INTO DATA(travel).
    IF travel-OverallStatus = zbp_r_traveltp=>c_status-accepted.
      APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.
      APPEND VALUE #( %tky = travel-%tky
                      %msg = new_message( id       = 'ZTRAVEL_MSG'
                                          number   = '020'
                                          severity = if_abap_behv_message=>severity-error
                                          v1       = travel-TravelId ) ) TO reported-travel.
    ENDIF.
  ENDLOOP.
ENDMETHOD.
```

> **Note on deletes:** in a `validation ... on save { delete; }` the instance is already marked for deletion in the buffer. `READ ENTITIES` still returns its data, which is what you need for the message. This works; it is one of the less obvious guarantees of the RAP buffer.

A `precheck` handler is the alternative when you want to reject the delete *before* it enters the buffer at all — see Chapter 14.

12.5 Execution order

Within one save, the order is:

```
INTERACTION PHASE
  ...consumer changes...
  determinations 'on modify'  (after each triggering change)
  validations    'on modify'  (after each triggering change)

EARLY SAVE PHASE
  finalize()
     -> determinations 'on save' run here
  check_before_save()
     -> validations 'on save' run here
  [ if failed -> cleanup_finalize(), back to interaction phase ]

=== POINT OF NO RETURN ===

LATE SAVE PHASE
  adjust_numbers()
  save() / save_modified()
  cleanup()
```

Order *within* a phase is not guaranteed

RAP does not promise that `validateDates` runs before `validateCustomer`. If validation B depends on validation A having passed, that is a design smell — restructure so each validation is independent, or merge them into one.

The same applies to determinations. If determination B needs the result of determination A:

- Either make A `on modify` and B `on save` (ordering by phase is guaranteed), or

- merge them, or
- have A trigger B via an `internal action`.

Determinations can trigger determinations

A determination that changes field X will trigger any determination declared `on modify { field X; }`. RAP handles this and re-runs the dependent determination. It also **detects direct self-triggering** and raises an error.

It does **not** detect indirect cycles:

```
determination A on modify { field X; }  ->  changes Y
determination B on modify { field Y; }  ->  changes X
```

This loops until the framework's iteration limit and then dumps. Draw the graph.

12.6 Performance rules

These matter because determinations and validations run on **every** save, on **every** instance.

Rule	Why
Never `SELECT SINGLE` inside a `LOOP AT keys`	A mass update of 5000 rows becomes 5000 round trips
Collect keys → one `SELECT ... FOR ALL ENTRIES` (or a range table) → `line_exists( )`	One round trip
`FIELDS ( ... )` in `READ ENTITIES`, not `ALL FIELDS`	Reads less
`CHECK <table> IS NOT INITIAL` before `MODIFY ENTITIES`	An empty modify still costs a framework round trip
`DELETE <table> WHERE <already set>` before writing	Idempotency **and** fewer writes
Use `LINK DATA(...)` instead of `RESULT DATA(...)` when you only need existence	Avoids reading child data
Prefer one determination triggered by several fields over several determinations	Fewer framework callbacks
Move expensive checks to `on save`	They then run once per transaction, not once per keystroke

---

12.7 Debugging determinations and validations

Setting a breakpoint that actually gets hit

The Fiori preview runs in a **different session** from your ADT debugger. Two options:

1. **External breakpoint** — in ADT, set the breakpoint, then right-click → *ABAP Debugger* → check that the *external* user matches the user running the browser. This is the normal way to debug from the Fiori preview.
2. **Drive the BO with EML from a class run** (`IF_OO_ADT_CLASSRUN`) and debug that with `F9`. Faster, reproducible, and it becomes a test. **Prefer this.**

Checking what actually ran

Add a temporary message with the method name:

```
APPEND VALUE #( %tky = key-%tky
                %msg = new_message_with_text(
                         severity = if_abap_behv_message=>severity-information
                         text     = |calculateTotalPrice ran for { key-TravelUUID }| ) )
       TO reported-travel.
```

It appears in the Fiori message popover. Remove before transport.

The four questions when "my determination does not run"

1. Is the trigger condition right? `field BookingFee` does not fire when `TotalPrice` changes.
2. Is it `on save` when you expected `on modify` (or vice versa)?
3. In a draft app: did the change happen on the **draft**, and are you reading with `%tky`?
4. Did you `DELETE` the instances out of your internal table with an over-eager idempotency check?

The three questions when "my validation does not block"

1. Did you fill `failed`? A message alone does not block.
2. Is the validation listed in `draft determine action Prepare`? (Otherwise it only runs at Activate.)
3. Is the trigger condition ever true? `on save { create; }` does not fire on update.

12.8 Checklist

- [ ] Every determination is idempotent (checks whether the value is already set).
- [ ] Every determination ends with `reported = CORRESPONDING #( DEEP ... )`.
- [ ] No validation changes data.
- [ ] Every rejecting validation fills **both** `failed` and `reported`.
- [ ] Every validation with a state message clears its `%state_area` first.
- [ ] Number ranges and external calls are in `on save`, never `on modify`.
- [ ] No `SELECT` inside a loop.
- [ ] No `SELECT` on the BO's own tables at all.
- [ ] Every validation the user should see early is in `draft determine action Prepare`.
- [ ] No indirect determination cycles.

12.9 Interview answer

> **"What are validations and determinations, and when do they run?"**
>
> Both are declared in the behavior definition with a trigger time and a trigger condition, and implemented in the behavior pool. A **determination** derives data — it fills a field automatically. A **validation** checks data and rejects it, and must never change data.
>
> The trigger time is `on modify` or `on save`. `on modify` runs in the interaction phase immediately after the triggering change; `on save` runs in the early save phase — determinations when `finalize` is reached, validations when `check_before_save` is reached. So an `on save` determination is guaranteed to run before an `on save` validation, but the order of two validations within the same phase is **not** guaranteed, which means they must be independent.
>
> Practically: determinations the user should see live are `on modify`, and anything that consumes a scarce resource like a number range is `on save`, otherwise every discarded draft burns a number. Validations are almost always `on save`, because an `on modify` validation complains while the user is still typing.
>
> Two things I always check. A validation blocks the save by adding the key to `failed` — a message in `reported` alone shows a red error and saves the record anyway. And a determination must be **idempotent**, because it can be called more than once in a transaction.

12.10 Production considerations

- **Declare them in the base BDEF**, not the projection — a projection-level rule is bypassed by EML callers, jobs and any second service.
- **No `SELECT` inside a loop.** These run on every save of every instance; a mass update turns a `SELECT SINGLE` into thousands of round trips.
- **No external calls from `on modify`.** An HTTP or RFC call there fires on every field change.
- **Clear the `%state_area` first** in every state-message validation, and use one area per validation so they do not clear each other's messages.
- **List every user-visible validation in `draft determine action Prepare`**, or the user only learns about the problem when they press Save.
- **Draw the determination dependency graph** before adding the second determination to an entity. RAP detects direct self-triggering but not an indirect A→B→A cycle, which dumps.
- **Declare a `side effect` for every field a determination derives**, or the value is right on the server and stale on screen.

Chapter 13 — Actions, Functions and Buttons

13.1 The four things you must do to get a working button

This is the checklist that turns an idea into a button on screen. Missing any one gives a silent failure.

```
1. BDEF (base)        action acceptTravel result [1] $self;
2. Behavior pool      METHOD acceptTravel. ... ENDMETHOD.
3. BDEF (projection)  use action acceptTravel;
4. Metadata extension @UI.lineItem: [{ type: #FOR_ACTION, dataAction: 'acceptTravel', label: '...' }]
```

Missing step	Symptom
1	Activation error in the behavior pool
2	Activation error in the BDEF ("method not implemented")
3	**Button never appears. No error anywhere.** ← by far the most common
4	Action exists in `$metadata` and is callable via OData/EML, but there is no button

13.2 Action types

```
action              acceptTravel;                      " instance action
static action       archiveOldTravels;                 " not bound to an instance
factory action      copyTravel [1];                    " creates new instances
internal action     recalculatePrices;                 " BO-internal only, never in OData
function            getPriceBreakdown result [1..*] ZE_PriceLine;  " read-only, returns data
```

Type	Bound to an instance?	Appears where in Fiori	Can change data
Instance action	Yes	Table toolbar (enabled when rows are selected) + Object Page header	Yes
Static action	No	Table toolbar, always enabled	Yes
Factory action	Optionally	Table toolbar	Creates instances
Internal action	Yes	Nowhere — not exposed	Yes
Function	Yes or no	Usually not rendered by default; called via OData/EML	**No**

13.3 Instance actions

Declaration

```
action ( features : instance ) acceptTravel result [1] $self;
```

Part	Meaning
`action`	Instance-bound
`( features : instance )`	Enable dynamic enable/disable per instance (Chapter 14)
`acceptTravel`	Name. Case matters in the MDE's `dataAction`.
`result [1] $self`	Returns one instance of the same entity. Makes the UI refresh automatically.

Result cardinality and type

Declaration	Returns
(nothing)	Nothing. The UI refreshes only what `side effects` says.
`result [1] $self`	The changed instance itself — **the normal choice**
`result [0..*] $self`	Several instances of the same entity
`result [0..*] ZR_BookingTP`	Instances of a different entity
`result [1] ZE_ActionOutput`	A non-persistent abstract entity — for returning computed data

Why `result [1] $self` matters: without a result, the Fiori UI does not know the instance changed. You would need a `side effects` clause for every affected field. With `$self`, the changed instance comes back in the response and the UI repaints it directly.

Implementation

```
METHOD acceptTravel.

  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      UPDATE FIELDS ( OverallStatus )
      WITH VALUE #( FOR key IN keys
                  ( %tky          = key-%tky
                    OverallStatus = zbp_r_traveltp=>c_status-accepted ) )
    FAILED   DATA(modify_failed)
    REPORTED DATA(modify_reported).

  failed   = CORRESPONDING #( DEEP modify_failed ).
  reported = CORRESPONDING #( DEEP modify_reported ).

  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      ALL FIELDS WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  result = VALUE #( FOR travel IN travels ( %tky   = travel-%tky
                                            %param = travel ) ).
ENDMETHOD.
```

`%param` carries the returned instance. For `result [1] $self`, `%param` is typed as the entity's structure. For `result [1] ZE_ActionOutput` it is that abstract entity's structure.

13.4 Actions with parameters (dialogs)

When the action needs input from the user, Fiori renders a **dialog**. The input structure is a CDS **abstract entity**.

The abstract entity

```
@EndUserText.label: 'Parameters for deductDiscount'
define abstract entity ZE_DeductDiscountParams
{
  @EndUserText.label: 'Discount %'
  @Consumption.valueHelpDefinition: [{ entity: { name: 'ZI_DiscountRates', element: 'Rate' } }]
  discount_percent : abap.int1;

  @EndUserText.label: 'Reason'
  reason_code      : zde_discount_reason;

  @EndUserText.label: 'Comment'
  comment          : abap.char(100);
}
```

Notes:

- `define abstract entity` — no `select from`, no persistence. It exists only as a type.
- Annotations on its fields drive the dialog: labels, value helps, mandatory flags.
- `@EndUserText.label` on the entity becomes the dialog title.

The BDEF

```
action ( features : instance ) deductDiscount
  parameter ZE_DeductDiscountParams
  result [1] $self;
```

The projection BDEF

```
use action deductDiscount;
```

The implementation

```
METHOD deductDiscount.

  " keys carries both the instance key and the parameter values
  DATA update_lines TYPE TABLE FOR UPDATE zr_traveltp.

  " --- validate the parameter BEFORE touching data ---
  LOOP AT keys INTO DATA(key) WHERE %param-discount_percent IS INITIAL
                                 OR %param-discount_percent > 80.
    APPEND VALUE #( %tky = key-%tky ) TO failed-travel.
    APPEND VALUE #( %tky = key-%tky
                    %msg = new_message( id       = 'ZTRAVEL_MSG'
```

```
                                    number   = '030'
                                    severity = if_abap_behv_message=>severity-error )
                  %element-BookingFee = if_abap_behv=>mk-on ) TO reported-travel.
  ENDLOOP.

  DATA(valid_keys) = keys.
  DELETE valid_keys WHERE %param-discount_percent IS INITIAL
                       OR %param-discount_percent > 80.
  CHECK valid_keys IS NOT INITIAL.

  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      FIELDS ( BookingFee )
      WITH CORRESPONDING #( valid_keys )
    RESULT DATA(travels).

  LOOP AT travels INTO DATA(travel).
    DATA(pct) = valid_keys[ KEY entity %tky = travel-%tky ]-%param-discount_percent.
    APPEND VALUE #( %tky       = travel-%tky
                    BookingFee = travel-BookingFee * ( 100 - pct ) / 100 ) TO update_lines.
  ENDLOOP.

  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      UPDATE FIELDS ( BookingFee )
      WITH update_lines
    REPORTED DATA(update_reported).

  reported = CORRESPONDING #( DEEP update_reported ).

  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel ALL FIELDS WITH CORRESPONDING #( valid_keys )
    RESULT DATA(travels_after).

  result = VALUE #( FOR t IN travels_after ( %tky = t-%tky %param = t ) ).
ENDMETHOD.
```

Always validate action parameters in the action itself. There is no validation mechanism for action input — `validation` only fires on entity fields.

Deep parameters

For an action whose input contains a nested table (e.g. "create a Travel with a list of Bookings"):

```
" the abstract entity tree
define abstract entity ZE_TravelTemplate
{
  key template_id : abap.char(10);
  agency_id       : /dmo/agency_id;

  _Bookings : composition [0..*] of ZE_BookingTemplate;
}

define abstract entity ZE_BookingTemplate
{
  key seq        : abap.int2;
  carrier_id     : /dmo/carrier_id;
  connection_id  : /dmo/connection_id;
}
```

```
" an abstract BDEF is needed for the hierarchy
abstract;
define behavior for ZE_TravelTemplate
with hierarchy
{
  association _Bookings { with draft; }
}
```

```
" the action
action createFromTemplate deep parameter ZE_TravelTemplate result [1] $self;
```

Deep parameters are powerful but Fiori Elements cannot render a dialog for them. They are for EML/API consumers. For a UI, use a flat parameter and let the user add children afterwards.

13.5 Static actions

Not bound to an instance — a global button.

```
static action archiveOldTravels parameter ZE_ArchiveParams;
```

```
METHODS archiveOldTravels FOR MODIFY
  IMPORTING keys FOR ACTION Travel~archiveOldTravels.
```

```
METHOD archiveOldTravels.

  " keys has ONE line, with %param filled from the dialog. No instance keys.
  DATA(params) = keys[ 1 ]-%param.

  SELECT FROM ztravel
    FIELDS travel_uuid
    WHERE end_date < @params-cutoff_date
      AND overall_status = 'A'
    INTO TABLE @DATA(uuids).

  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      UPDATE FIELDS ( OverallStatus )
      WITH VALUE #( FOR u IN uuids
                    ( %tky-TravelUUID = u-travel_uuid
                      %tky-%is_draft  = if_abap_behv=>mk-off
                      OverallStatus   = 'Z' ) )
    REPORTED DATA(update_reported).

  reported = CORRESPONDING #( DEEP update_reported ).
ENDMETHOD.
```

> The `SELECT` here is acceptable because we are **finding** instances to act on, not reading BO state for a decision. The subsequent change goes through EML.

In the MDE, a static action goes on the **entity level**, not on a field:

```
@UI: { lineItem: [ { type: #FOR_ACTION, dataAction: 'archiveOldTravels',
                     label: 'Archive Old Travels' } ] }
annotate entity ZC_TravelTP with { ... }
```

13.6 Factory actions

A factory action **creates** instances. The canonical use is "Copy".

```
factory action copyTravel [1];
```

`[1]` = one new instance per selected instance.

```
METHOD copyTravel.

  " 1. Read the source instances with their children
  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      ALL FIELDS WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel BY \_Booking
      ALL FIELDS WITH CORRESPONDING #( keys )
    RESULT DATA(bookings).

  " 2. Build the create payloads
  DATA travel_create  TYPE TABLE FOR CREATE zr_traveltp.
  DATA booking_create TYPE TABLE FOR CREATE zr_traveltp\_Booking.

  LOOP AT travels INTO DATA(travel).
    DATA(cid) = |COPY_{ sy-tabix }|.

    APPEND VALUE #( %cid         = cid
                    AgencyId     = travel-AgencyId
                    CustomerId   = travel-CustomerId
                    BeginDate    = travel-BeginDate
                    EndDate      = travel-EndDate
                    BookingFee   = travel-BookingFee
                    CurrencyCode = travel-CurrencyCode
                    Description  = |Copy of { travel-Description }|
                    OverallStatus = zbp_r_traveltp=>c_status-open )
           TO travel_create.

    " children, linked to the new parent via %cid_ref
    DATA(children) = VALUE #( FOR b IN bookings
                              WHERE ( TravelUUID = travel-TravelUUID )
                              ( %cid          = |{ cid }_B{ b-BookingId }|
                                CustomerId    = b-CustomerId
                                CarrierId     = b-CarrierId
                                ConnectionId  = b-ConnectionId
                                FlightDate    = b-FlightDate
                                FlightPrice   = b-FlightPrice
                                CurrencyCode  = b-CurrencyCode
                                BookingStatus = 'N' ) ).

    IF children IS NOT INITIAL.
      APPEND VALUE #( %cid_ref = cid
                      %target  = children ) TO booking_create.
    ENDIF.
```

```
  ENDLOOP.

  " 3. Create - deep, in one call
  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      CREATE FIELDS ( AgencyId CustomerId BeginDate EndDate BookingFee
                      CurrencyCode Description OverallStatus )
      WITH travel_create
      CREATE BY \_Booking
        FIELDS ( CustomerId CarrierId ConnectionId FlightDate
                 FlightPrice CurrencyCode BookingStatus )
        WITH booking_create
    MAPPED   DATA(create_mapped)
    FAILED   DATA(create_failed)
    REPORTED DATA(create_reported).

  " 4. Report the new keys back to the consumer
  mapped-travel  = create_mapped-travel.
  mapped-booking = create_mapped-booking.
  failed   = CORRESPONDING #( DEEP create_failed ).
  reported = CORRESPONDING #( DEEP create_reported ).

ENDMETHOD.
```

`%cid` and `%cid_ref` — how deep create works

The new instances have **no key yet** (the framework assigns UUIDs at create time). So how do you link a child to its parent?

Component	Meaning
`%cid`	*Content ID*. A string **you** invent to identify a not-yet-created instance within this request.
`%cid_ref`	On a child: "my parent is the instance whose `%cid` is this".
`%target`	On a `CREATE BY \_Assoc` : the table of children to create under that parent.
`mapped-travel[ %cid = 'X' ]-TravelUUID`	After the create, the framework reports the real key it assigned to `%cid = 'X'` .

`%cid` values must be unique **within one EML statement**. They have no meaning afterwards.

Filling `mapped` is mandatory in a factory action. The consumer (and the UI) uses it to navigate to the newly created instance. Without it the copy is created but the user is not taken to it.

13.7 Functions

A `function` is a **read-only** operation that returns data.

```
function getPriceBreakdown result [1..*] ZE_PriceLine;
```

```
METHODS getPriceBreakdown FOR READ
  IMPORTING keys FOR FUNCTION Travel~getPriceBreakdown RESULT result.
```

Note `FOR READ` , not `FOR MODIFY` . A function must not change data — the framework enforces this by giving the method no write access.

	Action	Function
Handler	`FOR MODIFY`	`FOR READ`
May change data	Yes	**No**
OData	`POST` to an action	`GET` on a function import
Fiori Elements renders a button	Yes	Not by default
Cacheable	No	Yes

Use functions for "calculate and show me something" operations that a Web API consumer will call. For a UI button, an action with `result` is usually the better fit because Fiori Elements renders it natively.

13.8 Putting the button on screen — the MDE

In the List Report table toolbar

```
@UI.lineItem: [ { position: 10, importance: #HIGH },
                { type: #FOR_ACTION, dataAction: 'acceptTravel', label: 'Accept' },
                { type: #FOR_ACTION, dataAction: 'rejectTravel', label: 'Reject' } ]
TravelId;
```

In the Object Page header

```
@UI.identification: [ { position: 10 },
                      { type: #FOR_ACTION, dataAction: 'acceptTravel', label: 'Accept' },
                      { type: #FOR_ACTION, dataAction: 'rejectTravel', label: 'Reject' } ]
TravelId;
```

In a specific Object Page section

Put the `#FOR_ACTION` entry in the `@UI.fieldGroup` of that section:

```
@UI.fieldGroup: [ { qualifier: 'PriceGroup', position: 10 },
                  { qualifier: 'PriceGroup', type: #FOR_ACTION,
                    dataAction: 'deductDiscount', label: 'Apply Discount' } ]
BookingFee;
```

Properties of a `#FOR_ACTION` entry

Property	Meaning
`type: #FOR_ACTION`	This entry is a button, not a column
`dataAction`	**The action name, exactly as in the BDEF.** Case-sensitive
`label`	Button text
`invocationGrouping`	`#CHANGE_SET` (all selected instances in one transaction — the default) or `#ISOLATED` (each independently)
`emphasized`	`true` makes it the primary (blue) button
`position`	Order among the buttons

`invocationGrouping` — worth understanding

Value	Behaviour with 10 selected rows
`#CHANGE_SET`	One call with 10 keys. If one fails, **all** fail.
`#ISOLATED`	Ten separate calls. Failures are independent.

For a "post to accounting" style action where partial success is wrong, use `#CHANGE_SET`. For a "send reminder email" style action, `#ISOLATED` is friendlier.

Making a button unavailable

- **Statically** (never available in this service) — omit `use action` in the projection BDEF.
- **Dynamically** (available but greyed out for some instances) — `( features : instance )` plus a `FOR INSTANCE FEATURES` handler. See Chapter 14.

13.9 Calling actions from ABAP

```
" instance action, no parameter
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    EXECUTE acceptTravel
    FROM VALUE #( ( %tky-TravelUUID = lv_uuid
                    %tky-%is_draft  = if_abap_behv=>mk-off ) )
  RESULT   DATA(action_result)
  FAILED   DATA(failed)
  REPORTED DATA(reported).

" instance action with a parameter
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    EXECUTE deductDiscount
    FROM VALUE #( ( %tky-TravelUUID = lv_uuid
                    %tky-%is_draft  = if_abap_behv=>mk-off
                    %param-discount_percent = 15
                    %param-reason_code      = 'LOYAL' ) )
  RESULT DATA(discount_result).

" static action
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    EXECUTE archiveOldTravels
    FROM VALUE #( ( %param-cutoff_date = '20240101' ) ).

" function
READ ENTITIES OF zr_traveltp
  ENTITY Travel
    EXECUTE getPriceBreakdown
    FROM VALUE #( ( %tky-TravelUUID = lv_uuid %tky-%is_draft = if_abap_behv=>mk-off ) )
  RESULT DATA(breakdown).
```

```
COMMIT ENTITIES.
```

Note: **actions use `MODIFY ENTITIES ... EXECUTE`; functions use `READ ENTITIES ... EXECUTE`.**

13.10 Calling actions over OData

```
POST /sap/opu/odata4/sap/zui_travel_o4/srvd/sap/zsd_travel/0001/
     Travel(TravelUUID=guid'ABC...',IsActiveEntity=true)/
     com.sap.gateway.srvd.zsd_travel.v0001.acceptTravel
Content-Type: application/json
{ }
```

With a parameter:

```
{ "discount_percent": 15, "reason_code": "LOYAL" }
```

The fully qualified action name comes from `$metadata`. If you are building an external integration, read it from `$metadata` rather than constructing it — the namespace segment changes with the service name and version.

13.11 Actions and the save sequence

An action runs in the **interaction phase**. It changes the transactional buffer, not the database.

Consequences:

- After `acceptTravel`, the status is in the buffer. `COMMIT ENTITIES` (or the UI's implicit save) still has to happen.
- Validations declared `on save { field OverallStatus; }` **will** run after your action, and can reject it. That is correct: an action is not above the BO's rules.
- If your action must do something at save time (write a change document, raise an event, call an API), do **not** do it in the action handler. Flag the instances in a `CLASS-DATA` on the global ABP class and act in `save_modified` — see Chapter 16.

13.12 Troubleshooting

Symptom	Cause
Button does not appear	`use action X;` missing in the projection BDEF (check this first); or `dataAction` spelling/case does not match; or browser cache
Button appears but is always greyed out	`( features : instance )` declared and `get_instance_features` returns nothing or `disabled`. Return `if_abap_behv=>fc-o-enabled`
Button works but the screen does not update	No `result [1] $self`, and no `side effects` declaration
"Action ... is not implemented"	Method missing in the behavior pool, or its signature does not match (regenerate with `Ctrl + 1`)
Parameter dialog does not appear	The abstract entity is not activated, or `parameter` is missing in the BDEF, or the abstract entity is not exposed where required
Parameter dialog appears with no fields	The abstract entity has no annotated fields, or the field types are not renderable
Action fails silently for some rows	You filled `failed` but not `reported` — the UI has nothing to display
`%param` is empty in the handler	The BDEF declares no `parameter`, or the consumer did not send one
Factory action creates the instance but the UI does not navigate to it	`mapped` not filled
Action changes data but a validation reverts the save	Correct behaviour — fix the data or the validation

---

13.13 Interview answer

> **"How do you implement an action in RAP?"**
>
> Four steps, and missing any one fails differently. Declare it in the base behavior definition — `action acceptTravel result [1] $self;`, where `result [1] $self` returns the changed instance so the UI repaints without an extra round trip. Implement the handler method, generated with `Ctrl + 1`: read the instances, apply the change with `MODIFY ENTITIES ... IN LOCAL MODE`, fill `result`. Expose it in the projection behavior definition with `use action acceptTravel;`. And annotate it in the metadata extension with `@UI.lineItem: [{ type: #FOR_ACTION, dataAction: 'acceptTravel', ... }]`.
>
> The step people forget is the third — without `use action` the button never appears and there is no error anywhere.
>
> The variants are: `static action` for something not bound to an instance, `factory action` for creating instances, which must fill `mapped` or the UI cannot navigate to the new record, `internal action` for BO-internal mechanisms that are never exposed, and `function`, which is read-only and uses `FOR READ` rather than `FOR MODIFY`.
>
> An action can take a `parameter` typed by a CDS abstract entity, which Fiori renders as a dialog — and you must validate those parameters inside the action, because `validation` only fires on entity fields.
>
> One architectural point: an action runs in the interaction phase and changes the buffer, so the BO's own validations still run afterwards and can reject it. An action is not above the rules.

13.14 Production considerations

- **Prefer `result [1] $self` over a side effect** for refreshing after an action — one round trip instead of two.
- **Choose `invocationGrouping` deliberately.** `#CHANGE_SET` means all selected instances fail together; `#ISOLATED` means they fail independently. For a posting action, all-or-nothing is usually right; for a notification, isolated is friendlier.
- **Re-check business preconditions inside the action.** Feature control only greys out the button; an API caller can still invoke it.
- **Do not put save-time work in the action handler.** Flag the instances and act in `save_modified` (Chapter 16), because the action runs in the interaction phase where the save has not happened.
- **Validate action parameters explicitly** — nothing else does.
- **Mass actions need the same set-based discipline** as determinations: no `SELECT` per key.
- **Deep parameters cannot be rendered by Fiori Elements.** They are for EML and API consumers; use a flat parameter for a UI dialog.

Chapter 14 — Feature Control and Authorization

14.1 Four different mechanisms, often confused

Mechanism	Question it answers	Where declared	Where implemented
DCL / Access Control	Which **rows** may this user *see*?	`@AccessControl.authorizationCheck: #CHECK` + a DCL role	DCL (declarative)
Global authorization	May this user perform this operation **at all**?	`authorization master ( global )`	`FOR GLOBAL AUTHORIZATION`
Instance authorization	May this user change **this specific** instance?	`authorization master ( instance )`	`FOR INSTANCE AUTHORIZATION`
Feature control	Is this field/button **available** right now (business state, not authorization)?	`field ( features : instance )` / `action ( features : instance )`	`FOR INSTANCE FEATURES` / `FOR GLOBAL FEATURES`

The distinction that matters most:

- **Authorization** = "you are not allowed". Enforced. A rejection is a security failure.
- **Feature control** = "this does not apply right now". A UI affordance. An already-accepted travel cannot be accepted again — that is not about permissions.

> Never implement authorization as feature control. Feature control only greys out the button; the OData call still reaches the backend and will succeed. Feature control is **not** a security boundary.

14.2 Read authorization — DCL

```
@EndUserText.label: 'Access control for Travel'
@MappingRole: true
define role ZC_TRAVELTP {
  grant select on ZC_TravelTP
    where ( AgencyId ) = aspect pfcg_auth( ZTRAVEL_AUTH, AGENCY_ID, ACTVT = '03' );
}
```

Clause	Meaning
`define role ZC_TRAVELTP`	The DCL object. Same name as the view is the convention
`@MappingRole: true`	Automatically applies to users holding the PFCG authorization
`grant select on ZC_TravelTP`	Read access to this entity
`where ( AgencyId ) = aspect pfcg_auth( ... )`	Restrict rows to the agencies the user is authorized for
`ZTRAVEL_AUTH`	The authorization object (SU21)
`AGENCY_ID`	The authorization field
`ACTVT = '03'`	Activity 03 = Display

Other DCL constructs

```
" unconditional grant
grant select on ZC_TravelTP where ( AgencyId ) = aspect pfcg_auth( ... )
                               or ( CreatedBy ) = aspect user;

" fixed value restriction
grant select on ZC_TravelTP where ( OverallStatus ) = 'A';

" inherit from another entity's DCL
grant select on ZC_BookingTP where inheriting conditions from entity ZC_TravelTP;
```

`inheriting conditions from entity` is the right pattern for child entities: a Booking is visible if its Travel is visible. Write the rule once, on the root.

Which layer to put the DCL on

There is no single correct layer. The rule belongs wherever the **authorization requirement** belongs, and that follows from your reuse and security architecture.

Option	`ZR_` interface	`ZC_` projection	Use when
Projection-level	`#NOT_REQUIRED`	`#CHECK` + DCL	Different services over the same BO need different row-level restrictions. The most common case for a single custom app family.
Interface-level	`#CHECK` + DCL	`#NOT_REQUIRED` (or `#CHECK` with no extra DCL)	The restriction is an invariant of the data model — no consumer, now or later, may read unrestricted. Typical for sensitive domains (HR, payroll, personal data) and for models you publish for reuse.

Option	`ZR_` interface	`ZC_` projection	Use when
Both layers	`#CHECK` + baseline DCL	`#CHECK` + narrower DCL	A mandatory baseline plus a service-specific narrowing. Conditions from both layers apply, so the effective visibility is the **intersection**.

Trade-offs to weigh:

Consideration	Favours projection-level	Favours interface-level
Several services, different rules	yes	no
A rule that must never be bypassed	no	yes
The model is consumed directly by other developers' views	no	yes
Rule likely to change per app	yes	no
Fewer objects to maintain	no (one DCL per service)	yes (one DCL total)
Easy to reason about "who can see what"	no	yes

The examples in this documentation use projection-level DCL because the running example is a single-app scenario with service-specific filtering. That is a choice, not a rule — if your architecture says the restriction belongs to the reusable model, put the DCL on `ZR_TravelTP` and set `#CHECK` there.

> Whichever layer you choose, **write the decision down** in the package documentation. It is cheap now and expensive to reverse once several services depend on the model.

The limit of DCL

DCL controls `SELECT` only. It does not stop an update, a delete, or an action. A user who can see nothing can still, in principle, `PATCH` an instance whose key they guessed — unless the BDEF authorization stops them. You need both.

14.3 Global authorization

"May this user create Travels at all?"

```
" BDEF
define behavior for ZR_TravelTP alias Travel
...
authorization master ( global )
{
  create ( authorization : global );
  update;
  delete;
}
```

```
METHODS get_global_authorizations FOR GLOBAL AUTHORIZATION
  IMPORTING REQUEST requested_authorizations FOR Travel RESULT result.
```

```
METHOD get_global_authorizations.

  " CREATE
  IF requested_authorizations-%create = if_abap_behv=>mk-on.
    AUTHORITY-CHECK OBJECT 'ZTRAVEL_AUTH'
      ID 'AGENCY_ID' DUMMY
      ID 'ACTVT'     FIELD '01'.
    IF sy-subrc = 0.
      result-%create = if_abap_behv=>auth-allowed.
    ELSE.
      result-%create = if_abap_behv=>auth-unauthorized.
      APPEND VALUE #( %msg = new_message( id       = 'ZTRAVEL_MSG'
                                          number   = '050'
                                          severity = if_abap_behv_message=>severity-error ) )
             TO reported-%other.
    ENDIF.
  ENDIF.

  " UPDATE
  IF requested_authorizations-%update = if_abap_behv=>mk-on.
    AUTHORITY-CHECK OBJECT 'ZTRAVEL_AUTH'
      ID 'AGENCY_ID' DUMMY
      ID 'ACTVT'     FIELD '02'.
    result-%update = COND #( WHEN sy-subrc = 0 THEN if_abap_behv=>auth-allowed
                                               ELSE if_abap_behv=>auth-unauthorized ).
  ENDIF.

  " DELETE
  IF requested_authorizations-%delete = if_abap_behv=>mk-on.
    AUTHORITY-CHECK OBJECT 'ZTRAVEL_AUTH'
      ID 'AGENCY_ID' DUMMY
      ID 'ACTVT'     FIELD '06'.
    result-%delete = COND #( WHEN sy-subrc = 0 THEN if_abap_behv=>auth-allowed
                                               ELSE if_abap_behv=>auth-unauthorized ).
```

```
  ENDIF.

  " ACTIONS
  IF requested_authorizations-%action-acceptTravel = if_abap_behv=>mk-on.
    AUTHORITY-CHECK OBJECT 'ZTRAVEL_AUTH'
      ID 'AGENCY_ID' DUMMY
      ID 'ACTVT'     FIELD '43'.        " 43 = Release / Approve
    result-%action-acceptTravel = COND #( WHEN sy-subrc = 0 THEN if_abap_behv=>auth-allowed
                                                            ELSE if_abap_behv=>auth-unauthorized ).
  ENDIF.

ENDMETHOD.
```

Why the `IF requested_authorizations-%... = mk-on` guards

The framework asks about **only what it needs right now**. Evaluating everything on every call costs `AUTHORITY-CHECK` round trips for no reason. Always guard.

Values

`if_abap_behv=>auth-allowed`	Permitted
`if_abap_behv=>auth-unauthorized`	Not permitted

Standard ACTVT values: `01` create, `02` change, `03` display, `06` delete, `43` release/approve.

14.4 Instance authorization

"May this user change *this* Travel?"

```
" BDEF
authorization master ( instance )
```

```
METHODS get_instance_authorizations FOR INSTANCE AUTHORIZATION
  IMPORTING keys REQUEST requested_authorizations FOR Travel RESULT result.
```

```
METHOD get_instance_authorizations.

  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      FIELDS ( AgencyId CreatedBy )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels)
    FAILED DATA(read_failed).

  DATA(current_user) = cl_abap_context_info=>get_user_technical_name( ).

  LOOP AT travels INTO DATA(travel).

    " one AUTHORITY-CHECK per distinct agency would be better for mass cases;
    " shown per instance here for clarity
    AUTHORITY-CHECK OBJECT 'ZTRAVEL_AUTH'
      ID 'AGENCY_ID' FIELD travel-AgencyId
      ID 'ACTVT'     FIELD '02'.
    DATA(agency_ok) = xsdbool( sy-subrc = 0 ).

    DATA(is_owner) = xsdbool( travel-CreatedBy = current_user ).

    DATA(may_change) = xsdbool( agency_ok = abap_true OR is_owner = abap_true ).

    APPEND VALUE #(
      %tky                      = travel-%tky
      %update                   = COND #( WHEN may_change = abap_true
                                          THEN if_abap_behv=>auth-allowed
                                          ELSE if_abap_behv=>auth-unauthorized )
      %delete                   = COND #( WHEN agency_ok = abap_true
                                          THEN if_abap_behv=>auth-allowed
                                          ELSE if_abap_behv=>auth-unauthorized )
      %action-acceptTravel      = COND #( WHEN agency_ok = abap_true
                                          THEN if_abap_behv=>auth-allowed
                                          ELSE if_abap_behv=>auth-unauthorized )
      %assoc-_Booking           = COND #( WHEN may_change = abap_true
                                          THEN if_abap_behv=>auth-allowed
                                          ELSE if_abap_behv=>auth-unauthorized )
    ) TO result.

  ENDLOOP.
ENDMETHOD.
```

Result component	Controls
`%update`	Updating the instance

Result component	Controls
`%delete`	Deleting it
`%action-<name>`	Executing that action on it
`%assoc-_Booking`	Create-by-association under it

Note there is no `%create` in instance authorization — creating an instance that does not exist yet cannot be instance-dependent. That is what global authorization is for.

Performance

Instance authorization is called for **every instance the user touches**, including every row the list report displays if the UI asks about permissions. For a mass scenario:

```
" collect distinct agencies, check once each, then map
DATA(agencies) = travels.
SORT agencies BY AgencyId.
DELETE ADJACENT DUPLICATES FROM agencies COMPARING AgencyId.

DATA authorized_agencies TYPE SORTED TABLE OF /dmo/agency_id WITH UNIQUE KEY table_line.
LOOP AT agencies INTO DATA(agency).
  AUTHORITY-CHECK OBJECT 'ZTRAVEL_AUTH' ID 'AGENCY_ID' FIELD agency-AgencyId ID 'ACTVT' FIELD '02'.
  IF sy-subrc = 0.
    INSERT agency-AgencyId INTO TABLE authorized_agencies.
  ENDIF.
ENDLOOP.
" then use line_exists( authorized_agencies[ table_line = travel-AgencyId ] )
```

14.5 Feature control — static

Declared in the BDEF and never changes:

```
field ( readonly ) TotalPrice, OverallStatus, CreatedBy, CreatedAt;
field ( mandatory ) AgencyId, CustomerId, BeginDate, EndDate;
field ( readonly : update ) CurrencyCode;      " set on create, frozen afterwards
```

No code. The framework enforces it and the UI renders accordingly.

14.6 Feature control — dynamic (instance features)

"An accepted Travel cannot be accepted again, and its dates can no longer be changed."

```
" BDEF
define behavior for ZR_TravelTP alias Travel
...
{
  update;
  delete;

  field ( features : instance ) BeginDate, EndDate, BookingFee;

  action ( features : instance ) acceptTravel result [1] $self;
  action ( features : instance ) rejectTravel result [1] $self;

  association _Booking { create ( features : instance ); with draft; }
}
```

```
METHODS get_instance_features FOR INSTANCE FEATURES
  IMPORTING keys REQUEST requested_features FOR Travel RESULT result.
```

```
METHOD get_instance_features.

  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      FIELDS ( OverallStatus BeginDate )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels)
    FAILED DATA(read_failed).

  result = VALUE #(
    FOR travel IN travels
    LET is_accepted = COND #( WHEN travel-OverallStatus = zbp_r_traveltp=>c_status-accepted
                              THEN if_abap_behv=>fc-o-disabled
                              ELSE if_abap_behv=>fc-o-enabled )
        is_rejected = COND #( WHEN travel-OverallStatus = zbp_r_traveltp=>c_status-rejected
                              THEN if_abap_behv=>fc-o-disabled
                              ELSE if_abap_behv=>fc-o-enabled )
        locked_flds = COND #( WHEN travel-OverallStatus <> zbp_r_traveltp=>c_status-open
                              THEN if_abap_behv=>fc-f-read_only
                              ELSE if_abap_behv=>fc-f-unrestricted )
```

```
    IN
    ( %tky                  = travel-%tky

      " operations
      %update               = COND #( WHEN travel-OverallStatus = zbp_r_traveltp=>c_status-rejected
                                      THEN if_abap_behv=>fc-o-disabled
                                      ELSE if_abap_behv=>fc-o-enabled )
      %delete               = COND #( WHEN travel-OverallStatus = zbp_r_traveltp=>c_status-open
                                      THEN if_abap_behv=>fc-o-enabled
                                      ELSE if_abap_behv=>fc-o-disabled )

      " actions
      %action-acceptTravel  = is_accepted
      %action-rejectTravel  = is_rejected

      " create-by-association
      %assoc-_Booking       = COND #( WHEN travel-OverallStatus = zbp_r_traveltp=>c_status-open
                                      THEN if_abap_behv=>fc-o-enabled
                                      ELSE if_abap_behv=>fc-o-disabled )

      " fields
      %field-BeginDate      = locked_flds
      %field-EndDate        = locked_flds
      %field-BookingFee     = locked_flds
    ) ).

ENDMETHOD.
```

The two constant families — do not mix them

For **operations, actions, associations** (`%update` , `%delete` , `%action-X` , `%assoc-_A`)	
`if_abap_behv=>fc-o-enabled`	Available
`if_abap_behv=>fc-o-disabled`	Greyed out

For **fields** (`%field-X`)	
`if_abap_behv=>fc-f-unrestricted`	Editable
`if_abap_behv=>fc-f-read_only`	Display only
`if_abap_behv=>fc-f-mandatory`	Must be filled

Using an `fc-o-` constant on a `%field-` (or vice versa) compiles and then behaves unpredictably. `strict ( 2 )` catches most of these; it is still worth being careful.

Global features

For operations that are disabled for everyone under some condition (a system-wide switch, a closed period):

```
" BDEF
create ( features : global );
```

```
METHODS get_global_features FOR GLOBAL FEATURES
  IMPORTING REQUEST requested_features FOR Travel RESULT result.
```

```
METHOD get_global_features.
  result-%create = COND #( WHEN zcl_travel_config=>is_period_open( )
                           THEN if_abap_behv=>fc-o-enabled
                           ELSE if_abap_behv=>fc-o-disabled ).
ENDMETHOD.
```

Refreshing feature control on screen

When a field change should re-evaluate button enablement, declare it:

```
side effects
{
  field OverallStatus affects permissions;
}
```

Without this, the user accepts a Travel and the *Accept* button stays enabled until they refresh.

14.7 Precheck

A `precheck` handler runs **before** the data enters the transactional buffer.

```
" BDEF
update ( precheck );
```

```
delete ( precheck );
association _Booking { create ( precheck ); with draft; }
```

```
METHODS precheck_update FOR PRECHECK IMPORTING entities FOR UPDATE Travel.
```

```
METHOD precheck_update.

  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel FIELDS ( OverallStatus ) WITH CORRESPONDING #( entities )
    RESULT DATA(travels).

  LOOP AT entities INTO DATA(entity).
    DATA(travel) = VALUE #( travels[ %tky = entity-%tky ] OPTIONAL ).

    IF travel-OverallStatus = zbp_r_traveltp=>c_status-rejected.
      APPEND VALUE #( %tky = entity-%tky ) TO failed-travel.
      APPEND VALUE #( %tky = entity-%tky
                      %msg = new_message( id = 'ZTRAVEL_MSG' number = '060'
                                          severity = if_abap_behv_message=>severity-error ) )
             TO reported-travel.
    ENDIF.
  ENDLOOP.
ENDMETHOD.
```

Precheck vs validation

	Precheck	Validation
Runs	Before the change enters the buffer	After (interaction) or at save
Sees	The **incoming request** (`entities` , with `%control`)	The **buffer state** (keys only; you read)
Prevents the change from happening	Yes	No — it rejects the *save*
Typical use	Reject a bad request early; check the *delta* being sent	Check the final document state

Use precheck when the incoming values themselves are the problem (e.g. "you may not change `CurrencyCode` on a posted document"). Use validation for document-level consistency.

14.8 The complete authorization picture for our example

Concern	Mechanism	Object
Which travels can a user see in the list?	DCL	`ZC_TRAVELTP` role, `pfcg_auth( ZTRAVEL_AUTH, AGENCY_ID, ACTVT='03' )`
May the user create any travel?	Global authorization	`get_global_authorizations` , ACTVT `01`
May the user change *this* travel?	Instance authorization	`get_instance_authorizations` , ACTVT `02` on the row's `AgencyId`
May the user accept *this* travel?	Instance authorization	`%action-acceptTravel` , ACTVT `43`
Can an accepted travel be accepted again?	Feature control	`%action-acceptTravel = fc-o-disabled`
Are the dates editable after acceptance?	Feature control	`%field-BeginDate = fc-f-read_only`
Can `TotalPrice` ever be typed by a user?	Static field control	`field ( readonly ) TotalPrice;`

The authorization object (SU21)

```
Object      : ZTRAVEL_AUTH
Class       : Z_CUSTOM
Fields      : AGENCY_ID  (referencing data element /DMO/AGENCY_ID)
              ACTVT      (ACTVT, allowed values 01, 02, 03, 06, 43)
```

Then a PFCG role with `ZTRAVEL_AUTH` and the relevant agency values, assigned to the user.

14.9 End-to-end worked scenario

One requirement, implemented across all four mechanisms, so the division of labour is concrete.

The requirement

> A travel agent may only work with travels of the agencies they are assigned to. Within those agencies: a **clerk** may create and change travels but not delete or accept them; a **supervisor** may do everything. Nobody may change a travel that has been rejected. A user with no assignment sees nothing at all.

The authorization object (SU21)

```
Object : ZTRAVEL_AUTH
Fields : AGENCY_ID   (data element /DMO/AGENCY_ID)
         ACTVT       (values 01 create, 02 change, 03 display, 06 delete, 43 release)
```

Two PFCG roles:

Role	`AGENCY_ID`	`ACTVT`
`Z_TRAVEL_CLERK`	assigned agencies	`01` , `02` , `03`
`Z_TRAVEL_SUPERVISOR`	assigned agencies	`01` , `02` , `03` , `06` , `43`

Step 1 — DCL: which rows are visible

```
@EndUserText.label: 'Access control for Travel'
@MappingRole: true
define role ZC_TRAVELTP {
  grant select on ZC_TravelTP
    where ( AgencyId ) = aspect pfcg_auth( ZTRAVEL_AUTH, AGENCY_ID, ACTVT = '03' );
}
```

```
@EndUserText.label: 'Access control for Booking'
@MappingRole: true
define role ZC_BOOKINGTP {
  grant select on ZC_BookingTP
    where inheriting conditions from entity ZC_TravelTP;
}
```

A user with no `ZTRAVEL_AUTH` values gets an empty list — correctly, and without any ABAP running.

Step 2 — BDEF declarations

```
define behavior for ZR_TravelTP alias Travel
persistent table ztravel
draft table ztravel_d
lock master
total etag LastChangedAt
authorization master ( global, instance )
etag master LocalLastChangedAt
{
  create ( authorization : global );
  update ( authorization : instance, features : instance );
  delete ( authorization : instance, features : instance );

  association _Booking { create ( authorization : instance ); with draft; }

  action ( authorization : instance, features : instance ) acceptTravel result [1] $self;
  action ( authorization : instance, features : instance ) rejectTravel result [1] $self;

  field ( features : instance ) BeginDate, EndDate, BookingFee;
  ...
}

define behavior for ZR_BookingTP alias Booking
...
authorization dependent by _Travel
{ ... }
```

Note `create ( authorization : global )` — creating an instance that does not exist yet cannot be instance-dependent.

Step 3 — global authorization: may this user create at all?

```
METHOD get_global_authorizations.

  IF requested_authorizations-%create = if_abap_behv=>mk-on.
    AUTHORITY-CHECK OBJECT 'ZTRAVEL_AUTH'
      ID 'AGENCY_ID' DUMMY
      ID 'ACTVT'     FIELD '01'.
    IF sy-subrc = 0.
      result-%create = if_abap_behv=>auth-allowed.
    ELSE.
      result-%create = if_abap_behv=>auth-unauthorized.
      APPEND VALUE #( %msg = new_message( id       = 'ZTRAVEL_MSG'
                                          number   = '050'    " 'Not authorized to create travels'
                                          severity = if_abap_behv_message=>severity-error ) )
             TO reported-%other.
    ENDIF.
  ENDIF.

ENDMETHOD.
```

`DUMMY` on `AGENCY_ID` means "any value" — at create time no agency is known yet, so the check is "may this user create for *some* agency". The agency-specific check happens on the subsequent update.

> The `IF requested_authorizations-%... = mk-on` guard matters: the framework asks about only what it needs, and evaluating everything on every call wastes `AUTHORITY-CHECK` round trips.

Step 4 — instance authorization: may this user touch *this* travel?

```
METHOD get_instance_authorizations.

  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      FIELDS ( AgencyId OverallStatus )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels)
    FAILED DATA(read_failed).

  " --- one AUTHORITY-CHECK per DISTINCT agency, not per row ---------------
  DATA(agencies) = travels.
  SORT agencies BY AgencyId.
  DELETE ADJACENT DUPLICATES FROM agencies COMPARING AgencyId.

  DATA: may_change TYPE HASHED TABLE OF /dmo/agency_id WITH UNIQUE KEY table_line,
        may_delete TYPE HASHED TABLE OF /dmo/agency_id WITH UNIQUE KEY table_line,
        may_accept TYPE HASHED TABLE OF /dmo/agency_id WITH UNIQUE KEY table_line.

  LOOP AT agencies INTO DATA(agency).

    IF requested_authorizations-%update              = if_abap_behv=>mk-on
    OR requested_authorizations-%assoc-_Booking      = if_abap_behv=>mk-on.
      AUTHORITY-CHECK OBJECT 'ZTRAVEL_AUTH'
        ID 'AGENCY_ID' FIELD agency-AgencyId
        ID 'ACTVT'     FIELD '02'.
      IF sy-subrc = 0. INSERT agency-AgencyId INTO TABLE may_change. ENDIF.
    ENDIF.

    IF requested_authorizations-%delete = if_abap_behv=>mk-on.
      AUTHORITY-CHECK OBJECT 'ZTRAVEL_AUTH'
        ID 'AGENCY_ID' FIELD agency-AgencyId
        ID 'ACTVT'     FIELD '06'.
      IF sy-subrc = 0. INSERT agency-AgencyId INTO TABLE may_delete. ENDIF.
    ENDIF.

    IF requested_authorizations-%action-acceptTravel = if_abap_behv=>mk-on
    OR requested_authorizations-%action-rejectTravel = if_abap_behv=>mk-on.
      AUTHORITY-CHECK OBJECT 'ZTRAVEL_AUTH'
        ID 'AGENCY_ID' FIELD agency-AgencyId
        ID 'ACTVT'     FIELD '43'.
      IF sy-subrc = 0. INSERT agency-AgencyId INTO TABLE may_accept. ENDIF.
    ENDIF.

  ENDLOOP.

  " --- map the results onto the instances ---------------------------------
  LOOP AT travels INTO DATA(travel).

    DATA(chg) = COND #( WHEN line_exists( may_change[ table_line = travel-AgencyId ] )
                        THEN if_abap_behv=>auth-allowed ELSE if_abap_behv=>auth-unauthorized ).
    DATA(del) = COND #( WHEN line_exists( may_delete[ table_line = travel-AgencyId ] )
                        THEN if_abap_behv=>auth-allowed ELSE if_abap_behv=>auth-unauthorized ).
    DATA(acc) = COND #( WHEN line_exists( may_accept[ table_line = travel-AgencyId ] )
                        THEN if_abap_behv=>auth-allowed ELSE if_abap_behv=>auth-unauthorized ).

    APPEND VALUE #( %tky                = travel-%tky
                    %update             = chg
                    %delete             = del
                    %assoc-_Booking     = chg
                    %action-acceptTravel = acc
                    %action-rejectTravel = acc ) TO result.

    " optional: explain the refusal, so the user is not left guessing
    IF chg = if_abap_behv=>auth-unauthorized.
      APPEND VALUE #( %tky = travel-%tky
                      %msg = new_message( id       = 'ZTRAVEL_MSG'
                                          number   = '051'  " 'Not authorized for agency &1'
                                          severity = if_abap_behv_message=>severity-error
                                          v1       = travel-AgencyId ) ) TO reported-travel.
    ENDIF.

  ENDLOOP.

ENDMETHOD.
```

Step 5 — feature control: business state, not permission

```
METHOD get_instance_features.

  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      FIELDS ( OverallStatus )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  result = VALUE #(
```

```
    FOR travel IN travels
    LET is_rejected = xsdbool( travel-OverallStatus = zbp_r_traveltp=>c_status-rejected )
        is_open     = xsdbool( travel-OverallStatus = zbp_r_traveltp=>c_status-open )
    IN
    ( %tky = travel-%tky

      " a rejected travel is frozen - nothing to do with permissions
      %update              = COND #( WHEN is_rejected = abap_true
                                     THEN if_abap_behv=>fc-o-disabled
                                     ELSE if_abap_behv=>fc-o-enabled )
      %delete              = COND #( WHEN is_open = abap_true
                                     THEN if_abap_behv=>fc-o-enabled
                                     ELSE if_abap_behv=>fc-o-disabled )

      " an accepted travel cannot be accepted again
      %action-acceptTravel = COND #( WHEN travel-OverallStatus = zbp_r_traveltp=>c_status-accepted
                                     THEN if_abap_behv=>fc-o-disabled
                                     ELSE if_abap_behv=>fc-o-enabled )
      %action-rejectTravel = COND #( WHEN is_rejected = abap_true
                                     THEN if_abap_behv=>fc-o-disabled
                                     ELSE if_abap_behv=>fc-o-enabled )

      " dates freeze once the travel leaves 'open'
      %field-BeginDate     = COND #( WHEN is_open = abap_true
                                     THEN if_abap_behv=>fc-f-unrestricted
                                     ELSE if_abap_behv=>fc-f-read_only )
      %field-EndDate       = COND #( WHEN is_open = abap_true
                                     THEN if_abap_behv=>fc-f-unrestricted
                                     ELSE if_abap_behv=>fc-f-read_only )
    ) ).

ENDMETHOD.
```

Note that "nobody may change a **rejected** travel" is implemented **here**, not in authorization — it is a state rule that applies to everyone, including supervisors. Authorization answers "*may you?*"; feature control answers "*does it apply right now?*".

Step 6 — rejecting with `failed` and `reported`

The authorization and feature handlers return a `result`; the framework turns an `auth-unauthorized` into a rejection for you. Where **you** must reject explicitly — in an action, a precheck or a validation — fill both response parameters and set a cause:

```
METHOD acceptTravel.

  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      FIELDS ( AgencyId OverallStatus )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  DATA update_lines TYPE TABLE FOR UPDATE zr_traveltp.

  LOOP AT travels INTO DATA(travel).

    " defence in depth: feature control is a UI hint, an API caller can still get here
    IF travel-OverallStatus = zbp_r_traveltp=>c_status-rejected.
      APPEND VALUE #( %tky        = travel-%tky
                      %fail-cause = if_abap_behv=>cause-disabled ) TO failed-travel.
      APPEND VALUE #( %tky = travel-%tky
                      %msg = new_message( id       = 'ZTRAVEL_MSG'
                                          number   = '052'   " 'Rejected travel cannot be accepted'
                                          severity = if_abap_behv_message=>severity-error ) )
             TO reported-travel.
      CONTINUE.
    ENDIF.

    AUTHORITY-CHECK OBJECT 'ZTRAVEL_AUTH'
      ID 'AGENCY_ID' FIELD travel-AgencyId
      ID 'ACTVT'     FIELD '43'.
    IF sy-subrc <> 0.
      APPEND VALUE #( %tky        = travel-%tky
                      %fail-cause = if_abap_behv=>cause-unauthorized ) TO failed-travel.
      APPEND VALUE #( %tky = travel-%tky
                      %msg = new_message( id       = 'ZTRAVEL_MSG'
                                          number   = '053'
                                          severity = if_abap_behv_message=>severity-error
                                          v1       = travel-AgencyId ) ) TO reported-travel.
      CONTINUE.
    ENDIF.

    APPEND VALUE #( %tky          = travel-%tky
                    OverallStatus = zbp_r_traveltp=>c_status-accepted ) TO update_lines.
  ENDLOOP.

  CHECK update_lines IS NOT INITIAL.

  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel UPDATE FIELDS ( OverallStatus ) WITH update_lines
    REPORTED DATA(mod_reported).
  reported = CORRESPONDING #( DEEP mod_reported ).

  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel ALL FIELDS WITH CORRESPONDING #( keys )
    RESULT DATA(travels_after).
```

```
    result = VALUE #( FOR t IN travels_after ( %tky = t-%tky %param = t ) ).

ENDMETHOD.
```

Re-checking the authorization inside the action is **defence in depth**. `( authorization : instance )` in the BDEF already makes the framework call `get_instance_authorizations`, so in normal operation this branch never fires — but it costs little and it documents the rule at the point of use.

What each user actually experiences

User	List	Create	Change own agency	Delete	Accept	Change a rejected travel
No `ZTRAVEL_AUTH`	**empty** (DCL)	refused (global auth)	—	—	—	—
Clerk, agency 070001	only 070001 rows (DCL)	allowed	allowed (instance auth)	button disabled (instance auth)	button disabled (instance auth)	disabled (feature control)
Clerk, viewing 070002	not visible (DCL)	—	—	—	—	—
Supervisor, agency 070001	only 070001 rows (DCL)	allowed	allowed	allowed	allowed	**disabled** (feature control applies to everyone)

DCL versus behavior authorization, in one sentence each

- **DCL** decided that the clerk's list contains only agency 070001 — a `WHERE` clause on every read.
- **Behavior authorization** decided that the clerk may change but not delete those rows — checked when a modification is attempted, whatever the client.
- **Feature control** decided that nobody edits a rejected travel — a state rule, and only a UI hint.

A user who can see nothing through DCL could still, in principle, `PATCH` an instance whose key they guessed. Only the behavior authorization stops them. That is why you need both.

14.10 Troubleshooting

Symptom	Cause
User sees no data at all	DCL filters everything; check the PFCG values and `@MappingRole: true`
User sees data but cannot edit anything	Global or instance authorization returns `unauthorized`; or feature control marks everything read-only
Button greyed out for everyone	`get_instance_features` does not set `%action-X`, so it defaults to disabled
Button enabled for everyone despite the status	`( features : instance )` missing in the BDEF
Button enablement does not refresh after a change	`field X affects permissions;` missing in `side effects`
Feature control works in the UI but EML bypasses it	Correct — `IN LOCAL MODE` deliberately skips feature control and authorization
Authorization check runs far too often	Not guarding with `IF requested_authorizations-%... = mk-on`
DCL has no effect	`@AccessControl.authorizationCheck` is `#NOT_REQUIRED` on the projection
Child rows visible although the parent is not	Child DCL missing; use `inheriting conditions from entity`

14.11 Checklist

- [ ] DCL exists on the **projection** with `#CHECK`; child inherits with `inheriting conditions from entity`.
- [ ] `authorization master ( global, instance )` on the root; `dependent by` on children.
- [ ] `get_global_authorizations` and `get_instance_authorizations` guard on `requested_authorizations`.
- [ ] Authorization returns `auth-allowed` / `auth-unauthorized`, never the `fc-` constants.
- [ ] Feature control uses `fc-o-` for operations and `fc-f-` for fields.
- [ ] `side effects ... affects permissions` declared where state drives enablement.
- [ ] No security rule is implemented **only** as feature control.
- [ ] `AUTHORITY-CHECK` is not called once per row in mass scenarios.

14.12 Interview answer

> **"How do you handle authorization in RAP?"**
>
> Four mechanisms, and the point is that they are not interchangeable.
>
> **DCL** — CDS access control — filters **reads**. A role with `aspect pfcg_auth(...)` becomes a `WHERE` clause on every select. It does not stop an update or a delete.
>
> **Global authorization**, `FOR GLOBAL AUTHORIZATION`, answers "may this user create at all" — instance-independent.
>
> **Instance authorization**, `FOR INSTANCE AUTHORIZATION`, answers "may this user change *this* instance" — typically an `AUTHORITY-CHECK` on a field of the row, returning `auth-allowed` or `auth-unauthorized` per operation, action and association.
>
> **Feature control** is different in kind: it answers "does this apply right now" — an accepted document cannot be accepted again. It uses the `fc-o-` and `fc-f-` constants and it is a **UI hint only**. The OData call still reaches the backend and still succeeds, so a security rule must never be implemented as feature control.
>
> The combination matters: a user who can see nothing through DCL could still `PATCH` an instance whose key they guessed, unless behavior authorization stops them. You need both.
>
> On performance — instance authorization runs for every row the list displays, so I do one `AUTHORITY-CHECK` per distinct authorization value and then a hashed lookup, and guard on `requested_authorizations` so only what the framework asked about is evaluated.

14.13 Production considerations

- **Test with real PFCG roles, never as a developer.** Authorization defects are the single most common "it worked in DEV" failure, and a developer with wide authorizations will never see them.
- **One test user per role in QA**, with a documented expected-result matrix (§14.9).
- **DCL depends on the role existing and being assigned in that system.** A DCL that filters everything in QA is almost always a missing role assignment, not a code defect.
- **Roles are usually a separate transport** and often a separate team. Plan the hand-off.
- **Do not use `PRIVILEGED` casually.** Where a job needs it, document why, and prefer `GET PERMISSIONS` to filter what the job may touch.
- **Authorization is a hot path.** Set-based `AUTHORITY-CHECK` and `requested_...` guards are performance requirements, not style.
- **Write down the DCL layer decision** (§14.2) — it is hard to reverse once several services depend on the model.

Chapter 15 — The Unmanaged Scenario

Status of the code in this chapter

The ABAP in this chapter is conceptual reference code. It has not been activated or executed in an SAP system. It shows the shape of an unmanaged implementation — which methods exist, what each is responsible for, and how the pieces fit — and the structure is sound. The exact syntax of individual statements must be validated in your target release before use.

Unmanaged RAP is the part of the model with the most hand-written framework-facing code, so it is also where small syntax differences bite hardest. Before adopting any example here, recheck:

Recheck	Why
`FOR READ` , `FOR MODIFY` , `FOR LOCK` method signatures	Generate them with `Ctrl + 1` from the BDEF rather than copying; the generated signature is authoritative for your release
Read-by-association (`rba_*`) and create-by-association (`cba_*`) signatures	`FULL result_requested` , `LINK association_links` , `%target` handling all have exact forms
`%tky` construction when building key tables by hand	Component names and whether `%is_draft` applies at all
`CORRESPONDING ... MAPPING FROM ENTITY` / `MAPPING TO ENTITY`	Depends on a correct `mapping for` block; behaviour differs with `control` and `corresponding`
`mapped` structures for create and create-by-association	Component names are the entity **aliases**, lower case
Buffer access from the saver class	The saver is a *separate local class* from the handler — verify the visibility of whatever you share, and that singleton state is reachable and correctly reset
Lock scope, `_scope` parameter and where `DEQUEUE` belongs	Lock-object generation and scope semantics are not RAP-specific but are easy to get wrong here
Error handling after the point of no return	Whether you need `CL_ABAP_BEHAVIOR_SAVER_FAILED` rather than `CL_ABAP_BEHAVIOR_SAVER`
Visibility sections of helper methods and types	Handler methods are `PRIVATE` , saver methods `PROTECTED` ; helper classes and types have their own rules

The **managed** chapters (08–14, 16) follow documented declarative syntax that was verified against SAP's published material; this chapter is the one to treat with the most caution.

The fastest way to get a known-good unmanaged implementation is to open the **ABAP Flight Reference Scenario** (`/DMO/` packages, shipped with the system) — it contains a complete, activated unmanaged BO for exactly this data model.

15.1 When unmanaged is the right answer

Unmanaged means: **you provide everything**. The transactional buffer, the read, the create, the update, the delete, the lock, the save. RAP provides only the orchestration and the protocol.

Choose unmanaged when **all** of these are true:

- There is existing transactional logic (function modules, BAPIs, update-task modules) that you must reuse rather than reimplement.
- That logic owns its own locking and its own persistence.
- Reimplementing it in a managed BO would be a rewrite with regression risk.

Do **not** choose unmanaged because:

Reason people give	Why it is wrong
"We already have tables"	Managed works on existing tables. `persistent table` accepts any table.
"The data model is complex"	Composition trees are a managed strength.
"We need custom validation logic"	Managed supports validations, determinations, actions.
"We need to call an API on save"	That is `with additional save` or `with unmanaged save` — Chapter 16, far less work.
"The key is not a UUID"	Managed supports semantic keys and external numbering.

If the only unmanaged thing you need is the save, use `managed with unmanaged save` (Chapter 16), not a full unmanaged BO. That is the single most useful piece of advice in this chapter — it saves several hundred lines.

15.2 What you have to implement

Handler	Mandatory	Purpose
`FOR READ`	Yes	Read instances from your buffer / persistence
`FOR MODIFY ... FOR CREATE`	If create is enabled	Create in the buffer
`FOR MODIFY ... FOR UPDATE`	If update is enabled	Update in the buffer
`FOR MODIFY ... FOR DELETE`	If delete is enabled	Mark for deletion in the buffer
`FOR LOCK`	Yes (unless `lock master unmanaged` is omitted)	Acquire the lock
`FOR READ ... \_Child` (rba)	If there are children	Read by association
`FOR MODIFY ... FOR CREATE ...\_Child` (cba)	If create-by-association is enabled	Create children
Saver `finalize`	Optional	Final adjustments
Saver `check_before_save`	Optional but recommended	Last consistency gate
Saver `save`	Yes	Persist the buffer
Saver `cleanup`	Yes	Clear the buffer
Saver `cleanup_finalize`	Yes	Clear the buffer after an early-save failure
Saver `adjust_numbers`	Only with `late numbering`	Assign final keys

Plus: the transactional buffer itself, which you design.

15.3 The BDEF

```
unmanaged implementation in class ZBP_R_TravelTP_U unique;
strict ( 2 );

define behavior for ZR_TravelTP_U alias Travel
implementation in class ZBP_R_TravelTP_U unique
lock master
authorization master ( instance )
etag master LastChangedAt
{
  create;
  update;
  delete;

  field ( readonly ) TravelUUID, CreatedBy, CreatedAt, LastChangedBy, LastChangedAt;
  field ( mandatory ) AgencyId, CustomerId, BeginDate, EndDate;

  association _Booking { create; }

  action acceptTravel result [1] $self;

  mapping for ztravel
  {
    TravelUUID = travel_uuid;
    TravelId   = travel_id;
    ...
  }
}

define behavior for ZR_BookingTP_U alias Booking
implementation in class ZBP_R_TravelTP_U unique
lock dependent by _Travel
authorization dependent by _Travel
{
  update;
  delete;
  field ( readonly ) BookingUUID, TravelUUID;
  association _Travel;
}
```

Differences from a managed BDEF

	Managed	Unmanaged
Header	`managed implementation in class ...`	`unmanaged implementation in class ...`
`persistent table`	Required	**Not present** — there is no framework save
`draft table`	For draft	Draft in unmanaged is possible but significantly more work
`field ( numbering : managed )`	Available	**Not available** — you assign keys
`lock master`	Framework locks	You implement `FOR LOCK`

	Managed	Unmanaged
`mapping for`	Used by the framework's save	Only useful if you use `CORRESPONDING ... MAPPING FROM ENTITY` yourself

15.4 The transactional buffer

Everything hinges on this. The buffer holds the state of the transaction between the interaction phase and the save.

```
CLASS lcl_buffer DEFINITION CREATE PRIVATE.

  PUBLIC SECTION.

    TYPES: BEGIN OF ty_travel_buffer,
             travel     TYPE ztravel,          " the data
             changed    TYPE abap_boolean,     " was it modified?
             created    TYPE abap_boolean,     " is it new?
             deleted    TYPE abap_boolean,     " marked for deletion?
             control    TYPE ztravel_x,        " which fields were sent (x-structure)
           END OF ty_travel_buffer,
           tt_travel_buffer TYPE SORTED TABLE OF ty_travel_buffer
                            WITH UNIQUE KEY travel-travel_uuid.

    TYPES: BEGIN OF ty_booking_buffer,
             booking TYPE zbooking,
             changed TYPE abap_boolean,
             created TYPE abap_boolean,
             deleted TYPE abap_boolean,
             control TYPE zbooking_x,
           END OF ty_booking_buffer,
           tt_booking_buffer TYPE SORTED TABLE OF ty_booking_buffer
                             WITH UNIQUE KEY booking-booking_uuid.

    CLASS-METHODS get_instance RETURNING VALUE(r_result) TYPE REF TO lcl_buffer.

    METHODS: load_travels  IMPORTING it_uuids TYPE ztravel_uuid_tab,
             get_travels   RETURNING VALUE(rt_result) TYPE tt_travel_buffer,
             get_bookings  RETURNING VALUE(rt_result) TYPE tt_booking_buffer,
             upsert_travel IMPORTING is_travel TYPE ty_travel_buffer,
             clear.

    DATA: mt_travels  TYPE tt_travel_buffer,
          mt_bookings TYPE tt_booking_buffer.

  PRIVATE SECTION.
    CLASS-DATA mo_instance TYPE REF TO lcl_buffer.
ENDCLASS.

CLASS lcl_buffer IMPLEMENTATION.

  METHOD get_instance.
    IF mo_instance IS NOT BOUND.
      mo_instance = NEW #( ).
    ENDIF.
    r_result = mo_instance.
  ENDMETHOD.

  METHOD load_travels.
    " Load only what is not already buffered. The buffer wins over the DB.
    DATA(lt_missing) = it_uuids.
    LOOP AT it_uuids INTO DATA(lv_uuid).
      IF line_exists( mt_travels[ travel-travel_uuid = lv_uuid ] ).
        DELETE lt_missing WHERE table_line = lv_uuid.
      ENDIF.
    ENDLOOP.
    CHECK lt_missing IS NOT INITIAL.

    SELECT * FROM ztravel
      FOR ALL ENTRIES IN @lt_missing
      WHERE travel_uuid = @lt_missing-table_line
      INTO TABLE @DATA(lt_db).

    LOOP AT lt_db INTO DATA(ls_db).
      INSERT VALUE #( travel = ls_db ) INTO TABLE mt_travels.
    ENDLOOP.
  ENDMETHOD.

  METHOD get_travels.   rt_result = mt_travels.  ENDMETHOD.
  METHOD get_bookings.  rt_result = mt_bookings. ENDMETHOD.

  METHOD upsert_travel.
    DELETE mt_travels WHERE travel-travel_uuid = is_travel-travel-travel_uuid.
    INSERT is_travel INTO TABLE mt_travels.
  ENDMETHOD.

  METHOD clear.
    CLEAR: mt_travels, mt_bookings.
  ENDMETHOD.

ENDCLASS.
```

Three non-negotiable buffer rules

1. **The buffer always wins over the database.** `load_travels` must not overwrite an instance that is already in the buffer — the buffered copy carries the user's uncommitted changes.
2. **The buffer must be cleared in `cleanup` and `cleanup_finalize`.** Otherwise the next transaction in the same session sees stale data. This is the most common unmanaged bug.
3. **Track the operation per instance** (`created` / `changed` / `deleted`) and the **field control** (`control`). Without the x-structure you cannot do a field-selective update, and a blanket `MODIFY` will overwrite fields the consumer never sent with initial values.

15.5 The handler methods

`FOR READ`

```
METHOD read.

  DATA(lo_buffer) = lcl_buffer=>get_instance( ).
  lo_buffer->load_travels( VALUE #( FOR k IN keys ( k-TravelUUID ) ) ).

  LOOP AT keys INTO DATA(key).
    DATA(ls_buffered) = VALUE #( lo_buffer->mt_travels[ travel-travel_uuid = key-TravelUUID ]
                                 OPTIONAL ).

    IF ls_buffered IS INITIAL OR ls_buffered-deleted = abap_true.
      APPEND VALUE #( %tky        = key-%tky
                      %fail-cause = if_abap_behv=>cause-not_found ) TO failed-travel.
      CONTINUE.
    ENDIF.

    APPEND CORRESPONDING #( ls_buffered-travel MAPPING TO ENTITY ) TO result
           ASSIGNING FIELD-SYMBOL(<result>).
    <result>-%tky = key-%tky.
  ENDLOOP.
ENDMETHOD.
```

`CORRESPONDING #( ... MAPPING TO ENTITY )` uses the `mapping for` block in the BDEF to translate DDIC field names to CDS element names. `MAPPING FROM ENTITY` goes the other way. These two RAP-specific additions to `CORRESPONDING` are what make `mapping for` worth writing.

> **Respect `%control`.** A well-behaved `read` returns only the fields the consumer requested (`keys-%control`). Returning everything works but is wasteful on wide entities.

`FOR MODIFY ... FOR CREATE`

```
METHOD create.

  DATA(lo_buffer) = lcl_buffer=>get_instance( ).

  LOOP AT entities INTO DATA(entity).

    " Unmanaged: YOU assign the key
    TRY.
        DATA(lv_uuid) = cl_system_uuid=>create_uuid_x16_static( ).
      CATCH cx_uuid_error.
        APPEND VALUE #( %cid = entity-%cid ) TO failed-travel.
        CONTINUE.
    ENDTRY.

    DATA(ls_travel) = CORRESPONDING ztravel( entity MAPPING FROM ENTITY ).
    ls_travel-travel_uuid     = lv_uuid.
    ls_travel-created_by      = cl_abap_context_info=>get_user_technical_name( ).
    GET TIME STAMP FIELD ls_travel-created_at.
    ls_travel-last_changed_by = ls_travel-created_by.
    ls_travel-last_changed_at = ls_travel-created_at.

    lo_buffer->upsert_travel( VALUE #( travel  = ls_travel
                                       created = abap_true ) ).

    " report the assigned key back to the consumer
    APPEND VALUE #( %cid       = entity-%cid
                    TravelUUID = lv_uuid ) TO mapped-travel.
  ENDLOOP.
ENDMETHOD.
```

Filling `mapped` is mandatory. The consumer sent a `%cid` and has no other way to learn the key.

`FOR MODIFY ... FOR UPDATE`

```
METHOD update.

  DATA(lo_buffer) = lcl_buffer=>get_instance( ).
  lo_buffer->load_travels( VALUE #( FOR e IN entities ( e-TravelUUID ) ) ).

  LOOP AT entities INTO DATA(entity).
```

```
    ASSIGN lo_buffer->mt_travels[ travel-travel_uuid = entity-TravelUUID ]
           TO FIELD-SYMBOL(<buffered>).
    IF sy-subrc <> 0.
      APPEND VALUE #( %tky        = entity-%tky
                      %fail-cause = if_abap_behv=>cause-not_found ) TO failed-travel.
      CONTINUE.
    ENDIF.

    " FIELD-SELECTIVE update: only what %control says was sent
    IF entity-%control-AgencyId   = if_abap_behv=>mk-on.
      <buffered>-travel-agency_id   = entity-AgencyId.
      <buffered>-control-agency_id  = abap_true.
    ENDIF.
    IF entity-%control-CustomerId = if_abap_behv=>mk-on.
      <buffered>-travel-customer_id  = entity-CustomerId.
      <buffered>-control-customer_id = abap_true.
    ENDIF.
    IF entity-%control-BeginDate  = if_abap_behv=>mk-on.
      <buffered>-travel-begin_date  = entity-BeginDate.
      <buffered>-control-begin_date = abap_true.
    ENDIF.
    " ... one block per field ...

    <buffered>-travel-last_changed_by = cl_abap_context_info=>get_user_technical_name( ).
    GET TIME STAMP FIELD <buffered>-travel-last_changed_at.
    <buffered>-changed = abap_true.

  ENDLOOP.
ENDMETHOD.
```

This is the code the managed runtime writes for you. One `IF %control` block per field, for every entity. On a 40-field entity that is 120 lines of mechanical code per entity. It is the clearest single illustration of what managed RAP is worth.

A shorter form using the x-structure mapping (see Chapter 08, §8.12):

```
DATA(ls_x) = CORRESPONDING ztravel_x( entity MAPPING FROM ENTITY ).
" then use ls_x-agency_id etc. as the control flags
```

`FOR MODIFY ... FOR DELETE`

```
METHOD delete.
  DATA(lo_buffer) = lcl_buffer=>get_instance( ).
  lo_buffer->load_travels( VALUE #( FOR k IN keys ( k-TravelUUID ) ) ).

  LOOP AT keys INTO DATA(key).
    ASSIGN lo_buffer->mt_travels[ travel-travel_uuid = key-TravelUUID ]
           TO FIELD-SYMBOL(<buffered>).
    IF sy-subrc <> 0.
      APPEND VALUE #( %tky        = key-%tky
                      %fail-cause = if_abap_behv=>cause-not_found ) TO failed-travel.
      CONTINUE.
    ENDIF.
    <buffered>-deleted = abap_true.
  ENDLOOP.
ENDMETHOD.
```

Mark, do not remove. The saver needs to know a delete is pending. Removing it from the buffer means the row survives on the database.

`FOR LOCK`

```
METHOD lock.
  LOOP AT keys INTO DATA(key).
    CALL FUNCTION 'ENQUEUE_EZTRAVEL'
      EXPORTING  travel_uuid    = key-TravelUUID
                 _scope         = '1'          " keep until COMMIT/DEQUEUE by us
      EXCEPTIONS foreign_lock   = 1
                 system_failure = 2
                 OTHERS         = 3.
    IF sy-subrc <> 0.
      APPEND VALUE #( %tky        = key-%tky
                      %fail-cause = if_abap_behv=>cause-locked ) TO failed-travel.
      APPEND VALUE #( %tky = key-%tky
                      %msg = new_message( id = 'ZTRAVEL_MSG' number = '070'
                                          severity = if_abap_behv_message=>severity-error
                                          v1 = sy-msgv1 ) ) TO reported-travel.
    ENDIF.
  ENDLOOP.
ENDMETHOD.
```

Prerequisite: a lock object `EZTRAVEL` created in SE11 over table `ZTRAVEL`.

Read by association (rba)

```
METHOD rba_Booking.

  DATA(lo_buffer) = lcl_buffer=>get_instance( ).
  lo_buffer->load_bookings_for_travels( VALUE #( FOR k IN keys_rba ( k-TravelUUID ) ) ).
```

```
  LOOP AT keys_rba INTO DATA(key).
    LOOP AT lo_buffer->mt_bookings INTO DATA(ls_booking)
         WHERE booking-travel_uuid = key-TravelUUID
           AND deleted = abap_false.

      " the LINK table: which parent maps to which child
      APPEND VALUE #( source-%tky-TravelUUID  = key-TravelUUID
                      target-%tky-BookingUUID = ls_booking-booking-booking_uuid )
             TO association_links.

      " the data, only if the consumer asked for it
      IF result_requested = abap_true.
        APPEND CORRESPONDING #( ls_booking-booking MAPPING TO ENTITY ) TO result
               ASSIGNING FIELD-SYMBOL(<res>).
        <res>-%tky-BookingUUID = ls_booking-booking-booking_uuid.
      ENDIF.
    ENDLOOP.
  ENDLOOP.
ENDMETHOD.
```

Two outputs:

- `association_links` — **always** filled. The parent→child mapping.
- `result` — only when `result_requested = abap_true`. The child data.

The consumer may want only the links (e.g. to count children). Honouring `result_requested` is part of the RAP BO contract.

Create by association (cba)

```
METHOD cba_Booking.

  DATA(lo_buffer) = lcl_buffer=>get_instance( ).

  LOOP AT entities_cba INTO DATA(entity_cba).

    " entity_cba-%tky identifies the PARENT
    " entity_cba-%target holds the children to create
    LOOP AT entity_cba-%target INTO DATA(target).

      TRY.
          DATA(lv_uuid) = cl_system_uuid=>create_uuid_x16_static( ).
        CATCH cx_uuid_error.
          APPEND VALUE #( %cid = target-%cid ) TO failed-booking.
          CONTINUE.
      ENDTRY.

      DATA(ls_booking) = CORRESPONDING zbooking( target MAPPING FROM ENTITY ).
      ls_booking-booking_uuid = lv_uuid.
      ls_booking-travel_uuid  = entity_cba-TravelUUID.    " link to the parent

      INSERT VALUE #( booking = ls_booking created = abap_true )
             INTO TABLE lo_buffer->mt_bookings.

      APPEND VALUE #( %cid        = target-%cid
                      BookingUUID = lv_uuid ) TO mapped-booking.
    ENDLOOP.
  ENDLOOP.
ENDMETHOD.
```

15.6 The saver class

```
CLASS lsc_ZR_TravelTP_U DEFINITION INHERITING FROM cl_abap_behavior_saver.
  PROTECTED SECTION.
    METHODS finalize          REDEFINITION.
    METHODS check_before_save REDEFINITION.
    METHODS save              REDEFINITION.
    METHODS cleanup           REDEFINITION.
    METHODS cleanup_finalize  REDEFINITION.
ENDCLASS.

CLASS lsc_ZR_TravelTP_U IMPLEMENTATION.

  METHOD finalize.
    " last chance to adjust data. Runs BEFORE check_before_save.
    " e.g. recalculate a total across the whole buffer.
  ENDMETHOD.

  METHOD check_before_save.
    DATA(lo_buffer) = lcl_buffer=>get_instance( ).
    LOOP AT lo_buffer->mt_travels INTO DATA(ls)
         WHERE deleted = abap_false
           AND ( created = abap_true OR changed = abap_true ).
      IF ls-travel-begin_date > ls-travel-end_date.
        APPEND VALUE #( %tky-TravelUUID = ls-travel-travel_uuid ) TO failed-travel.
        APPEND VALUE #( %tky-TravelUUID = ls-travel-travel_uuid
                        %msg = new_message( id = 'ZTRAVEL_MSG' number = '001'
```

```
                                        severity = if_abap_behv_message=>severity-error ) )
              TO reported-travel.
      ENDIF.
    ENDLOOP.
  ENDMETHOD.

  METHOD save.
    " LATE SAVE PHASE - the only place database changes are allowed
    DATA(lo_buffer) = lcl_buffer=>get_instance( ).

    DATA: lt_insert TYPE STANDARD TABLE OF ztravel,
          lt_update TYPE STANDARD TABLE OF ztravel,
          lt_delete TYPE STANDARD TABLE OF ztravel.

    LOOP AT lo_buffer->mt_travels INTO DATA(ls).
      CASE abap_true.
        WHEN ls-deleted. APPEND ls-travel TO lt_delete.
        WHEN ls-created. APPEND ls-travel TO lt_insert.
        WHEN ls-changed. APPEND ls-travel TO lt_update.
      ENDCASE.
    ENDLOOP.

    IF lt_insert IS NOT INITIAL.
      INSERT ztravel FROM TABLE @lt_insert.
    ENDIF.
    IF lt_update IS NOT INITIAL.
      UPDATE ztravel FROM TABLE @lt_update.
    ENDIF.
    IF lt_delete IS NOT INITIAL.
      DELETE ztravel FROM TABLE @lt_delete.
      " cascade: the children of a deleted parent
      DELETE FROM zbooking
        FOR ALL ENTRIES IN @lt_delete
        WHERE travel_uuid = @lt_delete-travel_uuid.
    ENDIF.

    " ... same for bookings ...

    " Release the locks
    LOOP AT lo_buffer->mt_travels INTO ls.
      CALL FUNCTION 'DEQUEUE_EZTRAVEL'
        EXPORTING travel_uuid = ls-travel-travel_uuid.
    ENDLOOP.
  ENDMETHOD.

  METHOD cleanup.
    lcl_buffer=>get_instance( )->clear( ).
  ENDMETHOD.

  METHOD cleanup_finalize.
    " called when finalize or check_before_save failed
    lcl_buffer=>get_instance( )->clear( ).
  ENDMETHOD.

ENDCLASS.
```

Notes on `save`

- **This is the only place `INSERT` / `UPDATE` / `DELETE` on the database is legal.** In any other phase it breaks the SAP LUW (Chapter 17, §17.6).
- **No `COMMIT WORK`.** The framework commits after `save` returns.
- **Set-based writes.** `INSERT ... FROM TABLE` once, not one `INSERT` per row.
- **Cascade deletes explicitly.** The managed runtime does this for you; unmanaged does not.
- **Do not raise exceptions.** A failure here is after the point of no return. If your save can genuinely fail, inherit from `CL_ABAP_BEHAVIOR_SAVER_FAILED` instead.

15.7 Wrapping an existing BAPI

The most common real reason for unmanaged. The shape:

```
METHOD create.
  " Interaction phase: buffer only. Do NOT call the BAPI here.
  LOOP AT entities INTO DATA(entity).
    lcl_buffer=>get_instance( )->add_pending_create( entity ).
    APPEND VALUE #( %cid = entity-%cid ) TO mapped-travel.   " key comes later
  ENDLOOP.
ENDMETHOD.

METHOD check_before_save.
  " Early save: run the BAPI in TEST MODE to surface errors while we can still go back.
  LOOP AT lcl_buffer=>get_instance( )->mt_pending INTO DATA(ls).
    CALL FUNCTION 'BAPI_TRAVEL_CREATE'
      EXPORTING  travel_data = ls-data
                 testrun     = abap_true
      TABLES     return      = DATA(lt_return).
    LOOP AT lt_return INTO DATA(ls_ret) WHERE type CA 'EAX'.
      APPEND VALUE #( %cid = ls-cid ) TO failed-travel.
      APPEND VALUE #( %cid = ls-cid
                      %msg = new_message( id = ls_ret-id number = ls_ret-number
```

```
                                    severity = if_abap_behv_message=>severity-error
                                    v1 = ls_ret-message_v1 ) ) TO reported-travel.
    ENDLOOP.
  ENDLOOP.
ENDMETHOD.

METHOD save.
  " Late save: the real call. No COMMIT WORK - RAP owns the commit.
  LOOP AT lcl_buffer=>get_instance( )->mt_pending INTO DATA(ls).
    CALL FUNCTION 'BAPI_TRAVEL_CREATE'
      EXPORTING  travel_data = ls-data
                 testrun     = abap_false
      IMPORTING  travelid    = DATA(lv_id)
      TABLES     return      = DATA(lt_return).
    " do NOT call BAPI_TRANSACTION_COMMIT
  ENDLOOP.
ENDMETHOD.
```

The three rules for BAPI wrapping

1. **Run the BAPI in test mode during `check_before_save`.** That is your only chance to reject before the point of no return. Skipping this means a BAPI error in `save` becomes a dump.
2. **Never call `BAPI_TRANSACTION_COMMIT`.** RAP commits. A nested commit corrupts the LUW.
3. **If the BAPI can still fail in `save` despite the test run**, use `CL_ABAP_BEHAVIOR_SAVER_FAILED` so you can report it, and expect `sy-subrc = 8` on the consumer's `COMMIT ENTITIES`.

If the BAPI assigns the key (late numbering), see Chapter 19 — `adjust_numbers` plus `CONVERT KEY`.

15.8 Unmanaged vs managed — the honest comparison

	Managed	Unmanaged
Lines of ABAP for plain CRUD on 2 entities	~0	~600–900
Buffer	Framework	You
Locking	`lock master`	`FOR LOCK` + lock object
ETag	`etag master F`	Manual, in read + save
Draft	`with draft` + a table	Substantial extra work
Key generation	`numbering : managed`	`cl_system_uuid` in `create`
Cascade delete	Automatic	Manual
Admin fields	`@Semantics` + `readonly`	Manual, in create and update
Field-selective update	Automatic	One `IF %control` block per field
Where bugs live	Your validations	Your buffer

Unmanaged is roughly an order of magnitude more code for the same CRUD. That is the honest number, and it is why the decision in Chapter 01, §1.3 matters so much.

15.9 Unmanaged pitfalls

Pitfall	Consequence	Fix
Buffer not cleared in `cleanup` / `cleanup_finalize`	The next transaction sees stale data; phantom saves	Clear in both
`load` overwrites buffered instances	The user's changes disappear	Load only what is missing
Ignoring `%control` in `update`	Fields the consumer never sent are overwritten with initial values	One `IF %control` block per field
`mapped` not filled in `create`	The consumer cannot address the new instance; the UI cannot navigate to it	Fill `mapped` with `%cid` → key
`DELETE` removes from the buffer instead of marking	The row survives on the database	Set a `deleted` flag
Database writes outside `save`	LUW broken; partial commits	Only in `save`
`COMMIT WORK` anywhere	Runtime error / corrupted LUW	Remove
Locks never released	Users blocked until the lock expires	`DEQUEUE` in `save` (and rely on `_scope = '1'` semantics)
Children not deleted with the parent	Orphan rows	Cascade explicitly in `save`
`association_links` not filled in rba	Navigation silently returns nothing	Always fill it
`result_requested` ignored	Unnecessary reads	Honour it
Errors first detected in `save`	Dump after the point of no return	Move the check to `check_before_save`

15.10 Interview answer

> **"When would you build an unmanaged RAP business object?"**
>
> When there is existing **transactional logic** that must be reused rather than reimplemented — an existing buffer, existing locking, existing update-task modules, reads that have to go through existing function modules. Typically a brownfield wrap of a module that has fifteen years of behaviour in it.
>
> Not because the tables already exist. `persistent table` accepts any DDIC table, so existing data is perfectly fine for a managed BO. That is the mistake I would push back on in a design review, because unmanaged is roughly an order of magnitude more code for the same CRUD: your own transactional buffer, `FOR READ`, `FOR MODIFY` create/update/delete, `FOR LOCK`, read- and create-by-association, plus a saver class — and you also take over ETag maintenance, cascading delete, admin fields, key generation, and the field-selective `%control` handling, which alone is one `IF` block per field per entity.
>
> And if only the **save** is special — persisting through a BAPI, or an API with late numbering — the right answer is `managed with unmanaged save`, not a full unmanaged BO. You keep the managed buffer, locking, ETag and draft, and only take over persistence.
>
> The two failure modes I would watch for in unmanaged code are the buffer not being cleared in both `cleanup` and `cleanup_finalize`, and `%control` being ignored on update, which silently wipes fields the user never touched.

15.11 Production considerations

- **Validate every example against your release before adopting it** — see the notice at the top of this chapter. Generate signatures with `Ctrl + 1`.
- **Start from the ABAP Flight Reference Scenario** (`/DMO/` packages). It ships an activated, working unmanaged BO for this exact model.
- **The buffer is where the bugs live.** Clear it in `cleanup` **and** `cleanup_finalize`, and never let a load overwrite a buffered instance.
- **Respect** `%control` **on update**, or users lose data they never edited.
- **Fill** `mapped` **on create**, or the consumer cannot address the new instance.
- **Cascade deletes explicitly.** The managed runtime does it for you; unmanaged does not.
- **Detect failures in** `check_before_save` **, not in** `save` **.** A test-mode BAPI call there is the difference between a clean rejection and a dump after the point of no return.
- **Budget honestly.** Plan unmanaged as a multiple of the managed effort, including test effort.

Chapter 16 — Managed with Unmanaged Save, and Additional Save

16.1 The two hybrid patterns

Between "the framework does everything" (managed) and "you do everything" (unmanaged) sit two options that cover most real requirements:

	`with additional save`	`with unmanaged save`
Who writes the BO's own table?	**The framework**	**You**
Your `save_modified` runs	**In addition** to the framework's save	**Instead of** the framework's save
`persistent table` in the BDEF	**Required**	**Forbidden**
Typical use	Change documents, application log, business events, calling a side-effect API	Persisting via a released API / BAPI, late numbering with `CONVERT KEY`, writing to a non-standard store
Everything else managed (buffer, lock, ETag, draft, field control, determinations, validations)	**Yes**	**Yes**

Both are declared the same way and both are implemented in the same method: `save_modified`.

> **This is the answer to most "we need managed but the save is special" requirements.** Reach for a full unmanaged BO only if the *interaction phase* also has to be custom.

16.2 Declaring it

In the BDEF header — applies to the whole BO

```
managed with additional save implementation in class ZBP_R_TravelTP unique;
strict ( 2 );
```

```
managed with unmanaged save implementation in class ZBP_R_TravelTP unique;
strict ( 2 );
```

Per entity — applies to that entity only

```
define behavior for ZR_TravelTP alias Travel
//persistent table ztravel          <- must be removed for unmanaged save
draft table ztravel_d
etag master LocalLastChangedAt
lock master total etag LastChangedAt
authorization master ( global )
with unmanaged save
{ ... }
```

Optional additions

```
managed with additional save with full data implementation in class ZBP_... unique;
managed with additional save and cleanup implementation in class ZBP_... unique;
```

Addition	Effect
`with full data`	`save_modified` receives **all** instance fields, not just keys and changed fields. Saves you a `READ ENTITIES`.
`and cleanup`	You must also redefine `cleanup` in the saver class. Use it when you keep your own state that has to be reset.

Both can be combined: `managed with additional save with full data and cleanup implementation in class ...`.

16.3 The three constraints that catch everyone

These are the practical rules, confirmed by SAP's own workshop material:

1. **`persistent table` and `with unmanaged save` cannot coexist.** Comment out or delete `persistent table ztravel` when you add `with unmanaged save`. Activation fails otherwise. (`with additional save` **keeps** `persistent table`.)
2. **With unmanaged/additional save, the behavior implementation class must be specified in the header, on the same line as `managed`.** If your BDEF currently has `implementation in class ... unique` on the *node* level (a common generated form), you must move it:

```abap " before managed implementation in class ZBP_R_TRAVELTP unique; define behavior for ZR_TravelTP alias Travel implementation in class ZBP_R_TRAVELTP unique <- node level persistent table ztravel ...

" after managed with unmanaged save implementation in class ZBP_R_TRAVELTP unique; define behavior for ZR_TravelTP alias Travel //implementation in class ZBP_R_TRAVELTP unique <- removed //persistent table ztravel <- removed draft table ztravel_d ... ```

3. **For a field-selective update you need a control structure.** `save_modified` gives you `%control` per instance. To turn that into a field-selective `UPDATE`, declare an x-structure and extend the mapping:

`abap mapping for ZTRAVEL control ztravel_x corresponding`

with

`abap @EndUserText.label : 'Control structure for ZTRAVEL' @AbapCatalog.enhancement.category : #NOT_EXTENSIBLE define structure ztravel_x { travel_id`

---

## 16.4 Generating the saver class

After activating the BDEF, put the cursor on the keyword `unmanaged` (or `additional`) and press `Ctrl + 1`. ADT adds the local saver class to the behavior pool's *Local Types*:

```
CLASS lsc_zr_traveltp DEFINITION INHERITING FROM cl_abap_behavior_saver.
  PROTECTED SECTION.
    METHODS save_modified REDEFINITION.
ENDCLASS.

CLASS lsc_zr_traveltp IMPLEMENTATION.
  METHOD save_modified.
  ENDMETHOD.
ENDCLASS.
```

---

## 16.5 The `save_modified` signature

```
METHODS save_modified
  IMPORTING create   TYPE DATA
            update   TYPE DATA
            delete   TYPE DATA
  CHANGING  reported TYPE DATA OPTIONAL
            failed   TYPE DATA OPTIONAL.
```

At runtime the parameters are typed:

Parameter	Derived type	Contains
`create`	`REQUEST FOR CHANGE zr_traveltp`	Instances to create, per entity: `create-travel`, `create-booking`
`update`	`REQUEST FOR CHANGE zr_traveltp`	Instances to update, with `%control`
`delete`	`REQUEST FOR DELETE zr_traveltp`	Keys to delete

Each is a **deep structure with one component per entity alias**, lowercase:

```
create-travel      " TABLE FOR CREATE zr_traveltp
create-booking     " TABLE FOR CREATE zr_traveltp\\Booking
update-travel
update-booking
delete-travel
delete-booking
```

By default `create` / `update` carry **keys plus changed fields only**. With `with full data` they carry everything.

16.6 Unmanaged save — full implementation

```
CLASS lsc_zr_traveltp DEFINITION INHERITING FROM cl_abap_behavior_saver.
  PROTECTED SECTION.
    METHODS save_modified REDEFINITION.
    METHODS cleanup       REDEFINITION.
ENDCLASS.

CLASS lsc_zr_traveltp IMPLEMENTATION.

  METHOD save_modified.

    " ---------------- CREATE ----------------
    IF create-travel IS NOT INITIAL.
      DATA(lt_insert) = CORRESPONDING ztravel_tab( create-travel MAPPING FROM ENTITY ).
      INSERT ztravel FROM TABLE @lt_insert.
    ENDIF.
```
```

```
    IF create-booking IS NOT INITIAL.
      DATA(lt_binsert) = CORRESPONDING zbooking_tab( create-booking MAPPING FROM ENTITY ).
      INSERT zbooking FROM TABLE @lt_binsert.
    ENDIF.

    " ---------------- UPDATE ----------------
    IF update-travel IS NOT INITIAL.

      " data ...
      DATA(lt_update)  = CORRESPONDING ztravel_tab(   update-travel MAPPING FROM ENTITY ).
      " ... and the control flags, via the x-structure from the BDEF mapping
      DATA(lt_control) = CORRESPONDING ztravel_x_tab( update-travel MAPPING FROM ENTITY ).

      LOOP AT lt_update INTO DATA(ls_upd).
        DATA(ls_ctrl) = lt_control[ sy-tabix ].

        " Field-selective UPDATE SET. Only the fields the consumer actually sent.
        UPDATE ztravel
          SET agency_id      = COND #( WHEN ls_ctrl-agency_id   = abap_true
                                       THEN @ls_upd-agency_id   ELSE @agency_id ),
              customer_id    = COND #( WHEN ls_ctrl-customer_id = abap_true
                                       THEN @ls_upd-customer_id ELSE @customer_id ),
              begin_date     = COND #( WHEN ls_ctrl-begin_date  = abap_true
                                       THEN @ls_upd-begin_date  ELSE @begin_date ),
              end_date       = COND #( WHEN ls_ctrl-end_date    = abap_true
                                       THEN @ls_upd-end_date    ELSE @end_date ),
              description    = COND #( WHEN ls_ctrl-description = abap_true
                                       THEN @ls_upd-description ELSE @description ),
              last_changed_by = @ls_upd-last_changed_by,
              last_changed_at = @ls_upd-last_changed_at
          WHERE travel_uuid = @ls_upd-travel_uuid.
      ENDLOOP.
    ENDIF.

    " ---------------- DELETE ----------------
    IF delete-travel IS NOT INITIAL.
      LOOP AT delete-travel INTO DATA(ls_del).
        DELETE FROM ztravel  WHERE travel_uuid = @ls_del-TravelUUID.
        DELETE FROM zbooking WHERE travel_uuid = @ls_del-TravelUUID.   " cascade
        DELETE FROM ztravel_d  WHERE traveluuid = @ls_del-TravelUUID.  " draft rows
        DELETE FROM zbooking_d WHERE traveluuid = @ls_del-TravelUUID.
      ENDLOOP.
    ENDIF.

    IF delete-booking IS NOT INITIAL.
      LOOP AT delete-booking INTO DATA(ls_bdel).
        DELETE FROM zbooking WHERE booking_uuid = @ls_bdel-BookingUUID.
      ENDLOOP.
    ENDIF.

  ENDMETHOD.

  METHOD cleanup.
    CLEAR zbp_r_traveltp=>mapped_purchase_requisition.
    CLEAR zbp_r_traveltp=>mt_travels_to_notify.
  ENDMETHOD.

ENDCLASS.
```

Notes

- `CORRESPONDING ... MAPPING FROM ENTITY` uses the BDEF's `mapping for` block to convert CDS element names to DDIC field names. Without a `mapping for`, this does nothing useful.
- **A simpler** `MODIFY ztravel FROM TABLE @lt_update` is possible and is what SAP's own workshop example uses. It is correct **only when the payload carries all fields** — i.e. with `with full data`, or when you first read the full instance. Without that, unsent fields are written as initial. The `COND` form above is the safe default; `with full data` + `MODIFY ... FROM TABLE` is the simpler one.
- **Draft rows must be deleted too.** The managed runtime does this; when you take over the save you inherit the responsibility.
- **No** `COMMIT WORK`**.** RAP commits after `save_modified` returns.

16.7 Additional save — full implementation

Here the framework still writes `ztravel`. Your method adds side effects.

```
CLASS lsc_zr_traveltp DEFINITION INHERITING FROM cl_abap_behavior_saver.
  PROTECTED SECTION.
    METHODS save_modified REDEFINITION.
    METHODS cleanup       REDEFINITION.
ENDCLASS.

CLASS lsc_zr_traveltp IMPLEMENTATION.

  METHOD save_modified.

    " ---- 1. Application log ----
    DATA(lo_log) = zcl_travel_applog=>get( ).
    LOOP AT create-travel INTO DATA(ls_created).
```

```
      lo_log->add_info( |Travel created: { ls_created-TravelUUID }| ).
    ENDLOOP.
    LOOP AT update-travel INTO DATA(ls_updated).
      lo_log->add_info( |Travel changed: { ls_updated-TravelUUID }| ).
    ENDLOOP.
    LOOP AT delete-travel INTO DATA(ls_deleted).
      lo_log->add_warning( |Travel deleted: { ls_deleted-TravelUUID }| ).
    ENDLOOP.
    lo_log->save( ).           " BAL_DB_SAVE - registered for the update task

    " ---- 2. Change documents ----
    LOOP AT update-travel INTO DATA(ls_chg).
      zcl_travel_changedoc=>write( travel_uuid = ls_chg-TravelUUID
                                   control     = CORRESPONDING ztravel_x( ls_chg
                                                     MAPPING FROM ENTITY ) ).
    ENDLOOP.

    " ---- 3. RAP business event ----
    LOOP AT update-travel INTO DATA(ls_evt)
         WHERE %control-OverallStatus = if_abap_behv=>mk-on.
      RAISE ENTITY EVENT zr_traveltp~TravelStatusChanged
        FROM VALUE #( ( %key-TravelUUID = ls_evt-TravelUUID
                        %param = VALUE #( travel_uuid = ls_evt-TravelUUID
                                          new_status  = ls_evt-OverallStatus ) ) ).
    ENDLOOP.

    " ---- 4. Notify an external system (flagged during the interaction phase) ----
    IF zbp_r_traveltp=>mt_travels_to_notify IS NOT INITIAL.
      zcl_travel_outbound=>enqueue( zbp_r_traveltp=>mt_travels_to_notify ).
    ENDIF.

  ENDMETHOD.

  METHOD cleanup.
    CLEAR zbp_r_traveltp=>mt_travels_to_notify.
  ENDMETHOD.

ENDCLASS.
```

`%control` in `update`

`ls_evt-%control-OverallStatus = if_abap_behv=>mk-on` tells you **which fields actually changed**. That is the difference between "log every save" and "log only status changes". It is the single most useful part of the `update` payload.

What belongs in `save_modified` and what does not

Belongs	Does not belong
Writing an application log	Business validation (→ `check_before_save`)
Change documents	Data derivation (→ a determination, or `finalize`)
Raising a business event	Anything that can fail and must be able to abort the save
Registering an update-task function module	`COMMIT WORK`
Enqueuing an outbound message	Synchronous RFC or `WAIT` (forbidden in the late phase)
`CONVERT KEY` for late numbering	A long-running HTTP call

The late save phase must not fail. Everything that can be checked must be checked in `check_before_save`.

16.8 The late-numbering pattern — the real reason for unmanaged save

This is SAP's own motivating example, and it is worth understanding because it is not obvious.

The problem: an action calls a released API (`I_PurchaseRequisitionTP`) that uses **late numbering**. The API returns a *preliminary* id (`%pid`), not the real document number. The real number only exists after the save sequence. And `CONVERT KEY` — the statement that resolves a `%pid` to a real key — **can only be called inside** `save_modified`.

So: the action runs in the interaction phase and gets a `%pid` ; the real number is available in the save sequence; and you must write it into your own table. That forces `with unmanaged save` (with `with additional save` you cannot write your own table — the framework already did).

Step 1 — a bridge variable in the global ABP class

```
CLASS zbp_r_traveltp DEFINITION
  PUBLIC ABSTRACT FINAL
  FOR BEHAVIOR OF zr_traveltp.

  PUBLIC SECTION.
    CLASS-DATA mapped_purchase_requisition
      TYPE RESPONSE FOR MAPPED i_purchaserequisitiontp.
ENDCLASS.
```

The handler and the saver are separate classes. A `CLASS-DATA` on the global ABP class is the supported channel between them.

Step 2 — the action stores the preliminary ids

```
METHOD createPurchaseRequisition.

  " ... build the payload from the selected Travels ...

  MODIFY ENTITIES OF i_purchaserequisitiontp
    ENTITY purchaserequisition
      CREATE FIELDS ( purchaserequisitiontype )
      WITH purchase_requisitions
      CREATE BY \_purchaserequisitionitem
        FIELDS ( plant requestedquantity material ... )
        WITH purchase_requisition_items
    MAPPED   DATA(mapped_create_pr)
    FAILED   DATA(failed_create_pr)
    REPORTED DATA(reported_create_pr).

  " hand the preliminary ids to the save sequence
  zbp_r_traveltp=>mapped_purchase_requisition-purchaserequisition
    = mapped_create_pr-purchaserequisition.

  " update our own instance so that the save sequence is actually triggered
  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      UPDATE FIELDS ( OverallStatus PurchRqnCreationDate )
      WITH update_lines
    REPORTED reported FAILED failed MAPPED mapped.

ENDMETHOD.
```

Note the last `MODIFY`: without changing something on our own BO, our `save_modified` would never be called and the `CONVERT KEY` would never run.

Step 3 — resolve the key in `save_modified`

```
METHOD save_modified.

  " ... normal create / update / delete handling of ztravel ...

  IF zbp_r_traveltp=>mapped_purchase_requisition IS NOT INITIAL
     AND update-travel IS NOT INITIAL.

    LOOP AT zbp_r_traveltp=>mapped_purchase_requisition-purchaserequisition
         ASSIGNING FIELD-SYMBOL(<pr_mapped>).

      " Only legal inside save_modified
      CONVERT KEY OF i_purchaserequisitiontp
        FROM <pr_mapped>-%pid
        TO   DATA(ls_pr_key).

      <pr_mapped>-purchaserequisition = ls_pr_key-purchaserequisition.
    ENDLOOP.

    LOOP AT update-travel INTO DATA(ls_travel)
         WHERE %control-OverallStatus = if_abap_behv=>mk-on.
      UPDATE ztravel
        SET purchase_requisition    = @ls_pr_key-purchaserequisition,
            purch_rqn_creation_date = @sy-datum
        WHERE travel_uuid = @ls_travel-TravelUUID.
    ENDLOOP.

  ENDIF.

ENDMETHOD.
```

Step 4 — clear the bridge

```
METHOD cleanup.
  CLEAR zbp_r_traveltp=>mapped_purchase_requisition.
ENDMETHOD.

METHOD cleanup_finalize.
  CLEAR zbp_r_traveltp=>mapped_purchase_requisition.
ENDMETHOD.
```

Do not skip this. `CLASS-DATA` survives the transaction. A second save in the same session would otherwise re-process stale preliminary ids.

16.9 Choosing between the four options — a worked decision

The one-line distinction

- `with additional save` — the framework persists the BO; your code performs **extra save-related work** alongside it.
- `with unmanaged save` — **your implementation is responsible for persistence**; the framework does not write the BO's table.

By business requirement

Requirement	Suitable approach	Why
Write an application log entry after a successful RAP save	**Additional save**	The BO's own persistence is unchanged; you are adding a side effect
Create a change document (who changed which field)	**Additional save**	Needs the `%control` flags from `update`, which `save_modified` provides, plus the framework's own write
Publish a RAP business event / notify a downstream process	**Additional save**	Events should fire on committed state; `save_modified` is the documented place
Register an update-task function module for an asynchronous follow-on	**Additional save**	Update-task registration is only legal in the late save phase
Write an outbound queue / IDoc / webhook payload	**Additional save**	Enqueue in the late phase, after the BO is known to be saving
Persist through a legacy BAPI instead of writing the table	**Unmanaged save**, or full **unmanaged RAP** if the reads must also go through legacy logic	The framework must not also write the table, or you get a double write
Persist into a non-RAP external store (external DB, REST API, file)	**Unmanaged save**, with careful error handling	No `persistent table` exists to be framework-managed
Call a released API that uses **late numbering** and store its key in your table	**Unmanaged save**	`CONVERT KEY` is only legal inside `save_modified`, and you need to write the resolved key yourself (§16.8)
Persist to a table whose layout does not match the BO 1:1 (splitting, denormalising, archiving)	**Unmanaged save**	The framework's 1:1 mapping cannot express it
Reuse existing **interaction and locking** logic, not just the save	**Full unmanaged RAP**	The interaction phase is custom too, so `managed` provides nothing you can use
Reads must go through existing function modules rather than the table	**Full unmanaged RAP**	You need `FOR READ`
Plain CRUD on your own Z tables	`managed`	Add nothing

Two cases worth spelling out

"Publish an event or call a side-effect process" is listed as *usually* additional save, and the qualifier matters. It is additional save when the side effect must happen **because the BO was saved** and must see committed state. It is **not** additional save when:

- the side effect can fail and should abort the save → the check belongs in `check_before_save`, and only the irreversible part in `save_modified`;
- the side effect is synchronous and slow (an HTTP call, an RFC) → synchronous RFC is **not** allowed in the late save phase; enqueue it instead and process it asynchronously;
- the side effect is really a data derivation → that is a determination, not a save enhancement.

"Persist through a legacy BAPI" splits by how much of the legacy logic you need:

You need	Choose
Only the BAPI's write, with your own model and buffer	`managed with unmanaged save`
The BAPI's write **and** its read/lock semantics	full `unmanaged`

Decision order

Work down this list and stop at the first match:

1. Is the **interaction phase** (read, lock, buffer) custom? → **unmanaged**.
2. Is the **persistence** custom (BAPI, external store, non-1:1 layout, late-numbering key)? → **managed with unmanaged save**.
3. Do you need **extra work at save time** on top of normal persistence? → **managed with additional save**.
4. Otherwise → **managed**.

Notice how far down the list you get before a full unmanaged BO is justified.

16.10 Troubleshooting

Symptom	Cause
"*persistent table is not allowed*"	`persistent table` left in place together with `with unmanaged save`
"*Implementation class must be specified in the header*"	The class is declared at node level; move it next to `managed`
`save_modified` never called	Nothing on **your** BO changed — an action that only calls another BO does not trigger your save. Add an update to your own instance
Data saved but fields are blank	`mapping for` missing, or `MODIFY ... FROM TABLE` used without `with full data`
Update wipes fields the user did not touch	`%control` ignored; use the x-structure and a field-selective `UPDATE SET`
`CONVERT KEY` raises an exception	Called outside `save_modified`; it is only legal there
Second save in the same session behaves oddly	`CLASS-DATA` not cleared in `cleanup` / `cleanup_finalize`
Draft rows remain after delete	With unmanaged save you must delete the draft table rows yourself

Symptom	Cause
Dump after the point of no return	Something that can fail was left in `save_modified` instead of `check_before_save`
`create-travel` is empty although a Travel was created	You are looking at the wrong component — the component name is the **alias**, lowercase

16.11 Interview answer

> **"What is the difference between additional save and unmanaged save, and when do you use each?"**
>
> Both are declared on a **managed** business object and both are implemented in `save_modified`, so you keep the managed buffer, locking, ETag, draft and field control. The difference is persistence.
>
> With `with additional save`, the framework still writes the BO's table. Your `save_modified` runs **in addition**, for save-related side effects — change documents, an application log entry, raising a RAP business event, registering an update-task module, enqueueing an outbound message. `persistent table` stays.
>
> With `with unmanaged save`, **you** are responsible for persistence. The framework does not write the table, and `persistent table` must be removed from the behavior definition — the two cannot coexist. That is for persisting through a BAPI, into an external store, or into a layout that is not 1:1 with the BO.
>
> The case that forces unmanaged save is **late numbering**. If you call a released API that only returns a preliminary ID, resolving it needs `CONVERT KEY`, which is legal only inside `save_modified` — and you need to write the resolved number into your own table, which additional save cannot do because the framework has already written it.
>
> `save_modified` gives you `create`, `update` and `delete`, and `update` carries `%control`, so you can tell exactly which fields changed — that is what makes change documents and status-change events practical.
>
> One rule applies to both: the late save phase must not fail. Anything that can fail belongs in `check_before_save`.

16.12 Production considerations

- **Two constraints that fail at activation:** `persistent table` cannot coexist with `with unmanaged save`, and the behavior pool class must be declared in the **header**, on the same line as `managed`, not at node level.
- **Clear every `CLASS-DATA` bridge variable in `cleanup` and `cleanup_finalize`.** Class data survives the session, and `cleanup_finalize` is the path taken when the save is rejected.
- **With unmanaged save you inherit the draft-table clean-up** — the framework no longer deletes draft rows on activation. Do it yourself.
- **Respect `%control` on update**, or use `with full data`, or you will write initial values over fields the user never touched.
- **No synchronous RFC or HTTP in `save_modified`** — it runs in the late save phase. Enqueue instead, and process asynchronously with retry and idempotency.
- **Raise business events here, not in the action**, so subscribers only see committed state.
- **Test the second save in the same session.** Stale bridge state is the classic defect and only shows up on the second transaction.

Chapter 17 — The RAP Transaction and the Save Sequence

17.1 The two phases of a RAP transaction

A RAP transaction has exactly **two** phases. The second splits into two sub-phases.

```
+=========================================================================+
|  1. INTERACTION PHASE                                                   |
|                                                                         |
|  RAP handler methods run (class inherits CL_ABAP_BEHAVIOR_HANDLER).     |
|  Instances are created in, or loaded into, the transactional buffer.    |
|  The buffer MAY be temporarily inconsistent. The database is untouched. |
+=========================================================================+
                                  |
                    COMMIT ENTITIES  (implicit in Fiori/OData)
                                  v
+=========================================================================+
|  2. SAVE SEQUENCE                                                       |
|     RAP saver methods run (class inherits CL_ABAP_BEHAVIOR_SAVER).      |
|                                                                         |
|  2a. EARLY SAVE PHASE                                                   |
|      Goal: make sure every BO in this transaction is consistent.        |
|                                                                         |
|      1. finalize()            <- determinations 'on save' run here      |
|      2. check_before_save()   <- validations   'on save' run here       |
|      3. cleanup_finalize()    <- only if 1 or 2 failed; then back to (1)|
|                                                                         |
|      Failure here: sy-subrc = 4 after COMMIT ENTITIES.                  |
|      The transaction RETURNS TO THE INTERACTION PHASE. Recoverable.     |
|                                                                         |
|  ==================== POINT OF NO RETURN ====================          |
|                                                                         |
|  2b. LATE SAVE PHASE                                                    |
|      Goal: write to the database.                                       |
|                                                                         |
|      4. adjust_numbers()      <- late numbering only                    |
|      5. save() / save_modified()                                        |
|         ... implicit COMMIT WORK ...                                    |
|      6. cleanup()             <- clears the transactional buffer        |
|                                                                         |
|      Failure here: either a runtime error, or (with                     |
|      CL_ABAP_BEHAVIOR_SAVER_FAILED) sy-subrc = 8 and a mandatory        |
|      ROLLBACK ENTITIES.                                                 |
+=========================================================================+
```

17.2 The saver methods, one by one

`finalize`

Phase	Early save
Purpose	Final calculations and data changes before the data can be persisted
May change data	**Yes** — this is the last place you can
May reject	Yes (via `failed`)
In managed scenarios	**Determinations declared `on save` are executed when this method is reached**
Scope	All BOs in the current RAP transaction

Use it for a cross-instance calculation that needs every change to be in place first — e.g. a document total across entities that no single determination can see.

```
METHOD finalize.
  " example: stamp a document version across the whole buffer
  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel ALL FIELDS WITH ... RESULT DATA(travels).
  " ... adjust ...
  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE ...
ENDMETHOD.
```

`check_before_save`

Phase	Early save — **the last gate before the point of no return**
Purpose	Check the consistency of the transactional buffer
May change data	**No**

May reject	**Yes** — this is its job
In managed scenarios	**Validations declared `on save` are executed when this method is reached**
On failure	`cleanup_finalize` is called and the transaction returns to the interaction phase

```
METHOD check_before_save.
  " Anything that can make the save fail MUST be detected here, not later.
  " Example: a test-mode BAPI call.
  LOOP AT ... .
    CALL FUNCTION 'BAPI_X_CREATE'
      EXPORTING testrun = abap_true
      TABLES    return  = DATA(lt_return).
    LOOP AT lt_return INTO DATA(ls) WHERE type CA 'EA'.
      APPEND VALUE #( %tky = ... ) TO failed-travel.
      APPEND VALUE #( %tky = ... %msg = ... ) TO reported-travel.
    ENDLOOP.
  ENDLOOP.
ENDMETHOD.
```

> **The rule the whole design rests on:** *everything that can fail must fail here.* A failure after this point cannot be recovered from.

`cleanup_finalize`

Phase	Early save, on failure
Purpose	Roll back changes made in `finalize`
Called when	`finalize` or `check_before_save` reported a failure
After it	The transaction returns to the interaction phase

If `finalize` changed anything outside the RAP buffer (your own state, a helper table), undo it here. Also clear any `CLASS-DATA` bridge variables.

`adjust_numbers`

Phase	Late save
Purpose	Replace preliminary ids (`%pid`) with final keys
Available	**Only** when the entity declares `late numbering`
May reject	No — we are past the point of no return

```
METHOD adjust_numbers.
  LOOP AT mapped-travel ASSIGNING FIELD-SYMBOL(<travel>).
    " draw the real number now
    <travel>-TravelId = zcl_travel_numbers=>next( ).
  ENDLOOP.
ENDMETHOD.
```

See Chapter 19 for the full late-numbering story.

`save` / `save_modified`

	`save`	`save_modified`
Used in	Unmanaged BOs	Managed BOs with `with additional save` or `with unmanaged save`
Parameters	none (read your own buffer)	`create` , `update` , `delete`
Job	Write the buffer to the database	Write / add to the write

This is the **only** phase where database modifications are allowed. See Chapters 15 and 16.

After `save` returns successfully, the framework issues the **implicit** `COMMIT WORK` .

`cleanup`

Phase	After the save, and after `ROLLBACK ENTITIES`
Purpose	Clear the transactional buffer and any own state
Mandatory in	Unmanaged BOs; and anywhere you keep `CLASS-DATA`

```
METHOD cleanup.
  lcl_buffer=>get_instance( )->clear( ).
```

```
  CLEAR zbp_r_traveltp=>mapped_purchase_requisition.
ENDMETHOD.
```

Not clearing state here is one of the most common and most confusing RAP bugs. The symptom is "the second save in the same session does something strange".

17.3 Where determinations and validations actually run

This table is the practical answer to "when does my code run":

BDEF declaration	Runs during	At which saver method
`determination X on modify`	Interaction phase	— (immediately after the triggering change)
`validation V on modify`	Interaction phase	—
`determination X on save`	Early save	when `finalize` is reached
`validation V on save`	Early save	when `check_before_save` is reached
`draft determine action Prepare`	Interaction phase (called by the UI), and before `Activate`	—
Action handler	Interaction phase	—
`FOR NUMBERING` (early numbering)	Interaction phase, at create	—
`adjust_numbers` (late numbering)	Late save	`adjust_numbers`

So: **your `on save` determination runs before your `on save` validation**, because `finalize` precedes `check_before_save`. That ordering is guaranteed. Ordering *within* a phase is not.

17.4 `COMMIT ENTITIES` and `ROLLBACK ENTITIES`

`COMMIT ENTITIES`

Triggers the save sequence and the final database commit.

```
" short form
COMMIT ENTITIES.

" with response, per BO
COMMIT ENTITIES RESPONSE OF zr_traveltp
  FAILED   DATA(failed)
  REPORTED DATA(reported).

" with the overall result
COMMIT ENTITIES RESPONSES
  FAILED   DATA(failed)
  REPORTED DATA(reported).

" simulation – runs the early save phase only, then rolls back.
" Excellent for "check before you commit" scenarios.
COMMIT ENTITIES IN SIMULATION MODE
  RESPONSE OF zr_traveltp
  FAILED   DATA(sim_failed)
  REPORTED DATA(sim_reported).
```

`sy-subrc` after `COMMIT ENTITIES`	Meaning
`0`	Success. Committed.
`4`	**Early save phase** failed. The transaction is back in the interaction phase. You can correct and retry.
`8`	**Late save phase** failed (only possible with `CL_ABAP_BEHAVIOR_SAVER_FAILED`). You **must** issue `ROLLBACK ENTITIES` before any further RAP operation.

Always check `sy-subrc` or `failed`. A silent `COMMIT ENTITIES.` that fails looks exactly like one that succeeded.

`ROLLBACK ENTITIES`

```
ROLLBACK ENTITIES.
```

Discards everything in the current RAP transaction, calls `cleanup`, and implicitly triggers `ROLLBACK WORK`.

`COMMIT ENTITIES` vs `COMMIT WORK`

	`COMMIT ENTITIES`	`COMMIT WORK`
Triggers the RAP save sequence	Yes	Yes

	`COMMIT ENTITIES`	`COMMIT WORK`
On an early-save failure	Returns to the interaction phase, `sy-subrc = 4`	**Runtime error**
RAP responses available	Yes (`FAILED` , `REPORTED`)	No
Key conversion for late numbering	Yes (`CONVERT KEY`)	No
Simulation mode	Yes	No
Update mode	Local update, falling back to synchronous. Never asynchronous	Consumer's choice

Use `COMMIT ENTITIES` . `COMMIT WORK` only behaves identically in the best case, and turns a recoverable situation into a dump in every other case.

`ROLLBACK ENTITIES` and `ROLLBACK WORK` are interchangeable in RAP.

17.5 Who calls `COMMIT ENTITIES`

Consumer	Who commits
Fiori / OData	**The RAP runtime engine, implicitly.** You never write it.
Your own ABAP class / report / job	**You**, explicitly, after your EML
A RAP handler or saver method	**Nobody.** Committing inside the BO is forbidden

This is why a handler method must never contain `COMMIT ENTITIES` or `COMMIT WORK` : the BO is not the transaction owner.

17.6 What is allowed in which phase

First, separate the concerns

"Database writes only happen in one method" is a useful slogan, but it compresses several different things. Keep these apart:

Concern	The actual rule
1. RAP transactional processing	Changes to *this BO's business data* are made in the **transactional buffer** during the interaction phase, via EML. They are not database writes at all at that point.
2. Direct SQL restrictions per phase	`INSERT` / `UPDATE` / `DELETE` / `MODIFY` issued directly by your code against the **standard database connection** belong in the **late save phase**. Earlier phases would create additional database transactions and break the single SAP LUW. Reads (`SELECT`) are allowed in every phase.
3. Framework-managed persistence	In a plain **managed** BO, *you write no persistence code at all*. The framework converts the buffer into `INSERT` / `UPDATE` / `DELETE` on `persistent table` during the late save phase. Concern 2 simply does not arise.
4. Additional save	The framework still persists the BO. Your `save_modified` runs **in addition**, in the late save phase, for side-effect work (change documents, application log, events).
5. Unmanaged save / unmanaged BO	**You** perform the persistence, in `save_modified` / `save` , in the late save phase. This is the case where concern 2 is your direct responsibility.
6. External APIs, update task, secondary connections	Governed by their own per-phase rules (matrix below), not by the "database writes" rule. A number range commits on a secondary connection and is legal in every phase; an update-task registration is legal only in the late save phase; a synchronous RFC is legal everywhere *except* the late save phase.

So the precise statement is:

> In RAP, business data changes are performed through the RAP transactional buffer during the interaction phase, and persistence is finalized by the RAP save sequence. Direct database modifications on the standard connection outside the late save phase can break transactional consistency and should be avoided.

Not: "you may only ever write to the database in one method." In a managed BO you write to the database nowhere; in an unmanaged save you write in `save_modified` ; and side-channel writes on a secondary connection (logs, number ranges, traces) follow different rules entirely.

The matrix

This comes from SAP's RAP BO contract. Violating it produces either a runtime error (under strict mode) or, worse, silent data inconsistency. Treat it as the authoritative version of the slogan above, and check it against your release's documentation for anything unusual.

Statement / operation	Interaction	Early save	Late save	Note
Database **read** (`SELECT`)	yes	yes	yes	On the BO's own data use EML, not SELECT
Database **modification** (`INSERT` / `UPDATE` / `DELETE` / `MODIFY`)	**no**	**no**	**yes**	Only the late phase, so there is exactly one DB transaction
Database commit on a **secondary connection**	yes	yes	yes	Legal for number ranges, application logs, traces
Database commit on the **standard connection**	yes	yes	**no**	Would split the LUW

Statement / operation	Interaction	Early save	Late save	Note
`CALL FUNCTION ... IN UPDATE TASK`	**no**	**no**	**yes**	Cannot be unregistered; registering earlier breaks draft
`CALL FUNCTION ... DESTINATION` (sRFC)	yes	yes	**no**	Not in the late phase
`CALL FUNCTION ... STARTING NEW TASK` (aRFC)	yes	yes	**no**	Same
`CALL FUNCTION ... IN BACKGROUND UNIT` (bgRFC)	**no**	**no**	**yes**	
`CALL FUNCTION ... IN BACKGROUND TASK` (tRFC/qRFC)	**no**	**no**	**no**	Obsolete
`PERFORM ON COMMIT` / `ON ROLLBACK`	(possible)	(possible)	yes	Signals poor RAP integration; prefer EML
`COMMIT WORK` / `ROLLBACK WORK`	**no**	**no**	**no**	Not where RAP is the transaction owner
`WAIT`	yes	yes	**no**	Implies a DB commit
Dynpro statements: `MESSAGE` without `INTO`, `WRITE`, `CALL SCREEN`, `SET SCREEN`, `LEAVE SCREEN`, `STOP`, `SUPPRESS DIALOG`	**no**	**no**	**no**	Runtime error
`CALL TRANSACTION`, `LEAVE TRANSACTION`, `SUBMIT`	**no**	**no**	**no**	
`EXIT FROM STEP-LOOP`, `LEAVE TO LIST-PROCESSING`	**no**	**no**	**no**	

The two rules worth memorising

1. **Business data changes go through the transactional buffer; persistence is finalized by the save sequence.** Where you write direct SQL at all (unmanaged, unmanaged save), it belongs in the late save phase, on the standard connection. Side-channel writes on a secondary connection are a separate case.
2. **Nothing that can fail should be left to the late save phase.** Detect it in `check_before_save`, while the transaction is still recoverable.

Rule 2 is close to absolute — it is the whole point of the early/late split. Rule 1 has the legitimate exceptions listed in the matrix, so read it as a default with known carve-outs rather than as a prohibition.

Secondary connections — the useful exception

A number range object commits on its own connection. That is why `cl_numberrange_runtime=>number_get( )` is legal in an `on save` determination even though it performs a commit: it does not touch the standard connection and therefore does not split your LUW.

The same applies to application logs written with `BAL_DB_SAVE` when configured for a secondary connection, and to trace/monitoring writes.

17.7 Failures in the late save phase

Normally, a failure after the point of no return is a runtime error. RAP offers a controlled escape for cases where it is genuinely unavoidable — e.g. wrapping a BAPI whose behaviour you cannot fully predict.

```
CLASS lsc_zr_traveltp DEFINITION INHERITING FROM cl_abap_behavior_saver_failed.
  PROTECTED SECTION.
    METHODS save_modified REDEFINITION.
ENDCLASS.
```

`CL_ABAP_BEHAVIOR_SAVER_FAILED` gives `save_modified` access to `failed` and `reported`, which the plain base class does not have in the late phase.

Consumer side:

```
COMMIT ENTITIES RESPONSE OF zr_traveltp
  FAILED   DATA(failed)
  REPORTED DATA(reported).

CASE sy-subrc.
  WHEN 0.
    " committed
  WHEN 4.
    " early save failed - correctable, we are back in the interaction phase
  WHEN 8.
    " LATE save failed. A further RAP operation would dump.
    ROLLBACK ENTITIES.
ENDCASE.
```

Use this sparingly. It is a mitigation, not a design pattern. The right answer is almost always to move the check into `check_before_save`.

17.8 A full trace of one save

User edits a Travel in Fiori and presses Save. With draft enabled:

```
1  PATCH /Travel(...,IsActiveEntity=false)  BookingFee = 30
     -> interaction phase
     -> determination calculateTotalPrice (on modify)  -> TotalPrice = 30 + sum(bookings)
```

```
     -> side effect tells the UI to re-read TotalPrice

2  POST .../Prepare
     -> draft determine action Prepare
     -> validation validateDates      runs -> ok
     -> validation validateCustomer   runs -> ok
     -> validation validateAgency     runs -> ok
     -> messages returned, UI shows none

3  POST .../Activate
     -> RAP runtime issues COMMIT ENTITIES implicitly

4  EARLY SAVE
     finalize()
       -> determination setTravelId (on save) runs
            -> cl_numberrange_runtime=>number_get( )   [secondary connection: legal]
            -> TravelId = 00000042
     check_before_save()
       -> validations on save re-run (skipped if 'Activate optimized' and Prepare covered them)
       -> no failures

5  ===== POINT OF NO RETURN =====

6  LATE SAVE
     adjust_numbers()      [not called - no late numbering declared]
     save()                [managed: framework writes ZTRAVEL and ZBOOKING,
                            deletes the draft rows from ZTRAVEL_D / ZBOOKING_D]
     implicit COMMIT WORK
     cleanup()             [buffer cleared, lock released]

7  HTTP 200 with the activated instance -> UI leaves edit mode
```

17.9 Debugging the save sequence

Breakpoints that are worth setting

Where	Tells you
`finalize`	Did the early save start? Are `on save` determinations running?
`check_before_save`	Is the buffer what you expect at the last gate?
`cleanup_finalize`	Something rejected the save. Inspect `failed` from the previous method
`save` / `save_modified`	What is actually being written
`cleanup`	Did the transaction complete?

The diagnostic that answers most questions

```
COMMIT ENTITIES RESPONSE OF zr_traveltp
  FAILED   DATA(failed)
  REPORTED DATA(reported).

out->write( |sy-subrc: { sy-subrc }| ).
out->write( failed ).
out->write( reported ).
```

`failed` tells you *which instance* was rejected; `reported` tells you *why*. Run this from an `IF_OO_ADT_CLASSRUN` class rather than through the UI — it is faster and reproducible.

Simulation mode

```
COMMIT ENTITIES IN SIMULATION MODE
  RESPONSE OF zr_traveltp
  FAILED DATA(sim_failed) REPORTED DATA(sim_reported).
```

Runs the early save phase (so all `on save` determinations and validations execute) and then rolls back. Nothing is written. Perfect for a "check document" button and for tests.

17.10 Common save-sequence problems

Symptom	Cause
Runtime error "*COMMIT WORK is not allowed here*"	`COMMIT WORK` inside a handler or saver
Runtime error in the late phase	Something that can fail was left in `save`. Move it to `check_before_save`
Data saved but a field is empty	The determination that fills it is `on modify` and its trigger never fired; or `mapping for` is incomplete
Second save in the same session behaves oddly	`CLASS-DATA` or an unmanaged buffer not cleared in `cleanup` / `cleanup_finalize`

Symptom	Cause
`sy-subrc = 4` and nothing in `reported`	A validation filled `failed` without a message
`sy-subrc = 8`	Late-phase failure. `ROLLBACK ENTITIES` is mandatory before continuing
Number range gaps	`setTravelId` is `on modify`; every discarded draft burned a number
`save_modified` never called	Nothing on **this** BO changed in the transaction
Update task function module never executes	Registered outside the late save phase
Everything works via EML but not via Fiori	Draft: your code uses `%key` instead of `%tky`, so it operates on the active instance

17.11 Interview answer

> **"Explain the RAP save sequence."**
>
> A RAP transaction has two phases. In the **interaction phase**, handler methods run and changes go into the transactional buffer. The database is untouched, and the buffer is allowed to be temporarily inconsistent.
>
> `COMMIT ENTITIES` — issued implicitly by the RAP runtime in a Fiori scenario — then triggers the **save sequence**, which splits in two.
>
> The **early save phase** runs `finalize`, where `on save` determinations execute and it is the last point at which data may be changed; then `check_before_save`, where `on save` validations execute and which is the last gate. If either reports a failure, `cleanup_finalize` runs and the transaction returns to the interaction phase — `sy-subrc` is 4 and it is recoverable.
>
> Then the **point of no return**, and the **late save phase**: `adjust_numbers` for late numbering, then `save` or `save_modified`, then the implicit `COMMIT WORK`, then `cleanup` clears the buffer.
>
> The design principle is that everything which can fail must fail in the early phase. A failure after the point of no return is a runtime error, unless you inherit from `CL_ABAP_BEHAVIOR_SAVER_FAILED`, in which case `sy-subrc` is 8 and the consumer must issue an explicit `ROLLBACK ENTITIES`.
>
> On phase rules: business data changes go through the buffer, and persistence is finalized by the save sequence. Where you write direct SQL at all — unmanaged, or unmanaged save — it belongs in the late phase on the standard connection. There are documented carve-outs: reads are allowed everywhere, and commits on a **secondary** connection, such as a number range or an application log, are legal in every phase. That is why drawing a number range in an `on save` determination does not break the LUW.
>
> And `COMMIT ENTITIES` rather than `COMMIT WORK`: on an early-save failure `COMMIT WORK` dumps, whereas `COMMIT ENTITIES` returns you to the interaction phase with a usable `failed` and `reported`.

17.12 Production considerations

- **Check `sy-subrc` after every `COMMIT ENTITIES`.** A silent commit that failed looks exactly like one that succeeded.
- **`sy-subrc = 8` requires an explicit `ROLLBACK ENTITIES`** before any further RAP operation, or the next call dumps.
- **Move every failure-capable check into `check_before_save`**, including test-mode BAPI calls. This is the single most important production rule in the whole model.
- `COMMIT ENTITIES IN SIMULATION MODE` is the right implementation for a "check document" button, and for tests — it runs the whole early phase and writes nothing.
- **Clear all own state in `cleanup` and `cleanup_finalize`.** The second transaction in a session is where stale state surfaces.
- **Never register an update-task module outside the late save phase** — it cannot be unregistered and it breaks draft.
- **Document the phase rules for the team.** This is where a well-meaning `COMMIT WORK` added later does the most damage.

Chapter 18 — EML (Entity Manipulation Language) Reference

18.1 What EML is

EML is the set of ABAP statements for calling a RAP business object from ABAP code. It is the *only* supported way to talk to a RAP BO from ABAP — not SQL, not an OData HTTP call to yourself.

Statement	Purpose
`READ ENTITIES`	Read instances, read by association, execute a function
`MODIFY ENTITIES`	Create / update / delete / create-by-association / execute an action
`COMMIT ENTITIES`	Trigger the save sequence and commit
`ROLLBACK ENTITIES`	Discard the transaction
`GET PERMISSIONS`	Ask what is allowed for given instances
`SET LOCKS`	Lock instances explicitly
`RAISE ENTITY EVENT`	Raise a RAP business event
`CONVERT KEY`	Resolve a preliminary id to a real key (late numbering, inside `save_modified`)

Two consumers:

- **Outside the BO** (a report, a job, a class, another BO) — full statements, then `COMMIT ENTITIES`.
- **Inside the BO** (a behavior pool) — always with `IN LOCAL MODE`, and **never** `COMMIT ENTITIES`.

18.2 Derived types

The framework generates ABAP types from the BDEF. You never declare these structures by hand.

```
" --- instance data ---
DATA lt_create  TYPE TABLE FOR CREATE zr_traveltp.
DATA ls_create  TYPE STRUCTURE FOR CREATE zr_traveltp.
DATA lt_update  TYPE TABLE FOR UPDATE zr_traveltp.
DATA lt_delete  TYPE TABLE FOR DELETE zr_traveltp.
DATA lt_read    TYPE TABLE FOR READ IMPORT zr_traveltp.
DATA lt_result  TYPE TABLE FOR READ RESULT zr_traveltp.

" --- child entity (double backslash) ---
DATA lt_bcreate TYPE TABLE FOR CREATE zr_traveltp\\Booking.

" --- create by association (single backslash + association name) ---
DATA lt_cba     TYPE TABLE FOR CREATE zr_traveltp\_Booking.

" --- actions ---
DATA lt_action  TYPE TABLE FOR ACTION IMPORT zr_traveltp~acceptTravel.
DATA lt_ares    TYPE TABLE FOR ACTION RESULT zr_traveltp~acceptTravel.

" --- keys ---
DATA lt_keys    TYPE TABLE FOR KEY OF zr_traveltp.

" --- responses ---
DATA ls_mapped   TYPE RESPONSE FOR MAPPED   zr_traveltp.
DATA ls_failed   TYPE RESPONSE FOR FAILED   zr_traveltp.
DATA ls_reported TYPE RESPONSE FOR REPORTED zr_traveltp.

" --- for save_modified (additional / unmanaged save) ---
DATA ls_change   TYPE REQUEST FOR CHANGE zr_traveltp.
DATA ls_del      TYPE REQUEST FOR DELETE zr_traveltp.

" --- permissions ---
DATA ls_perm     TYPE RESPONSE FOR PERMISSIONS REQUEST zr_traveltp.
```

The three separator characters

Written	Means
`zr_traveltp`	The root entity
`zr_traveltp\\Booking`	A **child entity** of the BO (double backslash)
`zr_traveltp\_Booking`	The **association** `_Booking` from the root (single backslash)
`zr_traveltp~acceptTravel`	An **action** of the root (tilde)
`zr_traveltp\\Booking~someAction`	An action of a child entity

Getting `\\` and `\_` confused is a frequent activation error. Mnemonic: a **single** backslash is a **navigation step**; a **double** backslash is a **node name**.

18.3 Special components (`%` components)

These appear in every derived type. They are the RAP protocol.

Component	Meaning	Where
`%key`	The key fields	Everywhere
`%tky`	*Transactional key* = `%key` + `%is_draft`. **Use this, not `%key`, in draft BOs**	Everywhere
`%is_draft`	`mk-on` = draft, `mk-off` = active	Draft BOs
`%cid`	Content id — a name you invent for a not-yet-created instance	Create
`%cid_ref`	"My parent is the instance with this `%cid`"	Create-by-association
`%target`	The table of children to create	Create-by-association
`%control`	Per-field flags: which fields were actually sent (`mk-on` / `mk-off`)	Update, read, `save_modified`
`%param`	Action/function parameter, or the action result payload	Actions, functions
`%data`	The instance data (in some response structures)	Responses
`%msg`	A message object implementing `IF_ABAP_BEHV_MESSAGE`	`reported`
`%element`	Which **field** a message refers to (highlighted in the UI)	`reported`
`%state_area`	A named area for persistent state messages	`reported`
`%fail-cause`	Why an instance failed	`failed`
`%path`	The path from the root to a child instance	Deep responses
`%pid`	Preliminary id, late numbering	`mapped`
`%pky`	Preliminary key	Late numbering
`%assoc-_X`	Permission/feature for an association	`GET PERMISSIONS`, features
`%action-X`	Permission/feature for an action	`GET PERMISSIONS`, features
`%field-X`	Feature for a field	Features
`%update`, `%delete`, `%create`	Permission/feature for an operation	`GET PERMISSIONS`, features
`%other`	Messages not tied to any instance	`reported-%other`

Constants

```
if_abap_behv=>mk-on          " '01' - set / true / is draft
if_abap_behv=>mk-off         " '00' - not set / false / is active

if_abap_behv=>fc-o-enabled   " operation/action enabled
if_abap_behv=>fc-o-disabled  " operation/action disabled
if_abap_behv=>fc-f-unrestricted  " field editable
if_abap_behv=>fc-f-read_only     " field read-only
if_abap_behv=>fc-f-mandatory     " field mandatory

if_abap_behv=>auth-allowed
if_abap_behv=>auth-unauthorized

if_abap_behv=>cause-unspecific
if_abap_behv=>cause-unauthorized
if_abap_behv=>cause-locked
if_abap_behv=>cause-disabled
if_abap_behv=>cause-not_found
if_abap_behv=>cause-conflict
if_abap_behv=>cause-dependency

if_abap_behv_message=>severity-error
if_abap_behv_message=>severity-warning
if_abap_behv_message=>severity-information
if_abap_behv_message=>severity-success
```

18.4 `MODIFY ENTITIES` — create

Short form (one entity, one operation)

```
MODIFY ENTITY zr_traveltp
  CREATE FIELDS ( AgencyId CustomerId BeginDate EndDate BookingFee CurrencyCode )
  WITH VALUE #( ( %cid         = 'CID_1'
```

```
                  AgencyId     = '070001'
                  CustomerId   = '000001'
                  BeginDate    = '20260601'
                  EndDate      = '20260610'
                  BookingFee   = '20.00'
                  CurrencyCode = 'EUR' ) )
  MAPPED   DATA(mapped)
  FAILED   DATA(failed)
  REPORTED DATA(reported).
```

Note `MODIFY ENTITY` (singular) in the short form, `MODIFY ENTITIES` (plural) in the long form.

Long form (several entities / operations in one statement)

```
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    CREATE FIELDS ( AgencyId CustomerId BeginDate EndDate CurrencyCode )
    WITH VALUE #( ( %cid = 'T1' AgencyId = '070001' CustomerId = '000001'
                    BeginDate = '20260601' EndDate = '20260610' CurrencyCode = 'EUR' ) )
  ENTITY Travel
    UPDATE FIELDS ( Description )
    WITH VALUE #( ( %tky-TravelUUID = lv_existing_uuid
                    %tky-%is_draft  = if_abap_behv=>mk-off
                    Description     = 'updated' ) )
  MAPPED   DATA(mapped)
  FAILED   DATA(failed)
  REPORTED DATA(reported).
```

`CREATE FIELDS ( ... )` vs `CREATE SET FIELDS WITH` vs `CREATE AUTO FILL CID`

Form	Meaning
`CREATE FIELDS ( A B C ) WITH tab`	Only fields A, B, C are sent. `%control` is derived from this list. **Preferred** — explicit.
`CREATE SET FIELDS WITH tab`	The `%control` values in `tab` decide what is sent. Use when you build `%control` dynamically.
`CREATE AUTO FILL CID WITH tab`	The framework generates `%cid` values for you. Convenient when you do not need to reference the created instances afterwards.
`CREATE AUTO FILL CID SET FIELDS WITH tab`	Both.

Deep create — parent and children in one statement

```
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    CREATE FIELDS ( AgencyId CustomerId BeginDate EndDate CurrencyCode )
    WITH VALUE #( ( %cid       = 'T1'
                    AgencyId   = '070001'
                    CustomerId = '000001'
                    BeginDate  = '20260601'
                    EndDate    = '20260610'
                    CurrencyCode = 'EUR' ) )

    CREATE BY \_Booking
      FIELDS ( CustomerId CarrierId ConnectionId FlightDate FlightPrice CurrencyCode )
      WITH VALUE #( ( %cid_ref = 'T1'                       " <- links to the parent above
                      %target  = VALUE #(
                        ( %cid = 'B1' CustomerId = '000001' CarrierId = 'LH'
                          ConnectionId = '0400' FlightDate = '20260601'
                          FlightPrice = '200.00' CurrencyCode = 'EUR' )
                        ( %cid = 'B2' CustomerId = '000001' CarrierId = 'LH'
                          ConnectionId = '0401' FlightDate = '20260610'
                          FlightPrice = '210.00' CurrencyCode = 'EUR' ) ) ) )
  MAPPED   DATA(mapped)
  FAILED   DATA(failed)
  REPORTED DATA(reported).

COMMIT ENTITIES.

" the real keys the framework assigned
DATA(travel_uuid)  = mapped-travel[  %cid = 'T1' ]-TravelUUID.
DATA(booking1_uuid) = mapped-booking[ %cid = 'B1' ]-BookingUUID.
```

This is the mechanism behind "create a document with its items in one transaction". `%cid` / `%cid_ref` / `%target` is the whole trick.

18.5 `MODIFY ENTITIES` — update and delete

```
" UPDATE
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    UPDATE FIELDS ( Description BookingFee )
    WITH VALUE #( ( %tky-TravelUUID = lv_uuid
                    %tky-%is_draft  = if_abap_behv=>mk-off
                    Description     = 'new text'
                    BookingFee      = '25.00' ) )
```

```
  FAILED   DATA(failed)
  REPORTED DATA(reported).

" UPDATE with explicit %control (dynamic field selection)
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    UPDATE SET FIELDS
    WITH VALUE #( ( %tky-TravelUUID = lv_uuid
                    %tky-%is_draft  = if_abap_behv=>mk-off
                    Description     = 'new text'
                    %control-Description = if_abap_behv=>mk-on
                    %control-BookingFee  = if_abap_behv=>mk-off ) ).

" DELETE
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    DELETE FROM VALUE #( ( %tky-TravelUUID = lv_uuid
                           %tky-%is_draft  = if_abap_behv=>mk-off ) )
  FAILED   DATA(failed)
  REPORTED DATA(reported).
```

Deleting a root **cascades** to its compositions in a managed BO. You do not delete the Bookings separately.

18.6 `MODIFY ENTITIES` — actions

```
" instance action, no parameter
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    EXECUTE acceptTravel
    FROM VALUE #( ( %tky-TravelUUID = lv_uuid
                    %tky-%is_draft  = if_abap_behv=>mk-off ) )
  RESULT   DATA(action_result)
  FAILED   DATA(failed)
  REPORTED DATA(reported).

" with a parameter
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    EXECUTE deductDiscount
    FROM VALUE #( ( %tky-TravelUUID = lv_uuid
                    %tky-%is_draft  = if_abap_behv=>mk-off
                    %param-discount_percent = 15
                    %param-reason_code      = 'LOYAL' ) )
  RESULT DATA(discount_result).

" static action
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    EXECUTE archiveOldTravels
    FROM VALUE #( ( %param-cutoff_date = '20240101' ) ).

" draft actions
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    EXECUTE Edit     FROM VALUE #( ( %tky-TravelUUID = lv_uuid %tky-%is_draft = if_abap_behv=>mk-off ) ).
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    EXECUTE Activate FROM VALUE #( ( %tky-TravelUUID = lv_uuid %tky-%is_draft = if_abap_behv=>mk-on ) ).
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    EXECUTE Discard  FROM VALUE #( ( %tky-TravelUUID = lv_uuid %tky-%is_draft = if_abap_behv=>mk-on ) ).
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    EXECUTE Resume   FROM VALUE #( ( %tky-TravelUUID = lv_uuid %tky-%is_draft = if_abap_behv=>mk-on ) ).
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel
    EXECUTE Prepare  FROM VALUE #( ( %tky-TravelUUID = lv_uuid %tky-%is_draft = if_abap_behv=>mk-on ) )
  REPORTED DATA(prepare_reported).
```

18.7 `READ ENTITIES`

```
" read one entity
READ ENTITIES OF zr_traveltp
  ENTITY Travel
    FIELDS ( TravelId AgencyId TotalPrice OverallStatus )
    WITH VALUE #( ( %tky-TravelUUID = lv_uuid
                    %tky-%is_draft  = if_abap_behv=>mk-off ) )
  RESULT   DATA(travels)
  FAILED   DATA(failed)
  REPORTED DATA(reported).

" all fields
READ ENTITIES OF zr_traveltp
  ENTITY Travel
    ALL FIELDS WITH VALUE #( ( %tky-TravelUUID = lv_uuid %tky-%is_draft = if_abap_behv=>mk-off ) )
```

```
  RESULT DATA(travels_full).

" read by association: the children of given parents
READ ENTITIES OF zr_traveltp
  ENTITY Travel BY \_Booking
    FIELDS ( BookingId FlightPrice CurrencyCode )
    WITH VALUE #( ( %tky-TravelUUID = lv_uuid %tky-%is_draft = if_abap_behv=>mk-off ) )
  RESULT DATA(bookings).

" read by association: only the LINKS, no data (cheaper)
READ ENTITIES OF zr_traveltp
  ENTITY Travel BY \_Booking
    FROM VALUE #( ( %tky-TravelUUID = lv_uuid %tky-%is_draft = if_abap_behv=>mk-off ) )
  LINK DATA(booking_links).

" read the parent from a child
READ ENTITIES OF zr_traveltp
  ENTITY Booking BY \_Travel
    FIELDS ( TravelId OverallStatus )
    WITH VALUE #( ( %tky-BookingUUID = lv_booking_uuid %tky-%is_draft = if_abap_behv=>mk-off ) )
  RESULT DATA(parent_travels).

" several reads in one statement
READ ENTITIES OF zr_traveltp
  ENTITY Travel
    FIELDS ( TravelId TotalPrice ) WITH CORRESPONDING #( keys )
    RESULT DATA(travels2)
  ENTITY Travel BY \_Booking
    FIELDS ( FlightPrice ) WITH CORRESPONDING #( keys )
    RESULT DATA(bookings2)
  FAILED DATA(failed2).

" execute a function
READ ENTITIES OF zr_traveltp
  ENTITY Travel
    EXECUTE getPriceBreakdown
    FROM VALUE #( ( %tky-TravelUUID = lv_uuid %tky-%is_draft = if_abap_behv=>mk-off ) )
  RESULT DATA(breakdown).
```

`WITH CORRESPONDING #( keys )`

Inside a handler method you receive `keys` typed for that specific operation. `WITH CORRESPONDING #( keys )` maps it to the type `READ ENTITIES` expects. It is the idiom you will write hundreds of times; it also carries `%tky` across correctly, which hand-built key tables often do not.

18.8 `IN LOCAL MODE` and `PRIVILEGED`

```
READ ENTITIES OF zr_traveltp IN LOCAL MODE ...
MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE ...
```

	Without	With `IN LOCAL MODE`
Authorization checks	Run	**Skipped**
Feature control	Applied	**Skipped**
`readonly` field restrictions	Enforced	**Lifted** — you can write them
Projection restrictions	Applied	Skipped
Recursion risk	Yes	No
Performance	Slower	Faster

Inside a behavior pool, always use `IN LOCAL MODE`. Outside, never — you want the checks.

```
READ ENTITIES OF zr_traveltp PRIVILEGED ...
MODIFY ENTITIES OF zr_traveltp PRIVILEGED ...
```

`PRIVILEGED` skips authorization checks from **outside** the BO. Only available in classes declared `FOR BEHAVIOR OF` (behavior pools and auxiliary classes). Use it for background jobs that must operate with full rights — and document why, every time.

```
" both
MODIFY ENTITIES OF zr_traveltp PRIVILEGED IN LOCAL MODE ...
```

`AUGMENTING`

```
MODIFY AUGMENTING ENTITY zc_traveltp ...
```

Used in a **projection** behavior pool to enrich an incoming request before it reaches the base BO — for example to fill a field the UI does not send. Declared in the projection BDEF with `augment`. Niche but occasionally the clean answer.

18.9 `GET PERMISSIONS`

Ask the BO what is currently allowed, without attempting it.

```
READ ENTITIES OF zr_traveltp
  ENTITY Travel
    REQUEST VALUE #( %update              = if_abap_behv=>mk-on
                     %delete              = if_abap_behv=>mk-on
                     %action-acceptTravel = if_abap_behv=>mk-on
                     %assoc-_Booking      = if_abap_behv=>mk-on
                     %field-BeginDate     = if_abap_behv=>mk-on )
    GET PERMISSIONS
    FROM VALUE #( ( %tky-TravelUUID = lv_uuid %tky-%is_draft = if_abap_behv=>mk-off ) )
  RESULT DATA(permissions)
  FAILED DATA(failed).

LOOP AT permissions-instances INTO DATA(perm).
  IF perm-%action-acceptTravel = if_abap_behv=>fc-o-enabled.
    " the Accept button would work for this instance
  ENDIF.
ENDLOOP.
```

`permissions` has three parts:

- `permissions-global` — global authorization + global features
- `permissions-instances` — per-instance authorization + features
- `permissions-permissions` — the combined result

This is exactly what the Fiori UI calls to decide button enablement. Use it in a batch job to filter the instances you can actually act on, instead of attempting each and handling failures.

18.10 `SET LOCKS`

```
SET LOCKS OF zr_traveltp
  ENTITY Travel
  FROM VALUE #( ( %tky-TravelUUID = lv_uuid %tky-%is_draft = if_abap_behv=>mk-off ) )
  FAILED   DATA(lock_failed)
  REPORTED DATA(lock_reported).

IF lock_failed IS NOT INITIAL.
  " someone else holds the lock
ENDIF.
```

Normally the framework locks automatically on modify. Use `SET LOCKS` when you want to reserve instances **before** doing expensive preparation, so you fail fast.

18.11 `RAISE ENTITY EVENT`

```
" BDEF
define behavior for ZR_TravelTP alias Travel
{
  ...
  event TravelStatusChanged parameter ZE_TravelStatusEvent;
}
```

```
" in save_modified (the recommended place for a managed BO with additional save)
RAISE ENTITY EVENT zr_traveltp~TravelStatusChanged
  FROM VALUE #( ( %key-TravelUUID = ls-TravelUUID
                  %param = VALUE #( travel_uuid = ls-TravelUUID
                                    new_status  = ls-OverallStatus ) ) ).
```

Events are raised in the **late save phase** so that subscribers only see committed state. SAP's guidance is to define the BO `with additional save` and raise events in `save_modified`.

18.12 Complete patterns

A) Background job: mass update with error handling

```
CLASS zcl_travel_mass_accept DEFINITION PUBLIC FINAL CREATE PUBLIC
  FOR BEHAVIOR OF zr_traveltp.          " needed for PRIVILEGED

  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
ENDCLASS.
```

```
CLASS zcl_travel_mass_accept IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.

    " 1. find candidates with plain SQL (finding, not reading BO state)
    SELECT FROM ztravel
      FIELDS travel_uuid
      WHERE overall_status = 'O'
        AND end_date < @sy-datum
      INTO TABLE @DATA(candidates)
      UP TO 1000 ROWS.

    CHECK candidates IS NOT INITIAL.

    DATA(keys) = VALUE ztravel_tky_tab(
      FOR c IN candidates ( TravelUUID = c-travel_uuid
                            %is_draft  = if_abap_behv=>mk-off ) ).

    " 2. ask what is actually permitted, rather than failing per instance
    READ ENTITIES OF zr_traveltp PRIVILEGED
      ENTITY Travel
        REQUEST VALUE #( %action-acceptTravel = if_abap_behv=>mk-on )
        GET PERMISSIONS
        FROM CORRESPONDING #( keys )
      RESULT DATA(perms).

    DATA(allowed) = VALUE ztravel_tky_tab( ).
    LOOP AT perms-instances INTO DATA(p)
         WHERE %action-acceptTravel = if_abap_behv=>fc-o-enabled.
      APPEND VALUE #( TravelUUID = p-TravelUUID
                      %is_draft  = if_abap_behv=>mk-off ) TO allowed.
    ENDLOOP.
    CHECK allowed IS NOT INITIAL.

    " 3. execute
    MODIFY ENTITIES OF zr_traveltp PRIVILEGED
      ENTITY Travel
        EXECUTE acceptTravel FROM CORRESPONDING #( allowed )
      FAILED   DATA(failed)
      REPORTED DATA(reported).

    " 4. commit and check
    COMMIT ENTITIES RESPONSE OF zr_traveltp
      FAILED   DATA(commit_failed)
      REPORTED DATA(commit_reported).

    CASE sy-subrc.
      WHEN 0.
        out->write( |Accepted { lines( allowed ) - lines( commit_failed-travel ) } travels| ).
      WHEN 4.
        out->write( 'Early save failed:' ).
        out->write( commit_reported ).
      WHEN 8.
        out->write( 'Late save failed - rolling back' ).
        ROLLBACK ENTITIES.
    ENDCASE.

  ENDMETHOD.
ENDCLASS.
```

B) Calling a released SAP BO (Purchase Requisition)

```
DATA pr_headers TYPE TABLE FOR CREATE i_purchaserequisitiontp.
DATA pr_items   TYPE TABLE FOR CREATE i_purchaserequisitiontp\_PurchaseRequisitionItem.

APPEND VALUE #( %cid                   = 'PR1'
                purchaserequisitiontype = 'NB'
                PurReqnDescription      = 'From the Travel app' ) TO pr_headers.

APPEND VALUE #( %cid_ref = 'PR1'
                %target  = VALUE #( ( %cid                     = 'PR1_I1'
                                      plant                    = '1010'
                                      accountassignmentcategory = 'U'
                                      requestedquantity        = 1
                                      baseunit                 = 'EA'
                                      material                 = 'TG11'
                                      materialgroup            = 'L001'
                                      purchasinggroup          = '001'
                                      purchasingorganization   = '1010'
                                      DeliveryDate             = sy-datum + 30 ) ) )
       TO pr_items.

MODIFY ENTITIES OF i_purchaserequisitiontp
  ENTITY purchaserequisition
    CREATE FIELDS ( purchaserequisitiontype PurReqnDescription )
    WITH pr_headers
    CREATE BY \_purchaserequisitionitem
      FIELDS ( plant accountassignmentcategory requestedquantity baseunit
               material materialgroup purchasinggroup purchasingorganization DeliveryDate )
      WITH pr_items
  MAPPED   DATA(pr_mapped)
  FAILED   DATA(pr_failed)
  REPORTED DATA(pr_reported).
```

```
IF pr_failed IS NOT INITIAL.
  " handle
ENDIF.

COMMIT ENTITIES RESPONSE OF i_purchaserequisitiontp FAILED DATA(pr_commit_failed).
```

`I_PurchaseRequisitionTP` uses **late numbering** — `pr_mapped` carries `%pid`, not the real document number. Resolving it requires `CONVERT KEY` inside `save_modified`. See Chapter 16, §16.8.

C) Reading a released BO instead of its tables

```
" instead of SELECT FROM ekko / ekpo
READ ENTITIES OF i_purchaseordertp
  ENTITY PurchaseOrder
    FIELDS ( PurchaseOrder Supplier PurchaseOrderDate )
    WITH VALUE #( ( PurchaseOrder = '4500000123' ) )
  RESULT DATA(pos)
  FAILED DATA(failed).
```

Reading via the BO honours its authorizations and its released contract. Reading `EKKO` directly does not, and is not a released API in ABAP Cloud.

18.13 EML performance

Rule	Why
One EML statement with many instances, never one per instance in a loop	Each statement is a framework round trip
`FIELDS ( ... )` rather than `ALL FIELDS`	Reads less
`LINK DATA( )` rather than `RESULT DATA( )` when you only need existence	Avoids reading child data
`IN LOCAL MODE` inside the pool	Skips authorization and feature control
Combine reads into one `READ ENTITIES ... ENTITY A ... ENTITY B ...`	One round trip
`GET PERMISSIONS` once for the whole set, before acting	Avoids attempt-and-fail per instance
`COMMIT ENTITIES` once per logical unit, not per instance	One save sequence
Chunk very large sets (e.g. 1000 instances per commit)	Keeps the buffer and the lock table bounded

The anti-pattern

```
" WRONG — 5000 EML statements and 5000 save sequences
LOOP AT lt_uuids INTO DATA(lv_uuid).
  MODIFY ENTITIES OF zr_traveltp ENTITY Travel
    UPDATE FIELDS ( OverallStatus )
    WITH VALUE #( ( %tky-TravelUUID = lv_uuid %tky-%is_draft = if_abap_behv=>mk-off
                    OverallStatus = 'A' ) ).
  COMMIT ENTITIES.
ENDLOOP.

" RIGHT — one statement, one commit
MODIFY ENTITIES OF zr_traveltp ENTITY Travel
  UPDATE FIELDS ( OverallStatus )
  WITH VALUE #( FOR u IN lt_uuids ( %tky-TravelUUID = u
                                    %tky-%is_draft  = if_abap_behv=>mk-off
                                    OverallStatus   = 'A' ) ).
COMMIT ENTITIES.
```

18.14 EML troubleshooting

Symptom	Cause
"*Entity does not exist*" at activation	Wrong separator: `\\` vs `\_` vs `~`
`RESULT` is empty	Wrong key. In a draft BO, `%key` was used instead of `%tky`
Create succeeds but `mapped` is empty	`%cid` not set, or `AUTO FILL CID` used and you looked for your own cid
Child created without a parent link	`%cid_ref` missing or pointing at a `%cid` that is not in the same statement
Update silently ignored	Field not listed in `FIELDS ( ... )`, so `%control` is off for it
"*Operation not allowed*"	`use update;` missing in the projection BDEF (if you called the projection), or feature control disabled it
Works with `IN LOCAL MODE`, fails without	An authorization check or feature control is rejecting it — which is correct behaviour
Nothing is saved	`COMMIT ENTITIES` missing
`COMMIT ENTITIES` "works" but nothing is in the table	`sy-subrc` was 4 and you did not check it

Symptom	Cause
"*COMMIT not allowed*"	`COMMIT ENTITIES` inside a behavior pool
Deep create fails with "parent not found"	`%cid_ref` refers to an instance created in a *different* EML statement. It must be the same one

18.15 Interview answer

> **"What is EML, and explain `%cid`, `%tky`, `%pid` and `%control`."**
>
> EML — Entity Manipulation Language — is the set of ABAP statements for calling a RAP business object: `READ ENTITIES`, `MODIFY ENTITIES`, `COMMIT ENTITIES`, `ROLLBACK ENTITIES`, `GET PERMISSIONS`, `SET LOCKS`, `RAISE ENTITY EVENT` and `CONVERT KEY`. It is the only supported way to reach a RAP BO from ABAP — you do not `SELECT` its tables and you do not call your own OData service over HTTP.
>
> `%cid` is the **content ID**: a name you invent for an instance that has no key yet. You send it on create and the framework tells you through `mapped` which real key it assigned. In a deep create the child uses `%cid_ref` to name its parent's `%cid`, and `%target` to carry the children — that is how you create a document and its items in one statement.
>
> `%tky` is the **transactional key** — the key plus `%is_draft`. In a draft-enabled BO the same key exists twice, active and draft, so `%key` alone is ambiguous. Using `%key` there reads the wrong copy, which usually presents as "my determination sees old data". `strict ( 2 )` makes it a syntax error.
>
> `%pid` is a **preliminary ID** in late-numbering scenarios, resolved to the real key with `CONVERT KEY` inside `save_modified`.
>
> `%control` is the set of per-field flags saying which fields the consumer actually sent. A managed BO honours it automatically; in an unmanaged BO or an unmanaged save you must respect it yourself, or a user who edits one field has every other field overwritten with initial values.
>
> One more thing worth saying: inside a behavior pool every EML call takes `IN LOCAL MODE` — buffer direct, skips authorization and feature control, and lets you write `readonly` fields. Outside the BO, never, because you want those checks.

18.16 Production considerations

- **One statement for many instances, never one per instance in a loop.** Each EML statement is a framework round trip, and a commit per instance is a save sequence per instance.
- **Chunk mass processing** (around 1,000 instances) so the buffer and the lock table stay bounded and the job is restartable.
- `GET PERMISSIONS` **before acting** in a batch, rather than attempting each instance and handling failures.
- `PRIVILEGED` **bypasses authorization.** Use it only where a job genuinely needs it, and document why each time.
- `FIELDS ( ... )` **not** `ALL FIELDS` ; `LINK` **not** `RESULT` when only existence matters.
- **Always check** `sy-subrc` **/** `failed` **after** `COMMIT ENTITIES`**.**
- **Never** `COMMIT ENTITIES` **inside a behavior pool** — the BO is not the transaction owner.
- **Read released BOs through EML, not their tables.** In ABAP Cloud the tables are not released APIs, and EML honours the BO's own authorizations.

Chapter 19 — Numbering, Locking and ETag

Part 1 — Numbering

19.1 The two dimensions

Key assignment in RAP is described by two independent questions:

When	
Early numbering	The final key is assigned during the create operation, in the **interaction phase**
Late numbering	The final key is assigned during the **save sequence**, in `adjust_numbers`

By whom	
External numbering	The **consumer** supplies the key
Internal numbering	The **provider** (framework or your code) supplies the key

Combining them gives the scenarios below. **The default, when nothing is declared, is external numbering** — the consumer must send a key.

19.2 The five numbering scenarios

A) External early numbering (the default)

The consumer sends the key on create.

```
define behavior for ZR_MaterialTP alias Material
persistent table zmaterial
{
  create;
  field ( mandatory : create, readonly : update ) MaterialId;
}
```

- The consumer must guarantee uniqueness. A duplicate is rejected at save.
- Key fields should be `mandatory : create, readonly : update` — writable when creating, frozen afterwards.
- Use when an external system or a business rule dictates the number.

B) Managed internal early numbering (UUID) — **the recommended default**

```
field ( numbering : managed, readonly ) TravelUUID;
```

- The framework generates a UUID at create time.
- Prerequisite: the key is a 16-byte field (`sysuuid_x16` / `RAW(16)`).
- `readonly` makes it fully internal. **Omitting** `readonly` gives *optional external numbering*: the framework fills it only if the consumer sends an initial value.
- Zero code.

This is why Chapter 02 recommends a UUID technical key: it makes numbering a one-word declaration.

C) Unmanaged internal early numbering

You assign the key in the interaction phase, in a `FOR NUMBERING` handler.

```
" BDEF
define behavior for ZR_TravelTP alias Travel
persistent table ztravel
early numbering
{
  create;
  field ( readonly ) TravelId;
  association _Booking { create; }
}
```

```
METHODS earlynumbering_create FOR NUMBERING
  IMPORTING entities FOR CREATE Travel.

METHODS earlynumbering_cba_Booking FOR NUMBERING
  IMPORTING entities FOR CREATE Travel\_Booking.
```

```
METHOD earlynumbering_create.

  " only instances without a key
  DATA(entities_wo_id) = entities.
  DELETE entities_wo_id WHERE TravelId IS NOT INITIAL.
  CHECK entities_wo_id IS NOT INITIAL.

  TRY.
      cl_numberrange_runtime=>number_get(
        EXPORTING nr_range_nr = '01'
                  object      = 'ZTRAVELID'
                  quantity    = CONV #( lines( entities_wo_id ) )
        IMPORTING number              = DATA(number_range_key)
                  returned_quantity   = DATA(returned_quantity) ).
    CATCH cx_number_ranges INTO DATA(lx).
      LOOP AT entities_wo_id INTO DATA(e).
        APPEND VALUE #( %cid = e-%cid %key = e-%key ) TO failed-travel.
        APPEND VALUE #( %cid = e-%cid %key = e-%key %msg = lx ) TO reported-travel.
      ENDLOOP.
      RETURN.
  ENDTRY.

  DATA(next) = number_range_key - returned_quantity.
  LOOP AT entities_wo_id INTO DATA(entity).
    next += 1.
    APPEND VALUE #( %cid     = entity-%cid
                    TravelId = next ) TO mapped-travel.
  ENDLOOP.

ENDMETHOD.
```

The key is reported through `mapped`, not written back into `entities`. That is the contract of `FOR NUMBERING`.

For children, the pattern is the same but the numbering is usually per-parent:

```
METHOD earlynumbering_cba_Booking.

  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel BY \_Booking
      FROM CORRESPONDING #( entities )
    LINK DATA(existing_links).

  LOOP AT entities INTO DATA(entity).

    " highest existing BookingId for this travel
    DATA(max_id) = REDUCE /dmo/booking_id(
      INIT m = CONV /dmo/booking_id( '0000' )
      FOR link IN existing_links WHERE ( source-%tky = entity-%tky )
      NEXT m = COND #( WHEN link-target-BookingId > m THEN link-target-BookingId ELSE m ) ).

    LOOP AT entity-%target INTO DATA(target).
      max_id += 10.
      APPEND VALUE #( %cid      = target-%cid
                      BookingId = max_id ) TO mapped-booking.
    ENDLOOP.
  ENDLOOP.
ENDMETHOD.
```

D) Late numbering

The key only exists in the late save phase.

```
define behavior for ZR_TravelTP alias Travel
persistent table ztravel
late numbering
{ ... }
```

```
METHODS adjust_numbers REDEFINITION.      " in the saver class
```

```
METHOD adjust_numbers.
  LOOP AT mapped-travel ASSIGNING FIELD-SYMBOL(<travel>).
    " <travel>-%pid is the preliminary id
    <travel>-TravelId = zcl_travel_numbers=>get_next( ).
  ENDLOOP.
ENDMETHOD.
```

When to use it:

- The number must be **gap-free** (legal requirement for some document types). Drawing it at save time minimises gaps.
- The number comes from a system that only issues it at commit time.
- You are wrapping an API that itself uses late numbering.

Costs:

- Draft tables need an extra key field `draftuuid` (`RAW(16)`).
- The consumer receives `%pid` from `mapped`, not a real key, until after the commit.

- Resolving `%pid` requires `CONVERT KEY`, which is only legal inside `save_modified`.

E) Optional external numbering

```
field ( numbering : managed ) TravelUUID;     " note: NO 'readonly'
```

The framework fills the key **only if** the consumer leaves it initial. Useful for a BO consumed both by a UI (no key) and by a migration program (keys supplied).

19.3 `CONVERT KEY` — resolving a preliminary id

Only valid inside `save_modified`.

```
METHOD save_modified.

  LOOP AT zbp_r_traveltp=>mapped_purchase_requisition-purchaserequisition
       ASSIGNING FIELD-SYMBOL(<pr>).

    CONVERT KEY OF i_purchaserequisitiontp
      FROM <pr>-%pid
      TO   DATA(ls_pr_key).

    <pr>-purchaserequisition = ls_pr_key-purchaserequisition.
  ENDLOOP.

  " now write the real number into our own table
  ...
ENDMETHOD.
```

The full worked example — including the bridge variable in the global ABP class — is in Chapter 16, §16.8. This is the single most common reason to choose `managed with unmanaged save`.

19.4 Choosing a numbering strategy

Requirement	Strategy
New app, no external constraint	**B** — `field ( numbering : managed, readonly )` on a UUID key
A human-readable document number is also needed	**B** for the technical key + a **determination** `on save` for the business number (Chapter 10, §10.3)
The number must be gap-free by law	**D** — late numbering
An external system supplies the number	**A** — external
The number comes from a released API that uses late numbering	**D** + `managed with unmanaged save` + `CONVERT KEY`
Both UI and migration must create instances	**E** — optional external
Child items numbered 10, 20, 30 within a parent	**C** — `FOR NUMBERING` on the cba

> **Do not draw a number range number in an `on modify` determination.** Every discarded draft burns a number. Use `on save`.

Part 2 — Locking

19.5 Pessimistic locking

```
define behavior for ZR_TravelTP alias Travel
lock master                     " the root owns the lock

define behavior for ZR_BookingTP alias Booking
lock dependent by _Travel       " children delegate to the parent
```

Declaration	Meaning
`lock master`	Managed: the framework acquires and releases the lock automatically
`lock master unmanaged`	You implement `FOR LOCK` yourself
`lock dependent by _Assoc`	The lock is the parent's
(nothing)	Not permitted under `strict ( 2 )` for modifiable entities

What the framework does

Before the first modification of an instance, a lock entry is written to the **global lock table** (the same infrastructure as `ENQUEUE`). A second request for the same instance is rejected. The entry is removed on commit, rollback, or session end.

The lock covers the **whole composition tree**: locking a Travel locks its Bookings, because they are `lock dependent by _Travel`.

Per-operation exemption

```
action ( lock : none ) printTravel;
```

For a read-only action that should not block other users.

Manual locking with `SET LOCKS`

```
SET LOCKS OF zr_traveltp
  ENTITY Travel
  FROM VALUE #( ( %tky-TravelUUID = lv_uuid %tky-%is_draft = if_abap_behv=>mk-off ) )
  FAILED DATA(lock_failed).
```

Use it to reserve instances before expensive preparation, so you fail fast instead of after doing the work.

Unmanaged locking

```
METHOD lock.
  LOOP AT keys INTO DATA(key).
    CALL FUNCTION 'ENQUEUE_EZTRAVEL'
      EXPORTING  travel_uuid  = key-TravelUUID
                 _scope       = '1'
      EXCEPTIONS foreign_lock = 1 OTHERS = 2.
    IF sy-subrc <> 0.
      APPEND VALUE #( %tky = key-%tky %fail-cause = if_abap_behv=>cause-locked )
             TO failed-travel.
    ENDIF.
  ENDLOOP.
ENDMETHOD.
```

Requires a lock object (SE11 → Lock Object `EZTRAVEL` over `ZTRAVEL`). Release with `DEQUEUE_EZTRAVEL` in `save`.

19.6 Draft locking — the two phases

Covered in detail in Chapter 11, §11.5. In summary:

Phase	Trigger	Behaviour
Exclusive lock	`Edit` / `Create`	Nobody else can edit. Survives the session ending. Expires after a configured timeout
Optimistic lock	Exclusive lock expired	Others may edit. On `Resume`, the draft's **total ETag** is compared with the active instance's. Mismatch → the draft is declared outdated

This is why `total etag` is mandatory for draft-enabled BOs.

Part 3 — ETag (optimistic concurrency)

19.7 What an ETag does

```
User A reads Travel 42          -> ETag = 2026-09-16T10:00:00
User B reads Travel 42          -> ETag = 2026-09-16T10:00:00
User B updates and saves        -> ETag becomes 2026-09-16T10:05:00
User A updates and saves        -> sends If-Match: 2026-09-16T10:00:00
                                -> MISMATCH -> rejected
                                   "Data has been changed by another user"
```

Without an ETag, User A's save silently overwrites User B's change. This is the *lost update* problem, and it is the single most common data-integrity defect in hand-written CRUD applications.

19.8 Declaring it

```
define behavior for ZR_TravelTP alias Travel
lock master
total etag LastChangedAt              " draft resumability
etag master LocalLastChangedAt        " optimistic concurrency
{
```

```
  field ( readonly ) LastChangedAt, LocalLastChangedAt;
}

define behavior for ZR_BookingTP alias Booking
lock dependent by _Travel
etag master LastChangedAt             " children may have their own
```

Declaration	Meaning
`etag master <field>`	This entity has its own ETag field
`etag dependent by _Assoc`	The ETag is taken from the parent
`total etag <field>`	Draft-only. Must come **after** `lock master`. Use a **different** field

Prerequisites

1. Field type `utclong`, `timestamp` or `timestampl`.
2. CDS annotation: - `@Semantics.systemDateTime.localInstanceLastChangedAt: true` for `etag master` - `@Semantics.systemDateTime.lastChangedAt: true` for `total etag`
3. `field ( readonly )` in the BDEF.

With all three in place, the **managed runtime maintains the ETag automatically**. No code.

In the projection BDEF

```
define behavior for ZC_TravelTP alias Travel
use etag                              " <- without this, no optimistic locking in this service
{ ... }
```

Forgetting `use etag` silently disables the protection for that service.

19.9 Lock vs ETag — they are not alternatives

	Lock (pessimistic)	ETag (optimistic)
Acquired	Before editing	Nothing is acquired
Checked	At acquisition	At save
Blocks other users	Yes	No
Protects against	Two users starting an edit	A stale overwrite
Fails when	Someone else is editing	Someone else already saved
Survives session end	Yes (draft: until timeout)	N/A
Needed in RAP	Yes	Yes

Use both. A lock stops concurrent editing sessions; an ETag stops a stale write that slips through when the lock has expired (the draft optimistic phase) or when there is no draft at all.

19.10 Unmanaged ETag handling

In an unmanaged BO the framework does nothing for you:

```
METHOD update.
  LOOP AT entities INTO DATA(entity).
    " 1. Compare the incoming ETag with what is stored
    SELECT SINGLE last_changed_at FROM ztravel
      WHERE travel_uuid = @entity-TravelUUID
      INTO @DATA(current_etag).

    IF entity-%control-LastChangedAt = if_abap_behv=>mk-on
       AND entity-LastChangedAt <> current_etag.
      APPEND VALUE #( %tky        = entity-%tky
                      %fail-cause = if_abap_behv=>cause-conflict ) TO failed-travel.
      CONTINUE.
    ENDIF.

    " 2. Apply the change and set a NEW ETag
    GET TIME STAMP FIELD DATA(new_etag).
    " ... write to the buffer with last_changed_at = new_etag ...
  ENDLOOP.
ENDMETHOD.
```

One more item on the list of what unmanaged costs you.

19.11 Troubleshooting

Symptom	Cause
"*Data has been changed by another user*" on the first save	Stale `$metadata` in the browser, or the ETag field is not `readonly` so the client sends its own value
ETag has no effect	`use etag` missing in the projection BDEF
BDEF activation: "*ETag field must be of type timestamp*"	Wrong data element; use `abp_locinst_lastchange_tstmpl`
BDEF activation: "*Total ETag is missing*"	`with draft;` declared without `total etag <field>`
"*total etag must be specified after lock master*"	Order of the clauses
"*Object is locked by user X*" and X is the same user	An orphaned draft from an earlier session holds the exclusive lock. Discard the draft, or wait for the timeout
Duplicate key on save with external numbering	The consumer supplied a key that already exists — expected behaviour
Number range gaps	The number is drawn `on modify` instead of `on save`
`%pid` in `mapped` instead of a real key	Late numbering — resolve with `CONVERT KEY` in `save_modified`
`FOR NUMBERING` never called	`early numbering` missing in the BDEF
`adjust_numbers` never called	`late numbering` missing in the BDEF

19.12 Interview answer

> **"How did you handle numbering, locking and ETags?"**
>
> Numbering is described by two independent questions: **when** the key is assigned — early, in the interaction phase, or late, in the save sequence — and **by whom** — the consumer, externally, or the provider, internally. The default with no declaration is external.
>
> I use managed internal early numbering for the technical key: a UUID with `field ( numbering : managed, readonly )`, which the framework generates with no code. The human-readable document number is separate, drawn from a number range in a determination declared `on save`, so discarded drafts do not consume numbers. Late numbering I would use only where the number must be gap-free by law, or where a called API itself uses late numbering — and then resolving the preliminary ID needs `CONVERT KEY` in `save_modified`, which forces `managed with unmanaged save`.
>
> Locking and ETags solve different problems and I use both. The **lock** is pessimistic — `lock master` on the root, `lock dependent by` on the children — and stops a second user starting an edit. The **ETag** is optimistic — `etag master LocalLastChangedAt` plus the matching `@Semantics` annotation and `readonly`, so the managed runtime maintains it — and stops a stale overwrite at the moment of saving. Plus `use etag` in the projection BDEF, without which the protection is silently off for that service.
>
> With draft there is a third: `total etag`, on a **different** field, which is what decides whether a draft is still resumable after the exclusive lock has expired and someone else may have changed the active instance.

19.13 Production considerations

- **Number range intervals are not transported.** Maintain them in `SNRO` per system and per client; a missing document number in QA is nearly always this.
- **Draw numbers `on save`.** Gaps from discarded drafts are an audit finding in regulated scenarios, not a cosmetic issue.
- **Number ranges commit on a secondary connection**, which is why they are legal in an `on save` determination — but handle `cx_number_ranges` and report it, or the save fails with no message.
- **Review the draft exclusive-lock timeout** against real editing times.
- **Use `( lock : none )` on read-only actions** so they do not block other users.
- **`SET LOCKS` before expensive preparation** in a job, so you fail fast.
- **Test the ETag conflict over HTTP** — the comparison happens in the OData layer, not in the buffer, so EML will not exercise it.
- **Lock table sizing matters for mass processing.** Every instance holds an entry until commit; chunk accordingly.

Chapter 20 — Messages, Troubleshooting and Cheat Sheet

Part 1 — Messages

20.1 The three ways to produce a message

A) `new_message_with_text( )` — prototyping only

```
APPEND VALUE #( %tky = key-%tky
                %msg = new_message_with_text(
                         severity = if_abap_behv_message=>severity-error
                         text     = 'End date must be after begin date' ) )
       TO reported-travel.
```

Not translatable. Fine while developing, never in a transport that leaves DEV.

B) `new_message( )` — translatable, no exception class

```
APPEND VALUE #( %tky = key-%tky
                %msg = new_message( id       = 'ZTRAVEL_MSG'
                                    number   = '001'
                                    severity = if_abap_behv_message=>severity-error
                                    v1       = CONV #( travel-BeginDate )
                                    v2       = CONV #( travel-EndDate ) ) )
       TO reported-travel.
```

Points at a message in SE91 message class `ZTRAVEL_MSG`. `v1` .. `v4` fill `&1` .. `&4`. **This is the pragmatic default** — translatable, and it needs no extra class.

C) An exception class implementing `IF_ABAP_BEHV_MESSAGE` — the full pattern

```
APPEND VALUE #( %tky = key-%tky
                %msg = NEW zcx_travel_messages(
                         severity  = if_abap_behv_message=>severity-error
                         textid    = zcx_travel_messages=>end_before_begin
                         begindate = travel-BeginDate
                         enddate   = travel-EndDate ) )
       TO reported-travel.
```

Full class source: Chapter 09, §9.6.

Use it when:

- the same message is raised from many places and you want one typed constructor,
- you also need to `RAISE` the exception in non-RAP code,
- you want compile-time safety on the message parameters.

20.2 The four components of a message entry

```
APPEND VALUE #(
  %tky               = key-%tky                       " 1. which instance
  %state_area        = 'VALIDATE_DATES'               " 2. persistent message area
  %msg               = <the message object>           " 3. the message
  %element-BeginDate = if_abap_behv=>mk-on            " 4. which field to highlight
  %element-EndDate   = if_abap_behv=>mk-on
) TO reported-travel.
```

Component	Effect in Fiori
`%tky`	Links the message to a row; clicking the message navigates to it
`%state_area`	Makes the message **persistent** until explicitly cleared
`%msg`	The text, severity and message class
`%element-<Field>`	The field is outlined in red and the message appears under it

Without `%element`, the message only appears in the message popover — the user has to guess which field is wrong.

20.3 Transient vs state messages

	Transient	State
Declared with	`%msg` only	`%msg` + `%state_area`
Lifetime	The current request	Until the same `%state_area` is cleared
Use for	Action results, information	Validation errors on a draft

The mandatory clear-first pattern

```
LOOP AT travels INTO DATA(travel).

  " 1. ALWAYS clear the area for this instance
  APPEND VALUE #( %tky        = travel-%tky
                  %state_area = 'VALIDATE_DATES' ) TO reported-travel.

  " 2. Then add a message only if the check fails
  IF <check fails>.
    APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.
    APPEND VALUE #( %tky        = travel-%tky
                    %state_area = 'VALIDATE_DATES'
                    %msg        = ...
                    %element-BeginDate = if_abap_behv=>mk-on ) TO reported-travel.
  ENDIF.
ENDLOOP.
```

Skipping step 1 leaves the error on screen after the user has fixed the data. Use **one** `%state_area` **per validation** so validations do not clear each other's messages.

20.4 `failed` vs `reported`

	`failed`	`reported`
Purpose	"This instance could not be processed"	"Here is a message about it"
Effect on the save	**Blocks it**	None on its own
Visible to the user	No	Yes

```
" ERROR that blocks the save
APPEND VALUE #( %tky = key-%tky ) TO failed-travel.
APPEND VALUE #( %tky = key-%tky %msg = ... severity-error ) TO reported-travel.

" WARNING that does not block
APPEND VALUE #( %tky = key-%tky %msg = ... severity-warning ) TO reported-travel.
```

`failed` **without** `reported` → the save is silently blocked and the user sees nothing. Worst case. `reported` **without** `failed` → a red message appears and the record saves anyway. Second worst.

Always pair them for errors.

`%fail-cause`

```
APPEND VALUE #( %tky        = key-%tky
                %fail-cause = if_abap_behv=>cause-unauthorized ) TO failed-travel.
```

Cause	Use for
`unspecific`	Default
`unauthorized`	Authorization failure
`locked`	Lock failure
`disabled`	Blocked by feature control
`not_found`	Instance does not exist
`conflict`	ETag mismatch
`dependency`	A dependent object blocked it

The consumer (and generic error handling) can react differently per cause.

20.5 Messages for a specific field of a child entity

```
APPEND VALUE #( %tky               = booking-%tky
                %msg               = ...
                %element-FlightDate = if_abap_behv=>mk-on ) TO reported-booking.
```

Note `reported-booking`, using the **alias in lower case**. For messages not tied to any instance:

```
APPEND VALUE #( %msg = ... ) TO reported-%other.
```

Part 2 — Troubleshooting index

20.6 Symptom → cause, by area

Activation errors

Message	Cause
"Entity must be a root entity"	`root` missing on the CDS view
"Association _X must be redirected"	`redirected to composition child` / `parent` missing in the projection
"No behavior definition found for ZC_..."	`provider contract transactional_query` without a projection BDEF
"Extension of view ... is not allowed"	`@Metadata.allowExtensions: true` missing
"Total ETag is missing"	`with draft;` without `total etag`
"ETag field must be of type timestamp"	Wrong data element
"persistent table is not allowed"	`persistent table` together with `with unmanaged save`
"Implementation class must be specified in the header"	Class declared at node level while using additional/unmanaged save
"Association must be draft enabled"	`with draft;` missing on an association in a draft BO
"Method ... is not implemented"	Handler method missing — regenerate with `Ctrl + 1`
"Method ... declared more than once"	The same handler method in two handler classes
"Use of ... not permitted in a projection view"	A join or aggregation in an `as projection on` view
Draft table will not activate	Hand-typed rather than generated. Regenerate with the ADT quick fix; the required structure is release-dependent

UI behaves wrongly

Symptom	Cause
Button does not appear	`use action X;` missing in the projection BDEF (check first); or `dataAction` spelling/case; or browser cache
Button greyed out for everyone	`get_instance_features` does not set `%action-X`
Button always enabled	`( features : instance )` missing
"+" missing in a child table	`association _Child { create; }` in the base, **and** `use association _Child { create; }` in the projection
Calculated field does not refresh	`side effects` not declared
Button enablement does not refresh	`field X affects permissions;` missing
Section renders empty	`targetQualifier` does not match any `qualifier`
Child table has no columns	The child MDE has no `@UI.lineItem`
Value help is empty	The value-help entity is not exposed in the service definition
Annotation change has no effect	Browser cache; MDE not activated; binding not re-published; annotated the wrong view
Mobile layout unusable	No `importance` on line items
List Report times out on open	No mandatory filter on a large table

Runtime / logic

Symptom	Cause
Determination reads stale data	Used `SELECT` instead of `READ ENTITIES ... IN LOCAL MODE`
`READ ENTITIES` returns nothing in a draft BO	Used `%key` instead of `%tky`
Validation message shown but the record saves	`failed` not filled

Symptom	Cause
Validation message persists after the fix	`%state_area` not cleared first
Validation only fires at Save	Not listed in `draft determine action Prepare`
Cannot write a `readonly` field from a determination	Missing `IN LOCAL MODE`
Second save in the session misbehaves	`CLASS-DATA` / unmanaged buffer not cleared in `cleanup` and `cleanup_finalize`
"COMMIT WORK not allowed"	`COMMIT WORK` / `COMMIT ENTITIES` inside a behavior pool
Runtime error in the late save phase	A failure-capable operation left in `save`; move the check to `check_before_save`
`save_modified` never called	Nothing on **this** BO changed
Number range gaps	Number drawn `on modify` instead of `on save`
Update wipes untouched fields	`%control` ignored (unmanaged / unmanaged save)
Nothing is persisted	`COMMIT ENTITIES` missing, or `sy-subrc = 4` unchecked
Determination loops / dumps	Indirect determination cycle (A changes B, B's determination changes A)
Works in EML but not in Fiori	Draft: `%key` vs `%tky`, or a missing `use` in the projection BDEF
Works in Fiori but not in EML	Authorization / feature control — EML without `IN LOCAL MODE` runs the full checks

20.7 The diagnostic sequence

When something does not work, go through this in order. It resolves most cases within three steps.

1. **Re-activate in dependency order**: tables → interface views (child first) → BDEF → ABP → projection views (child first) → projection BDEF → MDE → service definition → service binding → **Publish**.
2. **Clear the browser cache** (`F12` → *Empty cache and hard reload*).
3. **Look at `$metadata`.** If the field / action / annotation is not there, the problem is in CDS or the BDEF, not the UI.
4. **Drive the BO from an `IF_OO_ADT_CLASSRUN` class with EML** and print `failed` / `reported` / `sy-subrc`. This separates "the BO is wrong" from "the UI is wrong".
5. **Set a breakpoint** in the handler method and run the class with `F9`.
6. **Check the projection BDEF** for a missing `use`.

Part 3 — Cheat Sheet

20.8 Object naming

Prefix	Layer
`ZTRAVEL`	DDIC table
`ZTRAVEL_D`	Draft table
`ZI_`	Basic interface view
`ZR_...TP`	RAP root / interface view
`ZC_...TP`	Projection / consumption view
`ZBP_R_...TP`	Behavior pool
`ZSD_`	Service definition
`ZUI_..._O4`	Service binding, OData V4, UI
`ZE_`	CDS abstract entity
`lhc_<Alias>`	Local handler class
`lsc_<BDEF>`	Local saver class

20.9 BDEF skeleton

```
managed implementation in class ZBP_R_TravelTP unique;
strict ( 2 );
with draft;

define behavior for ZR_TravelTP alias Travel
persistent table ztravel
draft table ztravel_d
lock master
total etag LastChangedAt
```

```
authorization master ( instance )
etag master LocalLastChangedAt
{
  create; update; delete;

  field ( numbering : managed, readonly ) TravelUUID;
  field ( readonly )  CreatedBy, CreatedAt, LastChangedBy, LastChangedAt, LocalLastChangedAt;
  field ( mandatory ) AgencyId, CustomerId, BeginDate, EndDate;

  association _Booking { create; with draft; }

  determination setTravelId on save { create; }
  validation validateDates  on save { field BeginDate, EndDate; create; }
  action ( features : instance ) acceptTravel result [1] $self;

  side effects { field BookingFee affects field TotalPrice; }

  draft action Activate optimized;
  draft action Discard;
  draft action Edit;
  draft action Resume;
  draft determine action Prepare { validation validateDates; }

  mapping for ztravel corresponding;
}
```

20.10 Handler method signatures

```
FOR DETERMINE ON MODIFY IMPORTING keys FOR Travel~<det>.
FOR DETERMINE ON SAVE   IMPORTING keys FOR Travel~<det>.
FOR VALIDATE ON MODIFY  IMPORTING keys FOR Travel~<val>.
FOR VALIDATE ON SAVE    IMPORTING keys FOR Travel~<val>.
FOR MODIFY IMPORTING keys FOR ACTION Travel~<act> RESULT result.
FOR READ   IMPORTING keys FOR FUNCTION Travel~<fn> RESULT result.
FOR INSTANCE FEATURES       IMPORTING keys REQUEST requested_features       FOR Travel RESULT result.
FOR GLOBAL FEATURES         IMPORTING      REQUEST requested_features       FOR Travel RESULT result.
FOR INSTANCE AUTHORIZATION  IMPORTING keys REQUEST requested_authorizations FOR Travel RESULT result.
FOR GLOBAL AUTHORIZATION    IMPORTING      REQUEST requested_authorizations FOR Travel RESULT result.
FOR PRECHECK IMPORTING entities FOR UPDATE Travel.
FOR NUMBERING IMPORTING entities FOR CREATE Travel.
FOR MODIFY IMPORTING entities FOR CREATE Travel.
FOR MODIFY IMPORTING entities FOR UPDATE Travel.
FOR MODIFY IMPORTING keys     FOR DELETE Travel.
FOR READ   IMPORTING keys     FOR READ   Travel RESULT result.
FOR LOCK   IMPORTING keys     FOR LOCK   Travel.
FOR READ   IMPORTING keys_rba FOR READ Travel\_Booking FULL result_requested
             RESULT result LINK association_links.
FOR MODIFY IMPORTING entities_cba FOR CREATE Travel\_Booking.
```

20.11 Saver methods

Method	Phase	Purpose
`finalize`	Early save	Final data changes. `on save` determinations run here
`check_before_save`	Early save	Last gate. `on save` validations run here
`cleanup_finalize`	Early save, on failure	Undo `finalize`
`adjust_numbers`	Late save	Late numbering
`save`	Late save	Persist (unmanaged)
`save_modified`	Late save	Persist / add to persist (additional / unmanaged save)
`cleanup`	After save or rollback	Clear the buffer and own state

20.12 EML quick reference

```
READ ENTITIES OF <bo> [IN LOCAL MODE] [PRIVILEGED]
  ENTITY <Alias> FIELDS ( F1 F2 ) WITH CORRESPONDING #( keys ) RESULT DATA(r)
  ENTITY <Alias> ALL FIELDS WITH ... RESULT DATA(r2)
  ENTITY <Alias> BY \_Child FIELDS ( ... ) WITH ... RESULT DATA(c)
  ENTITY <Alias> BY \_Child FROM ... LINK DATA(links)
  ENTITY <Alias> EXECUTE <function> FROM ... RESULT DATA(fr)
  FAILED DATA(f) REPORTED DATA(rep).

MODIFY ENTITIES OF <bo> [IN LOCAL MODE] [PRIVILEGED]
  ENTITY <Alias> CREATE FIELDS ( ... ) WITH tab
                 CREATE BY \_Child FIELDS ( ... ) WITH cba_tab
  ENTITY <Alias> UPDATE FIELDS ( ... ) WITH tab
  ENTITY <Alias> DELETE FROM tab
  ENTITY <Alias> EXECUTE <action> FROM tab RESULT DATA(ar)
  MAPPED DATA(m) FAILED DATA(f) REPORTED DATA(rep).
```

```
COMMIT ENTITIES [RESPONSE OF <bo>] [IN SIMULATION MODE]
  FAILED DATA(f) REPORTED DATA(rep).

ROLLBACK ENTITIES.

READ ENTITIES OF <bo> ENTITY <Alias>
  REQUEST VALUE #( %update = if_abap_behv=>mk-on )
  GET PERMISSIONS FROM tab RESULT DATA(perm).

SET LOCKS OF <bo> ENTITY <Alias> FROM tab FAILED DATA(lf).

RAISE ENTITY EVENT <bo>~<Event> FROM tab.

CONVERT KEY OF <bo> FROM <pid> TO DATA(key).   " only inside save_modified
```

`sy-subrc` after `COMMIT ENTITIES` : `0` ok · `4` early save failed (recoverable) · `8` late save failed (`ROLLBACK ENTITIES` required).

20.13 Constants

```
if_abap_behv=>mk-on / mk-off
if_abap_behv=>fc-o-enabled / fc-o-disabled                      " operations & actions
if_abap_behv=>fc-f-unrestricted / fc-f-read_only / fc-f-mandatory " fields
if_abap_behv=>auth-allowed / auth-unauthorized
if_abap_behv=>cause-unauthorized / locked / disabled / not_found / conflict / dependency
if_abap_behv_message=>severity-error / warning / information / success
```

20.14 Derived-type separators

Written	Means
`zr_traveltp`	Root entity
`zr_traveltp\\Booking`	Child **entity**
`zr_traveltp\_Booking`	**Association**
`zr_traveltp~acceptTravel`	**Action**

20.15 The rules that prevent most bugs

1. **Never** `SELECT` **the BO's own tables** from a behavior pool. Use `READ ENTITIES ... IN LOCAL MODE`.
2. **Never** `COMMIT WORK` in a behavior pool.
3. **Database writes only in the late save phase.**
4. **Everything that can fail must fail in** `check_before_save`**.**
5. **Always use** `%tky` in a draft BO, never `%key`.
6. **Always** `IN LOCAL MODE` inside a behavior pool.
7. **A rejecting validation fills both** `failed` **and** `reported`**.**
8. **State messages clear their** `%state_area` **first.**
9. **Determinations must be idempotent.**
10. `use ...` **in the projection BDEF for every operation, association and action** the app needs.
11. `side effects` for every calculated field and every state-driven button.
12. **Clear** `CLASS-DATA` **and unmanaged buffers** in `cleanup` and `cleanup_finalize`.
13. `strict ( 2 )` on every new BDEF.
14. **No** `SELECT` **inside a loop.** One set-based read outside it.
15. **Generate method signatures with** `Ctrl + 1`, never by hand.

20.16 Build order

```
tables -> draft tables -> interface views (child first) -> BDEF -> behavior pool
  -> message class -> projection views (child first) -> projection BDEF
  -> metadata extensions -> service definition -> service binding -> Publish
```

20.17 Further reading

- SAP Help — ABAP RESTful Application Programming Model
- SAP Help — Save Options (additional / unmanaged save)
- SAP Help — Implementing the Interaction Phase and the Save Sequence

- SAP-samples/abap-cheat-sheets — RAP Behavior Definition Language
- SAP-samples/abap-cheat-sheets — EML / ABAP for RAP
- SAP-samples/abap-platform-rap-workshops
- SAP Help — Defining Unmanaged Save in the Behavior Definition
- ABAP Keyword Documentation — RAP saver methods
- ADT: the **ABAP Flight Reference Scenario** (`/DMO/` packages) — a complete, activatable managed *and* unmanaged implementation of exactly this model, shipped with the system.

Chapter 21 — Side Effects

21.1 What is it?

A **side effect** is a declaration in the behavior definition that tells the **consumer** (in practice: Fiori Elements) "*when X changes, the data you are holding for Y is now stale — go and read it again.*"

It changes nothing on the server. It changes what the client asks for.

```
side effects
{
  field BookingFee affects field TotalPrice;
}
```

Read it as: "*after the user changes* `BookingFee`*, re-request* `TotalPrice`*.*"

21.2 Why do we need it?

Because OData is stateless and the UI is not clairvoyant.

When the user types into `BookingFee`, Fiori Elements sends a `PATCH` containing **only that field** and gets back a response containing — by default — only what it asked about. Meanwhile, on the server, your determination recalculated `TotalPrice`. That new value sits in the transactional buffer, correct and invisible.

```
  UI                               Server
  --                               ------
  PATCH { BookingFee: 30 }   ---->  buffer: BookingFee = 30
                                    determination calculateTotalPrice runs
                                    buffer: TotalPrice = 430
  <----  200 { BookingFee: 30 }
  screen: BookingFee 30
          TotalPrice  400   <-- STALE. The user sees the old number.
```

The determination is not broken. The UI simply never asked.

> **This is the single most common "my determination doesn't work" false alarm in RAP.** Before debugging a determination, check whether a side effect is declared. If you can see the correct value after pressing F5, the determination is fine and the side effect is missing.

21.3 Where is it used?

Situation	Declare
A determination derives field B from field A	`field A affects field B;`
A determination derives a header total from item changes	`field Price affects entity _Travel;` (on the child)
An action changes fields on the instance	`action X affects field F;` — or give the action `result [1] $self` instead
An action creates or deletes child rows	`action X affects entity _Booking;`
An action produces messages the user must see	`action X affects messages;`
A field change enables or disables a button	`field Status affects permissions;`
Draft: `Prepare` produces validation messages	`determine action Prepare affects messages;`

Where to declare it — base BDEF or projection BDEF?

Both are legal. The useful split:

Put it in the…	For
Base BDEF	Side effects that follow from the **business logic** and are true for every consumer — e.g. a total that always recalculates
Projection BDEF	Side effects that exist because of **this service's UI** — e.g. refreshing a field that only this app displays

If in doubt, the base BDEF is the safer default: every projection inherits it.

21.4 Syntax

```
side effects
{
  // a field change invalidates another field
  field BookingFee               affects field TotalPrice;

  // one field invalidates several
  field CurrencyCode             affects field TotalPrice, BookingFee;

  // several fields invalidate one
  field BeginDate, EndDate       affects field TripDuration;

  // a field change invalidates a whole associated entity (the child table)
  field CurrencyCode             affects entity _Booking;

  // a field change invalidates the instance itself (everything)
  field OverallStatus            affects $self;

  // a field change requires re-evaluating enablement / feature control
  field OverallStatus            affects permissions;

  // an action invalidates fields, entities, messages, permissions
  action acceptTravel            affects field OverallStatus;
  action createBookingSet        affects entity _Booking;
  action validateDocument        affects messages;
  action acceptTravel            affects permissions;

  // draft: refresh the message list after Prepare
  determine action Prepare       affects messages;

  // a determination's effect (rarely needed - usually you name the field instead)
  determination calculateTotalPrice affects field TotalPrice;
}
```

The right-hand side

Target	Meaning
`field F1, F2`	Re-read exactly those fields
`entity _Assoc`	Re-read the associated entity (the whole child table)
`$self`	Re-read the entire instance
`messages`	Re-fetch the message list
`permissions`	Re-evaluate feature control and instance authorization

The left-hand side

Trigger	Meaning
`field F`	The consumer changed F
`action A`	The consumer executed A
`determine action Prepare`	Prepare ran (draft)
`determination D`	Determination D ran

21.5 Complete example

The full `side effects` block for the running Travel example, with a reason for every line:

```
define behavior for ZR_TravelTP alias Travel
...
{
  ...
  side effects
  {
    // --- price roll-up ------------------------------------------------
    // calculateTotalPrice recomputes TotalPrice whenever the fee or the
    // currency changes. Without these two lines the user sees a stale total.
    field BookingFee    affects field TotalPrice;
    field CurrencyCode  affects field TotalPrice;

    // A currency change also re-renders every child amount (decimal shift,
    // currency symbol), so the whole Bookings table must be re-read.
    field CurrencyCode  affects entity _Booking;

    // --- trip duration ------------------------------------------------
    field BeginDate, EndDate affects field TripDuration;

    // --- status actions -----------------------------------------------
```

```
    // acceptTravel / rejectTravel set OverallStatus and its criticality.
    // (These also return result [1] $self, which covers the instance itself;
    //  the 'permissions' line is the part $self does NOT cover.)
    action acceptTravel affects field OverallStatus, OverallStatusCriticality;
    action rejectTravel affects field OverallStatus, OverallStatusCriticality;

    // Once accepted, the Accept button must grey out and the date fields
    // must become read-only. That is feature control, not data.
    field  OverallStatus affects permissions;
    action acceptTravel  affects permissions;
    action rejectTravel  affects permissions;

    // --- an action that creates children ---------------------------------
    action createStandardBookings affects entity _Booking;

    // --- draft -----------------------------------------------------------
    // Prepare runs the validations; their messages must reach the UI.
    determine action Prepare affects messages;
  }
}
```

And on the **child**, so that changing a booking price refreshes the parent's total:

```
define behavior for ZR_BookingTP alias Booking
...
{
  ...
  side effects
  {
    // FlightPrice feeds the parent's TotalPrice via the internal action
    // recalculatePrices. The parent's TotalPrice is what must be re-read.
    field FlightPrice   affects field _Travel.TotalPrice;
    field CurrencyCode  affects field _Travel.TotalPrice;
  }
}
```

> `field _Travel.TotalPrice` — a **path expression** on the right-hand side, following the `association to parent`. This is how a child declares an effect on its parent. Support for path expressions in side effects depends on your release; if it is rejected, fall back to `field FlightPrice affects entity _Travel;` (coarser, re-reads the whole parent).

21.6 Runtime behavior

What actually happens, step by step:

```
1. BDEF activated
     -> side effects are written into the OData $metadata as
        com.sap.vocabularies.Common.v1.SideEffects annotations

2. Fiori Elements loads $metadata and builds a map:
        "BookingFee changed"  ->  re-request TargetProperties [ TotalPrice ]

3. User edits BookingFee, field loses focus
     -> PATCH { BookingFee: 30 }

4. Server: determination runs, buffer updated

5. UI consults its map, issues a follow-up GET for TotalPrice
     -> GET .../Travel(...)?$select=TotalPrice

6. TotalPrice repaints
```

Consequences worth knowing:

Behaviour	Implication
Side effects cost an **extra round trip**	Do not declare `affects $self` where `affects field X` will do
They are **client-side hints**	EML consumers ignore them entirely. A background job does not need and does not use them
They are **declarative only**	There is no handler method. Nothing to implement, nothing to debug in ABAP
They fire **after** the triggering request completes	The server has already run the determination by then
`affects permissions` triggers a `GET PERMISSIONS`-style request	More expensive than a field re-read; declare it only where enablement really changes

Side effect vs determination — they are not alternatives

	Determination	Side effect
Where it runs	Server (behavior pool)	Client (Fiori Elements)
What it does	**Calculates** the new value	**Asks for** the new value
Implementation	ABAP method	One line of BDL

	Determination	Side effect
Without it	The value is wrong	The value is right but **displayed** wrong

You almost always need **both**: the determination to compute, the side effect to display.

21.7 Common errors

Symptom	Cause	Fix
Calculated field shows the old value until F5	No side effect declared	`field A affects field B;`
Child table does not refresh after a header change	`affects entity _Booking` missing	Add it
Button stays enabled after the status changes	`affects permissions` missing	Add it
Validation messages do not appear after Prepare	`determine action Prepare affects messages;` missing	Add it
Action result is correct in the response but the screen is stale	Action has no `result [1] $self` **and** no side effect	Add `result [1] $self` (preferred) or a side effect
Every keystroke triggers several requests; the app feels slow	Over-declared side effects — `affects $self` or `affects entity` where a field would do	Narrow the targets
Side effect ignored in a test class	Correct — EML consumers do not use side effects	Assert on the buffer with `READ ENTITIES`, not on refresh behaviour
Change had no effect after adding the declaration	BDEF not re-activated, binding not re-published, or browser cache	Re-activate, re-publish, hard-reload
Activation error: unknown field on the right-hand side	The field is not exposed in the projection, or the path expression form is not supported on this release	Expose it, or use the coarser `affects entity` form

Diagnosing "is it the determination or the side effect?"

```
Change the field in the UI.  Is the displayed value wrong?
        |
        +-- Press F5.  Is it now right?
                |
                +-- YES -> the determination is fine. The SIDE EFFECT is missing.
                |
                +-- NO  -> the determination did not run, or ran wrongly.
                           Debug the determination (Chapter 12, §12.7).
```

This 10-second test saves hours. Run it before opening the debugger.

21.8 Interview answer

> **"What are side effects in RAP?"**
>
> A side effect is a declaration in the behavior definition that tells the consuming UI which data has become stale after a change, so that Fiori Elements knows to re-read it. For example, if a determination recalculates a total when the booking fee changes, the UI has no way of knowing that — it only sent the fee. Declaring `field BookingFee affects field TotalPrice;` makes the UI issue a follow-up read.
>
> It is purely declarative; there is no handler method. It is surfaced in the OData metadata as the standard `Common.SideEffects` annotation, so it only affects UI consumers — an EML caller ignores it.
>
> The practical point is that a side effect and a determination solve different halves of the same problem: the determination computes the value on the server, the side effect makes the client ask for it. If a calculated field looks stale but is correct after a refresh, the determination is working and the side effect is missing.

21.9 Production considerations

- **Declare the narrowest target that works.** `affects field TotalPrice` is one small `$select`; `affects $self` re-reads the whole instance; `affects entity _Booking` re-reads the child table. On an Object Page with 40 fields and 200 items, the difference is very visible.
- **`affects permissions` is the expensive one.** It triggers a permissions round trip. Declare it only where the enablement of a button or the read-only-ness of a field genuinely changes.
- **Audit the block when you add a determination.** Every new derived field needs a matching side effect, and this is routinely forgotten in change requests months after go-live.
- **Do not chain them into a storm.** `A affects B`, `B affects C`, `C affects entity _X` means one keystroke produces three follow-up requests. Collapse them where you can.
- **They are not a substitute for `result [1] $self` on actions.** Returning the instance from the action is one round trip; a side effect is two.

- **Review them on mobile.** Extra round trips that are invisible on a LAN are noticeable on a phone over mobile data.

Chapter 22 — Value Helps and Text Associations

22.1 What is it?

Two related but distinct mechanisms:

Mechanism	Answers	Annotation
Value help	"What may I type into this field?"	`@Consumption.valueHelpDefinition`
Text association	"What does this code mean?"	`@ObjectModel.text.element` + `@UI.textArrangement`

A field usually needs **both**: a dropdown to pick `070001`, and a text so the user sees *Aurora Travel* rather than a number.

22.2 Why do we need it?

- Users do not know code values. `AgencyId = 070001` is meaningless on screen.
- Free-text entry into a code field guarantees bad data. A value help turns a validation problem into an input constraint.
- A well-built value help can **fill several fields at once** (pick a flight → carrier, price and currency fill themselves), removing both typing and a determination.
- It is entirely declarative. A dependent, filtered, searchable value help with write-back is annotations only — no ABAP.

22.3 Where is it used?

Layer	What goes there
Interface view `ZR_` / `ZI_`	The **association** to the master-data entity, and the text field if it is exposed via a path expression
Projection view `ZC_`	`@Consumption.valueHelpDefinition`, `@ObjectModel.text.element`, `@Consumption.filter`
Metadata extension	`@UI.textArrangement`, hiding the text element, `@UI.lineItem` on the code field
Service definition	`expose` of every value-help and text entity — the step people forget

> **The single most common cause of an empty F4 in a custom app is a value-help entity that is not exposed in the service definition.** Check that before anything else.

22.4 Syntax

Value help — the four forms

```
// 1. Simplest: one entity, one element
@Consumption.valueHelpDefinition: [{ entity: { name: '/DMO/I_Agency', element: 'AgencyID' } }]
AgencyId,

// 2. Several alternative value helps on one field (the UI offers a chooser)
@Consumption.valueHelpDefinition: [
  { entity: { name: 'ZI_ActiveCustomer',   element: 'CustomerId' }, label: 'Active customers' },
  { entity: { name: 'ZI_ProspectCustomer', element: 'CustomerId' }, label: 'Prospects' } ]
CustomerId,

// 3. Dependent / parameterised: the local value restricts the list,
//    and the chosen row writes back into several local fields
@Consumption.valueHelpDefinition: [{
  entity           : { name: '/DMO/I_Connection', element: 'ConnectionID' },
  additionalBinding: [
    { localElement: 'CarrierId',    element: 'AirlineID', usage: #FILTER_AND_RESULT },
    { localElement: 'FlightPrice',  element: 'Price',     usage: #RESULT },
    { localElement: 'CurrencyCode', element: 'Currency',  usage: #RESULT } ] }]
ConnectionId,

// 4. Value help on a CDS view with parameters
@Consumption.valueHelpDefinition: [{
  entity           : { name: 'ZI_PriceListVH', element: 'Material' },
  additionalBinding: [ { localElement: 'BeginDate', parameter: 'p_KeyDate' } ] }]
Material,
```

`additionalBinding.usage`

Value	Effect
`#FILTER`	The local field's value **restricts** the value-help list
`#RESULT`	The selected row **writes back** into the local field
`#FILTER_AND_RESULT`	Both

Text association

```
// in the projection
@ObjectModel.text.element: ['AgencyName']
AgencyId,
_Agency.Name as AgencyName,
```

```
// in the metadata extension
@UI.textArrangement: #TEXT_FIRST
AgencyId;

@UI.hidden: true
AgencyName;
```

`@UI.textArrangement`	Renders
`#TEXT_FIRST`	`Aurora Travel (070001)`
`#TEXT_LAST`	`070001 (Aurora Travel)`
`#TEXT_ONLY`	`Aurora Travel`
`#TEXT_SEPARATE`	Two independent columns

Domain fixed values — the free option

If the field's data element has **domain fixed values**, Fiori renders a dropdown with the translated texts and **no annotation is needed at all**. Reach for `@Consumption.valueHelpDefinition` only when the values live in a table.

22.5 Complete example

A custom, searchable, filtered value help for **Agency**, built from scratch, with text, search and a dependent second field.

Step 1 — the value-help provider view

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Agency - value help'
@Metadata.ignorePropagatedAnnotations: true
@ObjectModel.usageType: { serviceQuality: #C, sizeCategory: #S, dataClass: #MASTER }

@Search.searchable: true                        // enables the search box in the F4 dialog
@ObjectModel.resultSet.sizeCategory: #XS        // hint: small result set

define view entity ZI_AgencyVH
  as select from /dmo/agency as Agency
{
      @Search.defaultSearchElement: true
      @Search.ranking: #HIGH
  key agency_id      as AgencyId,

      @Search.defaultSearchElement: true
      @Search.fuzzinessThreshold: 0.8
      name           as Name,

      @Search.defaultSearchElement: true
      city           as City,

      country_code   as CountryCode,
      postal_code    as PostalCode,
      phone_number   as PhoneNumber,
      email_address  as EmailAddress
}
```

Notes:

Line	Why
`@Search.searchable: true` on the view	Turns on the free-text search field inside the F4 dialog
`@Search.defaultSearchElement: true` on several fields	The user can find an agency by id, name **or** city
`@Search.fuzzinessThreshold: 0.8`	HANA fuzzy search — finds *Barcelona* when the user types *Barcelna*. Below ~0.7 produces noise

Line	Why
`@Search.ranking: #HIGH`	An exact id match ranks above a name match
`@ObjectModel.resultSet.sizeCategory: #XS`	Tells the runtime this list is small, so it can be loaded eagerly
No `@UI` annotations	The F4 dialog builds its own columns; add `@UI.lineItem` only if you want to control which columns appear

Step 2 — the association in the interface view

```
define root view entity ZR_TravelTP
  as select from ztravel as Travel

  composition [0..*] of ZR_BookingTP as _Booking

  association [0..1] to ZI_AgencyVH as _Agency
    on $projection.AgencyId = _Agency.AgencyId
{
  key travel_uuid as TravelUUID,
      agency_id   as AgencyId,
      ...
      _Booking,
      _Agency                       // must be exposed
}
```

Step 3 — the projection: value help + text + filter

```
define root view entity ZC_TravelTP
  provider contract transactional_query
  as projection on ZR_TravelTP
{
  key TravelUUID,

      // ---- value help ------------------------------------------------
      @Consumption.valueHelpDefinition: [{
        entity: { name: 'ZI_AgencyVH', element: 'AgencyId' } }]
      // ---- text association ------------------------------------------
      @ObjectModel.text.element: ['AgencyName']
      // ---- filter-bar behaviour --------------------------------------
      @Consumption.filter: { selectionType: #SINGLE, multipleSelections: true }
      AgencyId,

      _Agency.Name as AgencyName,          // the text, read-only

      // ---- dependent value help: Connection depends on Carrier --------
      @Consumption.valueHelpDefinition: [{
        entity: { name: '/DMO/I_Carrier', element: 'AirlineID' } }]
      @ObjectModel.text.element: ['CarrierName']
      CarrierId,
      _Carrier.Name as CarrierName,

      @Consumption.valueHelpDefinition: [{
        entity           : { name: '/DMO/I_Connection', element: 'ConnectionID' },
        additionalBinding: [
          { localElement: 'CarrierId',    element: 'AirlineID',    usage: #FILTER_AND_RESULT },
          { localElement: 'FlightPrice',  element: 'Price',        usage: #RESULT },
          { localElement: 'CurrencyCode', element: 'Currency',     usage: #RESULT },
          { localElement: 'DepartureAirport', element: 'AirportFromID', usage: #RESULT } ] }]
      ConnectionId,

      FlightPrice,
      CurrencyCode,
      DepartureAirport,

      /* Associations */
      _Booking : redirected to composition child ZC_BookingTP,
      _Agency,
      _Carrier,
      _Connection
}
```

Step 4 — the metadata extension

```
@Metadata.layer: #CORE
annotate entity ZC_TravelTP with
{
  @UI.lineItem:        [ { position: 20, importance: #HIGH } ]
  @UI.identification:  [ { position: 20 } ]
  @UI.selectionField:  [ { position: 20 } ]
  @UI.textArrangement: #TEXT_FIRST          // "Aurora Travel (070001)"
  AgencyId;

  @UI.hidden: true                          // the text is shown *through* AgencyId
  AgencyName;

  @UI.lineItem:        [ { position: 30, importance: #HIGH } ]
  @UI.identification:  [ { position: 30 } ]
  @UI.textArrangement: #TEXT_FIRST
  CarrierId;
```

```
  @UI.hidden: true
  CarrierName;

  @UI.lineItem:       [ { position: 40, importance: #HIGH } ]
  @UI.identification: [ { position: 40 } ]
  ConnectionId;
}
```

Step 5 — expose everything in the service definition

```
@EndUserText.label: 'Service definition for Travel'
define service ZSD_TRAVEL {
  expose ZC_TravelTP       as Travel;
  expose ZC_BookingTP      as Booking;

  // value-help and text providers - REQUIRED, or the F4 is empty
  expose ZI_AgencyVH       as Agency;
  expose /DMO/I_Carrier    as Carrier;
  expose /DMO/I_Connection as Connection;
  expose I_Currency        as Currency;
}
```

What the user gets

1. Clicking the Agency field opens a dialog with a search box; typing *barcelna* still finds *Barcelona* (fuzzy search).
2. The Agency column shows **Aurora Travel (070001)**.
3. Selecting a Carrier restricts the Connection value help to that carrier's flights.
4. Selecting a Connection fills `FlightPrice`, `CurrencyCode` and `DepartureAirport` automatically — no determination, no ABAP.
5. Agency appears in the filter bar with multi-select.

22.6 Runtime behavior

```
1. User opens the value help on ConnectionId
2. Fiori reads the ValueList annotation from $metadata:
     CollectionPath = "Connection"
     Parameters     = [ In(CarrierId -> AirlineID),
                        Out(Price -> FlightPrice),
                        Out(Currency -> CurrencyCode),
                        InOut(ConnectionId -> ConnectionID) ]
3. It issues: GET /Connection?$filter=AirlineID eq 'LH'&$top=100
4. The user types in the search box:
     GET /Connection?$search=barcel&$filter=AirlineID eq 'LH'
5. The user picks a row; Fiori writes the Out parameters into the local model
6. A PATCH sends the changed fields to the server
```

Points that follow from this:

Behaviour	Implication
The value help is a **normal OData read** of the exposed entity	It obeys that entity's DCL. A user sees only agencies they are authorized to read
It is **paged**	A value help over a million rows is fine, provided the entity is indexed and searchable
`#FILTER` bindings become `$filter`	The dependent list is filtered **on the database**, not in the browser
`#RESULT` bindings write into the **client model**	The server sees them in the subsequent `PATCH`, as if the user had typed them
`$search` requires `@Search.searchable`	Without it the dialog has no search box
A value help does **not** validate	A user can still `PATCH` an invalid code via the API. You still need a validation (Chapter 12)

> **A value help is an input aid, not a constraint.** It improves data quality for UI users and does nothing for API consumers. Referential integrity still belongs in a validation.

22.7 Common errors

Symptom	Cause	Fix
F4 dialog is empty	Value-help entity not exposed in the service definition	`expose ZI_AgencyVH as Agency;`
F4 dialog does not open at all	Annotation on the interface view instead of the projection, or the field is `readonly`	Move the annotation; check field control
No search box in the F4 dialog	`@Search.searchable: true` missing on the value-help view	Add it, plus at least one `@Search.defaultSearchElement`
Text does not appear next to the code	`@ObjectModel.text.element` missing, or the text element is not exposed, or `@UI.textArrangement` missing	All three are needed
Text column appears twice	The text element is not `@UI.hidden: true`	Hide it

Symptom	Cause	Fix
Dependent value help shows everything	`usage: #FILTER` missing, or `localElement` spelling is wrong	Check both names against the projection
Write-back fields stay empty	`usage: #RESULT` missing, or the target field is `readonly` in the BDEF	A `#RESULT` target must be writable
Value help works in the List Report filter bar but not on the Object Page	The field is read-only in edit mode via feature control	Check `get_instance_features`
Fuzzy search returns nonsense	`fuzzinessThreshold` too low	Raise it to 0.8 or above
Value help slow to open	The provider view joins large tables, or has no `sizeCategory` hint	Simplify the view; add `@ObjectModel.resultSet.sizeCategory`
Works in DEV, empty in QA	The value-help entity's DCL filters everything for that user	Check the PFCG role in the target system
Activation error: element not found	`element:` refers to the value-help entity's element name, not the local one — the two are easy to swap	`entity.element` = remote, `localElement` = local

22.8 Interview answer

> **"How do you implement a value help in RAP?"**
>
> With the `@Consumption.valueHelpDefinition` annotation on the field in the **projection view**, pointing at a CDS entity that supplies the values. The entity must also be exposed in the service definition, otherwise the F4 comes back empty — that is the usual cause of a value help that silently does nothing.
>
> For a dependent value help you add `additionalBinding` entries. `usage: #FILTER` means a local field restricts the list; `usage: #RESULT` means the selected row writes back into a local field. So selecting a flight connection can automatically fill the price and currency without any ABAP.
>
> Separately from the value help, you show the description with `@ObjectModel.text.element` pointing at a text field — usually exposed through an association — and control how it renders with `@UI.textArrangement`, for example `#TEXT_FIRST`.
>
> If the field's domain has fixed values you need no annotation at all: Fiori renders a dropdown automatically.
>
> One thing worth saying: a value help is an input aid, not a constraint. It does not stop an API consumer sending an invalid code, so referential integrity still belongs in a validation in the behavior pool.

22.9 Production considerations

- **Build a dedicated `ZI_..._VH` view** rather than pointing the value help at a big transactional view. It keeps the F4 fast and lets you filter out obsolete or blocked master data (`where DeletionFlag = ''`).
- **Filter out inactive records in the value-help view**, not in the UI. Users should not be offered a blocked customer at all.
- **Check the DCL on the value-help entity.** It is a normal read, so it is filtered per user. A value help that is full in DEV and empty in PROD is nearly always this.
- **Expose value-help entities in every service that uses them.** Easy to miss when you add a second service over the same projection.
- **Use `@ObjectModel.resultSet.sizeCategory`** honestly — it drives whether the UI loads the list eagerly or lazily.
- **Fuzzy search costs CPU.** Enable it on the one or two fields where it helps, not on everything.
- **Translate the texts.** A text association pointing at a non-translated field looks fine in English and wrong everywhere else.
- **Value helps are part of the transport.** A missing `expose` line is a classic QA-only defect; include an F4 check in the post-import test list (Chapter 25).
- **Think about volume.** A value help over a table with millions of rows needs a searchable, indexed provider view and should never be marked `#XS`.

Chapter 23 — Testing and Debugging

23.1 What is it?

A RAP application has four testable surfaces, and they fail in different ways:

```
+-----------------------------------------------------+
| 4. UI          Fiori Elements preview / real app | layout, buttons, side effects
+-----------------------------------------------------+
| 3. Service     OData: $metadata, GET, POST       | exposure, annotations, protocol
+-----------------------------------------------------+
| 2. Behavior    EML from ABAP                     | validations, determinations, actions, save
+-----------------------------------------------------+
| 1. Model       CDS views, Data Preview           | joins, fields, associations, DCL
+-----------------------------------------------------+
```

Test from the bottom up. A bug found at layer 1 costs a minute; the same bug found at layer 4 costs an afternoon, because the UI gives you almost no diagnostic information.

23.2 Why do we need it?

RAP's declarative parts fail *silently*. A missing `use action` produces no error anywhere — the button simply is not there. A validation that fills `reported` but not `failed` shows a red message and saves the record anyway. Neither is visible without a deliberate test.

And the Fiori preview is the **worst** place to discover this: it caches aggressively, it hides server messages, and a stale browser cache looks exactly like a code defect.

23.3 Where is it used?

Stage	Test
While writing a CDS view	Data Preview
While writing a behavior pool	EML console class (`IF_OO_ADT_CLASSRUN`)
Before committing to a transport	ABAP Unit + ATC
Before handing to QA	Fiori Elements preview, all flows
In QA	Real Launchpad app, real roles, real data volume
After go-live	SQL Monitor, application log, dump analysis

23.4 Layer 1 — testing the CDS model

Data Preview

Right-click the CDS view → **Open With** → **Data Preview** (or `F8`).

Check	Why
Row count is plausible	A wrong association cardinality silently multiplies rows
No unexpected duplicates	The classic symptom of a `[0..1]` association that is really `[0..*]`
Amounts show a currency, quantities a unit	`@Semantics` annotations missing
Text fields populated	Path expression / association join condition wrong
Filter on a key returns exactly one row	Key definition wrong

Data Preview runs **as you**, with DCL applied. To see whether DCL is the reason a row is missing, compare with the same `SELECT` in a console class.

Verifying DCL

```
" What the user actually sees (DCL applied)
SELECT FROM zc_traveltp FIELDS COUNT(*) INTO @DATA(with_dcl).

" What exists (interface view, no DCL if #NOT_REQUIRED)
SELECT FROM zr_traveltp FIELDS COUNT(*) INTO @DATA(without_dcl).

out->write( |visible: { with_dcl } of { without_dcl }| ).
```

If the two differ and you did not expect them to, the DCL is doing something you did not intend.

23.5 Layer 2 — the EML console class

This is the single most valuable testing tool in RAP. Before touching the UI, drive the business object from ABAP where you can see `failed`, `reported` and `sy-subrc`.

```
CLASS zcl_travel_playground DEFINITION PUBLIC FINAL CREATE PUBLIC.
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
ENDCLASS.

CLASS zcl_travel_playground IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.

    " ---------- CREATE ----------
    MODIFY ENTITIES OF zr_traveltp
      ENTITY Travel
        CREATE FIELDS ( AgencyId CustomerId BeginDate EndDate BookingFee CurrencyCode Description )
        WITH VALUE #( ( %cid         = 'C1'
                        AgencyId     = '070001'
                        CustomerId   = '000001'
                        BeginDate    = cl_abap_context_info=>get_system_date( ) + 30
                        EndDate      = cl_abap_context_info=>get_system_date( ) + 40
                        BookingFee   = '20.00'
                        CurrencyCode = 'EUR'
                        Description  = 'EML playground' ) )
      MAPPED   DATA(mapped)
      FAILED   DATA(failed)
      REPORTED DATA(reported).

    out->write( |--- after MODIFY ---| ).
    out->write( |failed:  { COND #( WHEN failed IS INITIAL THEN 'none' ELSE 'YES' ) }| ).
    out->write( reported ).

    IF failed IS NOT INITIAL.
      out->write( 'aborting - interaction phase rejected the create' ).
      RETURN.
    ENDIF.

    " ---------- COMMIT ----------
    COMMIT ENTITIES RESPONSE OF zr_traveltp
      FAILED   DATA(commit_failed)
      REPORTED DATA(commit_reported).

    out->write( |--- after COMMIT --- sy-subrc = { sy-subrc }| ).
    CASE sy-subrc.
      WHEN 0. out->write( |committed. UUID = { mapped-travel[ 1 ]-TravelUUID }| ).
      WHEN 4. out->write( 'EARLY SAVE rejected - a validation failed:' ).
              out->write( commit_reported ).
      WHEN 8. out->write( 'LATE SAVE failed - rolling back' ).
              ROLLBACK ENTITIES.
    ENDCASE.

  ENDMETHOD.
ENDCLASS.
```

Run with `F9`. Output appears in the ADT console.

Why this beats the UI for diagnosis

	Fiori preview	EML console class
See `failed`	No	Yes
See `reported` in full	Partly (popover)	Yes, complete
See `sy-subrc` after commit	No	Yes
Reproducible	No (cache, drafts, session)	Yes
Debuggable with `F9`	Awkward (external breakpoints)	Directly
Becomes a regression test	No	Yes — promote it to ABAP Unit

Useful playground snippets

```
" --- test a validation in isolation, without committing ------------------
COMMIT ENTITIES IN SIMULATION MODE
  RESPONSE OF zr_traveltp
  FAILED DATA(sim_failed) REPORTED DATA(sim_reported).
out->write( sim_reported ).          " all on-save validations ran; nothing was written

" --- test a determination: read the buffer after the modify ---------------
READ ENTITIES OF zr_traveltp
  ENTITY Travel ALL FIELDS
  WITH VALUE #( ( %tky-TravelUUID = uuid %tky-%is_draft = if_abap_behv=>mk-off ) )
  RESULT DATA(after).
```

```
out->write( after ).                " is TotalPrice what you expect?

" --- test an action ------------------------------------------------------
MODIFY ENTITIES OF zr_traveltp
  ENTITY Travel EXECUTE acceptTravel
  FROM VALUE #( ( %tky-TravelUUID = uuid %tky-%is_draft = if_abap_behv=>mk-off ) )
  RESULT DATA(action_result) FAILED DATA(af) REPORTED DATA(ar).
out->write( action_result ).

" --- test feature control / authorization without attempting the operation -
READ ENTITIES OF zr_traveltp
  ENTITY Travel
    REQUEST VALUE #( %update = if_abap_behv=>mk-on
                     %delete = if_abap_behv=>mk-on
                     %action-acceptTravel = if_abap_behv=>mk-on )
    GET PERMISSIONS
    FROM VALUE #( ( %tky-TravelUUID = uuid %tky-%is_draft = if_abap_behv=>mk-off ) )
  RESULT DATA(perms).
out->write( perms ).                " exactly what the UI uses to enable buttons
```

`GET PERMISSIONS` is the honest answer to "why is my button greyed out" — it returns precisely what Fiori asked for.

23.6 ABAP Unit — isolating the behavior

The test doubles

Class	Replaces	Use for
`cl_cds_test_environment`	The tables behind a **CDS entity**	Testing a RAP BO against in-memory data
`cl_osql_test_environment`	**Database tables** directly	Testing plain SQL / helper classes
RAP BO test double framework (BOTD)	A **dependent RAP BO** you call via EML	Testing code that calls `I_PurchaseRequisitionTP` without creating real documents

> The BOTD class names differ by release (they are in the `CL_BOTD_*` family). Look them up with `Ctrl + Shift + A` in your system before using them; the concept below is stable, the names are what to verify.

Skeleton

```
CLASS ltc_travel DEFINITION FINAL FOR TESTING
  DURATION SHORT
  RISK LEVEL HARMLESS.

  PRIVATE SECTION.
    CLASS-DATA cds_env TYPE REF TO if_cds_test_environment.

    CLASS-METHODS class_setup.
    CLASS-METHODS class_teardown.
    METHODS setup.
    METHODS teardown.

    METHODS create_valid_travel          FOR TESTING RAISING cx_static_check.
    METHODS end_before_begin_rejected    FOR TESTING RAISING cx_static_check.
    METHODS unknown_customer_rejected    FOR TESTING RAISING cx_static_check.
    METHODS total_price_is_rolled_up     FOR TESTING RAISING cx_static_check.
    METHODS accept_sets_status           FOR TESTING RAISING cx_static_check.
    METHODS accept_disabled_when_accepted FOR TESTING RAISING cx_static_check.
ENDCLASS.

CLASS ltc_travel IMPLEMENTATION.

  METHOD class_setup.
    cds_env = cl_cds_test_environment=>create( i_for_entity = 'ZR_TRAVELTP' ).
  ENDMETHOD.

  METHOD class_teardown.
    cds_env->destroy( ).
  ENDMETHOD.

  METHOD setup.
    cds_env->clear_doubles( ).
  ENDMETHOD.

  METHOD teardown.
    ROLLBACK ENTITIES.
  ENDMETHOD.
  ...
ENDCLASS.
```

`ROLLBACK ENTITIES` in `teardown` is important: it clears the transactional buffer between tests. Without it, one test's uncommitted instances leak into the next.

Testing a validation

```
METHOD end_before_begin_rejected.

  MODIFY ENTITIES OF zr_traveltp
    ENTITY Travel
      CREATE FIELDS ( AgencyId CustomerId BeginDate EndDate CurrencyCode )
      WITH VALUE #( ( %cid         = 'T1'
                      AgencyId     = '070001'
                      CustomerId   = '000001'
                      BeginDate    = '20260601'
                      EndDate      = '20260501'      " <-- before begin
                      CurrencyCode = 'EUR' ) )
    FAILED   DATA(failed)
    REPORTED DATA(reported).

  " The interaction phase accepts it; the ON SAVE validation rejects it.
  cl_abap_unit_assert=>assert_initial(
    act = failed
    msg = 'create itself should not fail - validateDates is ON SAVE' ).

  COMMIT ENTITIES RESPONSE OF zr_traveltp
    FAILED   DATA(commit_failed)
    REPORTED DATA(commit_reported).

  cl_abap_unit_assert=>assert_equals(
    exp = 4  act = sy-subrc
    msg = 'early save phase should have rejected the document' ).

  cl_abap_unit_assert=>assert_not_initial(
    act = commit_failed-travel
    msg = 'validateDates should have filled failed-travel' ).

  cl_abap_unit_assert=>assert_not_initial(
    act = commit_reported-travel
    msg = 'validateDates should also have produced a message' ).

ENDMETHOD.
```

> Asserting on **both** `failed` and `reported` is deliberate. It is exactly the defect described in Chapter 20, §20.4: a validation that reports without failing lets the record save. A test that only checks `sy-subrc` will not catch the reverse case — a validation that fails silently with no message.

Testing a determination

```
METHOD total_price_is_rolled_up.

  MODIFY ENTITIES OF zr_traveltp
    ENTITY Travel
      CREATE FIELDS ( AgencyId CustomerId BeginDate EndDate BookingFee CurrencyCode )
      WITH VALUE #( ( %cid = 'T1' AgencyId = '070001' CustomerId = '000001'
                      BeginDate = '20260601' EndDate = '20260610'
                      BookingFee = '20.00' CurrencyCode = 'EUR' ) )
      CREATE BY \_Booking
        FIELDS ( CarrierId ConnectionId FlightDate FlightPrice CurrencyCode )
        WITH VALUE #( ( %cid_ref = 'T1'
                        %target = VALUE #(
                          ( %cid = 'B1' CarrierId = 'LH' ConnectionId = '0400'
                            FlightDate = '20260601' FlightPrice = '200.00' CurrencyCode = 'EUR' )
                          ( %cid = 'B2' CarrierId = 'LH' ConnectionId = '0401'
                            FlightDate = '20260610' FlightPrice = '210.00' CurrencyCode = 'EUR' ) ) ) )
    MAPPED DATA(mapped)
    FAILED DATA(failed).

  cl_abap_unit_assert=>assert_initial( act = failed ).

  " read the BUFFER - no commit needed to test an ON MODIFY determination
  READ ENTITIES OF zr_traveltp
    ENTITY Travel
      FIELDS ( TotalPrice )
      WITH VALUE #( ( %tky-TravelUUID = mapped-travel[ 1 ]-TravelUUID
                      %tky-%is_draft  = if_abap_behv=>mk-off ) )
    RESULT DATA(travels).

  cl_abap_unit_assert=>assert_equals(
    exp = CONV /dmo/total_price( '430.00' )     " 20 fee + 200 + 210
    act = travels[ 1 ]-TotalPrice
    msg = 'calculateTotalPrice should roll up the booking prices' ).

ENDMETHOD.
```

Note: **no** `COMMIT ENTITIES`**.** An `on modify` determination has already run; reading the buffer is enough and keeps the test fast.

Testing an action and its feature control

```
METHOD accept_disabled_when_accepted.

  DATA(uuid) = create_accepted_travel( ).      " helper that creates + accepts

  READ ENTITIES OF zr_traveltp
```

```
    ENTITY Travel
      REQUEST VALUE #( %action-acceptTravel = if_abap_behv=>mk-on )
      GET PERMISSIONS
      FROM VALUE #( ( %tky-TravelUUID = uuid %tky-%is_draft = if_abap_behv=>mk-off ) )
    RESULT DATA(perms).

  cl_abap_unit_assert=>assert_equals(
    exp = if_abap_behv=>fc-o-disabled
    act = perms-instances[ 1 ]-%action-acceptTravel
    msg = 'an already-accepted travel must not be acceptable again' ).

ENDMETHOD.
```

Testing draft behaviour

```
METHOD draft_edit_change_activate.

  DATA(uuid) = create_active_travel( ).

  " Edit: on the ACTIVE instance
  MODIFY ENTITIES OF zr_traveltp
    ENTITY Travel
      EXECUTE Edit FROM VALUE #( ( %tky-TravelUUID = uuid
                                   %tky-%is_draft  = if_abap_behv=>mk-off ) )
    FAILED DATA(edit_failed).
  cl_abap_unit_assert=>assert_initial( act = edit_failed msg = 'Edit should succeed' ).

  " Change: on the DRAFT
  MODIFY ENTITIES OF zr_traveltp
    ENTITY Travel
      UPDATE FIELDS ( Description )
      WITH VALUE #( ( %tky-TravelUUID = uuid
                      %tky-%is_draft  = if_abap_behv=>mk-on
                      Description     = 'changed in draft' ) ).

  " The ACTIVE instance must still be unchanged at this point
  READ ENTITIES OF zr_traveltp
    ENTITY Travel FIELDS ( Description )
      WITH VALUE #( ( %tky-TravelUUID = uuid %tky-%is_draft = if_abap_behv=>mk-off ) )
    RESULT DATA(active_before).
  cl_abap_unit_assert=>assert_differs(
    exp = 'changed in draft' act = active_before[ 1 ]-Description
    msg = 'the active instance must not change before Activate' ).

  " Prepare: collect validation messages without activating
  MODIFY ENTITIES OF zr_traveltp
    ENTITY Travel
      EXECUTE Prepare FROM VALUE #( ( %tky-TravelUUID = uuid
                                      %tky-%is_draft  = if_abap_behv=>mk-on ) )
    REPORTED DATA(prepare_reported).

  " Activate: on the DRAFT
  MODIFY ENTITIES OF zr_traveltp
    ENTITY Travel
      EXECUTE Activate FROM VALUE #( ( %tky-TravelUUID = uuid
                                       %tky-%is_draft  = if_abap_behv=>mk-on ) )
    FAILED DATA(act_failed).
  cl_abap_unit_assert=>assert_initial( act = act_failed ).

  COMMIT ENTITIES.

  READ ENTITIES OF zr_traveltp
    ENTITY Travel FIELDS ( Description )
      WITH VALUE #( ( %tky-TravelUUID = uuid %tky-%is_draft = if_abap_behv=>mk-off ) )
    RESULT DATA(active_after).
  cl_abap_unit_assert=>assert_equals(
    exp = 'changed in draft' act = active_after[ 1 ]-Description ).

ENDMETHOD.
```

The assertion in the middle — that the active instance is **not** changed before `Activate` — is the one that catches a genuinely broken draft setup.

Testing failed save and rollback

```
METHOD failed_save_leaves_db_clean.

  SELECT COUNT(*) FROM ztravel INTO @DATA(before).

  " create something a validation will reject
  MODIFY ENTITIES OF zr_traveltp
    ENTITY Travel
      CREATE FIELDS ( AgencyId CustomerId BeginDate EndDate CurrencyCode )
      WITH VALUE #( ( %cid = 'BAD' AgencyId = '070001' CustomerId = 'NOPE'
                      BeginDate = '20260601' EndDate = '20260610' CurrencyCode = 'EUR' ) ).

  COMMIT ENTITIES RESPONSE OF zr_traveltp FAILED DATA(cf).

  cl_abap_unit_assert=>assert_equals( exp = 4 act = sy-subrc ).

  SELECT COUNT(*) FROM ztravel INTO @DATA(after).
  cl_abap_unit_assert=>assert_equals(
```

```
    exp = before act = after
    msg = 'a rejected save must not have written anything' ).

  ROLLBACK ENTITIES.
ENDMETHOD.
```

Testing authorization — an honest note

Authorization is the hardest RAP layer to unit-test, because `AUTHORITY-CHECK` evaluates the **running user's** profile and there is no supported way to impersonate another user inside ABAP Unit.

Practical options, best first:

1. **Extract the decision.** Put the rule in a small class with an injectable checker:
   `abap INTERFACE zif_travel_auth_check. METHODS may_change IMPORTING agency_id TYPE /dmo/agency_id RETURNING VALUE(r) TYPE abap_bool. ENDINTERFACE.`
   The handler calls the interface; the test injects a double (`cl_abap_testdouble`). You then unit-test the *mapping* from check result to `%update` / `%action-X` , which is where the bugs actually are.
2. **Integration-test with real users** in QA: one test user per role, a checklist per role. This is the only way to test the PFCG role content itself.
3. **Test DCL separately** by comparing row counts as different users (23.4).

Do not pretend a unit test covers authorization when it only covers the code path.

Running the tests

Action	Shortcut
Run ABAP Unit for the current object	`Ctrl + Shift + F10`
Run with coverage	`Ctrl + Shift + F11`
Run ATC checks	Right-click → *Run As* → *ABAP Test Cockpit*

23.7 Layer 3 — testing the OData service

`$metadata` is the source of truth

```
GET /sap/opu/odata4/sap/zui_travel_o4/srvd/sap/zsd_travel/0001/$metadata
```

If something is missing here, the problem is in CDS or the BDEF — **stop looking at the UI.**

Look for	Confirms
The `EntityType` and its properties	Projection view exposure
`<Action Name="acceptTravel">`	Base BDEF action **and** `use action` in the projection
`Common.SideEffects` annotations	`side effects` block
`Common.ValueList` annotations	`@Consumption.valueHelpDefinition` **and** the entity is exposed
`UI.LineItem` / `UI.Facets`	Metadata extension activated
`IsActiveEntity` in the key	Draft is enabled
`Core.OptimisticConcurrency` / ETag on the entity	`use etag` in the projection BDEF

Request checklist

Request	Checks
`GET .../Travel?$top=5`	Read, DCL
`GET .../Travel?$filter=OverallStatus eq 'O'`	Filtering pushed to HANA
`GET .../Travel?$expand=_Booking`	Composition, redirection
`GET .../Travel?$select=TravelId,TotalPrice`	Projection
`POST .../Travel`	Create, determinations, mandatory field control
`PATCH .../Travel(...)`	Update, ETag (`If-Match`)
`POST .../Travel(...)/...acceptTravel`	Action
`DELETE .../Travel(...)`	Delete, cascade

Testing an ETag conflict

This is one of the few things you **cannot** properly test through EML — the ETag comparison happens in the OData layer, not in the transactional buffer. Test it over HTTP:

1. `GET` an instance; note the `ETag` response header.

2. `PATCH` it once with `If-Match: <that etag>` → succeeds, and the ETag changes.
3. `PATCH` again with the **old** ETag → expect **HTTP 412 Precondition Failed**.

In the UI the equivalent is: open the same object in two browser sessions, change and save in the first, then save in the second. The second must be refused.

Testing concurrency / locking

Two ADT sessions, or two browser sessions with different users:

Test	Expected
User A presses Edit, user B presses Edit	B gets "*Object is locked by user A*"
A discards, B presses Edit	B succeeds
A edits and closes the browser; B waits for the lock timeout, then Edits	B succeeds (optimistic phase)
A returns and presses Resume after B has saved	A is told the draft is outdated

---

23.8 Layer 4 — testing the UI

The Fiori Elements preview

Service binding → select the entity set → **Preview**.

Before you conclude anything, clear the cache. `F12` → right-click reload → *Empty cache and hard reload*. More than half of "the annotation has no effect" reports are this.

UI test checklist

Area	Test
List Report	Filter bar fields present; mandatory filters enforced; search works; export to spreadsheet works
Create	Object Page opens in edit mode; a draft row appears
Mandatory fields	Empty mandatory field blocks save, field marked
Validation	Bad data shows a message **on the right field**; fixing it clears the message
Determination	Calculated field updates without a manual refresh (if not → side effect missing, Chapter 21)
Child table	"+" adds a row; delete removes it; the parent total updates
Actions	Buttons present, enabled/disabled per status, message shown on failure
Draft	Cancel discards; reopening offers Resume; two sessions conflict correctly
Delete	Parent delete removes children
Responsive	Resize to phone width — only `#HIGH` importance columns remain
Messages	Message popover lists everything; clicking a message navigates to the field

Where UI errors actually surface

Tool	Shows
Browser DevTools → **Network**	The real OData request and the **full JSON error body** — far more detail than the UI toast
Browser DevTools → **Console**	UI5 errors, annotation problems
`/IWFND/ERROR_LOG`	Gateway errors (mainly OData V2; still useful on-premise)
`ST22`	Short dumps from the backend
`SLG1`	Your own application log

The Network tab is the most under-used RAP debugging tool. A failed `POST` returns a JSON body containing the full `reported` message list, including the ones the UI chose not to display.

23.9 Debugging

Getting a breakpoint to fire from the Fiori preview

The preview runs in a different session from your ADT debugger. You need an **external breakpoint**:

1. Set the breakpoint in ADT.
2. Check the breakpoint is external: ADT → *Window* → *Preferences* → *ABAP Development* → *Debug* → confirm the external debugging user matches the user running the browser.

3. Trigger the action in the browser.

If it still does not fire, the code is not being reached — which is itself the answer.

Better: skip the browser. Reproduce with an EML console class and debug with `F9`. Faster, repeatable, and it becomes a test.

Conditional breakpoints for mass scenarios

In a determination processing 5000 instances, a plain breakpoint is useless. Right-click the breakpoint → *Breakpoint Properties* → condition:

```
travel-TravelId = '00000042'
```

Breakpoints worth knowing

Where	Tells you
Start of each determination/validation	Whether it runs at all, and with which keys
`finalize`	Early save started; `on save` determinations are running
`check_before_save`	The final buffer state, before the point of no return
`cleanup_finalize`	Something rejected the save — inspect `failed`
`save` / `save_modified`	What is actually written
`cleanup`	The transaction completed
`get_instance_features`	Why a button is enabled or disabled

Temporary message tracing

```
APPEND VALUE #( %tky = key-%tky
                %msg = new_message_with_text(
                         severity = if_abap_behv_message=>severity-information
                         text     = |calculateTotalPrice: { lines( keys ) } keys| ) )
       TO reported-travel.
```

Appears in the Fiori message popover. **Remove before transport** — and add an ATC exemption-free check for `new_message_with_text` in your review, because it is not translatable.

23.10 Performance tracing tools

Tool	Transaction	Use for
SQL Trace	`ST05`	The actual SQL a CDS view or EML call issues. Start trace → run → stop → display
Runtime Analysis	`SAT`	Where ABAP time goes inside a behavior pool
SQL Monitor	`SQLM` / `SQLMD`	Production-safe, always-on statement statistics. The right tool for "which statement is slow in PROD"
SQL Performance Tuning Worklist	`SWLT`	Combines SQLM data with ATC findings to rank what to fix
ABAP Profiler	ADT	Profile an EML console class directly (*Run As → ABAP Application (Profiling)*)
ATC	ADT / `ATC`	Static checks including HANA-readiness and nested-SELECT findings
Application Log	`SLG1`	Your own logs written in `save_modified`
Short dumps	`ST22`	Late-save-phase failures, `COMMIT WORK` violations
Gateway error log	`/IWFND/ERROR_LOG`	OData-layer errors (on-premise)

The one trace worth running on every new RAP app

`ST05` around a single List Report page load. Look for:

- a `SELECT` without a `WHERE` on a large table → missing mandatory filter
- the **same** statement repeated N times → a `SELECT` inside a loop in a determination or instance-authorization handler
- a join returning far more rows than the page displays → wrong association cardinality

---

23.11 Common errors

Symptom	Cause
Breakpoint never fires from the preview	Not an external breakpoint, or the code genuinely is not reached
Test passes but the UI misbehaves	The defect is in the projection BDEF (`use ...`), the MDE, or the browser cache — none of which EML exercises

Symptom	Cause
UI works but the test fails	The test calls the BO without `IN LOCAL MODE` and hits authorization/feature control, which the UI passed
Test data leaks between tests	`clear_doubles( )` missing in `setup`, or `ROLLBACK ENTITIES` missing in `teardown`
`cl_cds_test_environment` create fails	Entity name must be upper case and must exist; check nested CDS dependencies
Determination "not running" in a test	It is `on save` and the test never commits — use `COMMIT ENTITIES IN SIMULATION MODE`
Everything passes in DEV, fails in QA	Data-dependent logic, missing transport object, or different roles — see Chapter 25
Cannot reproduce a user's error	Ask for the exact time, then check `ST22` and `SLG1` for that window

23.12 Interview answer

> **"How did you test your RAP application?"**
>
> On four levels.
>
> First the CDS model, with Data Preview — checking row counts, no duplicates from wrong association cardinalities, and that DCL filters what I expect.
>
> Second, and most importantly, the behavior through **EML from a console class implementing** `IF_OO_ADT_CLASSRUN`. That is the only place you can actually see `failed`, `reported` and `sy-subrc` after `COMMIT ENTITIES`. `sy-subrc = 4` means the early save phase rejected it and it is recoverable; `8` means a late-phase failure needing an explicit rollback. I also use `COMMIT ENTITIES IN SIMULATION MODE` to run the on-save validations without writing anything.
>
> Those console classes then become **ABAP Unit tests** with `cl_cds_test_environment`, which replaces the underlying tables with in-memory doubles so the tests are fast and touch no real data. For validations I assert on both `failed` and `reported` — a validation that reports without failing lets the record save, which is a bug a `sy-subrc` check alone would miss. For feature control I assert via `GET PERMISSIONS`, which is exactly what the UI calls.
>
> Third, the OData service: `$metadata` first. If an action or annotation is not in `$metadata`, the problem is in CDS or the BDEF and there is no point looking at the UI.
>
> Fourth, the Fiori Elements preview for the flows — always after clearing the browser cache, because a stale cache looks exactly like a code defect. ETag conflicts and locking I test over HTTP or with two browser sessions, since those happen above the EML layer.
>
> For performance I use `ST05` on a page load to catch SELECTs inside loops, and `SQLM` in production.

23.13 Production considerations

- **Promote every playground class to an ABAP Unit test** before you delete it. The cost is minutes; the value is a regression suite.
- **Keep unit tests** `RISK LEVEL HARMLESS` **and** `DURATION SHORT` so they can run in a CI/transport gate. Tests that write real data cannot.
- **Run ATC before every transport release**, with the HANA-readiness and security check variants.
- **Do not ship** `new_message_with_text`**.** It is not translatable. Grep for it as a release check.
- **Test with production-like volumes.** A List Report that is fine with 200 rows in DEV can time out with 2 million in PROD. Copy a realistic data subset into QA.
- **Test with real roles, not** `SAP_ALL`**.** Authorization defects are the single most common "worked in DEV" failure, and a developer with wide authorizations will never see them.
- **Enable** `SQLM` **in production for a window after go-live.** It is low-overhead and it will tell you what actually needs tuning, rather than what you guessed.
- **Write down a per-role UI test checklist** and re-run it after every transport to QA. The declarative parts of RAP fail silently; only a checklist catches a missing `use action`.

Chapter 24 — Performance and Production Readiness

24.1 What is it?

The set of design and coding decisions that determine whether a RAP application that works in DEV with 200 rows still works in PROD with 20 million.

Almost all RAP performance problems fall into five categories:

#	Category	Typical cost
1	Row-by-row database access inside a loop	N round trips instead of 1
2	Reading more than you need (`ALL FIELDS` , unfiltered lists)	Large result sets, memory
3	Work done per instance that could be done per set (authorization, feature control)	N `AUTHORITY-CHECK` s
4	Joins and associations with wrong cardinality	Row multiplication
5	Chatty UI (over-declared side effects, uncontrolled `$expand`)	Round trips

24.2 Why do we need it?

Because RAP hides the database. The framework issues the SQL, so a developer can write a perfectly reasonable-looking determination that turns into 5,000 `SELECT SINGLE` s at runtime and never notice until a user opens the app with a full year of data.

24.3 Set-based reads and updates

The rule

Never access the database or the transactional buffer once per instance. Collect keys, act once.

```
" ---------- WRONG: one SELECT per instance ----------
LOOP AT travels INTO DATA(travel).
  SELECT SINGLE customer_id FROM /dmo/customer
    WHERE customer_id = @travel-CustomerId
    INTO @DATA(found).
  IF sy-subrc <> 0.
    APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.
  ENDIF.
ENDLOOP.
```

With 5,000 travels this is 5,000 database round trips.

```
" ---------- RIGHT: one SELECT for the whole set ----------
DATA(customers) = travels.
SORT customers BY CustomerId.
DELETE ADJACENT DUPLICATES FROM customers COMPARING CustomerId.
DELETE customers WHERE CustomerId IS INITIAL.

IF customers IS NOT INITIAL.
  SELECT FROM /dmo/customer
    FIELDS customer_id
    FOR ALL ENTRIES IN @customers
    WHERE customer_id = @customers-CustomerId
    INTO TABLE @DATA(valid_customers).
ENDIF.

LOOP AT travels INTO DATA(travel).
  IF NOT line_exists( valid_customers[ customer_id = travel-CustomerId ] ).
    APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.
  ENDIF.
ENDLOOP.
```

One round trip, and the `LOOP` is now pure in-memory work.

> For very large key sets, prefer a **range table** (`FOR ALL ENTRIES` has its own pitfalls: it de-duplicates, it returns nothing for an empty driver table, and it can be split into several statements by the database interface). Always guard with `IF ... IS NOT INITIAL` .

The same rule for EML

```
" WRONG - 5000 EML statements and 5000 save sequences
LOOP AT uuids INTO DATA(uuid).
  MODIFY ENTITIES OF zr_traveltp ENTITY Travel
    UPDATE FIELDS ( OverallStatus )
```

```
    WITH VALUE #( ( %tky-TravelUUID = uuid %tky-%is_draft = if_abap_behv=>mk-off
                    OverallStatus = 'A' ) ).
  COMMIT ENTITIES.
ENDLOOP.

" RIGHT — one statement, one save sequence
MODIFY ENTITIES OF zr_traveltp ENTITY Travel
  UPDATE FIELDS ( OverallStatus )
  WITH VALUE #( FOR u IN uuids ( %tky-TravelUUID = u
                                 %tky-%is_draft  = if_abap_behv=>mk-off
                                 OverallStatus   = 'A' ) ).
COMMIT ENTITIES.
```

Internal table access

Inside the loop, `line_exists( tab[ key = x ] )` on a **standard** table is a linear scan — so the "fixed" version above is still O(n×m). For large sets, use a sorted or hashed table:

```
DATA valid_customers TYPE HASHED TABLE OF /dmo/customer_id WITH UNIQUE KEY table_line.
" ... fill it once ...
IF line_exists( valid_customers[ table_line = travel-CustomerId ] ).   " O(1)
```

24.4 Reading only what you need

Instead of	Write	Saves
`ALL FIELDS WITH ...`	`FIELDS ( F1 F2 ) WITH ...`	Columns, memory, network
`RESULT DATA(children)` when you only need existence	`LINK DATA(links)`	The entire child read
Exposing 60 fields in the projection	Expose the 15 the app uses	Every read, every `$expand`
Several `READ ENTITIES` statements	One statement with several `ENTITY` blocks	Framework round trips

```
" one statement, two reads
READ ENTITIES OF zr_traveltp IN LOCAL MODE
  ENTITY Travel
    FIELDS ( BookingFee CurrencyCode ) WITH CORRESPONDING #( keys )
    RESULT DATA(travels)
  ENTITY Travel BY \_Booking
    FIELDS ( FlightPrice CurrencyCode ) WITH CORRESPONDING #( keys )
    RESULT DATA(bookings)
  FAILED DATA(failed).
```

`ALL FIELDS` is correct in exactly one common place: building an action `result` that must return the complete instance.

24.5 Filtering, paging and the List Report

Mandatory filters

The single highest-value performance annotation in a report:

```
@Consumption.filter: { selectionType: #INTERVAL, mandatory: true }
CreationDate,
```

Without it, opening the List Report issues an unrestricted `SELECT` on the base table. On `VBAP` that is the difference between 0.4 seconds and a timeout.

Make mandatory the field that is **most selective and most natural** for the user — usually a date range, a plant, or a sales organisation.

Paging

OData paging (`$top` / `$skip`) is automatic and is pushed to the database. You do not implement it. What you can break:

Breaks paging	Why
Sorting or filtering on a **virtual element**	Calculated in ABAP after the read, so the full result set must be fetched first
A projection `where` clause on a non-indexed field	Full scan before paging
Client-side aggregation	Same

Virtual elements are display-only. If a field must be filterable or sortable, calculate it in the interface view in CDS so HANA can push it down.

`$expand`

`GET /Travel?$expand=_Booking` reads every child of every travel on the page. On a header with hundreds of items this is expensive, and Fiori Elements does it automatically for an Object Page.

- Keep child projections narrow.
- Do not expose grandchildren in a service that does not display them.
- Check the actual request in DevTools → Network; the `$expand` the UI sends is often wider than you assumed.

Search annotations

```
@Search.searchable: true
@Search.defaultSearchElement: true
@Search.fuzzinessThreshold: 0.8
```

Fuzzy search is a real HANA feature and costs real CPU. Enable it on the one or two text fields where it helps (a description, a name) — not on codes, dates or every field in the view.

24.6 Associations vs joins

	Association	Join
Executed	Only when used (lazy)	Always
Cost when unused	**Zero**	Full
Risk of row multiplication	Only when used, and visible	Silent
Use for	Optional navigation, value help, texts	Fields needed on **every** row

Exposing ten associations on an interface view costs nothing. Adding ten `LEFT OUTER JOIN`s costs every read.

The cardinality trap

```
association [0..1] to ZI_Something as _S on ...    " declared 0..1
```

If the underlying data actually has several matching rows, CDS does **not** enforce the `[0..1]`. Using the association in a path expression then **multiplies rows** — silently. Row counts that are a suspiciously round multiple of what you expect are the symptom.

Verify cardinalities with Data Preview and a `COUNT(*)` comparison (Chapter 23, §23.4).

24.7 Authorization and feature-control performance

These handlers run for **every instance the user touches** — including every row a List Report displays, when the UI asks about permissions. They are the most commonly overlooked hot path in RAP.

```
" WRONG - one AUTHORITY-CHECK per row
LOOP AT travels INTO DATA(travel).
  AUTHORITY-CHECK OBJECT 'ZTRAVEL_AUTH'
    ID 'AGENCY_ID' FIELD travel-AgencyId
    ID 'ACTVT'     FIELD '02'.
  ...
ENDLOOP.
```

```
" RIGHT - one check per DISTINCT agency, then a hashed lookup
DATA(agencies) = travels.
SORT agencies BY AgencyId.
DELETE ADJACENT DUPLICATES FROM agencies COMPARING AgencyId.

DATA authorized TYPE HASHED TABLE OF /dmo/agency_id WITH UNIQUE KEY table_line.
LOOP AT agencies INTO DATA(agency).
  AUTHORITY-CHECK OBJECT 'ZTRAVEL_AUTH'
    ID 'AGENCY_ID' FIELD agency-AgencyId
    ID 'ACTVT'     FIELD '02'.
  IF sy-subrc = 0.
    INSERT agency-AgencyId INTO TABLE authorized.
  ENDIF.
ENDLOOP.

LOOP AT travels INTO DATA(travel).
  DATA(ok) = xsdbool( line_exists( authorized[ table_line = travel-AgencyId ] ) ).
  ...
ENDLOOP.
```

Also:

- **Guard on `requested_authorizations` / `requested_features`.** The framework asks about only what it needs; evaluating everything on every call is wasted work.
- **Declare `affects permissions` sparingly.** Each one triggers a permissions round trip (Chapter 21, §21.9).

24.8 Determinations and validations

Rule	Why
Move expensive checks to `on save`	They then run **once per transaction**, not after every change
Make determinations idempotent (`DELETE tab WHERE <already set>`)	Fewer writes, and correctness
`CHECK tab IS NOT INITIAL` before `MODIFY ENTITIES`	An empty modify still costs a framework round trip
One determination triggered by several fields, not several determinations	Fewer framework callbacks
Never call an external service (HTTP, RFC) from `on modify`	Runs on every keystroke-level change
Avoid indirect determination cycles	A → B → A loops until the iteration limit, then dumps

24.9 Draft, locking and EML at scale

Draft

- Drafts accumulate. **Schedule the standard draft clean-up job** — see Chapter 11, §11.8. Without it the draft tables grow without bound.
- Never `SELECT` or `DELETE` on a draft table directly; use `EXECUTE Discard`.
- For an entity with very many columns and rows, weigh the cost of the second table before enabling draft.

Lock duration

- A draft holds an **exclusive lock** from Edit until Save/Cancel, and beyond — it survives the session ending and expires on a timeout. A user who opens a document and goes to lunch blocks everyone else.
- Keep the lock timeout aligned with how long an edit realistically takes.
- Use `( lock : none )` on read-only actions so they do not lock.
- `SET LOCKS` before expensive preparation, so you fail fast rather than after doing the work.

Large-volume EML processing

```
DATA(chunk_size) = 1000.
DATA(offset) = 0.

WHILE offset < lines( all_uuids ).
  DATA(chunk) = VALUE ztravel_tky_tab( ).
  LOOP AT all_uuids INTO DATA(u) FROM offset + 1 TO offset + chunk_size.
    APPEND VALUE #( TravelUUID = u %is_draft = if_abap_behv=>mk-off ) TO chunk.
  ENDLOOP.

  MODIFY ENTITIES OF zr_traveltp PRIVILEGED
    ENTITY Travel EXECUTE acceptTravel FROM chunk
    FAILED DATA(failed) REPORTED DATA(reported).

  COMMIT ENTITIES RESPONSE OF zr_traveltp FAILED DATA(commit_failed).
  IF sy-subrc = 8.
    ROLLBACK ENTITIES.
  ENDIF.

  " log per chunk, then continue
  offset += chunk_size.
ENDWHILE.
```

Why chunk:

Reason	Detail
Buffer size	The whole chunk lives in memory until commit
Lock table	Every instance holds a lock entry until commit; a 500,000-row commit can exhaust it
Restartability	A failure loses one chunk, not the whole run
Progress reporting	A job with no visible progress is unsupportable

1,000–5,000 is a reasonable starting range; tune with measurement, not intuition.

Background jobs

- Use `PRIVILEGED` deliberately, and document why — it bypasses authorization.
- Use `GET PERMISSIONS` **before** acting to filter out instances the job may not touch, rather than attempting each and handling failures.
- Never create drafts from a job. Jobs write active instances directly.
- Write an application log (`SLG1`) per run: started, chunk results, failures, finished.
- Make the job restartable and idempotent — re-running after a failure must not double-process.

24.10 A performance checklist for go-live

- [ ] At least one **mandatory, selective filter** on every List Report over a large table.
- [ ] No `SELECT` inside a `LOOP` anywhere in the behavior pool.
- [ ] No `SELECT` on the BO's own tables at all (use EML).
- [ ] `FIELDS ( ... )` rather than `ALL FIELDS` except where the full instance is genuinely needed.
- [ ] `LINK DATA( )` where only existence is needed.
- [ ] `AUTHORITY-CHECK` performed per distinct value, not per row.
- [ ] Handlers guard on `requested_authorizations` / `requested_features`.
- [ ] Virtual elements are display-only; nothing filterable or sortable depends on one.
- [ ] Association cardinalities verified against real data (no row multiplication).
- [ ] Expensive logic is `on save`, not `on modify`.
- [ ] No external calls (HTTP/RFC) in `on modify`, and none at all in the late save phase.
- [ ] `side effects` targets are as narrow as possible; `affects permissions` used sparingly.
- [ ] EML mass processing is chunked and restartable.
- [ ] Draft clean-up job scheduled.
- [ ] `ST05` trace of one List Report page load reviewed.
- [ ] ATC run with the HANA-readiness variant, findings triaged.
- [ ] `SQLM` enabled for a window after go-live.
- [ ] Tested with production-like data volume in QA.

24.11 Common errors

Symptom	Likely cause
List Report times out on open	No mandatory filter
App fast in DEV, slow in PROD	Data volume; or DCL adding an expensive filter on a non-indexed field
Save is slow with many items	`SELECT` inside a loop in a determination or validation
Opening the list is slow but scrolling is fast	Instance authorization / feature control running per row
Row counts are a multiple of what you expect	Association cardinality wrong
Sorting on one column is far slower than others	It is a virtual element
Memory dump (`TSV_TNEW_PAGE_ALLOC_FAILED`) in a job	Unchunked mass EML
Lock table overflow	Commit interval too large
Draft tables enormous	No clean-up job
UI feels laggy on every keystroke	Over-declared side effects

24.12 Interview answer

> **"How did you optimize CDS and EML performance?"**
>
> Mainly by keeping everything set-based and pushing work to the database.
>
> In CDS: associations rather than joins, because an association is lazy and costs nothing when it is not used; calculations in the interface view rather than as virtual elements, because a virtual element is computed in ABAP after the read and therefore cannot be filtered or sorted on HANA; and a mandatory, selective filter on any List Report over a large table — that one annotation is usually the difference between a sub-second report and a timeout.
>
> In the behavior pool: no `SELECT` inside a loop. I collect the distinct foreign keys, do one `SELECT ... FOR ALL ENTRIES`, and then use a hashed internal table for the lookup. The same applies to `AUTHORITY-CHECK` — one check per distinct authorization value, not per row, because instance authorization and feature control run for every row the list displays.
>
> In EML: one statement for many instances rather than a statement per instance, `FIELDS ( )` instead of `ALL FIELDS`, `LINK` instead of `RESULT` when I only need to know whether children exist, and chunked commits — around a thousand instances — for mass processing, so the transactional buffer and the lock table stay bounded and the job is restartable.
>
> To find problems I use `ST05` on a single page load — repeated identical statements immediately expose a SELECT in a loop — and `SQLM` in production because it is low-overhead and tells you what is actually slow rather than what you guessed.

24.13 Production considerations

Beyond performance, the things that decide whether an app survives contact with production:

Area	Requirement
Volume testing	QA must hold a production-like data subset. A 200-row DEV test proves nothing
Real roles	Test with the actual PFCG roles, never `SAP_ALL`
Monitoring	Application log per background job; `SQLM` window after go-live; a dump-check routine
Housekeeping	Draft clean-up job; application-log deletion job; archiving strategy for the BO's own tables
Restartability	Every job restartable and idempotent
Error visibility	Failures must reach a human — a job that logs to `SLG1` and nothing else will be ignored
Capacity	Lock table sizing if you do mass processing; work-process availability for long jobs
Documentation	The commit chunk size, the mandatory filter, and the lock timeout are all decisions the next developer must not undo blindly

Chapter 25 — Transport and Deployment (DEV → QA → PROD)

25.1 What is it?

Everything between "it works in my ADT session" and "a business user opens the tile in production".

RAP adds several artifacts that classic ABAP developers have not transported before — behavior definitions, service definitions, service bindings, metadata extensions — plus a set of **non-transportable** steps that must be repeated per system.

25.2 Why do we need it?

Because a RAP app can transport cleanly and still not work. The usual reasons:

Transported fine	Still broken because
All repository objects	The **service binding was never published** in the target system
All repository objects	The **ICF node / OData service** is not active (on-premise)
All repository objects	The **PFCG roles** were not transported or not assigned
All repository objects	The **Launchpad catalog / space / tile** does not exist
All repository objects	The **number range object** has no interval in the target client
All repository objects	**Customizing / master data** the app depends on is missing

Transporting the code is maybe 60% of a RAP deployment.

25.3 Package design

Get this right on day one; it is painful to change later.

```
ZRAP_TRAVEL                    (main package, transport layer assigned)
 +-- ZRAP_TRAVEL_DB            tables, data elements, domains, lock objects
 +-- ZRAP_TRAVEL_MODEL         interface views (ZR_, ZI_), DCL for the model layer
 +-- ZRAP_TRAVEL_BEHAVIOR      BDEF, behavior pools, message class, exception classes
 +-- ZRAP_TRAVEL_UI            projection views, projection BDEF, MDEs, DCL for the service
 +-- ZRAP_TRAVEL_SRV           service definitions, service bindings
 +-- ZRAP_TRAVEL_TEST          test classes, test data helpers
```

Decision	Guidance
Package type	**Development package**, not a structure package, for the leaves
Software component	`HOME` / `ZLOCAL` for custom development, unless your landscape says otherwise
Transport layer	Must be assigned, or objects land in `$TMP` and cannot be transported
Package interfaces	Worth defining if other teams will reuse `ZR_` / `ZI_` views
`$TMP`	**Never** for anything that must reach QA. Check before you start, not after

> A very common first mistake: developing in `$TMP` "just to try it" and then discovering that reassigning a package changes the object's transport request handling. Create the package first.

25.4 What transports, and what does not

Transportable repository objects

Object	Notes
DDIC tables, data elements, domains, structures	Table changes may require conversion in the target
Lock objects	Needed for unmanaged locking
CDS views (interface, projection)	
Metadata extensions	Separate objects from the views
Access controls (DCL)	

Object	Notes
Behavior definitions	
Behavior pool classes, exception classes, auxiliary classes	
Message class (SE91)	
Service definitions	
Service bindings	The object transports; **publication state may not** — see 25.6
Number range **object** (SNRO)	The **intervals** do not — see below
Authorization object (SU21)	
PFCG roles	Transported via role transport, often a separate request
Fiori app (BSP) if you built a UI5 project	Deployed separately via `npm run deploy` or `/UI5/UI5_REPOSITORY_LOAD`

Not transported — repeat in every system

Item	Where
Number range intervals	`SNRO` → *Number ranges* → maintain per client. Transporting the object does **not** carry the interval
Service binding publication	Publish in each system (see 25.6)
ICF service activation	`SICF`
OData service registration	`/IWFND/MAINT_SERVICE` (V2/on-premise scenarios)
Launchpad content (catalog, group, space, page, tile)	Maintained per system, or transported via its own mechanism
Role assignment to users	Per system, usually by the security team
Customizing	Its own customizing request
Master data / test data	Never transported
Communication arrangements / scenarios	Per system (cloud); RFC destinations per system (on-premise)
Background job scheduling	Per system

25.5 Transport organisation and sequence

One request or several?

For a new application: **one request with several tasks** is usually easiest, because RAP objects are tightly coupled — a BDEF cannot activate without its CDS view.

Splitting becomes necessary when:

- DDIC changes must go ahead of the rest (a table extension released early),
- roles go on a separate request by policy,
- a hotfix must move independently.

If you split, the import sequence matters. The safe order is the same as the build order:

```
1. DDIC (tables, data elements, domains, lock objects)
2. Interface views (children before parents)
3. Behavior definitions + behavior pools + message class
4. Projection views + projection BDEF
5. Metadata extensions + DCL
6. Service definitions
7. Service bindings
8. Authorization objects and roles
```

Importing a request containing a projection view before the request containing its interface view produces activation errors in the target system that look like code defects.

Release checklist (DEV)

- [ ] Everything activates with **zero errors and zero warnings**.
- [ ] ATC run, findings triaged (HANA-readiness + security variants).
- [ ] ABAP Unit tests green.
- [ ] No object left in `$TMP` (`SE03` → *Search for Objects in Requests* / check the package).
- [ ] No `new_message_with_text` left in the code (not translatable).
- [ ] No temporary debug messages or hard-coded test values.
- [ ] Texts translated, or at least flagged for translation.
- [ ] The transport description says what the change does, not "RAP changes".

25.6 Service binding activation in the target system

This is the step most often missed.

The service binding object transports. Whether it arrives **published** depends on your release and how the binding was created. Treat it as a post-import step:

1. Open the service binding in ADT, connected to the target system.
2. Check the status. If it is not **Published**, press **Publish**.
3. Confirm the Service URL is generated.

> In many landscapes, developers have no change authorization in QA/PROD. Then publication must be handled by whoever does — agree this **before** the first import, not on go-live day. It is a frequent cause of "the transport went in but the app is dead".

On-premise: ICF and gateway

For a service to be reachable over HTTP on-premise:

Step	Transaction	Notes
Activate the ICF node	`SICF`	Usually under `/sap/opu/odata4/...`; often already active for the whole branch
Register / activate the OData service	`/IWFND/MAINT_SERVICE`	Mainly relevant for V2 and for gateway-hub scenarios. A V4 service published locally may not need it
Check the gateway error log after first use	`/IWFND/ERROR_LOG`	First place to look if calls 500
Clear the metadata cache	`/IWFND/CACHE_CLEANUP`	After changing annotations in a system where caching is aggressive

The exact requirements vary by release and by whether you run an embedded or hub gateway. Confirm with your Basis team once, and then write it into your own deployment runbook.

25.7 Authorizations

Step	Where	Notes
Create the authorization object	`SU21`	Transportable
Add it to a PFCG role	`PFCG`	Maintain the field values (e.g. the agency ids) per system if they differ
Add the OData service authorization	`PFCG` → `S_SERVICE` (and the service in the role menu)	Needed for the user to call the service at all
Add the Launchpad catalog to the role	`PFCG` → role menu	Controls tile visibility
Assign the role to users	`SU01` / identity management	Per system
Test with a real role	—	Never validate authorization as a developer with wide authorizations

> The DCL (`@MappingRole: true`) binds automatically to users who hold the PFCG authorization — but **only if the role exists and is assigned in that system**. A DCL that filters everything is the normal symptom of a missing role assignment in QA.

25.8 Launchpad

Development preview and a productive Launchpad tile are **not the same thing**:

	ADT Preview	Fiori Launchpad
How it starts	Service binding → *Preview*	Tile in a catalog/space
Who can use it	Developers with ADT	End users
Needs a UI5 app / BSP	No	Usually yes
Needs catalog/group/space/page	No	Yes
Needs `S_SERVICE` and role menu entries	No (dev authorizations)	Yes
Theming, navigation, personalisation	Minimal	Full
Suitable for user acceptance testing	**No**	Yes

Do not sign off UAT on the ADT preview. It bypasses Launchpad configuration, role checks and navigation — exactly the areas that break in QA.

Getting to a real tile

1. Generate a Fiori Elements app in SAP Business Application Studio (or VS Code) from the OData service — *List Report Object Page* template.
2. Deploy to ABAP (`npm run deploy`) → creates a BSP application, on a transport request.
3. Create/extend the **catalog** and **tile** (Launchpad App Manager app, or `/UI2/FLPD_CUST`).

4. Assign the catalog to a **space/page** (or group, on older launchpads).
5. Add the catalog to the **PFCG role**.
6. Transport the BSP and the Launchpad content.

25.9 APIs and integration

If the deliverable is a Web API binding rather than a UI:

Step	On-premise	Cloud / ABAP Environment
Expose	Service binding, type *OData V4 - Web API*	Same
Reach it	ICF node + `S_SERVICE` authorization	**Communication scenario** (transportable) + **communication arrangement** (per system)
Credentials	Technical user, RFC destination	Communication user, communication system
Versioning	Service binding version (`0001` , `0002`)	Same
Contract	Publish `$metadata` to the consumer	Same

For cloud: the **communication scenario** is a transportable development object; the **communication arrangement** (which binds a scenario to a system and user) is configuration and is maintained per system. Plan the arrangement with the integration team before the import.

For outbound integration (your RAP app calling somebody else), remember the phase rules from Chapter 17: no synchronous RFC or HTTP in the late save phase. Enqueue and process asynchronously, with retry and idempotency.

25.10 Post-import checklist (run in every target system)

Technical

- [ ] Import log shows return code 0 or 4 (investigate any 4; never accept 8).
- [ ] All objects active — `SE03` → *Check Objects* / activate inactive objects.
- [ ] Table conversions completed (`SE14` if a table changed).
- [ ] Service binding is **Published**; Service URL present.
- [ ] `$metadata` returns 200 and contains the expected entities, actions and annotations.
- [ ] ICF node active (on-premise).
- [ ] Metadata/OData caches cleared.
- [ ] Number range intervals maintained in the target **client**.
- [ ] Draft clean-up job scheduled.
- [ ] Any application jobs scheduled.

Functional

- [ ] Log on as a **business test user**, not as a developer.
- [ ] Tile visible in the Launchpad.
- [ ] List Report opens; mandatory filters behave.
- [ ] **Value helps return data** (the classic QA-only failure — a missing `expose` , or DCL).
- [ ] Create → save works; number range assigns a document number.
- [ ] Validations fire with the right messages.
- [ ] Determinations produce the right calculated values, and the UI refreshes (side effects).
- [ ] Action buttons appear, are enabled/disabled correctly, and work.
- [ ] Draft: cancel, resume, and a two-session lock conflict.
- [ ] Delete cascades to children.
- [ ] Texts appear in the target language.

Authorization

- [ ] A user **with** the role sees the expected rows.
- [ ] A user **without** it sees nothing / is refused.
- [ ] Instance-level restriction works (a user restricted to one agency cannot change another's).
- [ ] Buttons requiring a specific activity are hidden/disabled for users without it.

25.11 Common errors

Symptom in QA/PROD	Cause
Tile missing	Catalog not in the PFCG role, or Launchpad content not transported
Tile present, app does not start	Service binding not published; ICF node inactive
App starts, no data	DCL + missing role assignment; or no data in that client
Value helps empty	Value-help entity not exposed, or DCL on it
Document number not assigned	Number range interval not maintained in the target client
Action button missing	Metadata cache; or the projection BDEF request was not part of the import
Activation errors after import	Requests imported in the wrong order
Works for the developer, not for users	Developer has wide authorizations; role incomplete
Texts in English only	Translation not transported
App worked, broke after the next import	A partially imported change set — check that all related requests moved together
Background job never runs	Not scheduled in the target system

25.12 Interview answer

> **"How did you transport and activate the service in QA and production?"**
>
> The repository objects — tables, CDS views, DCL, behavior definitions and pools, metadata extensions, service definition and service binding — all go on transport requests from a proper development package, never `$TMP`. If the change set is split across requests, the import order has to follow the build order: DDIC, then interface views, then behavior, then projections, then metadata extensions, then the service objects — otherwise you get activation errors in the target that look like code defects.
>
> The part people underestimate is what does **not** transport. The service binding object moves, but I treat **publishing it in the target system** as an explicit post-import step, and I agree up front who does it, because developers usually have no change authorization in QA and PROD. Number range **intervals** are not transported either — the SNRO object is, the interval is not — so a missing document number in QA is almost always that. Launchpad catalogs and tiles, role assignments, ICF activation, communication arrangements and job scheduling are all per-system too.
>
> Then I run a post-import checklist as a **business test user**, not as a developer: tile visible, `$metadata` correct, value helps returning data, create and save producing a document number, validations firing, action buttons enabled correctly, and the authorization cases — a user with the role and a user without it. Testing authorization as a developer with wide authorizations proves nothing.
>
> And UAT is signed off on the real Launchpad app, never on the ADT service-binding preview, because the preview bypasses exactly the Launchpad and role configuration that tends to break.

25.13 Production considerations

- **Write a deployment runbook** for the application, listing every non-transportable step. Do it after the first QA import, while the pain is fresh.
- **Automate what you can.** The post-import checks that can be scripted (service reachable, `$metadata` contains expected actions, number range interval exists) are worth a small ABAP report.
- **Keep DEV, QA and PROD on the same release.** RAP syntax availability differs between releases; code that activates in DEV on a newer stack will not activate in an older PROD.
- **Plan a rollback.** For a RAP app the realistic rollback is a corrective transport, not an un-import — design changes so that the previous version still runs if the new one is disabled.
- **Version Web API bindings** if external systems consume them. Breaking a published `$metadata` contract without a new version breaks the consumer.
- **Record who publishes bindings and who assigns roles.** These two hand-offs cause more go-live delays than any code problem.

Chapter 26 — Anti-Patterns: What Not To Do

26.1 How to use this chapter

Every entry is a **wrong / right pair** with the symptom it produces. These are the defects that actually appear in RAP code reviews, ordered roughly by how much damage they do.

Use it as a review checklist as well as a reading chapter.

#	Anti-pattern	Damage
1	`COMMIT WORK` in a handler or saver	Corrupts the LUW
2	`COMMIT ENTITIES` inside a behavior implementation	Same
3	Database modification inside `FOR MODIFY`	Bypasses the buffer; split transaction
4	Calling a BAPI that commits itself	Corrupts the LUW
5	`SELECT` on the BO's own tables	Reads stale data
6	`SELECT` inside a loop	N round trips
7	Message without `failed`	Record saves despite the error
8	Ignoring `%control`	Silently wipes fields
9	EML without `IN LOCAL MODE` inside the pool	Recursion, blocked writes, slow
10	Authorization only in the UI	Security hole
11	`@UI` on a reusable interface view	Leaks layout into every consumer
12	Number range in an `on modify` determination	Permanent number gaps
13	Non-idempotent determination	Overwrites user input
14	State message without clearing the state area	Error stays after the fix
15	Unmanaged buffer not cleared in `cleanup`	Second save misbehaves
16	`%key` instead of `%tky` in a draft BO	Operates on the wrong copy
17	Business logic in the projection layer	Bypassed by other consumers
18	Choosing `unmanaged` because tables already exist	10× the code for no benefit

26.2 Transaction control

1. `COMMIT WORK` in a handler or saver

```
" WRONG
METHOD acceptTravel.
  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel UPDATE FIELDS ( OverallStatus ) WITH ...
  COMMIT WORK.                          " <-- never
ENDMETHOD.
```

```
" RIGHT
METHOD acceptTravel.
  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel UPDATE FIELDS ( OverallStatus ) WITH ...
  " no commit. The RAP runtime owns the transaction.
ENDMETHOD.
```

Symptom: runtime error "*COMMIT WORK is not allowed here*" under strict mode; on older releases, a silently split LUW and half-saved documents.

Why: the behavior object is not the transaction owner. The consumer (Fiori runtime, or your calling program) decides when to commit.

2. `COMMIT ENTITIES` inside a behavior implementation

```
" WRONG
METHOD createPurchaseRequisition.
  MODIFY ENTITIES OF i_purchaserequisitiontp ENTITY purchaserequisition CREATE ...
  COMMIT ENTITIES.                      " <-- never inside the BO
ENDMETHOD.
```

```
" RIGHT
METHOD createPurchaseRequisition.
```

```
  MODIFY ENTITIES OF i_purchaserequisitiontp ENTITY purchaserequisition CREATE ...
  " both BOs commit together, in the caller's single LUW
ENDMETHOD.
```

Symptom: the called BO commits while yours has not; a failure afterwards leaves the two out of step and unrecoverable.

The rule: `COMMIT ENTITIES` belongs to whoever *started* the transaction — the RAP runtime for OData, or your own report/job. Never to a handler or saver.

3. Database modification inside `FOR MODIFY`

```
" WRONG
METHOD acceptTravel.
  LOOP AT keys INTO DATA(key).
    UPDATE ztravel SET overall_status = 'A'
      WHERE travel_uuid = @key-TravelUUID.       " <-- interaction phase!
  ENDLOOP.
ENDMETHOD.
```

```
" RIGHT
METHOD acceptTravel.
  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      UPDATE FIELDS ( OverallStatus )
      WITH VALUE #( FOR key IN keys ( %tky = key-%tky OverallStatus = 'A' ) )
    REPORTED DATA(rep).
  reported = CORRESPONDING #( DEEP rep ).
ENDMETHOD.
```

Symptom: the change is written even when the user later cancels; in a draft app it writes to the active instance while the user is editing a draft; the framework's own save then overwrites it or conflicts.

Why: direct database modification belongs in the **late save phase** only (Chapter 17, §17.6). In a managed BO you write no persistence code at all.

4. Calling a BAPI that commits itself

```
" WRONG
METHOD save_modified.
  CALL FUNCTION 'BAPI_TRAVEL_CREATE' EXPORTING ... TABLES return = lt_return.
  CALL FUNCTION 'BAPI_TRANSACTION_COMMIT'.        " <-- never
ENDMETHOD.
```

```
" RIGHT
METHOD check_before_save.
  " surface errors while we can still go back
  CALL FUNCTION 'BAPI_TRAVEL_CREATE'
    EXPORTING testrun = abap_true
    TABLES    return  = DATA(lt_return).
  LOOP AT lt_return INTO DATA(ls) WHERE type CA 'EA'.
    APPEND VALUE #( %tky = ... ) TO failed-travel.
    APPEND VALUE #( %tky = ... %msg = ... ) TO reported-travel.
  ENDLOOP.
ENDMETHOD.

METHOD save_modified.
  CALL FUNCTION 'BAPI_TRAVEL_CREATE'
    EXPORTING testrun = abap_false
    TABLES    return  = DATA(lt_return).
  " no BAPI_TRANSACTION_COMMIT - RAP commits
ENDMETHOD.
```

Symptom: the BAPI's commit ends the LUW mid-save; the RAP framework's own write then belongs to a different transaction.

Also note the test run. Without the `testrun = abap_true` call in `check_before_save`, a BAPI error in `save_modified` occurs after the point of no return and becomes a dump.

26.3 Reading and writing the wrong way

5. `SELECT` on the BO's own tables

```
" WRONG
METHOD validateDates.
  LOOP AT keys INTO DATA(key).
    SELECT SINGLE begin_date, end_date FROM ztravel
      WHERE travel_uuid = @key-TravelUUID
      INTO @DATA(ls).                          " <-- reads the DATABASE
    ...
  ENDLOOP.
ENDMETHOD.
```

```
" RIGHT
METHOD validateDates.
  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel
      FIELDS ( BeginDate EndDate )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).                          " <-- reads the BUFFER
  LOOP AT travels INTO DATA(travel).
    ...
  ENDLOOP.
ENDMETHOD.
```

Symptom: the validation passes on data the user just corrected, or fails on data they just fixed. In a draft app it reads the *active* row while the user works on the *draft* — so it validates values that are not on screen.

The boundary: `SELECT` on **other** tables (master data, customizing) is correct and normal. `SELECT` on the BO's **own** persistent or draft tables is the defect.

6. `SELECT` inside a loop

```
" WRONG - 5000 travels = 5000 round trips
LOOP AT travels INTO DATA(travel).
  SELECT SINGLE customer_id FROM /dmo/customer
    WHERE customer_id = @travel-CustomerId INTO @DATA(found).
  IF sy-subrc <> 0.
    APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.
  ENDIF.
ENDLOOP.
```

```
" RIGHT - one round trip, then O(1) lookups
DATA(customers) = travels.
SORT customers BY CustomerId.
DELETE ADJACENT DUPLICATES FROM customers COMPARING CustomerId.
DELETE customers WHERE CustomerId IS INITIAL.

IF customers IS NOT INITIAL.
  SELECT FROM /dmo/customer FIELDS customer_id
    FOR ALL ENTRIES IN @customers
    WHERE customer_id = @customers-CustomerId
    INTO TABLE @DATA(valid).
ENDIF.

LOOP AT travels INTO DATA(travel).
  IF NOT line_exists( valid[ customer_id = travel-CustomerId ] ).
    APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.
  ENDIF.
ENDLOOP.
```

Symptom: the app is fine in DEV and unusable in PROD. Visible instantly in an `ST05` trace as the same statement repeated N times.

9. EML without `IN LOCAL MODE` inside the behavior pool

```
" WRONG - inside a determination
MODIFY ENTITIES OF zr_traveltp                     " <-- no IN LOCAL MODE
  ENTITY Travel UPDATE FIELDS ( TotalPrice ) WITH ...
```

```
" RIGHT
MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
  ENTITY Travel UPDATE FIELDS ( TotalPrice ) WITH ...
```

Symptoms, all three of them:

- `TotalPrice` is `field ( readonly )`, so the write is **rejected** — the determination appears to do nothing;
- the call goes through the full BO stack, re-triggering authorization, feature control and potentially the determination itself (recursion);
- it is measurably slower.

The rule: inside a behavior pool, always `IN LOCAL MODE`. Outside it, never — you want the checks.

26.4 Getting the RAP contract wrong

7. A message without a `failed` entry

```
" WRONG - the user sees a red error and the record saves anyway
IF travel-EndDate < travel-BeginDate.
  APPEND VALUE #( %tky = travel-%tky
                  %msg = new_message( id = 'ZTRAVEL_MSG' number = '001'
                                      severity = if_abap_behv_message=>severity-error ) )
         TO reported-travel.
ENDIF.
```

```
" RIGHT - failed blocks the save, reported explains why
IF travel-EndDate < travel-BeginDate.
  APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.          " <-- blocks
  APPEND VALUE #( %tky = travel-%tky
                  %msg = new_message( id = 'ZTRAVEL_MSG' number = '001'
                                      severity = if_abap_behv_message=>severity-error )
                  %element-BeginDate = if_abap_behv=>mk-on
                  %element-EndDate   = if_abap_behv=>mk-on )
        TO reported-travel.                                        " <-- explains
ENDIF.
```

The inverse is just as bad: `failed` without `reported` blocks the save and tells the user nothing. For an error, always fill **both**. For a warning, fill `reported` only — deliberately.

8. Ignoring `%control`

```
" WRONG - in an unmanaged update or save_modified
LOOP AT entities INTO DATA(entity).
  <buffered>-travel = CORRESPONDING #( entity MAPPING FROM ENTITY ).
ENDLOOP.
```

The consumer sent only `Description`. Every other field in `entity` is initial — so this writes blanks over `AgencyId`, `CustomerId`, `BeginDate` and the rest.

```
" RIGHT - apply only what was actually sent
LOOP AT entities INTO DATA(entity).
  IF entity-%control-Description = if_abap_behv=>mk-on.
    <buffered>-travel-description = entity-Description.
  ENDIF.
  IF entity-%control-AgencyId = if_abap_behv=>mk-on.
    <buffered>-travel-agency_id = entity-AgencyId.
  ENDIF.
  " ... one block per field ...
ENDLOOP.
```

Or, with a control structure declared in the BDEF (`mapping for ztravel control ztravel_x corresponding`), map `%control` into an x-structure and drive a field-selective `UPDATE SET` — see Chapter 16, §16.6.

Symptom: a user edits one field and unrelated fields are wiped. Reported as "data loss", which it is.

This does not apply to managed BOs — the framework honours `%control` for you. It is an unmanaged / unmanaged-save concern.

14. State message without clearing the state area

```
" WRONG - the error stays on screen after the user fixes the data
LOOP AT travels INTO DATA(travel).
  IF travel-EndDate < travel-BeginDate.
    APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.
    APPEND VALUE #( %tky = travel-%tky %state_area = 'VALIDATE_DATES' %msg = ... )
           TO reported-travel.
  ENDIF.
ENDLOOP.
```

```
" RIGHT - clear the area first, unconditionally, then add if still invalid
LOOP AT travels INTO DATA(travel).

  APPEND VALUE #( %tky = travel-%tky %state_area = 'VALIDATE_DATES' ) TO reported-travel.

  IF travel-EndDate < travel-BeginDate.
    APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.
    APPEND VALUE #( %tky = travel-%tky %state_area = 'VALIDATE_DATES' %msg = ... )
           TO reported-travel.
  ENDIF.
ENDLOOP.
```

Use **one** `%state_area` **per validation**, or validations will clear each other's messages.

16. `%key` instead of `%tky` in a draft BO

```
" WRONG
READ ENTITIES OF zr_traveltp IN LOCAL MODE
  ENTITY Travel FIELDS ( TotalPrice )
  WITH VALUE #( ( TravelUUID = key-TravelUUID ) )     " which copy? active or draft?
  RESULT DATA(travels).
```

```
" RIGHT
READ ENTITIES OF zr_traveltp IN LOCAL MODE
  ENTITY Travel FIELDS ( TotalPrice )
  WITH CORRESPONDING #( keys )                        " keys carries %tky
  RESULT DATA(travels).
```

Symptom: the determination reads the *active* instance while the user is editing the *draft*, so it sees old values, or returns nothing at all. `strict ( 2 )` catches most of these at activation — another reason to switch it on.

13. Non-idempotent determination

```
" WRONG – overwrites whatever the user typed, every time it runs
METHOD setInitialOverallStatus.
  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel UPDATE FIELDS ( OverallStatus )
    WITH VALUE #( FOR key IN keys ( %tky = key-%tky OverallStatus = 'O' ) ).
ENDMETHOD.
```

```
" RIGHT – only fill what is still empty
METHOD setInitialOverallStatus.
  READ ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel FIELDS ( OverallStatus ) WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  DELETE travels WHERE OverallStatus IS NOT INITIAL.
  CHECK travels IS NOT INITIAL.

  MODIFY ENTITIES OF zr_traveltp IN LOCAL MODE
    ENTITY Travel UPDATE FIELDS ( OverallStatus )
    WITH VALUE #( FOR t IN travels ( %tky = t-%tky OverallStatus = 'O' ) ).
ENDMETHOD.
```

A determination can be called more than once in a transaction. Assume it will be.

12. Number range in an `on modify` determination

```
" WRONG
determination setTravelId on modify { create; }
```

```
" RIGHT
determination setTravelId on save { create; }
```

Symptom: permanent gaps in the document numbering, because every draft the user discards has already consumed a number. In regulated scenarios that is an audit finding, not a cosmetic issue.

15. Unmanaged buffer / `CLASS-DATA` not cleared

```
" WRONG – cleanup implemented, cleanup_finalize forgotten
METHOD cleanup.
  lcl_buffer=>get_instance( )->clear( ).
ENDMETHOD.
```

```
" RIGHT – both paths clear the state
METHOD cleanup.
  lcl_buffer=>get_instance( )->clear( ).
  CLEAR zbp_r_traveltp=>mapped_purchase_requisition.
ENDMETHOD.

METHOD cleanup_finalize.
  lcl_buffer=>get_instance( )->clear( ).
  CLEAR zbp_r_traveltp=>mapped_purchase_requisition.
ENDMETHOD.
```

`CLASS-DATA` lives for the whole ABAP session, not the transaction. `cleanup_finalize` is the path taken when the early save phase **rejects** the save — exactly the case where stale state then leaks into the user's next attempt.

26.5 Architecture and layering

10. Authorization only in the UI

```
" WRONG – the only 'protection' is feature control
METHOD get_instance_features.
  result = VALUE #( FOR t IN travels
    ( %tky = t-%tky
      %action-acceptTravel = COND #( WHEN user_is_manager( ) THEN fc-o-enabled
                                                             ELSE fc-o-disabled ) ) ).
ENDMETHOD.
```

Feature control only greys out the button. A `POST` to the action URL still reaches the backend and still succeeds. Anyone who can open DevTools can execute it.

```
" RIGHT – authorization enforces, feature control informs
METHOD get_instance_authorizations.
  " ... AUTHORITY-CHECK ...
  APPEND VALUE #( %tky = travel-%tky
                  %action-acceptTravel = COND #( WHEN authorized = abap_true
                                                 THEN if_abap_behv=>auth-allowed
                                                 ELSE if_abap_behv=>auth-unauthorized ) )
```

```
        TO result.
ENDMETHOD.

METHOD get_instance_features.
  " business state only: an accepted travel cannot be accepted again
  APPEND VALUE #( %tky = travel-%tky
                  %action-acceptTravel = COND #( WHEN travel-OverallStatus = 'A'
                                                 THEN if_abap_behv=>fc-o-disabled
                                                 ELSE if_abap_behv=>fc-o-enabled ) )
        TO result.
ENDMETHOD.
```

The distinction to hold on to:

	Authorization	Feature control
Question	"May you?"	"Does it apply right now?"
Enforced by the framework	Yes	No — UI hint only
Bypassable by an API caller	No	Yes
Constants	`auth-allowed` / `auth-unauthorized`	`fc-o-enabled` / `fc-o-disabled`

And remember DCL covers **reads** only. A user who can see nothing can still `PATCH` a guessed key unless BDEF authorization stops them.

11. `@UI` annotations on a reusable interface view

```
" WRONG - in ZR_TravelTP
@UI.lineItem: [ { position: 10, importance: #HIGH } ]
travel_id as TravelId,
```

Every projection over this model now inherits that layout, including services built by other teams years later.

```
" RIGHT - ZR_ stays neutral
@Metadata.ignorePropagatedAnnotations: true
define root view entity ZR_TravelTP ...
{
  travel_id as TravelId,
}
```

with the layout in the **metadata extension** on `ZC_TravelTP`.

The layering rule: business semantics go **down** (`@Semantics`, criticality calculations, associations); presentation goes **up** (`@UI`, `@Search` for one app, value helps for one service).

17. Business logic in the projection layer

```
" WRONG - a validation only in the projection BDEF
projection;
define behavior for ZC_TravelTP alias Travel
{
  validation validateCreditLimit on save { create; }   " <-- only this service checks it
}
```

The rule is then **not** applied to an EML caller, a background job, a second service, or an API binding over the same BO.

```
" RIGHT - the rule lives with the business object
managed implementation in class ZBP_R_TravelTP unique;
define behavior for ZR_TravelTP alias Travel
{
  validation validateCreditLimit on save { create; }   " every consumer
}
```

Projection-level validations and determinations are legitimate — but only for genuinely **service-specific** behaviour (a default that only makes sense in this app). A business invariant belongs in the base BDEF.

18. Choosing `unmanaged` because tables already exist

```
" WRONG reasoning: "we have existing tables, so we must go unmanaged"
unmanaged implementation in class ZBP_R_TravelTP unique;
```

```
" RIGHT: existing tables are fine for managed
managed implementation in class ZBP_R_TravelTP unique;
...
persistent table ztravel
```

`persistent table` accepts any DDIC table. What forces unmanaged is existing **transactional logic** — an existing buffer, existing locking, existing update-task modules — not existing data.

Cost of getting this wrong: roughly 600–900 lines of hand-written buffer, lock, read, create, update, delete and save code that the managed runtime would have provided for nothing (Chapter 15, §15.8).

And if only the **save** is special, use `managed with unmanaged save` (Chapter 16), not a full unmanaged BO.

26.6 Smaller but common

Anti-pattern	Instead
`new_message_with_text( )` in production code	`new_message( id = ... number = ... )` — translatable
Hard-coded status literals (`'A'` , `'O'`) scattered through the pool	Constants in the global ABP class
`reported = CORRESPONDING #( DEEP ... )` omitted after an inner `MODIFY`	Always propagate; otherwise messages vanish
`ALL FIELDS` everywhere	`FIELDS ( ... )` except when returning a full instance
`RESULT DATA( )` when only existence matters	`LINK DATA( )`
Empty `MODIFY ENTITIES` with no rows	`CHECK tab IS NOT INITIAL` first
An empty saver class "just in case"	Do not create one unless a saver method is actually needed
Validation declared `on modify`	`on save` — `on modify` nags the user mid-typing
Validation not listed in `draft determine action Prepare`	List it, or errors appear only at Save
Calculated field with no `side effects` declaration	Declare it, or the UI shows a stale value
Hand-typed handler method signatures	Generate with `Ctrl + 1`
Hand-designed draft tables	Generate with the ADT quick fix
BDEF without `strict ( 2 )`	Add it; every error it raises is a real latent defect
Developing in `$TMP`	Create the package first
UAT signed off on the ADT preview	Use the real Launchpad app
Authorization tested as a developer	Test with a real business role

---

26.7 Review checklist

Run this over any RAP change before it leaves DEV:

- [ ] No `COMMIT WORK` / `ROLLBACK WORK` / `COMMIT ENTITIES` anywhere in the behavior pool.
- [ ] No database modification outside the late save phase.
- [ ] No `SELECT` on the BO's own persistent or draft tables.
- [ ] No `SELECT` inside a loop.
- [ ] Every EML call inside the pool uses `IN LOCAL MODE` .
- [ ] Every rejecting validation fills **both** `failed` and `reported` .
- [ ] Every state message clears its `%state_area` first.
- [ ] Every determination is idempotent.
- [ ] `reported = CORRESPONDING #( DEEP ... )` after every inner `MODIFY` .
- [ ] `%control` respected in unmanaged / unmanaged-save update logic.
- [ ] `%tky` used throughout in draft-enabled BOs.
- [ ] Number ranges drawn `on save` , never `on modify` .
- [ ] `cleanup` **and** `cleanup_finalize` clear all own state.
- [ ] Authorization enforced in `FOR ... AUTHORIZATION` , not only in feature control.
- [ ] No `@UI` annotations on interface views.
- [ ] Business invariants in the base BDEF, not the projection.
- [ ] `strict ( 2 )` on.
- [ ] No `new_message_with_text` left in the code.
- [ ] No objects in `$TMP` .

Chapter 27 — Interview Preparation

27.1 How to use this chapter

Two sections:

- **27.3 Core questions** — knowledge questions with a concise spoken answer, plus the follow-up the interviewer will ask next.
- **27.4 Experience questions** — "how did *you* do X". These are about **your** project, so what follows is the **shape** of a strong answer plus a worked example based on this guide's running scenario. Fill in your own specifics; do not recite someone else's project.

Three rules for delivery:

1. **Answer in three layers**: one sentence of definition, two or three of mechanism, one of trade-off or gotcha. Then stop. Interviewers interrupt when they want more.
2. **Name the failure mode.** Saying "*a validation that fills* `reported` *but not* `failed` *shows an error and saves anyway*" signals real experience more than any definition does.
3. **Say what you do not know.** "I have used managed and managed-with-unmanaged-save; I have read about full unmanaged but not built one" is a better answer than a confident wrong one.

27.2 The two-minute overview

If you get exactly one question, it will be this one.

> RAP is SAP's programming model for building OData services and Fiori apps on the ABAP platform. You define the data model in CDS, declare the transactional behavior in a behavior definition, and implement the business logic in an ABAP behavior pool. The framework handles the transaction, the buffer, locking, ETags, draft and the OData protocol.
>
> The layers are: database table, CDS interface view (the business object model), behavior definition plus behavior pool (the behavior), CDS projection view plus projection behavior definition (the service-specific subset), metadata extension (the UI layout), and service definition plus service binding (what is exposed, and over which protocol).
>
> The big shift from classic ABAP is that it is **declarative first**. Locking is one word, `lock master`. Optimistic concurrency is one word, `etag master`. Draft is one word. If you find yourself writing a lot of code for something RAP offers declaratively, you are fighting the framework.

27.3 Core questions

1. What is RAP?

> See 27.2. Keep it to the layer stack plus the "declarative first" point.

Follow-up: "*Which layer would you put a validation in?*" → the **base** behavior definition, so every consumer gets it, not just one service.

2. Managed versus unmanaged RAP

> They are implementation types declared in the behavior definition header. The difference is who provides the **transactional buffer** and who performs the **save**.
>
> In a **managed** BO the framework does both. You declare `persistent table`, and create, update, delete and read work out of the box. You only write the business logic — validations, determinations, actions.
>
> In an **unmanaged** BO you provide everything: your own buffer, and handler methods for `FOR READ`, `FOR MODIFY` create/update/delete, `FOR LOCK`, read-by-association and create-by-association, plus a saver class that persists the buffer.
>
> Managed is for greenfield development. Unmanaged is for wrapping **existing transactional logic** you must reuse — existing locking, existing update-task modules. Existing *tables* are not a reason: `persistent table` accepts any table.
>
> There are also two hybrids: `managed with additional save`, where the framework still persists the BO and your `save_modified` does extra work such as change documents or business events, and `managed with unmanaged save`, where you take over persistence — for example to write through a BAPI — but keep the managed buffer, locking, ETag and draft.

Follow-up: "*How much more code is unmanaged?*" → roughly an order of magnitude for the same CRUD. The field-selective `%control` handling alone is one `IF` block per field per entity.

3. Why use a projection view? / 4. Interface vs projection

> The interface view, `ZR_...`, is the business object model: the fields, the composition tree, the associations, the semantic annotations. It is neutral and reusable — no UI annotations, nothing service-specific.
>
> The projection view, `ZC_...`, is a **service-specific window** onto that model, written with `as projection on`. It exposes a subset of fields, carries the value helps and the `@Consumption` annotations, and is the attachment point for the metadata extension and the DCL.
>
> Three reasons for the split. First, one business object can serve several services with different fields and different enabled operations — a manager app and a self-service app. Second, you can add a field to the model without touching any running service. Third, it keeps business semantics and presentation in different layers.
>
> The projection behavior definition follows the same logic: it contains `use create`, `use update`, `use action X` and so on. An operation or action not listed there does not exist in that service, no matter what the base behavior definition says.

Follow-up: "*What can a projection BDEF not do?*" → it can only subset. It cannot add `create` if the base has none, and it cannot change locking, ETag, authorization or numbering.

---

5. Purpose of a behavior definition

> It is the contract of the business object, written in BDL. It declares the implementation type, the persistent and draft tables, locking, ETag, authorization, which operations exist, field characteristics such as `readonly` and `mandatory`, create-by-association, validations, determinations, actions, side effects, the draft actions, and the mapping between CDS element names and database field names.
>
> It is a declaration, not code. The behavior pool implements what it declares.

Follow-up: "*What does `strict ( 2 )` do?*" → turns on the strictest syntax and semantic checks: mandatory lock and authorization declarations, `%tky` instead of `%key` in draft scenarios, stricter feature control and EML checks. Every error it raises on a legacy BDEF is a real latent defect.

---

6. What is a behavior pool?

> A global class declared `FOR BEHAVIOR OF <bdef>`. It is `ABSTRACT FINAL` because the RAP runtime instantiates the local classes inside it, never the global class.
>
> The real content is in the local types: one or more **handler** classes inheriting `CL_ABAP_BEHAVIOR_HANDLER`, which implement the interaction phase — determinations, validations, actions, feature control, authorization — and, where needed, a **saver** class inheriting `CL_ABAP_BEHAVIOR_SAVER`, which implements the save sequence.
>
> The global class's public section is also the only supported channel between the handler and the saver, since they are separate classes. If you use `CLASS-DATA` for that, you have to clear it in both `cleanup` and `cleanup_finalize` — it lives for the whole session, not the transaction.

---

7. Validations and determinations

> Both are declared in the behavior definition with a trigger time and a trigger condition, and implemented in the behavior pool.
>
> A **determination** derives data: it fills a field automatically. A **validation** checks data and rejects it. A validation must never change data.
>
> The trigger time is `on modify` or `on save`. `on modify` runs during the interaction phase, immediately after the triggering change; `on save` runs in the early save phase — determinations when `finalize` is reached, validations when `check_before_save` is reached.
>
> Practically: determinations the user should see live are `on modify`; anything that consumes a scarce resource, like a number range, is `on save`, otherwise every discarded draft burns a number. Validations are almost always `on save`, because an `on modify` validation nags the user while they are still typing.
>
> The gotcha worth mentioning: a validation blocks the save by adding the key to `failed`. A message in `reported` alone shows a red error and saves the record anyway.

Follow-up: "*Is the order of two validations guaranteed?*" → no. Order within a phase is not guaranteed, so validations must be independent of each other.

---

8. Draft handling

> A draft is a second copy of the instance, stored in its own draft table, that the user edits before committing. Both rows share the same key and are distinguished by `IsActiveEntity` — which is why the RAP transactional key is `%tky`, the key plus `%is_draft`.
>
> You enable it with `with draft;` in the header, a `draft table` per entity, `total etag`, and the four draft actions: `Edit`, `Activate`, `Discard`, `Resume`. Plus `draft determine action Prepare`, which lists the validations that should run **while** the draft is open so errors appear inline rather than only at save.
>
> Locking has two phases. From `Edit`, the active instance holds an **exclusive** lock that survives the session ending and expires on a timeout. After that the **optimistic** phase begins: others can edit, and when the original user resumes, RAP compares the draft's stored **total ETag** with the active instance's. If they differ, the draft is declared outdated. That is what `total etag` is for, and why it should be a different field from `etag master`.
>
> Draft is a UI concept. A background job or an API creates active instances directly.

Follow-up: "*When would you not use draft?*" → APIs and jobs; high-frequency single-field updates where the round trips cost more than the operation; very wide, very large entities where a second table is expensive.

---

9. What is EML?

Entity Manipulation Language — the ABAP statements for calling a RAP business object: `READ ENTITIES`, `MODIFY ENTITIES`, `COMMIT ENTITIES`, `ROLLBACK ENTITIES`, `GET PERMISSIONS`, `SET LOCKS`, `RAISE ENTITY EVENT`, and `CONVERT KEY` for late numbering.

It is the only supported way to talk to a RAP BO from ABAP. You do not `SELECT` the BO's tables and you do not call your own OData service over HTTP.

Inside a behavior pool you always add `IN LOCAL MODE`: it goes straight to the transactional buffer, skips authorization and feature control, and — importantly — lets you write `readonly` fields, which is how a determination sets a field the consumer may not touch. Outside the BO you never use it, because you want those checks.

And `COMMIT ENTITIES` is only ever issued by whoever owns the transaction — the RAP runtime for OData, or your own report or job. Never inside a handler or saver.

---

10. `%cid`, `%tky`, `%pid`, `%control`

`%cid` is the **content ID** — a name you invent for an instance that does not have a key yet. On create you send a `%cid`; the framework reports back through `mapped` which real key it assigned. In a deep create, the child uses `%cid_ref` to say which `%cid` is its parent, and `%target` to carry the children. That is how you create a document with its items in one statement.

`%tky` is the **transactional key**: the key plus `%is_draft`. In a draft-enabled BO you always use `%tky`, never `%key`, because the same key exists twice — once active, once draft. Using `%key` reads the wrong copy, which usually looks like "my determination sees old data".

`%pid` is a **preliminary ID** in late-numbering scenarios. The instance gets a real key only in the save sequence, and you resolve `%pid` to the real key with `CONVERT KEY`, which is only legal inside `save_modified`.

`%control` is a set of **per-field flags** saying which fields the consumer actually sent, using `if_abap_behv=>mk-on` / `mk-off`. A managed BO honours it automatically. In an unmanaged BO or an unmanaged save you must respect it yourself — otherwise a user who edits one field has every other field overwritten with initial values.

---

11. `failed`, `reported`, `mapped`

The three RAP response parameters. Every handler method has them as implicit changing parameters.

`failed` says *which instances could not be processed*. Adding a key to `failed` is what actually blocks the operation.

`reported` carries the *messages* — the text, severity, and optionally `%element-<Field>` so Fiori highlights the right field, and `%state_area` for messages that persist until cleared.

`mapped` carries *key mapping information* — which real key was assigned to which `%cid`. It only applies to create operations. A managed BO fills it; in an unmanaged BO or a factory action you fill it yourself, and if you forget, the consumer cannot address the new instance and the UI cannot navigate to it.

The pairing matters: for an error, fill both `failed` and `reported`. `failed` without `reported` blocks the save and tells the user nothing; `reported` without `failed` shows an error and saves anyway. For a warning, fill `reported` only — deliberately.

---

12. When do you use additional save?

When the framework's persistence is correct and you need **extra work at save time**: writing a change document, an application log entry, raising a RAP business event, registering an update-task module, or enqueueing an outbound message. You declare `managed with additional save` and implement `save_modified`, which gives you the created, updated and deleted instances — including `%control`, so you can tell which fields actually changed.

The contrast is `with unmanaged save`, where you take over persistence entirely: the framework does not write the table, and you must remove `persistent table` from the behavior definition. That is for persisting through a BAPI, into an external store, or when you need `CONVERT KEY` for a late-numbering API — because `CONVERT KEY` is only legal in `save_modified`.

One rule applies to both: the late save phase must not fail. Anything that can fail belongs in `check_before_save`, while the transaction is still recoverable.

---

13. What is the RAP save sequence?

A RAP transaction has two phases. During the **interaction phase**, handler methods run and changes go into the transactional buffer; the database is untouched and the buffer may be temporarily inconsistent.

Then `COMMIT ENTITIES` — issued implicitly by the RAP runtime in a Fiori scenario — triggers the **save sequence**, which splits in two.

The **early save phase** runs `finalize`, where `on save` determinations execute and it is the last place data may be changed; then `check_before_save`, where `on save` validations execute and which is the last gate. If either reports a failure, `cleanup_finalize` runs and the transaction returns to the interaction phase — `sy-subrc` is 4 after `COMMIT ENTITIES` and it is recoverable.

Then comes the **point of no return**, and the **late save phase**: `adjust_numbers` for late numbering, then `save` or `save_modified`, then the implicit `COMMIT WORK`, then `cleanup` clears the buffer.

The design principle is that everything which can fail must fail in the early phase. A failure after the point of no return is a runtime error unless you inherit from `CL_ABAP_BEHAVIOR_SAVER_FAILED`, in which case `sy-subrc` is 8 and the consumer must issue an explicit `ROLLBACK ENTITIES`.

Follow-up: "*Where can you do database updates?*" → in the late save phase, and only where you own persistence at all. In a plain managed BO you write no persistence code; the framework does it. Reads are allowed everywhere; commits on a **secondary** connection — number ranges, application logs — are allowed in every phase, which is why a number range is legal in an `on save` determination.

---

14. How do you implement actions?

> Four steps, and missing any one fails silently in a different way.
>
> One: declare it in the base behavior definition — `action acceptTravel result [1] $self;` . `result [1] $self` returns the changed instance so the UI repaints without an extra round trip. Two: implement the handler method, generated with `Ctrl + 1` — read the instances, apply the change with `MODIFY ENTITIES ... IN LOCAL MODE` , fill `result` . Three: expose it in the projection behavior definition with `use action acceptTravel;` . Four: annotate it in the metadata extension with `@UI.lineItem: [{ type: #FOR_ACTION, dataAction: 'acceptTravel', label: '...' }]` .
>
> The step people forget is three. Without `use action` , the button never appears and there is no error anywhere.
>
> Variants: `static action` is not bound to an instance; `factory action` creates instances and must fill `mapped` ; `internal action` is callable only from inside the BO; a `function` is read-only and uses `FOR READ` rather than `FOR MODIFY` . An action can take a `parameter` typed by a CDS abstract entity, which Fiori renders as a dialog — and you must validate those parameters inside the action, because `validation` only fires on entity fields.
>
> Enabling and disabling per instance is `( features : instance )` plus a `FOR INSTANCE FEATURES` handler — but that is a UI hint, not security. Authorization for the action belongs in `FOR INSTANCE AUTHORIZATION` .

---

15. How do you handle authorization?

> Four mechanisms, and the interview point is that they are not interchangeable.
>
> **DCL** — CDS access control — filters **reads**. `@AccessControl.authorizationCheck: #CHECK` plus a role with `aspect pfcg_auth(...)` restricts which rows a user sees. Which layer it goes on depends on the design: projection-level when different services need different restrictions, interface-level when the restriction is an invariant of the model. DCL does **not** stop an update or a delete.
>
> **Global authorization**, `FOR GLOBAL AUTHORIZATION` , answers "may this user create at all" — instance-independent.
>
> **Instance authorization**, `FOR INSTANCE AUTHORIZATION` , answers "may this user change *this* instance" — typically an `AUTHORITY-CHECK` on a field of the row. It returns `auth-allowed` or `auth-unauthorized` per operation, per action, per association.
>
> **Feature control** is different in kind. It answers "does this apply right now" — an accepted document cannot be accepted again. It uses the `fc-o-` and `fc-f-` constants and it is a **UI hint only**: the OData call still reaches the backend and still succeeds. Never implement a security rule as feature control.
>
> Performance is worth mentioning: instance authorization runs for every row the list displays, so do one `AUTHORITY-CHECK` per distinct value and then a hashed lookup, and guard on `requested_authorizations` so you only evaluate what the framework asked about.

---

16. How do you expose a RAP BO through OData V4?

> Two objects. A **service definition** lists what is exposed — `expose ZC_TravelTP as Travel;` — including every child entity and every value-help and text entity, otherwise the F4 comes back empty. Then a **service binding** of type *OData V4 - UI* over that definition, which you activate and then **Publish**; publishing generates the runtime artifacts and the service URL, and gives you the Fiori Elements preview.
>
> One service definition can feed several bindings — for example a V4 UI binding for a Fiori app and a V4 Web API binding for machine-to-machine integration, where the UI annotations are stripped out.
>
> The prerequisite on the CDS side is `provider contract transactional_query` on the root projection and a projection behavior definition. Leave the contract off and you get a read-only service, which is exactly what you want for a report.
>
> For deployment, the binding object transports but I treat **publishing in the target system** as an explicit post-import step, and on-premise you also need the ICF node active and the Launchpad content and roles in place.

27.4 Experience questions

These are about your project. Below is the **shape** of a strong answer, then a worked example based on this guide's scenario. Replace the example with yours.

Why did you choose managed RAP for your project?

Shape: state the decision criteria, then how your project scored against them, then what you would have chosen otherwise.

> The deciding question is whether there is existing **transactional logic** to reuse — an existing buffer, existing locking, existing update-task modules. If there is, unmanaged. If not, managed. Existing tables on their own are not a reason, because `persistent table` accepts any table.
>
> In our case it was greenfield on our own Z tables with no legacy logic, so managed. That gave us CRUD, UUID key generation, the five admin fields, locking, ETag and draft declaratively, and we only wrote the business logic — three determinations, three validations and two actions, about 200 lines. Unmanaged would have been roughly an order of magnitude more code for the same result.
>
> We did use `managed with additional save` for one piece: writing an application log entry and raising an event on status change, which has to happen in the late save phase. And if the persistence itself had needed to go through a released API, the answer would have been `managed with unmanaged save` rather than a full unmanaged BO.

Why not SEGW?

> SEGW is the classic Gateway Service Builder. It generates an OData **V2** service where you implement CRUD by redefining methods in a `DPC_EXT` class — `GET_ENTITYSET`, `CREATE_ENTITY` and so on. There is no transactional framework: no buffer, no draft, no declarative locking or ETag, no validation or determination concept. Everything is hand-written, and the Fiori annotations live in a separate annotation model rather than coming from CDS.
>
> RAP gives all of that declaratively, targets OData V4, and is the model that is ABAP Cloud compliant — SEGW is not available in the restricted ABAP language scope, so anything built in it is a dead end for a move to the cloud or a clean-core strategy.
>
> The honest caveat is that SEGW still exists for maintaining older services, and if you have to extend an existing V2 service consumed by an app you do not control, you extend it rather than rewrite. But for new development there is no case for it.

How did you handle locking and ETags?

Shape: say they solve different problems, then name your declarations, then the draft nuance.

> They are complementary, not alternatives. The lock is **pessimistic** — it stops a second user starting an edit. The ETag is **optimistic** — it stops a stale write at the moment of saving.
>
> We declared `lock master` on the root and `lock dependent by _Travel` on the child, so the whole composition tree locks together, and the managed runtime acquires and releases it automatically.
>
> For the ETag, `etag master LocalLastChangedAt`, with `@Semantics.systemDateTime.localInstanceLastChangedAt: true` on the field and `readonly` in the behavior definition, so the framework maintains it. The client gets it as an HTTP ETag and sends it back in `If-Match`; a stale value is rejected. And `use etag` in the projection behavior definition — without that the protection is silently off for that service.
>
> Because we use draft we also needed `total etag LastChangedAt`, on a **different** field. That covers draft resumability: the exclusive lock expires after a timeout, and after that RAP compares the draft's stored total ETag against the active instance to decide whether the draft is still resumable.
>
> We tested the ETag case over HTTP rather than through EML, because the comparison happens in the OData layer — `GET`, `PATCH` with the current ETag, then `PATCH` again with the old one and expect a 412.

How did you prevent duplicate processing?

Shape: name the layer each mechanism sits in — key uniqueness, business-key validation, status guard, idempotent job design.

> On several levels, because they fail differently.
>
> The technical key is a UUID with `numbering : managed`, so duplicates are impossible at that level. The **business** key is a separate concern: we draw it from a number range in a determination declared `on save`, not `on modify`, so a discarded draft does not consume a number and we do not get gaps.
>
> Against duplicate business documents we used a validation on save checking the natural key against existing records — set-based, one `SELECT ... FOR ALL ENTRIES` outside the loop, not a `SELECT SINGLE` per instance.
>
> For the action that creates a follow-on document, the guard is a **status field** plus feature control: once the document is created the status changes, the instance authorization and feature control disable the action, and the action implementation itself re-reads the status and skips instances that already have a follow-on — because feature control is only a UI hint and an API caller can still invoke the action.
>
> For background processing we made the job idempotent: it selects by status, so a re-run after a failure picks up only what is still open, and it commits in chunks so a failure loses one chunk rather than the whole run.

How did you implement validations and determinations?

> Declared in the base behavior definition so every consumer gets them, not in the projection — a projection-level validation is bypassed by EML callers, jobs and any second service.
>
> Validations `on save`, so the user is not nagged mid-typing, and every one of them listed in `draft determine action Prepare` so the messages appear while the draft is open rather than only at activation. Each one fills **both** `failed` and `reported`, and uses a `%state_area` that it clears at the start of the loop — otherwise the message stays on screen after the user fixes the data. Messages carry `%element-<Field>` so Fiori highlights the right field.
>
> Determinations: `on modify` for anything the user should see immediately, like the price roll-up, and `on save` for the number range. All of them idempotent — they read first and skip instances where the value is already set, because a determination can be called more than once.
>
> Everything reads the buffer with `READ ENTITIES ... IN LOCAL MODE`, never `SELECT` on our own tables, because in a draft app a `SELECT` reads the active row while the user is editing the draft. Master-data lookups are `SELECT`, but set-based and outside the loop.
>
> And for every calculated field we declared a `side effect`, otherwise the value is correct on the server and stale on screen — which looks exactly like a broken determination.

How did you debug a failed save?

Shape: describe a method, not a tool list.

> The first thing is to get out of the browser. I reproduce it in a console class implementing `IF_OO_ADT_CLASSRUN`, driving the BO with EML, and print `failed`, `reported` and `sy-subrc` after `COMMIT ENTITIES`. That immediately tells me which half of the problem I have: `sy-subrc = 4` means the **early save phase** rejected it and it is recoverable — a validation — and `reported` names which one. `sy-subrc = 8` means a **late-phase** failure, which needs an explicit `ROLLBACK ENTITIES` and means something that could fail was left in `save_modified` instead of `check_before_save`.
>
> `COMMIT ENTITIES IN SIMULATION MODE` is very useful here: it runs the whole early save phase, including all the on-save validations, and rolls back, so I can iterate without writing data.
>
> If `failed` is filled but `reported` is empty, the validation is rejecting without a message — that is the defect, not the data.
>
> For phase-level problems I set breakpoints in `finalize`, `check_before_save` and `cleanup_finalize`. Landing in `cleanup_finalize` tells me the early phase rejected the save, and `failed` from the previous method tells me which instance.
>
> From the UI side, the browser Network tab is underused: a failed `POST` returns a JSON body with the full message list, including messages the UI chose not to display.
>
> One real case: a determination appeared not to run. It ran; the field was `readonly` and the EML call was missing `IN LOCAL MODE`, so the write was silently rejected.

How did you handle external API integration?

Shape: phase rules first, then error handling, then idempotency.

> The constraint that drives the design is the phase rules. Synchronous RFC and HTTP calls are **not allowed in the late save phase**, and update-task registration is only allowed there. So a synchronous call to an external system cannot sit in `save_modified`.
>
> What we did depended on the direction. For calling another RAP BO — a released `I_...TP` API — we used EML in the action handler, in the interaction phase, so both business objects commit in the same LUW and there is no `COMMIT ENTITIES` inside our behavior pool.
>
> That API used **late numbering**, so it returned a preliminary ID rather than a document number. Resolving it needs `CONVERT KEY`, which is only legal inside `save_modified` — which is precisely why that BO had to be `managed with unmanaged save`. We passed the preliminary IDs from the action to the saver through a `CLASS-DATA` on the global behavior pool class, resolved them in `save_modified`, wrote the real number into our table, and cleared the variable in both `cleanup` and `cleanup_finalize` — because class data survives the transaction and would otherwise leak into the next save.
>
> For outbound calls to a non-SAP system we did **not** call synchronously during the save. We wrote the payload to an outbound queue in `save_modified` under `with additional save`, and a separate job processes it with retry, timeout handling and idempotency keys, so a failed call does not roll back the business document and a retry does not create duplicates.
>
> Where a call had to influence whether the save is allowed at all, the check went in `check_before_save` — a test-mode call — so the transaction was still recoverable.

How did you transport and activate the service in QA and production?

> See Chapter 25, §25.12 for the full answer. The compressed version: all repository objects on transport requests from a real development package, never `$TMP`; import order following the build order if the change set is split; then the non-transportable steps — **publishing the service binding** in each system, number range **intervals**, ICF activation, Launchpad catalog and tiles, role assignment, job scheduling. Then a post-import checklist run as a **business test user**, not as a developer, and UAT on the real Launchpad app rather than the ADT preview.

How did you optimize CDS and EML performance?

> See Chapter 24, §24.12. The compressed version: associations rather than joins because associations are lazy; calculations in the interface view rather than virtual elements because virtual elements are computed in ABAP and cannot be filtered or sorted on HANA; a mandatory selective filter on any List Report over a large table; no `SELECT` inside a loop — collect distinct keys, one `FOR ALL ENTRIES`, hashed lookup; `AUTHORITY-CHECK` per distinct value rather than per row, because instance authorization runs for every row the list displays; `FIELDS ( )` instead of `ALL FIELDS`; `LINK` instead of `RESULT` when only existence matters; and chunked commits for mass EML so the buffer and lock table stay bounded and the job is restartable. `ST05` on a page load finds SELECTs in loops immediately; `SQLM` in production tells you what is actually slow.

How did you test the application?

> See Chapter 23, §23.12. The compressed version: four levels — Data Preview for the CDS model, EML console classes for the behavior (the only place you see `failed`, `reported` and `sy-subrc`), those promoted to ABAP Unit tests with `cl_cds_test_environment`, `$metadata` for the service, and the Fiori preview for the flows. Validations asserted on **both** `failed` and `reported`; feature control asserted via `GET PERMISSIONS`; ETag and locking tested over HTTP or with two sessions because they happen above the EML layer; and authorization integration-tested with real roles, because there is no supported way to impersonate a user in ABAP Unit.

27.5 Questions to ask them

Asking good questions is part of a senior interview. These are specific enough to show you have built something:

- Is the landscape ABAP Cloud / clean core, or classic on-premise with unrestricted ABAP?
- Are new developments managed RAP, or are you wrapping a lot of existing logic unmanaged?
- Do you use draft, and who owns the draft clean-up job?
- How do you handle service binding publication in QA and production — do developers have authorization there, or is it a Basis hand-off?
- Is there an existing convention for package layout and for `ZR_` / `ZC_` naming?
- Do you run ATC and ABAP Unit as a gate on transport release?
- How is authorization tested — is there a set of role-specific test users in QA?

27.6 Ten-minute revision sheet

Term	One line
RAP layers	table → interface view → BDEF + ABP → projection + projection BDEF → MDE → service definition → binding
managed	framework provides buffer and save
unmanaged	you provide buffer, CRUD handlers, lock and save
additional save	framework saves; your `save_modified` does extra work
unmanaged save	you save, in `save_modified`; no `persistent table`
Interface view	the BO model; neutral, reusable, no `@UI`
Projection view	service-specific subset; `as projection on`; value helps, DCL, MDE attach here
BDEF	the contract: implementation type, tables, lock, etag, auth, operations, validations, determinations, actions
ABP	`FOR BEHAVIOR OF`; local handler (`CL_ABAP_BEHAVIOR_HANDLER`) + local saver (`CL_ABAP_BEHAVIOR_SAVER`)
Determination	derives data; `on modify` or `on save`
Validation	checks data; must not change it; blocks via `failed`
Action	operation beyond CRUD; needs `use action` in the projection BDEF to exist
Side effect	tells the UI what to re-read; declarative; no handler
Draft	second copy in a draft table; `%tky`; `total etag`; Edit/Activate/Discard/Resume/Prepare
Save sequence	finalize → check_before_save → [point of no return] → adjust_numbers → save → cleanup
`sy-subrc` **after COMMIT ENTITIES**	0 ok · 4 early save failed (recoverable) · 8 late save failed (rollback required)
`%cid`	consumer-invented id before a key exists
`%tky`	key + `%is_draft`; always use it in draft BOs
`%pid`	preliminary id, late numbering; resolve with `CONVERT KEY` in `save_modified`
`%control`	which fields the consumer sent
`failed`	blocks the operation
`reported`	carries the messages
`mapped`	reports the assigned keys (create only)

Term	One line
DCL	filters reads; does not stop writes
BDEF authorization	controls modification; `auth-allowed` / `auth-unauthorized`
Feature control	UI hint only; `fc-o-` for operations, `fc-f-` for fields
EML	`READ` / `MODIFY` / `COMMIT` / `ROLLBACK ENTITIES`, `GET PERMISSIONS`, `SET LOCKS`
`IN LOCAL MODE`	inside the pool, always: buffer-direct, skips checks, allows writing `readonly` fields
`strict ( 2 )`	strictest checks; always on for new development